

Biología Computacional

De la entidad biológica a la decisión

Álvaro Serrano Navarro

Edición 2

@toc spanish

Índice general

1. La información biológica y su representación en Python	1
1.1. Por qué la biología se dejó leer como información	1
1.2. El Dogma Central y lo que lo rompe	2
1.3. Ácidos nucleicos: el soporte del texto	3
1.4. Proteínas: la maquinaria	4
1.5. Carbohidratos: energía, andamio y etiqueta	6
1.6. Lípidos: la frontera de la célula	7
1.7. Qué hace Python cuando guarda una secuencia	8
1.8. Cuatro operaciones sobre secuencias	9
1.9. Un módulo propio de secuencias	11
1.10. Un genoma que no cabía en sí mismo	12
1.11. Tres errores que va a cometer	13
1.12. Síntesis y tablas de diagnóstico	13
1.13. Glosario	14
1.14. Con qué sigue este capítulo	15
1.15. Fuentes y reproducibilidad	15
Anexo práctico · Escribir y verificar un módulo de secuencias	15
2. Biomoléculas como objetos computables	19
2.1. Biomoléculas como objetos computables: encapsulación e invariantes	19
2.2. El paradigma Secuencia → Estructura → Función	20
2.3. La biofísica cuántica del enlace peptídico	21
2.4. El diagrama de Ramachandran y conformación del esqueleto	22
2.5. Jerarquía estructural y fuerzas biofísicas estabilizadoras	23
2.6. Propiedades fisicoquímicas computables	25
2.7. Escala de hidropatía de Kyte-Doolittle y perfiles	26
2.8. Desnaturalización proteica y estabilidad conformacional	27
2.9. Propiedades emergentes: alosterismo y cooperatividad	28
2.10. Diseño computacional en Python: Módulos de proteínas	28
2.11. Peligros computacionales y actividad de depuración	29
2.12. Síntesis operativa y tablas de diagnóstico	30
2.13. Glosario conceptual	30
2.14. Conexiones y mapa del curso	31
2.15. Fuentes y reproducibilidad	31
Anexo práctico · Modelado computacional de péptidos y propiedades biofísicas	31

3. Genomas, epigenética y variantes	34
3.1. La arquitectura física y funcional del genoma eucariota	34
3.2. Estructura de la cromatina: Nucleosomas y empaquetamiento	36
3.3. El código epigenético y modificaciones de histonas	37
3.4. Variantes genéticas: Clasificación física y espectro mutacional	39
3.5. Consecuencias funcionales en regiones codificantes	39
3.6. El formato estándar Variant Call Format (VCFv4.2)	40
3.7. Variantes estructurales y firmas mutacionales	40
3.8. Diseño computacional en Python: Procesador de variantes y CpG	41
3.9. Peligros computacionales y actividad de depuración	41
3.10. Síntesis operativa y tablas de diagnóstico	42
3.11. Glosario conceptual	43
3.12. Conexiones y mapa del curso	43
3.13. Fuentes y reproducibilidad	43
Anexo práctico · Análisis computacional de variantes genómicas y firmas epigenéticas	43
4. Transcripción, procesamiento de ARN y expresión génica	46
4.1. El transcriptoma como estado funcional dinámico	47
4.2. Mecanismo de la transcripción y complejo de pre-iniciación	47
4.3. Diversidad funcional del ARN: codificantes y no codificantes	49
4.4. Procesamiento co-transcripcional del pre-ARNm eucariota	49
4.5. La traducción: cómo el ribosoma lee el mensaje	50
4.6. El código genético y sesgo de codones (GC3)	51
4.7. Marcos abiertos de lectura (ORFs) en seis fases	51
4.8. Cuantificación transcriptómica: Normalización TPM	52
4.9. Redes de regulación transcripcional y lógica combinatoria	53
4.10. Diseño de módulos transcriptómicos en Python	54
4.11. Peligros computacionales y actividad de depuración	54
4.12. Síntesis operativa y tablas de diagnóstico	55
4.13. Glosario conceptual	55
4.14. Conexiones y mapa del curso	55
4.15. Fuentes y reproducibilidad	56
Anexo práctico · Flujo transcripcional, traducción en 6 marcos y cuantificación TPM	56
5. Secuencia, estructura y función de proteínas	58
5.1. El paradigma Secuencia a Estructura a Función: marco y límites	59
5.2. Representación cristalográfica: El formato Protein Data Bank (PDB)	61
5.3. Geometría molecular 3D: Distancias, centroides y contactos	62

5.4.	Mapas de contacto y matrices de distancia	63
5.5.	Comparación estructural: RMSD y el algoritmo de Kabsch	63
5.6.	Jerarquía de dominios y clasificaciones CATH/SCOP	64
5.7.	Diseño computacional en Python: Módulo de geometría PDB	65
5.8.	Peligros computacionales y actividad de depuración	66
5.9.	Síntesis operativa y tablas de diagnóstico	67
5.10.	Glosario conceptual	67
5.11.	Conexiones y cierre de la Parte I	68
5.12.	Fuentes y reproducibilidad	68
Anexo práctico · Estructura 3D de proteínas, matrices de contacto y RMSD		68
6. Tecnologías de secuenciación y diseño experimental		71
6.1.	Del corte químico a la lectura masiva	72
6.2.	Antes de la máquina: la preparación de la librería	72
6.3.	Segunda generación: Secuenciación masiva en paralelo (NGS)	73
6.4.	Tercera generación: Molécula única y lecturas largas	74
6.5.	Métricas teóricas: Cobertura y profundidad genómica	74
6.6.	El modelo estadístico de Lander-Waterman	75
6.7.	Dimensionamiento experimental y control de sesgo GC	75
6.8.	Diseño experimental riguroso en genómica	76
6.9.	Diseño computacional en Python: Calculadora genómica	77
6.10.	Peligros computacionales y actividad de depuración	78
6.11.	Síntesis operativa y tablas de diagnóstico	78
6.12.	Glosario conceptual	79
6.13.	Conexiones y mapa del curso	79
6.14.	Fuentes y reproducibilidad	79
Anexo práctico · Tecnologías de secuenciación, cobertura Lander-Waterman y diseño experimental		80
7. Formatos de secuencias y control de calidad		82
7.1.	El formato FASTQ y la llamada de bases	83
7.2.	La escala de calidad Phred y codificación ASCII	84
7.3.	Diagnóstico con FastQC: Perfiles típicos y anomalías	85
7.4.	Preprocesamiento computacional: Recorte por ventana deslizante	86
7.5.	Genomas de referencia y sistemas de coordenadas	87
7.6.	Diseño computacional en Python: Procesador FASTQ	88
7.7.	Peligros computacionales y actividad de depuración	89
7.8.	Síntesis operativa y tablas de diagnóstico	89
7.9.	Glosario conceptual	90

7.10. Conexiones y mapa del curso	90
7.11. Fuentes y reproducibilidad	90
Anexo práctico · Formato FASTQ, decodificación Phred y filtrado por ventana deslizante	90
8. Alineamiento de lecturas a secuencias de referencia	92
8.1. El desafío computacional del mapeo genómico	93
8.2. Indexación por tablas hash y la heurística Seed-and-Extend	94
8.3. La Transformada de Burrows-Wheeler y el FM-Index	95
8.4. El formato Sequence Alignment/Map (SAM/BAM)	95
8.5. Flujos de trabajo completos NGS y alineamiento de transcriptomas	98
8.6. Diseño computacional en Python: Mapeador y parser CIGAR	98
8.7. Peligros computacionales y actividad de depuración	99
8.8. Síntesis operativa y tablas de diagnóstico	100
8.9. Glosario conceptual	100
8.10. Conexiones y cierre de la Parte II	100
8.11. Fuentes y reproducibilidad	101
Anexo práctico · Indexado de genomas, algoritmo Seed-and-Extend y parsing CIGAR	101
9. De secuencias homólogas a hipótesis evolutivas	103
9.1. La inferencia comienza con una pregunta	104
9.2. Leer árboles sin añadir historia que no contienen	104
9.3. Antes del alineamiento: ¿qué relación comparamos?	108
9.4. Homología posicional: el alineamiento es una hipótesis	110
9.5. De diferencias observadas a distancias de modelo	111
9.6. UPGMA: una traza transparente y un supuesto fuerte	115
9.7. Unión de vecinos: distancias sin reloj estricto	117
9.8. De una matriz resumida a los caracteres	118
9.9. Modelo, selección y herramienta	121
9.10. Apoyo no significa verdad	123
9.11. Sensibilidad: perturbar una decisión con intención	125
9.12. Errores y límites que un árbol bonito puede ocultar	125
9.13. Python como cuaderno verificable	127
9.14. Un caso integrado: de la pregunta a una afirmación limitada	129
9.15. Síntesis: qué puede defenderse y qué queda abierto	129
Anexo práctico · De secuencias a una hipótesis filogenética	130
10. Regulación génica, clonación y vectores recombinantes	133
10.1. Regulación génica procariota: La lógica molecular de los operones	134

10.2. Vectores de clonación y sistemas de expresión recombinante	136
10.3. Anatomía de un plásmido de expresión de proteínas	136
10.4. Diseño computacional de cebadores de PCR	137
10.5. Detección de dianas de restricción y métodos de ensamblaje	139
10.6. Diseño computacional en Python: Módulo de diseño recombinante	139
10.7. Peligros computacionales y actividad de depuración	141
10.8. Síntesis operativa y tablas de diagnóstico	141
10.9. Glosario conceptual	142
10.10. Conexiones y cierre de la Parte III	142
10.11. Fuentes y reproducibilidad	142
Anexo práctico · Diseño in silico de cebadores, operones procariotas y ensamblaje recombinante	143
11. Estructura tridimensional de proteínas: Métodos y validación	146
11.1. La estructura macromolecular y la evidencia experimental	146
11.2. Técnicas biofísicas de determinación estructural	147
11.3. Formatos estructurales: Del PDB clásico al estándar mmCIF	147
11.4. Geometría tridimensional, RMSD y factores B	149
11.5. Validación estereoquímica y el diagrama de Ramachandran	150
11.6. Clasificación jerárquica de dominios: CATH y SCOP	151
11.7. Diseño computacional en Python: Procesador PDB y métricas	151
11.8. Peligros computacionales y actividad de depuración	152
11.9. Síntesis operativa y tablas de diagnóstico	152
11.10. Glosario conceptual	153
11.11. Conexiones y mapa del curso	153
11.12. Fuentes y reproducibilidad	153
Anexo práctico · Parsing posicional PDB, métricas de distancia 3D, RMSD y factores B	153
12. Modelado computacional de estructuras	156
12.1. El problema del plegamiento y paisajes conformacionales	157
12.2. Modelado comparativo por homología	158
12.3. Reconocimiento de pliegues y enhebrado (<i>Threading</i>)	160
12.4. Modelado ab initio y ensamblaje de fragmentos (Rosetta)	160
12.5. Evaluación de calidad y validación de modelos	161
12.6. Diseño computacional en Python: Módulo de predicción y validación	162
12.7. Peligros computacionales y actividad de depuración	163
12.8. Síntesis operativa y tablas de diagnóstico	163
12.9. Glosario conceptual	164
12.10. Conexiones y mapa del curso	164

12.11. Fuentes y reproducibilidad	164
Anexo práctico · Métricas de predicción estructural, criterio de Metropolis y evaluación de choques	165
13. Aprendizaje profundo y AlphaFold en biología estructural	168
13.1. Qué cambió con el aprendizaje profundo	169
13.2. Coevolución molecular en alineamientos múltiples (MSAs)	169
13.3. Arquitectura de AlphaFold2: Del Evoformer al Structure Module	169
13.4. Métricas de confianza: pLDDT y matrices PAE	171
13.5. Fronteras emergentes: AlphaFold3 y Modelos de Lenguaje	172
13.6. Diseño computacional en Python: Clasificador pLDDT y PAE	172
13.7. Peligros computacionales y actividad de depuración	174
13.8. Ética y auditoría de las predicciones	174
13.9. Síntesis operativa y tablas de diagnóstico	175
13.10. Glosario conceptual	175
13.11. Conexiones y mapa del curso	175
13.12. Fuentes y reproducibilidad	176
Anexo práctico · Métricas de confianza pLDDT, matrices PAE y auditoría de modelos de IA	176
14. Cribado virtual y diseño computacional de fármacos	179
14.1. Del cribado masivo al diseño racional	180
14.2. Cribado virtual: el flujo de trabajo	180
14.3. Bases de datos y representación de moléculas pequeñas	181
14.4. Acoplamiento molecular	182
14.5. AutoDock Vina	183
14.6. Cribado basado en ligandos	184
14.7. Filtros de propiedades	186
14.8. Curado por dinámica molecular	187
14.9. Ranking, controles y señuelos	189
14.10. El módulo en Python	189
14.11. Actividad de depuración	190
14.12. La síntesis del libro	191
14.13. Glosario	191
14.14. Cierre	192
14.15. Fuentes y reproducibilidad	192
Anexo práctico · Un cribado virtual completo, con sus controles	192

La información biológica y su representación en Python

BC-CH01 – Biología Computacional

Edición 2

Pregunta del tema. Una cadena de texto con cuatro letras distintas puede ser un fichero cualquiera o el programa completo de un ser vivo. ¿Qué convierte una secuencia de ADN en información, y qué operaciones sobre ella significan algo biológico y no solo sintáctico?

Producto final. Un módulo propio en Python que calcule contenido GC, complemento reverso, transcripción y perfiles por ventana, con una guarda que rechace lo que no es ADN.

Mapa del tema

1.1. Por qué la biología se dejó leer como información	1
1.2. El Dogma Central y lo que lo rompe	2
1.3. Ácidos nucleicos: el soporte del texto	3
1.4. Proteínas: la maquinaria	4
1.5. Carbohidratos: energía, andamio y etiqueta	6
1.6. Lípidos: la frontera de la célula	7
1.7. Qué hace Python cuando guarda una secuencia	8
1.8. Cuatro operaciones sobre secuencias	9
1.9. Un módulo propio de secuencias	11
1.10. Un genoma que no cabía en sí mismo	12
1.11. Tres errores que va a cometer	13
1.12. Síntesis y tablas de diagnóstico	13
1.13. Glosario	14
1.14. Con qué sigue este capítulo	15
1.15. Fuentes y reproducibilidad	15

Cómo usar este tema

Este capítulo tiene dos mitades que conviene no separar. La primera es química: qué son las moléculas que vamos a representar y qué propiedades suyas importan. La segunda es computacional: cómo guarda Python esa información y qué cuesta cada operación. Están juntas porque las decisiones de la segunda solo se entienden desde la primera. Que una secuencia se escriba siempre de 5' a 3' no es una convención de formato: es la razón por la que el complemento reverso lleva una inversión y no solo un cambio de letras.

Al terminar sabrá: explicar por qué una secuencia de ADN es información y no solo una cadena; nombrar las cuatro familias de biomoléculas y decir qué representa cada una en un programa; predecir qué ocurre en memoria cuando modifica una lista de secuencias; calcular a mano el contenido GC y el complemento reverso, y comprobar su cálculo con el oráculo del capítulo; y escribir funciones que rechacen una entrada inválida en lugar de producir un número sin sentido.

La ruta **esencial** son las cuatro familias de biomoléculas, el modelo de memoria de Python, las cuatro operaciones sobre secuencias y la ventana deslizante en tiempo $O(N)$. La ruta de **estudio** añade la estereoquímica de los azúcares, la física de la bicapa lipídica, el caso de $\Phi X174$, la actividad de depuración y el anexo. Antes de ejecutar cada orden, escriba lo que espera obtener: el capítulo está construido sobre ese contraste.

1.1 Por qué la biología se dejó leer como información

ESENCIAL. Durante casi todo el siglo XIX la biología describía. Clasificaba organismos por su forma, seguía el balance químico de una reacción, medía. Era una ciencia de sustancias y de procesos, no de mensajes. Que hoy hablemos de «leer» un genoma, de «editar» o de «programar» una célula procede de tres trabajos que en su momento no tenían nada que ver entre sí.

El primero es de Turing, en 1936 [71]. Turing no estudiaba células: quería saber qué problemas puede resolver un procedimiento mecánico. Para responderlo describió una máquina que lee y escribe símbolos discretos sobre una cinta, y demostró que ese artefacto tan pobre basta para cualquier cómputo. La consecuencia que nos interesa es

indirecta pero decisiva: si un proceso manipula símbolos discretos según reglas fijas, es computación, sea cual sea el material del que esté hecho.

El segundo es de Shannon, en 1948 [66]. Shannon trabajaba en telefonía y necesitaba medir cuánta información lleva un mensaje. Su respuesta, la cantidad de incertidumbre que el mensaje elimina, tiene una propiedad extraña: no depende del soporte. La misma información viaja por un cable, por el aire o por una molécula. Nada obliga a que el soporte de un mensaje sea electrónico.

El tercero es de Watson, Crick y Franklin, en 1953 [74]. La doble hélice mostró que el material hereditario es una secuencia lineal de cuatro piezas químicas distintas que se repiten sin patrón periódico. Es decir: exactamente el tipo de objeto del que hablaban los dos primeros.

Puestos uno junto a otro, los tres dicen algo que ninguno decía por separado. Una célula almacena instrucciones en un polímero, las copia, las transmite a su descendencia y las ejecuta. Eso es procesamiento de información, con un soporte químico en lugar de electrónico.

Atención. Conviene tomarse la metáfora con cuidado. Que el ADN se comporte como un texto no significa que la célula sea un ordenador. Un ordenador separa el programa de la máquina que lo ejecuta; una célula no siempre lo hace, como veremos en cuanto miremos las excepciones del Dogma Central. La representación digital es una herramienta que funciona muy bien para casi todo lo que haremos en este libro, y este capítulo se ocupa también de señalar dónde deja de funcionar.

De ahí sale el programa de trabajo de la biología computacional, que en la práctica exige tres cosas a la vez. Hay que entender el sustrato: qué fuerzas y qué restricciones termodinámicas gobiernan las moléculas, porque son las que hacen que unas operaciones tengan sentido y otras no. Hay que formalizar: convertir el fenómeno en un objeto discreto, una cadena, un grafo, una distribución. Y hay que implementar de forma que el cálculo termine, lo que con genomas de 10^6 a 10^{10} bases no es una cuestión de estilo sino de viabilidad.

La biología computacional representa la información biológica mediante objetos discretos y algoritmos deterministas cuyo coste puede acotarse.

1.2 El Dogma Central y lo que lo rompe

ESENCIAL Si la célula guarda instrucciones, ¿por dónde circulan? La respuesta canónica la enunció Crick en 1958 y consta de tres procesos, que la figura 1.1 resume.

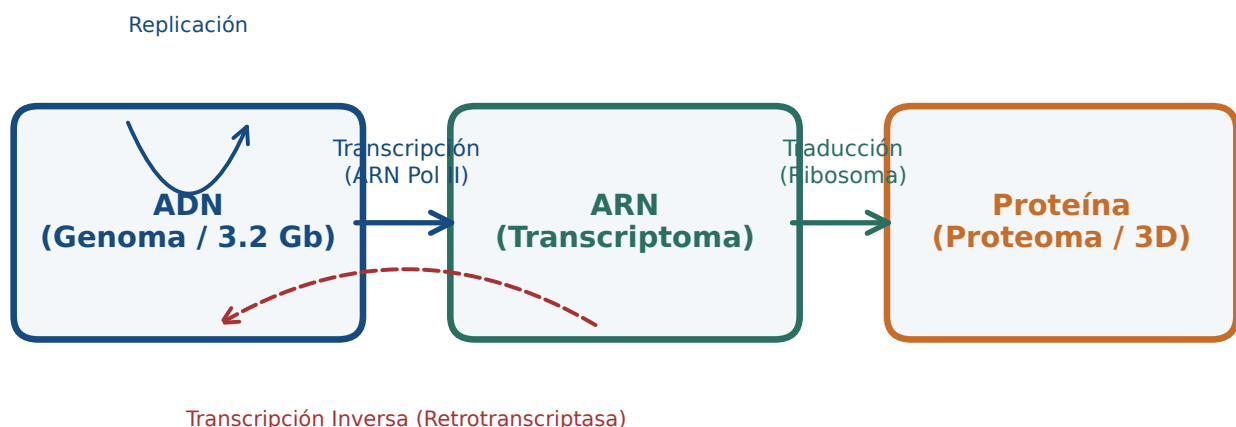


Figura 1.1: Los tres flujos del Dogma Central y dos de sus excepciones. Figura original generada por `verification/make_figures.py`.

- **Replicación**, de ADN a ADN. Es la copia del archivo completo antes de cada división celular.
- **Transcripción**, de ADN a ARN. Es la lectura de un fragmento concreto, un gen, para obtener una copia de trabajo.

- **Traducción**, de ARN a proteína. El ribosoma ejecuta esa copia y convierte una secuencia de nucleótidos en una secuencia de aminoácidos.

Enunciado así, el dogma reparte los papeles con nitidez: los ácidos nucleicos guardan y transmiten, las proteínas ejecutan. El archivo por un lado, la maquinaria por otro. Es un reparto cómodo, y es el que justifica que este libro represente las secuencias como cadenas de texto.

Cuatro fenómenos conocidos desdibujan ese reparto. No son curiosidades de examen: son justamente los casos donde la representación que vamos a usar se queda corta, y por eso vale la pena conocerlos desde el principio.

- La **transcripción inversa**, de ARN a ADN, que realizan los retrovirus y las retrotransposasas. La flecha va hacia atrás.
- La **replicación del ARN**, en virus cuyo genoma es ARN. El archivo nunca pasa por ADN.
- Las **ribozimas**, moléculas de ARN con actividad catalítica. El propio centro peptidil-transferasa del ribosoma, el que forma el enlace peptídico, es una de ellas. Aquí el supuesto portador de información es la maquinaria.
- Los **priones**, proteínas capaces de imponer su conformación anómala a otras copias de la misma proteína. La información se propaga sin ácido nucleico de por medio, en el plegamiento y no en la secuencia.

Atención. Las dos últimas importan especialmente para lo que sigue. Un prión demuestra que hay información biológica que la secuencia no captura: dos moléculas con la misma secuencia y distinto plegamiento se comportan de forma distinta, y ninguna cadena de texto distingue una de otra. Esa limitación de la representación reaparecerá en los capítulos de estructura de proteínas, donde la secuencia deja de bastar.

1.3 Ácidos nucleicos: el soporte del texto

ESENCIAL La materia viva se organiza en cuatro grandes familias de moléculas. Cada una tiene una lógica propia, y esa lógica determina cómo la representamos en un programa [3]. Empezamos por la que este libro usa más.

Las cuatro familias biomoleculares, ácidos nucleicos, proteínas, carbohidratos y lípidos, tienen propiedades estructurales y funcionales distintas que condicionan su representación computacional.

Los ácidos nucleicos, ADN y ARN, son los portadores del material hereditario en los tres dominios de la vida y en los virus. Son polímeros lineales sin ramificar, hechos de **nucleótidos**, y cada nucleótido tiene tres piezas unidas covalentemente:

1. Un azúcar de cinco carbonos en forma de anillo: **2-desoxi-D-ribosa** en el ADN, que carece del hidroxilo en la posición 2', y **D-ribosa** en el ARN, que sí lo tiene. Ese hidroxilo de más es la razón química de que el ARN sea mucho más frágil: hace la molécula más reactiva y sensible a la hidrólisis. Cuando en el laboratorio el ARN se degrada y el ADN no, el culpable es ese único grupo.
2. Un grupo fosfato unido al carbono 5' del azúcar.
3. Una base nitrogenada unida al carbono 1' por un enlace β -N-glucosídico.

Las bases se reparten en dos familias por su forma: **purinas**, con dos anillos fusionados, que son la adenina (A) y la guanina (G); y **pirimidinas**, con un solo anillo, que son la citosina (C), la timina (T, solo en el ADN) y el uracilo (U, solo en el ARN). Que una purina se empareje siempre con una pirimidina no es casualidad: es lo que mantiene constante la anchura de la doble hélice.

Un nucleótido consta de fosfato, pentosa y base nitrogenada; el ADN bicatenario con desoxirribosa y timina difiere del ARN monocatenario con ribosa y uracilo.

1.3.1 Por qué una secuencia tiene dirección

Los nucleótidos se enlazan mediante **enlaces fosfodiéster**: un fosfato une el carbono 3' del nucleótido anterior con el carbono 5' del siguiente. Como los dos extremos que se unen son químicamente distintos, la cadena resultante no es simétrica. Tiene un **extremo 5'**, que termina en fosfato, y un **extremo 3'**, que termina en un

hidroxilo libre. Ese hidroxilo del extremo 3' es el punto donde las polimerasas añaden el nucleótido siguiente, y por eso toda síntesis de ADN avanza en el sentido 5' → 3' y nunca al revés.

De ahí sale la convención que gobierna todo lo que haremos:

Concepto. Toda secuencia de ácido nucleico se escribe, se almacena y se procesa en sentido 5' → 3', salvo que se declare explícitamente lo contrario. Cuando alguien le entrega la cadena ATGC sin más aclaración, le está entregando 5'-ATGC-3'. Esta convención es la razón de que más adelante el complemento reverso lleve una inversión: sin ella, el resultado estaría escrito al revés de como todo el mundo lo lee.

1.3.2 El apareamiento de bases y la fuerza que lo mantiene

En el ADN de doble cadena, las dos hebras corren **antiparalelas**, una en sentido 5' → 3' y la otra en sentido 3' → 5', y se enrollan en una hélice dextrógira de unos 2,0 nm de diámetro. Lo que mantiene enfrentadas las bases correctas son puentes de hidrógeno:

- **Adenina con timina:** dos puentes de hidrógeno, entre N1 y H-N3, y entre N6-H y O4.
- **Guanina con citosina:** tres puentes de hidrógeno, entre O6 y H-N4, entre N1-H y N3, y entre N2-H y O2.

Los tres puentes del par G-C estabilizan más que los dos del par A-T, y por eso una región rica en G y C funde a mayor temperatura. Pero conviene no confundir estos puentes con enlaces covalentes, ni escribirlos como si lo fueran.

Atención. Un puente de hidrógeno es una interacción *no* covalente, entre uno y dos órdenes de magnitud más débil que el enlace covalente que une los átomos dentro de cada base. Esa debilidad no es un defecto: es exactamente lo que permite que la doble hélice se abra y se cierre durante la replicación y la transcripción sin romper ninguna molécula. Si los pares de bases estuvieran unidos covalentemente, copiar el genoma exigiría romper y rehacer enlaces químicos, y la célula no podría hacerlo miles de veces al día.

Hay un segundo punto donde es fácil equivocarse. Los valores de energía que se manejan en la práctica, del orden de -1 a -2 kcal/mol para pares A-T y de -2,5 a -3,5 para pares G-C, no son la energía de los puentes de hidrógeno de un par aislado. Corresponden a pares de bases *contiguos* e incluyen el apilamiento aromático entre bases vecinas, que aporta una parte sustancial de la estabilidad. Por eso el modelo estándar para predecir la temperatura de fusión de un oligonucleótido se llama de **vecino más próximo**: mira parejas de pares, no pares sueltos [64].

Concepto. Retenga la consecuencia práctica: el contenido GC de una secuencia predice bastante bien su estabilidad, pero no la determina. Dos secuencias con el mismo contenido GC y distinto orden de bases funden a temperaturas distintas, porque el apilamiento depende de qué base sigue a cuál. El capítulo sobre diseño de cebadores retoma esta idea con el cálculo completo.

1.4 Proteínas: la maquinaria

Si los ácidos nucleicos son el archivo, las **proteínas** son quien hace el trabajo. Catalizan reacciones, transportan moléculas a través de membranas, reciben señales, mueven la célula, la sostienen y reconocen lo que es propio y lo que no.

Una proteína es un polímero lineal de **aminoácidos** unidos por enlaces peptídicos. Los aminoácidos que la vida usa para construirlas son veinte, y todos comparten el mismo esqueleto: un carbono central, el C α , con cuatro sustituyentes, un hidrógeno, un grupo carboxilo (-COOH), un grupo amino (-NH₂) y una cadena lateral variable que se abrevia -R. Toda la variedad funcional de las proteínas sale de esa cuarta posición.

Los veinte aminoácidos estándar comparten un carbono alfa con grupos amino y carboxilo y se diferencian por una cadena lateral que determina si son no polares, polares, ácidos o básicos.

1.4.1 Todos son L, y eso quiere decir algo concreto

Con la excepción de la glicina, el C α tiene cuatro sustituyentes distintos y es por tanto un centro quiral: la molécula y su imagen especular no son superponibles. Existen dos configuraciones posibles, L y D, y las proteínas de

todos los seres vivos conocidos usan solo la L.

Concepto. Que un aminoácido sea de la serie L no es una etiqueta arbitraria sino una descripción geométrica. En la proyección de Fischer, con el carboxilo dibujado arriba y la cadena lateral abajo, un aminoácido L tiene el grupo amino a la izquierda, y uno D lo tiene a la derecha. Esa homociralidad es una de las regularidades más llamativas de la vida terrestre. Los aminoácidos D no son imposibles: aparecen en las paredes celulares bacterianas y en algunos péptidos antibióticos, precisamente donde interesa que las enzimas del hospedador no los reconozcan.

1.4.2 Carga eléctrica y punto isoelectrónico

ESTUDIO A pH fisiológico, entre 7,2 y 7,4, un aminoácido libre no es neutro en sus dos extremos: el carboxilo ha perdido su protón ($-\text{COO}^-$, con $\text{p}K_{a1}$ entre 1,8 y 2,4) y el amino lo ha ganado ($-\text{NH}_3^+$, con $\text{p}K_{a2}$ entre 8,8 y 10,5). La molécula tiene dos cargas opuestas y carga neta cero. A esa forma se la llama **zwitterión**.

El **punto isoelectrónico** (pI) es el pH al que la carga neta media es exactamente cero. Para un aminoácido sin cadena lateral ionizable se calcula promediando sus dos $\text{p}K_a$:

$$\text{pI} = \frac{\text{p}K_{a1} + \text{p}K_{a2}}{2}$$

Cuando la cadena lateral también se ioniza hay tres valores, y se promedian los dos que flanquean la forma neutra. Para los ácidos (Asp, Glu) el pI queda entre 2,8 y 3,2, de modo que a pH fisiológico llevan carga -1 . Para los básicos (Lys, Arg) queda entre 9,7 y 10,8, y a pH fisiológico llevan carga $+1$. De ahí sale una regla útil: a pH fisiológico, la superficie de una proteína está dominada por los residuos cargados, y su carga neta decide cómo se comporta en una columna de intercambio iónico o en un gel.

1.4.3 Las cuatro clases de cadena lateral

1. **No polares alifáticas:** Glicina (Gly, G), Alanina (Ala, A), Valina (Val, V), Leucina (Leu, L), Isoleucina (Ile, I), Metionina (Met, M) y Prolina (Pro, P). La glicina abre la lista por una razón: su cadena lateral es un único átomo de hidrógeno. Es el más pequeño de los veinte, el único aciral, y el que aporta flexibilidad al esqueleto allí donde cualquier otra cadena lateral estorbaría; por eso aparece en los giros cerrados y en las bisagras. La prolina está en el extremo opuesto: su cadena lateral se cierra sobre el propio nitrógeno formando un anillo, lo que le impide adoptar la mayoría de los ángulos y rompe las hélices.
2. **Aromáticas:** Fenilalanina (Phe, F), Tirosina (Tyr, Y, que lleva un hidroxilo y es por tanto anfipática) y Triptófano (Trp, W, con anillo de indol). Las tres absorben luz ultravioleta cerca de 280 nm, y esa es la razón por la que la concentración de una proteína se mide a esa longitud de onda.
3. **Polares sin carga:** Serina (Ser, S), Treonina (Thr, T), Cisteína (Cys, C), Asparagina (Asn, N) y Glutamina (Gln, Q). La cisteína merece atención aparte: su grupo tiol ($-\text{SH}$) puede oxidarse y formar un puente disulfuro covalente con otra cisteína, dentro de la misma cadena o entre cadenas distintas. Es el único enlace covalente que estabiliza la estructura terciaria, y abunda en las proteínas que trabajan fuera de la célula.
4. **Cargadas a pH fisiológico:** las ácidas, con carga negativa, son el aspartato (Asp, D) y el glutamato (Glu, E); las básicas, con carga positiva, son la lisina (Lys, K), la arginina (Arg, R) y la histidina (His, H). La histidina es un caso especial: el $\text{p}K_a$ de su anillo de imidazol ronda 6,0, muy cerca del pH fisiológico, así que puede estar protonada o desprotonada según el entorno local. Esa ambigüedad es justamente lo que la hace útil como lanzadera de protones en el centro activo de muchas enzimas.

La clasificación de aminoácidos incluye los veinte estándar; el aspartato tiene cadena lateral ácida con carga negativa y la lisina cadena básica con carga positiva a pH fisiológico.

Se reproduce con `verification/chapter_checks.py aa_class D`.

Se reproduce con `verification/chapter_checks.py aa_class K`.

Comprueba. Cuente los aminoácidos de la lista anterior antes de seguir. Si no le salen veinte, vuelva atrás: una clasificación a la que le falta un residuo es un error que se arrastra hasta el examen. La glicina es la que se pierde más a menudo, porque su cadena lateral es tan pequeña que cuesta verla como una cadena lateral.

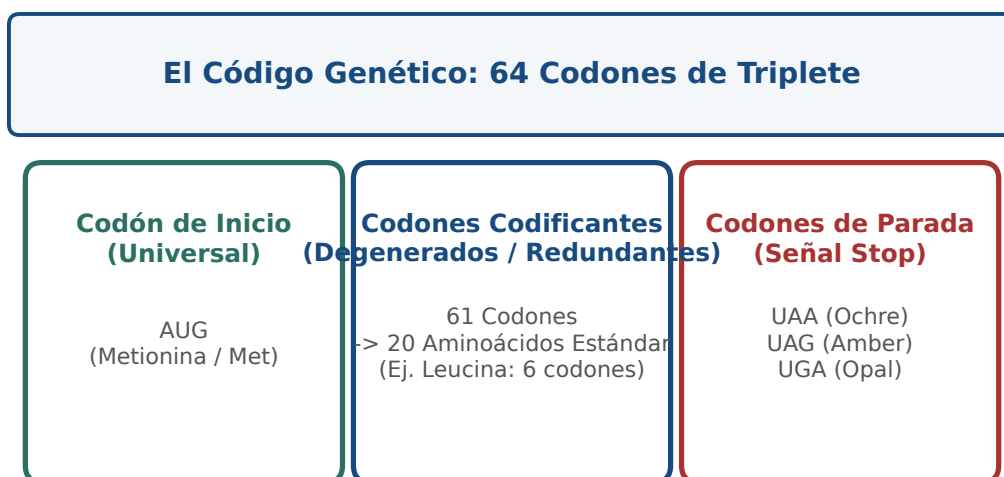


Figura 1.2: Los 64 codones del código genético y su redundancia. Figura original generada por `verification/make_figures.py`.

1.5 Carbohidratos: energía, andamio y etiqueta

ESTUDIO Los carbohidratos tienen mala fama de ser «solo azúcares», y es injusta. Químicamente son polihidroxialdehídos o polihidroxiketonas, o compuestos que al hidrolizarse los producen, y en los azúcares simples responden a la fórmula $C_n(H_2O)_n$. Hacen cuatro cosas distintas en la célula:

1. **Combustible inmediato.** La D-glucosa es el metabolito central de la glucólisis.
2. **Almacén a medio plazo.** Polímeros ramificados de glucosa: almidón en plantas, glucógeno en animales y hongos.
3. **Estructura.** Celulosa en la pared vegetal, quitina en el exoesqueleto de los artrópodos.
4. **Etiquetas de reconocimiento.** Cadenas de oligosacáridos unidas a proteínas y lípidos en la superficie celular. De ellas dependen los grupos sanguíneos del sistema AB0, buena parte del reconocimiento inmunitario y el tropismo de muchos virus, que se agarran a esos azúcares para entrar.

1.5.1 Cómo se nombran y por qué son casi todos D

Los monosacáridos se clasifican por el número de carbonos (triosas, tetrasas, pentosas, hexosas) y por dónde está el grupo carbonilo. Si está en el extremo de la cadena, en C1, es un aldehído y el azúcar es una **aldosa**: D-gliceraldehído, D-ribosa, D-glucosa. Si está en un carbono interior, normalmente C2, es una cetona y el azúcar es una **cetosa**: dihidroxiacetona, D-fructosa.

El criterio D/L es el mismo de Fischer que vimos en los aminoácidos, aplicado a otro carbono: un azúcar es de la **serie D** si el hidroxilo del centro quiral más alejado del carbonilo, que en las hexosas es el C5, queda a la derecha en la proyección vertical. Casi todos los monosacáridos biológicos son D, igual que casi todos los aminoácidos son L. Las dos homocirquialidades son independientes y ninguna tiene una explicación química obvia; son huellas del origen común de la vida terrestre.

1.5.2 El anillo, el carbono anomérico y por qué la celulosa no se come

En agua, un monosacárido de cinco o más carbonos no se queda como cadena abierta: se cierra sobre sí mismo, porque el anillo es más estable. El oxígeno del hidroxilo de C5 ataca al carbono del carbonilo y forma un enlace **hemiacetal** en las aldosas, con un anillo de seis miembros llamado **piranosa**, o **hemiacetal** en las cetosas, con un anillo de cinco llamado **furanosa**.

Al cerrarse, el carbono del carbonilo pasa de plano (sp^2) a tetraédrico (sp^3) y se convierte en un centro quiral nuevo: el **carbono anomérico**. Puede quedar de dos maneras, que se llaman anómeros α y β según hacia dónde

apunte su hidroxilo. En la proyección de Haworth de un azúcar D, el anómero α lo tiene hacia abajo y el β hacia arriba.

Esa diferencia parece un detalle de dibujo. No lo es: decide las propiedades mecánicas del polímero entero.

- **Enlace $\alpha(1 \rightarrow 4)$, en almidón y glucógeno.** El ángulo que impone hace que la cadena se enrolle en una hélice abierta y flexible. El resultado son gránulos densos pero hidratados, a los que las enzimas hidrolíticas llegan sin dificultad. El glucógeno añade ramificaciones $\alpha(1 \rightarrow 6)$ cada 8 a 12 residuos, con lo que un solo gránulo ofrece cientos de extremos por los que empezar a liberar glucosa a la vez.
- **Enlace $\beta(1 \rightarrow 4)$, en celulosa.** Para acomodar la geometría β , cada glucosa gira 180° respecto a su vecina. La cadena sale recta y rígida, como una cinta. Las cintas se apilan lateralmente con redes densas de puentes de hidrógeno, expulsan el agua y forman microfibrillas cristalinas cuya resistencia a la tracción es comparable a la del acero. Y como la mayoría de los animales no producen celulasas, esa misma glucosa que en el almidón es alimento, en la celulosa es indigerible. La diferencia entre las dos moléculas es la orientación de un hidroxilo.
- **Quitina.** Polímero lineal de N-acetilglucosamina con el mismo enlace $\beta(1 \rightarrow 4)$, en el exoesqueleto de los artrópodos y la pared de los hongos.

REFERENCIA Entre los disacáridos conviene recordar tres. La **sacarosa**, α -D-Glc(1 $\rightarrow$ 2) β -D-Fru, tiene los dos carbonos anoméricos comprometidos en el enlace, así que no es reductora y no reacciona por ese extremo: es el azúcar que las plantas transportan por la savia, precisamente porque es químicamente inerte. La **lactosa**, β -D-Gal(1 $\rightarrow$ 4)D-Glc, sí deja libre el carbono anomérico de la glucosa y es reductora; para digerirla hace falta lactasa en el borde intestinal, y su ausencia en la edad adulta es la norma, no la excepción, en la mayoría de las poblaciones humanas. La **maltosa**, α -D-Glc(1 $\rightarrow$ 4)D-Glc, aparece como producto intermedio de la digestión del almidón.

1.6 Lípidos: la frontera de la célula

ESTUDIO Los lípidos son la familia rara de las cuatro. Las otras tres se definen por su monómero; los lípidos, no. Se definen por una propiedad física: se disuelven en disolventes no polares y no en agua. Bajo esa etiqueta caben moléculas muy distintas entre sí, y lo que las une es cómo se comportan frente al agua.

1.6.1 Ácidos grasos: por qué el aceite es líquido y la mantequilla no

Un ácido graso es una cadena de hidrocarburo con un grupo carboxilo en un extremo, casi siempre con un número par de carbonos, entre 12 y 24. La diferencia que importa es si tiene dobles enlaces o no.

- **Saturados**, como el palmítico (16:0) o el esteárico (18:0). Sin dobles enlaces, la cadena es recta y extendida, y las cadenas vecinas se apilan muy juntas maximizando las fuerzas de Van der Waals. A temperatura corporal son sólidos.
- **Insaturados**, como el oleico (18:1 *cis*-9). Cada doble enlace en configuración *cis* introduce un codo rígido de unos 30° . Ese codo impide el apilamiento regular y debilita las atracciones entre cadenas. Funden a temperatura mucho más baja y son líquidos a temperatura ambiente.

Concepto. Toda la diferencia entre una grasa animal sólida y un aceite vegetal líquido está en la forma de la cadena, no en su composición elemental. Es un buen recordatorio de algo que reaparecerá en proteínas: en biología, la geometría suele explicar más que la fórmula.

1.6.2 Triglicéridos: por qué la grasa almacena tanto

Un triglicérido es glicerol esterificado con tres ácidos grasos. Al no tener grupos ionizables, es completamente hidrofóbico y se almacena en los adipocitos como gotas anhidras, sin agua asociada. Su oxidación completa rinde unos 38 kJ/g, frente a unos 17 kJ/g de glúcidos y proteínas. La ventaja real es todavía mayor de lo que sugieren esas cifras: un gramo de glucógeno arrastra unos dos gramos de agua de hidratación, y la grasa no arrastra ninguna. Por eso la reserva a largo plazo de los animales es grasa y no almidón.

1.6.3 Fosfolípidos: la molécula que construye una frontera sola

Un fosfolípido tiene glicerol unido a dos ácidos grasos hidrofóbicos, en las posiciones *sn*-1 y *sn*-2, y a un grupo fosfato en *sn*-3 que a su vez lleva una cabeza polar: colina, etanolamina o serina. El resultado es una molécula **anfipática**, con una cabeza que quiere agua y dos colas que la rehúyen.

Puesta en agua, esa molécula no necesita ayuda para organizarse. Se autoensambla en **bicapas** cerradas de unos 4 a 5 nm de grosor, con las colas hacia dentro y las cabezas hacia fuera. Lo interesante es la razón termodinámica: no es que las colas se atraigan entre sí, es que el agua ordenada alrededor de una cola aislada tiene entropía baja, y expulsar las colas del agua libera esa entropía. La membrana se forma porque al agua le conviene, no porque a los lípidos les convenga. A ese mecanismo se le llama **efecto hidrofóbico**, y es el mismo que pliega las proteínas.

1.6.4 Colesterol: un amortiguador que funciona en los dos sentidos

REFERENCIA El colesterol pertenece a los esteroides, lípidos contruidos sobre cuatro anillos fusionados y rígidos. Intercalado entre los fosfolípidos de las membranas animales, hace algo que parece contradictorio: a temperatura alta, sus anillos rígidos estorban el movimiento de las colas y la membrana se vuelve menos fluida y menos permeable; a temperatura baja, se interpone entre las cadenas saturadas e impide que se empaqueten en una fase sólida, con lo que la membrana sigue siendo fluida. No fluidifica ni solidifica: amortigua en la dirección en que haga falta.

Además, el colesterol es el precursor obligado de las sales biliares, de la vitamina D y de todas las hormonas esteroideas, del cortisol a la testosterona y el estradiol. Es una molécula estructural y una materia prima a la vez.

1.7 Qué hace Python cuando guarda una secuencia

ESENCIAL Un cromosoma humano tiene del orden de 10^8 bases. A partir de ahí, la diferencia entre un programa que funciona y uno que agota la memoria del servidor no está en el algoritmo sino en cómo se guardan los datos. Esta sección explica lo justo del modelo de memoria de Python para poder tomar esas decisiones con criterio [72].

Los tipos primitivos de Python representan de forma natural los polímeros biológicos; en particular, las cadenas inmutables modelan secuencias que no deben alterarse.

1.7.1 Una variable no contiene un valor: apunta a él

En CPython, la implementación que casi todo el mundo usa, una variable no guarda un dato: guarda la dirección de un objeto que vive en el montículo (*heap*). Ese objeto, un `PyObject`, lleva al menos tres cosas: un contador de cuántas variables lo están apuntando (`ob_refcnt`), un puntero a su tipo (`ob_type`) y los datos propiamente dichos.

Cuando una variable deja de apuntar a un objeto, porque se le asigna otra cosa o porque se hace `del`, el contador baja. Si llega a cero, la memoria se libera en ese mismo instante. Solo cuando hay ciclos de referencias, objetos que se apuntan entre sí, interviene el recolector de basura cíclico (`gc`), que pasa cada cierto tiempo a buscar grupos inalcanzables.

1.7.2 Mutable e inmutable, y el error que esto provoca

Python reparte sus tipos en dos grupos:

- **Inmutables** (`str`, `tuple`, `int`, `bytes`): su contenido no se puede cambiar. Cualquier «modificación» crea en realidad un objeto nuevo en otra dirección.
- **Mutable** (`list`, `dict`, `set`, `bytearray`): su contenido se cambia en el sitio, sin que el objeto cambie de identidad.

Ahora viene el error. Asignar una lista a otra variable no copia nada: las dos variables apuntan al mismo objeto. A eso se le llama *aliasing*.

```
genoma_control = ['A', 'C', 'G', 'T']
```

```

genoma_mutado = genoma_control    # no copia: es el mismo objeto
genoma_mutado[1] = 'T'           # modifica los dos
print(genoma_control)            # ['A', 'T', 'G', 'T']

```

Atención. Este fallo es especialmente peligroso porque no lanza ninguna excepción. El programa sigue, los números salen, y lo que se ha corrompido es la referencia contra la que iba a comparar. Si necesita una copia, pídala explícitamente con `genoma_control.copy()` o `list(genoma_control)`; y si la secuencia no debe cambiar nunca, guárdela en un `str` o una `tuple`, que no admiten modificación y le avisarán con un error si lo intenta.

Las colecciones de Python se corresponden con usos biológicos concretos: listas para ensamblajes que crecen, tuplas para coordenadas genómicas fijas, conjuntos para alfabetos y diccionarios para tablas de codones.

1.7.3 Diccionarios y conjuntos: por qué buscar en ellos es gratis

Buscar un elemento en una lista obliga a recorrerla: coste $O(N)$. Los conjuntos y los diccionarios usan una tabla de dispersión, que calcula a partir del propio valor dónde debería estar, y por eso insertan, buscan y borran en tiempo constante promedio $O(1)$. Con un alfabeto de cuatro letras la diferencia es irrelevante; con un índice de millones de k -meros decide si el programa termina hoy o la semana que viene.

```

COMPLEMENTO_ADN = {'A': 'T', 'T': 'A', 'C': 'G', 'G': 'C'}
ALFABETO_ADN = {'A', 'C', 'G', 'T'}

```

1.7.4 Generadores: leer un cromosoma sin cargarlo

Cuando el fichero es grande, `f.read()` y `f.readlines()` son una mala idea: traen todo el contenido a memoria de golpe. Un **generador**, una función que produce valores con `yield` en lugar de devolverlos con `return`, entrega un registro cada vez y mantiene la huella de memoria constante, $O(1)$, independientemente del tamaño del fichero.

```

def leer_fasta(ruta):
    """Produce pares (cabecera, secuencia) de uno en uno."""
    cabecera = None
    trozos = []
    with open(ruta, 'r') as f:
        for linea in f:
            linea = linea.strip()
            if not linea:
                continue
            if linea.startswith('>'):
                if cabecera:
                    yield (cabecera, "".join(trozos))
                    cabecera = linea[1:]
                    trozos = []
                else:
                    trozos.append(linea.upper())
            else:
                trozos.append(linea.upper())
    if cabecera:
        yield (cabecera, "".join(trozos))

```

Comprueba. Fíjese en las dos líneas con `yield`. La segunda, fuera del bucle, no es un descuido: sin ella se perdería el último registro del fichero, porque nadie escribe un `>` después de la última secuencia. Es el error más frecuente al escribir un lector de FASTA, y no falla ruidosamente: simplemente devuelve una secuencia menos.

Las funciones que validan su entrada y lanzan `ValueError` ante caracteres no válidos evitan que un error de datos se propague como un resultado numérico plausible.

1.8 Cuatro operaciones sobre secuencias

ESENCIAL Con la química y el modelo de memoria en la mano, ya podemos escribir las cuatro operaciones que

aparecerán en todos los capítulos siguientes. Cada una es corta; lo que importa es de dónde sale y qué la puede estropear.

1.8.1 Contenido GC

El contenido de guanina y citosina de una secuencia es la proporción de esas dos bases sobre el total. Interesa porque, como vimos, se relaciona con la estabilidad de la doble hélice, y además correlaciona con el sesgo de uso de codones y con la presencia de islas reguladoras en promotores.

Para una secuencia S de longitud N sobre el alfabeto $\Sigma = \{A, C, G, T\}$:

$$\text{GC}\%(S) = \frac{N(G) + N(C)}{N} \times 100$$

Hágalo primero a mano con $S = \text{ATGCGATCG}$, que tiene nueve bases. Los conteos son $N(A) = 2$, $N(C) = 2$, $N(G) = 3$ y $N(T) = 2$; suman nueve, que es la primera comprobación que debe hacer siempre. La suma GC es $3 + 2 = 5$, y $5/9 \times 100 = 55,555 \dots$, que redondeado a seis decimales es 55,555556 por ciento.

Para la secuencia ATGCGATCG de longitud 9 los conteos son A:2, C:2, G:3, T:2 y el contenido GC es 55,555556 por ciento.

Se reproduce con `verification/chapter_checks.py gc_content ATGCGATCG`.

Atención. Un detalle que arruina más análisis de los que parece: si la secuencia trae caracteres que no son ACGT y usted los descarta antes de contar, el denominador ya no es la longitud original. Divida siempre por la longitud de la secuencia *limpia*, la que realmente ha contado. Y deje constancia de cuántos caracteres descartó, porque si son muchos el problema no es el cálculo, es el fichero.

1.8.2 Complemento reverso

Aquí es donde la convención $5' \rightarrow 3'$ deja de ser un detalle. La cadena complementaria de una secuencia no se obtiene solo cambiando cada base por su pareja: como las dos hebras son antiparalelas, hay que además invertir el orden, para que el resultado quede escrito en el sentido en que todo el mundo lee.

Formalmente, si $S = s_1 s_2 \dots s_N$, su complemento reverso R cumple

$$r_i = \text{comp}(s_{N-i+1})$$

Con $S = \text{ATGCGATC}$, el proceso tiene dos pasos. Primero se complementa base a base y sale TACGCTAG. Después se invierte de extremo a extremo y sale GATCGCAT. Ese es el resultado correcto.

El complemento reverso de la secuencia ATGCGATC, escrita de 5 prima a 3 prima, es GATCGCAT.

Se reproduce con `verification/chapter_checks.py reverse_complement ATGCGATC`.

Comprueba. Antes de ejecutar nada, aplique dos veces el complemento reverso a una secuencia cualquiera en papel. ¿Qué obtiene? Si no le sale la secuencia de partida, ha olvidado la inversión o la ha aplicado dos veces. Esta propiedad, que la operación es su propia inversa, es la mejor prueba que puede escribir para su función, mejor que memorizar un resultado concreto.

Atención. La figura muestra otra fuente de errores tan frecuente que tiene nombre propio, el error de desplazamiento en uno. Las coordenadas biológicas empiezan en 1 e incluyen los dos extremos; los índices de Python empiezan en 0 y excluyen el final. La base que un fichero GFF llama posición 100 es `seq[99]` en Python. Escriba en un comentario qué convención usa cada función, y conviértalas en un único punto del programa, no en cada llamada.

1.8.3 Transcripción

Transcribir es sustituir cada timina por un uracilo, porque el ARN usa uracilo donde el ADN usa timina:

$$\text{transcribir}(S) = S[T \mapsto U]$$

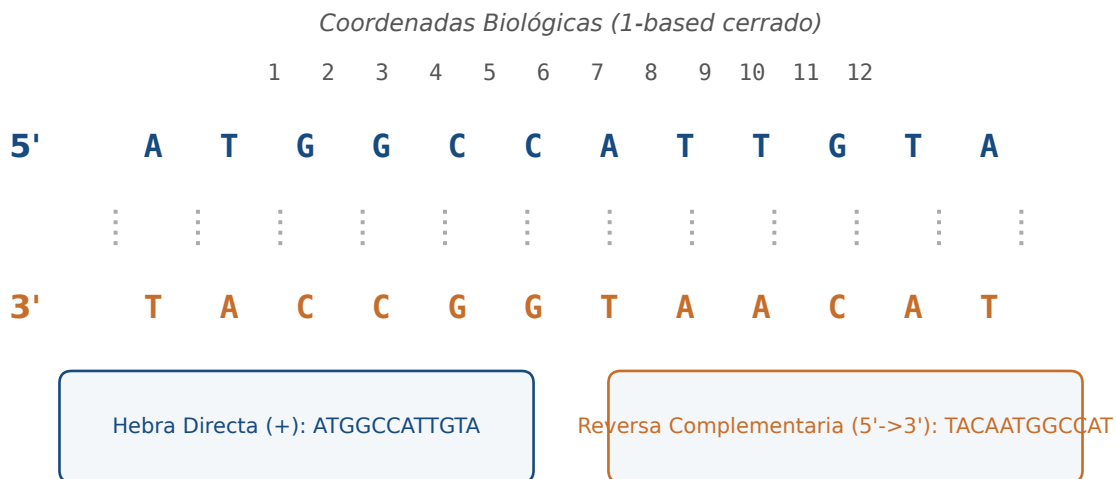


Figura 1.3: Orientación antiparalela del dúplex y coordenadas biológicas, que empiezan en 1 y no en 0. Figura original generada por `verification/make_figures.py`.

Es la operación más sencilla de las cuatro y también la más engañosa, porque el cambio de una letra oculta un cambio de molécula. El resultado ya no es ADN: no tiene sentido pedirle el complemento reverso con la tabla del ADN, ni pasárselo a una función que valide el alfabeto ACGT. Esa es exactamente la clase de descuido que la guarda de dominio del anexo está pensada para atrapar.

La transcripción sustituye timinas por uracilos y convierte ATGCGATC en AUGCGAUC.

Se reproduce con `verification/chapter_checks.py transcribe ATGCGATC`.

1.8.4 Ventana deslizante: el mismo resultado por dos precios

El contenido GC de un cromosoma entero dice poco: lo interesante es cómo varía a lo largo de la secuencia. Para verlo se recorre la secuencia con una **ventana** de anchura fija W que avanza de S en S posiciones, y se calcula el contenido GC dentro de cada ventana.

La forma directa de programarlo es contar las G y las C de cada ventana desde cero. Funciona, y su coste es $O(N \times W)$. Póngale números: para un cromosoma humano con $N = 10^8$ bases y una ventana de $W = 10\,000$, son del orden de 10^{12} operaciones. No es lento: es inviable.

La alternativa cuesta un poco más de pensar y mucho menos de ejecutar. Se calcula el contenido de la primera ventana una sola vez, en $O(W)$. Para cada avance, en lugar de recomtar, se ajusta: se resta la base que sale por la izquierda y se suma la que entra por la derecha.

$$GC_{k+1} = GC_k - \mathbb{I}(s_k \in \{G,C\}) + \mathbb{I}(s_{k+W} \in \{G,C\})$$

Cada paso cuesta $O(1)$ y el total es $O(W + N) = O(N)$. Los mismos 10^8 pasos que antes tardaban horas se resuelven en segundos, y el resultado es idéntico hasta el último decimal.

Concepto. Este patrón, guardar un resultado parcial y actualizarlo en lugar de recalcularlo, se llama ventana rodante y reaparece en todo el libro: en el conteo de k -meros, en el filtrado de calidad de lecturas y en los alineamientos por bandas. Cuando vea un bucle que recorre lo mismo dos veces, pregúntese qué parte del cálculo anterior podría reutilizar.

El análisis por ventana de tamaño 4 y paso 2 sobre ATGCGATCG da 0-4 ATGC con 50 por ciento, 2-6 GCGA con 75 por ciento y 4-8 GATC con 50 por ciento.

Se reproduce con `verification/chapter_checks.py sliding_window ATGCGATCG --window 4 --step 2`.

Comprueba. Con $N = 9$, $W = 4$ y $S = 2$ salen tres ventanas. ¿Sabe decir por qué no cuatro, y qué le ocurre a la última base de la secuencia? Escriba la fórmula que da el número de ventanas antes de mirar la salida del oráculo. La necesitará en el anexo.

1.9 Un módulo propio de secuencias

ESENCIAL Las cuatro operaciones se escriben juntas en un módulo, con anotaciones de tipo y con una guarda que valide antes de calcular. El orden importa: validar primero, contar después. Así ninguna función tiene que preguntarse si su entrada es razonable, porque ya lo comprobó la guarda. Aquí van la guarda y la primera operación completas, como modelo de lo que se espera:

```
from typing import List

ALFABETO_ADN = {'A', 'C', 'G', 'T'}
TABLA_COMPLEMENTO = {'A': 'T', 'T': 'A', 'C': 'G', 'G': 'C'}

def validar_adn(seq: str) -> str:
    """Normaliza y valida. Devuelve la secuencia limpia o lanza ValueError."""
    limpia = seq.strip().upper()
    if not limpia:
        raise ValueError("La secuencia de ADN no puede estar vacia.")
    invalidos = set(limpia) - ALFABETO_ADN
    if invalidos:
        raise ValueError(f"Bases no validas: {sorted(invalidos)}")
    return limpia

def contenido_gc(seq: str) -> float:
    """Porcentaje de G y C sobre la secuencia ya validada."""
    s = validar_adn(seq)
    return (sum(1 for b in s if b in {'G', 'C'}) / len(s)) * 100.0
```

Las otras tres funciones las escribirá usted en el anexo. Aquí queda su contrato, que es lo que el verificador comprobará; el código es cosa suya.

- `complemento_reverso(seq: str) -> str`. Valida, complementa cada base con `TABLA_COMPLEMENTO` e invierte el orden. Aplicada dos veces a la misma secuencia debe devolver la de partida.
- `transcribir(seq: str) -> str`. Valida y sustituye cada T por una U. Conserva la longitud y no deja ninguna T en el resultado.
- `perfil_gc(seq, ventana, paso=1)`, que devuelve una lista de números en coma flotante. Valida, rechaza con `ValueError` una ventana mayor que la secuencia o un paso no positivo, y devuelve el contenido GC de cada ventana usando la actualización rodante de la sección anterior, no un recuento desde cero en cada posición. El primer valor de la lista debe coincidir con el contenido GC de las primeras ventana bases.

1.9.1 Validar no es desconfiar de usted: es desconfiar del fichero

Dar por bueno lo que entra es la causa más común de fallo silencioso en bioinformática. Un fichero descargado puede traer una N donde el secuenciador no supo decidir, una U porque en realidad es ARN, un espacio al final de la línea, o toda la secuencia en minúsculas porque alguien marcó así las regiones repetidas. Ninguna de esas cuatro cosas hace fallar a un programa que no valida: hacen que devuelva un número equivocado.

Por eso `validar_adn` hace dos cosas a la vez, normalizar y comprobar, y devuelve la secuencia limpia en lugar de un simple `True`. Si devolviera un booleano, quien la llame tendría que acordarse de limpiar por su cuenta, y tarde o temprano no se acordará.

La guarda de dominio rechaza con `ValueError` una secuencia que contiene bases no canónicas, como la X de `ATGCX`, y acepta las que solo contienen A, C, G y T.

Se reproduce con `verification/chapter_checks.py validate_dna ATGCX`.

1.10 Un genoma que no cabía en sí mismo

ESTUDIO El bacteriófago Φ X174 es un virus que infecta a *Escherichia coli* y cuyo genoma es una molécula de ADN

de una sola cadena, circular, de 5386 nucleótidos. En 1977 el equipo de Fred Sanger determinó su secuencia completa con el método de terminadores dideoxi: fue el primer genoma de ADN secuenciado de la historia [63].

Y en cuanto se pudo leer, apareció un problema de contabilidad. Con 5386 bases no hay sitio para las once proteínas que el virus fabrica, si cada una ocupa su propio tramo de secuencia. Salen las cuentas mal por bastante.

La solución que usa el virus es **solapar los genes**. El gen *B* está contenido íntegramente dentro del gen *A*, leído en otro marco de lectura, y el gen *E* lo está dentro del gen *D* de la misma manera. El genoma no yuxtapone sus genes: los superpone. Una misma secuencia de bases codifica dos proteínas distintas según en qué posición empiece a leerse.

Concepto. Fíjese en lo que esto implica para nosotros. Una secuencia de ADN no tiene una única interpretación: tiene seis marcos de lectura posibles, tres en cada hebra, y cuál de ellos es el correcto no está escrito en la secuencia. Encontrar genes es, por eso, un problema de inferencia y no de lectura, y ocupará un capítulo entero más adelante. También explica por qué una mutación en un genoma tan compacto puede estropear dos proteínas a la vez: no hay bases sobrantes donde acomodar un error.

El contenido GC global de este genoma es del 44,76 por ciento, con variaciones locales que un perfil por ventana deslizante detecta y que coinciden con el origen de replicación y con los promotores tempranos. Es exactamente el análisis que usted programará en el anexo, aplicado al primer genoma que se pudo leer entero.

1.11 Tres errores que va a cometer

ESTUDIO La siguiente función parece razonable y hace tres cosas mal. Léala antes de seguir e intente localizarlas; las tres son errores reales, de los que aparecen en código de trabajo, no ejercicios artificiales.

```
def procesar_lecturas(lista_adn):
    limpias = lista_adn
    for i in range(len(limpias)):
        limpias[i] = limpias[i].upper()

    genoma = ""
    for seq in limpias:
        genoma = genoma + seq

    tabla = str.maketrans("ATCG", "TAGC")
    return genoma.translate(tabla)
```

1. **Modifica la lista de quien la llamó.** `limpias = lista_adn` no crea una lista nueva, sino un segundo nombre para la misma. Al escribir en `limpias[i]` se escribe en la lista original, que quizá era la referencia del análisis. La corrección es construir una lista nueva: `limpias = [s.upper() for s in lista_adn]`.
2. **Concatena en un bucle.** Como `str` es inmutable, cada `+` reserva un búfer nuevo y copia todo lo acumulado hasta ese momento. Con M fragmentos de tamaño L el coste total es $\sum_{i=1}^M i \cdot L$, es decir $O(M^2L)$. La corrección es `"".join(limpias)`, que mide una vez el total y copia una vez.
3. **Complementa sin invertir.** `translate` cambia las bases pero conserva el orden, así que devuelve la hebra complementaria escrita de 3' a 5', no el complemento reverso. La corrección es añadir `[::-1]`.

Atención. Los tres fallan en silencio. Ninguno lanza una excepción, ninguno deja el programa a medias, y los tres devuelven una cadena de la longitud correcta con las letras esperadas. Es la peor categoría de error: la que solo se detecta comparando con un resultado conocido. Por eso el método de este curso empieza siempre por predecir la salida antes de ejecutar.

El aliasing sobre listas mutables, la concatenación cuadrática de cadenas y el complemento sin inversión producen resultados incorrectos sin lanzar ninguna excepción.

1.12 Síntesis y tablas de diagnóstico

REFERENCIA La primera tabla recoge los fallos que más aparecen al procesar secuencias, con su causa y su corrección.

Búsqueda cuando algo no cuadre, antes de empezar a cambiar código a ciegas.

Síntoma observable	Causa probable	Comprobación y corrección
MemoryError al abrir un FASTA grande	se cargó el fichero entero con <code>read()</code> o <code>readlines()</code>	recorrerlo con un generador que produzca un registro cada vez
El alineamiento con la hebra reversa no puntúa	se complementó sin invertir	aplicar <code>[: :-1]</code> después de complementar; comprobar que la operación aplicada dos veces devuelve el original
El contenido GC no coincide con el esperado	se dividió por la longitud original tras descartar caracteres inválidos	dividir por la longitud de la secuencia validada
La secuencia de referencia cambió sola	<i>aliasing</i> : se pasó una lista mutable y se modificó dentro	copiar explícitamente o guardar la referencia en <code>str</code> o <code>tuple</code>
La anotación señala la base contigua a la esperada	mezcla de coordenadas 1-based cerradas con índices 0-based	convertir en un solo punto del programa y documentar la convención

La segunda tabla resume qué estructura de datos conviene a cada uso. La columna del coste de búsqueda es la que decide en cuanto los datos crecen.

Tipo	Mutabilidad	Coste de búsqueda	Uso típico en bioinformática
<code>str</code>	inmutable	$O(N)$	secuencia de referencia que no debe alterarse
<code>list</code>	mutable	$O(N)$	colección ordenada de lecturas que crece
<code>tuple</code>	inmutable	$O(N)$	coordenadas genómicas, claves compuestas
<code>set</code>	mutable	$O(1)$ promedio	alfabetos, vocabularios de k -meros
<code>dict</code>	mutable	$O(1)$ promedio	tabla de codones, índice de k -meros
<code>deque</code>	mutable	$O(1)$ en los extremos	ventanas rodantes

1.13 Glosario

- **Ácido nucleico:** polímero lineal de nucleótidos que se escribe de 5' a 3'.
- **Enlace fosfodiéster:** unión covalente entre el 3'-OH de un nucleótido y el 5'-fosfato del siguiente.
- **Complementariedad:** apareamiento A con T por dos puentes de hidrógeno y G con C por tres.
- **Puente de hidrógeno:** interacción no covalente, mucho más débil que un enlace covalente, que puede formarse y romperse sin dañar la molécula.
- **Contenido GC:** porcentaje de guaninas y citosinas sobre el total de bases.
- **Complemento reverso:** la hebra complementaria escrita en sentido 5' $\rightarrow$ 3'; se complementa y después se invierte.
- **Ventana deslizante:** recorrido de una secuencia por tramos de anchura fija W separados por un paso S .
- **Ventana rodante:** implementación de lo anterior que actualiza el conteo en lugar de recalcularlo, en tiempo $O(N)$.
- **Inmutabilidad:** propiedad de un objeto cuyo contenido no puede modificarse después de crearlo.
- **Aliasing:** dos o más nombres que apuntan al mismo objeto en memoria.
- **Generador:** función que produce valores de uno en uno con `yield` y no los guarda todos.
- **Zwitterión:** molécula con dos cargas opuestas y carga neta cero, forma habitual de un aminoácido a pH fisiológico.
- **Anómero:** cada una de las dos configuraciones, α y β , del carbono anomérico de un azúcar cíclico.
- **Efecto hidrofóbico:** tendencia de las moléculas apolares a agruparse en agua, impulsada por la entropía del disolvente.
- **Marco de lectura:** cada una de las seis formas de agrupar una secuencia en tripletes, tres por hebra.

1.14 Con qué sigue este capítulo

Lo que aquí se establece se da por sabido en el resto del libro. La representación de una secuencia como cadena inmutable y la guarda de dominio reaparecen en cuanto empezamos a leer formatos reales, donde los ficheros vienen sucios de verdad. El análisis de coste de la ventana rodante es el mismo razonamiento que gobierna los algoritmos de alineamiento. Y la limitación que señalamos al hablar de los priones, que la secuencia no captura el plegamiento, es el punto de partida de los capítulos de estructura.

Los contratos de inmutabilidad, validación de entrada y coste acotado establecidos en este capítulo se aplican en el resto del libro sin volver a justificarse.

1.15 Fuentes y reproducibilidad

La química de las cuatro familias de biomoléculas sigue a Alberts y colaboradores [3]; el modelo de ejecución de Python, al manual de referencia del lenguaje [72]; los parámetros termodinámicos de vecino más próximo, a SantaLucia [64]. Las herramientas de bioinformática en Python que este capítulo no usa, para escribir el módulo desde cero, están recogidas en Biopython [12], y las figuras se generan con Matplotlib [33] mediante `verification/make_figures.py`, con fuentes incrustadas y sin retoque manual.

Los datos de `data/` son sintéticos, de elaboración propia y con licencia CC0; su procedencia y su contenido están descritos en `data/README.md`. Todos los cálculos que aparecen en el texto se reproducen con `verification/chapter_checks.py` tal como están impresos, sin argumentos ocultos.

Anexo práctico · Escribir y verificar un módulo de secuencias

Pregunta, producto y separación de materiales

¿Puede escribir, con la biblioteca estándar de Python y sin ningún paquete externo, un módulo que calcule contenido GC, complemento reverso, transcripción y perfiles por ventana, y que rechace lo que no es ADN en lugar de devolver un número sin sentido? Este laboratorio responde que sí, y lo comprueba con secuencias que usted no ha visto.

El producto individual es un fichero, `mi_modulo_secuencias.py`, con las cinco funciones del contrato de la sección [Un módulo propio de secuencias](#). Le acompañan: la tabla de predicciones y resultados, las salidas guardadas en `resultados/`, un dictamen escrito sobre las cuatro lecturas crudas, un `README.md` con lo que ejecutó y sus supuestos, la defensa conceptual y un paquete `.tar.gz` verificado.

Este anexo es autónomo: no necesita red ni datos externos. Usa `data/` del capítulo como entrada de solo lectura y un directorio de entrega que usted crea. Los valores esperados no se imprimen aquí: los calcula usted primero a mano y los contrasta después con la herramienta que el generador deja en `herramienta/`.

Mapa de ruta y productos entregables

Bloque	Producto
1 preparar el espacio de trabajo	directorio de entrega creado sin tocar <code>data/</code>
2 comprobar los datos de partida	manifiesto verificado y datos inspeccionados
3 cálculo a mano de la secuencia ancla	GC y complemento reverso escritos antes de ejecutar nada
4 escribir el módulo	<code>mi_modulo_secuencias.py</code> con las cinco funciones
5 contrastar con el oráculo	salidas en <code>resultados/</code> y filas completas en <code>notas/respuestas.tsv</code>
6 dictaminar las lecturas crudas	<code>notas/lecturas.md</code> con las cuatro decisiones
7 empaquetar y verificar	<code>.tar.gz</code> y verificador en <code>ENTREGA_OK</code>

La ruta es única. Antes de ejecutar cada orden, escriba en `notas/respuestas.tsv` lo que espera obtener. Lo que se evalúa es el contraste entre su predicción y el resultado, no el resultado solo.

1 • Preparar el espacio de trabajo

Desde el directorio del capítulo, compruebe su versión de Python y cree el directorio de entrega:

```
python3 --version
python3 verification/chapter_checks.py gc_content ATGCGATCG
```

La segunda orden reproduce el cálculo que aparece en la sección [Cuatro operaciones sobre secuencias](#). Si no coincide con lo impreso allí, deténgase: hay algo mal en su instalación antes de empezar.

```
./practice_assets/create_workspace.sh BC01_entrega
cd BC01_entrega
ls -la
```

El generador crea `datos/` con copias de los ficheros del capítulo y su manifiesto, `notas/respuestas.tsv` solo con la cabecera, `resultados/` vacía, `herramienta/` con el oráculo y el comprobador, y `mi_modulo_secuencias.py` con las cinco funciones sin escribir. Si el directorio ya existe, se niega a sobrescribirlo. No copia ninguna solución: todas las funciones de la plantilla lanzan `NotImplementedError` hasta que usted las escriba.

2 • Comprobar los datos antes de usarlos

```
cd datos && sha256sum -c manifest.sha256 && cd ..
head -n 4 datos/bc01_secuencias.fasta
cat datos/bc01_lecturas_crudas.txt
```

Comprueba. Mire las cuatro lecturas crudas y decida, antes de programar nada, qué debería hacer su guarda con cada una: aceptarla tal cual, aceptarla después de normalizarla o rechazarla. Anote las cuatro decisiones y su motivo; en el bloque 6 tendrá que defenderlas por escrito.

3 • El cálculo ancla, primero a mano

Tome la secuencia de doce bases, que es el último registro del fichero de secuencias y vale 5'-ATGCATGCGTCG-3', y con papel:

1. cuente cada una de las cuatro bases y compruebe que suman doce;
2. calcule el contenido GC exacto y anótelos con seis decimales;
3. escriba el complemento reverso, complementando primero e invirtiendo después;
4. escriba la transcripción a ARN;
5. con ventana $W = 6$ y paso $S = 3$, diga cuántas ventanas salen y cuál es el contenido GC de cada una.

Escriba estos cinco resultados en la columna `prediccion` de `notas/respuestas.tsv` antes de continuar. No ejecute todavía nada: una predicción escrita después de ver la salida no es una predicción.

4 • Escribir el módulo

Abra `mi_modulo_secuencias.py` e implemente las cinco funciones. La guarda y el contenido GC aparecen completos en la sección [Un módulo propio de secuencias](#) y puede partir de ahí; las otras tres son suyas. Recuerde que `perfil_gc` debe actualizar el conteo al avanzar, no recontar cada ventana desde cero.

Compruebe su avance tantas veces como quiera:

```
bash herramienta/check_modulo.sh
```


El comprobador no compara con valores memorizables: prueba cinco propiedades sobre secuencias que no aparecen en el capítulo. Que el complemento reverso aplicado dos veces devuelva la secuencia de partida. Que el contenido GC de una secuencia coincida con el de su complemento reverso, y con un recuento directo. Que la transcripción conserve la longitud y no deje timinas. Que el perfil devuelva tantas ventanas como impone el paso, y que la primera coincida con el contenido GC de las primeras ventana bases. Y que la guarda rechace una N, una U, un espacio interior y la cadena vacía, aceptando en cambio una secuencia en minúsculas y devolviéndola normalizada.

Atención. Copiar el oráculo del capítulo no sirve: los nombres de sus funciones y sus tipos de retorno son distintos de los que pide el contrato. El comprobador lo detecta en la primera línea de su salida.

5 • Contrastar con el oráculo

Cuando su módulo pase las cinco propiedades, y solo entonces, contraste sus cálculos a mano con la herramienta y guarde las salidas:

```
python3 herramienta/chapter_checks.py gc_content ATGCATGCGTCG > resultados/gc_ancla.txt
python3 herramienta/chapter_checks.py reverse_complement ATGCATGCGTCG > resultados/revcomp_ancla.txt
python3 herramienta/chapter_checks.py transcribe ATGCATGCGTCG > resultados/arn_ancla.txt
python3 herramienta/chapter_checks.py sliding_window ATGCATGCGTCG --window 6 --step 3 > resultados/perfil_w6_s3.txt
cat resultados/gc_ancla.txt resultados/revcomp_ancla.txt resultados/arn_ancla.txt resultados/perfil_w6_s3.txt
```

Complete ahora las columnas resultado y coincide de notas/respuestas.tsv. Si alguna no coincide, no corrija el número: averigüe cuál de los dos cálculos está mal y por qué. Anótelos, porque esa explicación vale más que el acierto.

Comprueba. Compruebe además que su propio módulo y el oráculo dan lo mismo sobre las secuencias largas de datos/, no solo sobre la de doce bases. Una implementación puede acertar en un caso pequeño y fallar en cuanto la ventana rueda muchas veces.

6 • Dictaminar las lecturas crudas

Pase por su guarda las cuatro lecturas del fichero de lecturas crudas y compruebe qué hace con cada una. Estas dos órdenes le dan el criterio del oráculo para dos de ellas:

```
python3 herramienta/chapter_checks.py validate_dna ATGCNNATC > resultados/guarda_lect_02.txt
python3 herramienta/chapter_checks.py validate_dna ATGCCAUCG > resultados/guarda_lect_04.txt
cat resultados/guarda_lect_02.txt resultados/guarda_lect_04.txt
```

Escriba en notas/lecturas.md un dictamen breve para lect_01, lect_02, lect_03 y lect_04: qué hace su guarda con cada una y si es lo correcto. Distinga los dos casos que no son iguales aunque lo parezcan: una N es una posición que el secuenciador no supo leer, y una U indica que alguien ha mezclado ARN en un fichero que dice contener ADN. Rechazar las dos es defendible; tratarlas igual, no.

7 • Clases de aminoácidos

Compruebe la clase fisicoquímica de tres residuos, uno de cada extremo de la clasificación:

```
python3 herramienta/chapter_checks.py aa_class G > resultados/clases_aa.txt
python3 herramienta/chapter_checks.py aa_class D >> resultados/clases_aa.txt
python3 herramienta/chapter_checks.py aa_class K >> resultados/clases_aa.txt
cat resultados/clases_aa.txt
```

8 • Empaquetar y verificar la entrega

Escriba README.md con las órdenes que ejecutó y los supuestos que hizo, y notas/defensa.md con la defensa que se pide más abajo. Después vuelva al directorio del capítulo, empaquete y verifique:

```
cd ..
tar -czf BC01_entrega.tar.gz BC01_entrega
tar -tzf BC01_entrega.tar.gz | head -n 12
sha256sum BC01_entrega.tar.gz
./practice_assets/check_entrega.sh BC01_entrega BC01_entrega.tar.gz
```

El verificador comprueba que su módulo cumple las cinco propiedades con secuencias que no ha visto, que los datos de partida no se han modificado, que la tabla de respuestas tiene al menos cinco filas completas, que resultados/ no está vacía, que el dictamen cubre las cuatro lecturas, que el README y la defensa existen con la extensión mínima, y que el paquete se extrae con el módulo dentro. Si falta algo lo enumera y termina en ENTREGA_INCOMPLETA; si está todo, en ENTREGA_OK. No puntúa: la nota la pone quien corrige.

Rúbrica de evaluación

Sobre 10 puntos, repartidos entre lo que el verificador puede comprobar y lo que solo puede valorar quien corrige.

- **3,0 puntos, el módulo funciona:** las cinco funciones cumplen el contrato, el verificador termina en ENTREGA_OK y el paquete se extrae con el producto dentro.
- **2,5 puntos, el módulo está bien escrito:** perfil_gc actualiza el conteo al avanzar en lugar de recontar; las funciones validan antes de calcular y no repiten la validación; los mensajes de ValueError dicen qué carácter sobra.
- **2,0 puntos, predicción y contraste:** las cinco filas de respuestas llevan predicción escrita antes de ejecutar; las discrepancias están explicadas, no corregidas en silencio.
- **1,5 puntos, dictamen de las lecturas crudas:** las cuatro decisiones están justificadas y distinguen el caso de la N del caso de la U.
- **1,0 punto, defensa:** concisa, correcta y apoyada en un ejemplo de su propia entrega.

Terminar en ENTREGA_OK es requisito previo, no garantía de nota: un módulo que aprueba con un perfil que recuenta desde cero cumple el contrato y pierde la mitad del segundo apartado.

Lista de autocorrección

- | | |
|--|---|
| ■ mis cinco predicciones estaban escritas antes de ejecutar nada; | un booleano; |
| ■ el complemento reverso aplicado dos veces me devuelve la secuencia original; | ■ sé decir qué hace mi guarda con una N y por qué no es lo mismo que una U; |
| ■ mi perfil_gc no vuelve a contar la ventana entera en cada paso; | ■ no he modificado ningún fichero de datos/; |
| ■ mi guarda devuelve la secuencia normalizada, no | ■ las salidas que cito están guardadas en resultados/; |
| | ■ el verificador termina en ENTREGA_OK. |

Defensa

En notas/defensa.md, entre 100 y 150 palabras, explique por qué las cadenas de texto de Python son inmutables y qué le aporta esa propiedad al manipular una secuencia de referencia, frente al riesgo de compartir una lista mutable. Use un ejemplo concreto de su propia entrega, no una afirmación general: diga qué habría pasado en su código si la secuencia de partida hubiera sido una lista y una de sus funciones la hubiera modificado.

Fuentes

Los datos de data/ son sintéticos, de elaboración propia y con licencia CC0; su contenido está descrito en data/README.md. El módulo se escribe solo con la biblioteca estándar [72]; las funciones equivalentes de

una biblioteca establecida, para comparar después, están en Biopython [12]. El oráculo del capítulo está en `verification/` y el comprobador del producto, en `practice_assets/`.

Biomoléculas como objetos computables: Biofísica, modelado orientado a objetos y propiedades conformacionales

BC-CH02 – Biología Computacional

Edición 2

Pregunta del tema. Una cadena de aminoácidos podría plegarse de más maneras que átomos hay en el universo, y sin embargo encuentra su forma correcta en microsegundos. ¿Qué restringe ese espacio, y qué propiedades de la proteína se pueden calcular sabiendo solo su secuencia?

Producto final. Una clase en Python que represente una proteína, valide su alfabeto al construirla y calcule masa, composición y perfil de hidropatía, aplicada a localizar una región transmembrana.

Mapa del tema

2.1. Biomoléculas como objetos computables: encapsulación e invariantes	19
2.2. El paradigma Secuencia → Estructura → Función	20
2.3. La biofísica cuántica del enlace peptídico	21
2.4. El diagrama de Ramachandran y conformación del esqueleto	22
2.5. Jerarquía estructural y fuerzas biofísicas estabilizadoras	23
2.6. Propiedades fisicoquímicas computables	25
2.7. Escala de hidropatía de Kyte-Doolittle y perfiles	26
2.8. Desnaturalización proteica y estabilidad conformacional	27
2.9. Propiedades emergentes: alosterismo y cooperatividad	28
2.10. Diseño computacional en Python: Módulos de proteínas	28
2.11. Peligros computacionales y actividad de depuración	29
2.12. Síntesis operativa y tablas de diagnóstico	30
2.13. Glosario conceptual	30
2.14. Conexiones y mapa del curso	31
2.15. Fuentes y reproducibilidad	31

Cómo usar este tema

Este capítulo trata las proteínas como objetos con invariantes: cosas que se pueden comprobar y magnitudes que se pueden calcular a partir de la secuencia. Empieza por el problema que hace interesante todo lo demás, la paradoja de Levinthal, y sigue por lo que restringe el espacio de plegamientos: la química del enlace peptídico, los ángulos que la geometría permite y las fuerzas que estabilizan el resultado.

Al terminar sabrá: explicar por qué el plegamiento no puede ser una búsqueda exhaustiva; leer un diagrama de ángulos permitidos y decir qué región corresponde a qué estructura; calcular la masa de un péptido sin equivocarse con el agua terminal; y usar un perfil de hidropatía para proponer, con la cautela debida, dónde hay una región transmembrana.

La ruta **esencial** son las biomoléculas como objetos, el enlace peptídico, la jerarquía estructural y las propiedades computables. La ruta de **estudio** añade los motivos supersecundarios, la desnaturalización, el alosterismo y el anexo.

2.1 Biomoléculas como objetos computables: encapsulación e invariantes

ESENCIAL En el capítulo inicial establecimos que la vida celular puede comprenderse de forma rigurosa como un vasto sistema de procesamiento de información biológica y que los biopolímeros operan análogamente a cintas de datos discretos. Sin embargo, cuando nos enfrentamos al desarrollo de algoritmos y herramientas bioinformáticas en el mundo real, constatar una proteína o un ácido nucleico como una simple cadena alfanumérica de texto plano (`str`) constituye una abstracción excesivamente reduccionista que oculta su auténtica naturaleza biofísica.

Una macromolécula biológica no es un mensaje tipográfico estático suspendido en el vacío. Es una entidad física tridimensional dinámica, inmersa en un solvente acuoso a temperatura fisiológica, dotada de masa molecular cuantificable, cargas electrostáticas variables según el pH circundante, restricciones espaciales impuestas por la mecánica cuántica de sus enlaces covalentes, patrones de hidropatía que gobiernan su solvatación y una intrincada red de interacciones no covalentes que estabilizan su arquitectura activa.

2.1.1 El modelado por dominios y la Programación Orientada a Objetos (POO)

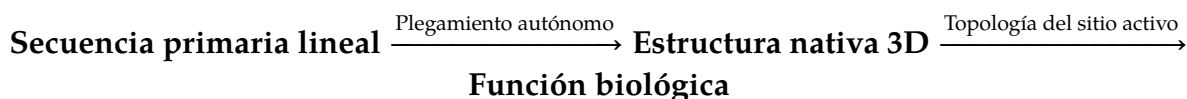
El paradigma de la **Programación Orientada a Objetos (POO)** junto con el diseño guiado por el dominio (*Domain-Driven Design*) ofrece el marco computacional ideal para salvar la brecha entre la abstracción digital y la realidad bioquímica. Al modelar una proteína como una clase en Python, logramos agrupar su secuencia primaria junto con sus propiedades fisicoquímicas calculables, sus métodos de transformación y, de forma crucial, sus **contratos invariantes de validación defensiva**:

1. **Encapsulación y tipado estricto:** Los atributos constitutivos de la molécula (identificador único, secuencia limpia, organismo de origen, anotaciones de dominio funcional) se protegen mediante propiedades inmutables (`@property`) o estructuras de datos congeladas (`@dataclass(frozen=True)`). Esto erradica por diseño los errores silenciosos derivados de la mutación accidental de datos en memorias compartidas.
2. **Evaluación perezosa (*lazy evaluation*) y memorización:** Ciertos descriptores moleculares (como la masa molecular exacta, el punto isoeléctrico teórico, los perfiles de hidropatía o los mapas de carga neta) requieren cálculos iterativos. Mediante decoradores de caché (`@cached_property`), se calculan bajo demanda la primera vez que se solicitan y se conservan en memoria, optimizando drásticamente el rendimiento en análisis a escala genómica.
3. **Defensa incondicional de invariantes biológicos:** El constructor de la clase actúa como un guardián de dominio. En el momento exacto de la instanciación, verifica que la cadena de caracteres pertenezca estrictamente al alfabeto de los 20 aminoácidos estándar según la nomenclatura internacional IUPAC. Cualquier carácter no canónico o símbolo espurio es rechazado de inmediato lanzando una excepción controlada (`ValueError`), garantizando que ningún objeto en el sistema pueda hallarse en un estado biológicamente inválido.

Las biomoléculas como objetos computacionales: encapsular datos de secuencia, propiedades físicas e invariantes dentro de clases de Python proporciona un modelado biológico riguroso y verificable.

2.2 El paradigma Secuencia → Estructura → Función

El pilar conceptual sobre el que descansa toda la biología molecular estructural y la bioinformática de macromoléculas se resume en el **paradigma clásico**:



Este postulado, refrendado experimentalmente en los célebres trabajos de Christian Anfinsen en 1961 con la ribonucleasa A bovina, establece que **toda la información termodinámica necesaria y suficiente para especificar la conformación tridimensional nativa de una proteína globular en condiciones fisiológicas reside exclusivamente en su secuencia lineal de aminoácidos**.

2.2.1 La paradoja de Levinthal y el embudo de energía libre

Si un polipéptido de 100 residuos intentara plegarse mediante un muestreo aleatorio ciego de todas sus posibles combinaciones de ángulos diedros del esqueleto (asumiendo de forma conservadora solo 3 conformaciones posibles por residuo, lo que equivale a $3^{100} \approx 5 \times 10^{47}$ estados conformacionales), y si cada transición tomara apenas 10^{-13} segundos (el tiempo de una vibración de enlace), la proteína tardaría más de 10^{27} años en hallar su conformación nativa, un lapso inmensamente superior a la edad del universo ($\approx 1.38 \times 10^{10}$ años). Esta contradicción física se conoce como la **paradoja de Levinthal**.

La resolución de la paradoja radica en que el plegamiento proteico no es una búsqueda estocástica no guiada. Es un proceso de colapso termodinámico dirigido a través de un **embudo de energía libre** (*folding funnel*):

- En la parte superior del embudo, el polipéptido recién traducido por el ribosoma posee una enorme entropía conformacional y un estado de alta energía libre.
- A medida que se forman microdominios de estructura secundaria locales y se produce el colapso hidrofóbico inicial, la proteína desciende por las pendientes del embudo pasando por intermediarios compactos (*molten globule*).
- En la base del embudo se encuentra el estado nativo biológicamente activo, que corresponde al **mínimo global de energía libre de Gibbs** ($\Delta G_{\text{nativo}} = \text{mínimo}$), donde los empaquetamientos atómicos de Van der Waals y las redes de puentes de hidrógeno alcanzan su máxima complementariedad geométrica.

Una vez plegada, la proteína expone hendiduras catalíticas, bolsas hidrofóbicas y superficies electrostáticas que reconocen ligandos con exquisita especificidad química, transformando la información genética unidimensional en actividad funcional en la célula.

El paradigma central Secuencia → Estructura → Función postula que la secuencia primaria lineal de aminoácidos determina termodinámicamente el plegamiento tridimensional nativo y la función biológica.

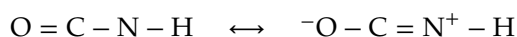
2.3 La biofísica cuántica del enlace peptídico

La polimerización de aminoácidos para constituir cadenas polipeptídicas ocurre mediante la formación secuencial de **enlaces peptídicos**. Esta unión covalente de tipo amida se sintetiza en el ribosoma a través de una reacción de condensación por ataque nucleofílico del grupo α -amino ($-\text{NH}_2$) de un aminoácido entrante sobre el carbono carbonílico del grupo α -carboxilo ($-\text{COOH}$) del aminoácido precedente, con la liberación estequiométrica de una molécula de agua (H_2O).

2.3.1 Resonancia electrónica y carácter parcial de doble enlace

Desde la perspectiva de la química física y la teoría de orbitales moleculares, el enlace peptídico C-N dista mucho de comportarse como un enlace simple ordinario con libre rotación. El par de electrones no enlazantes localizado en el orbital p del átomo de nitrógeno amídico se encuentra fuertemente deslocalizado por conjugación electrónica con el sistema π del grupo carbonilo contiguo ($\text{C}=\text{O}$).

Este fenómeno se describe cuánticamente mediante un híbrido de resonancia entre dos estructuras canónicas límite:



La contribución de la forma zwitteriónica deslocalizada confiere al enlace peptídico propiedades físico-químicas extraordinarias que condicionan toda la arquitectura de las proteínas:

1. **Carácter parcial de doble enlace** ($\approx 40\%$): Como resultado directo de la deslocalización de densidad electrónica π , la longitud del enlace C-N amídico es de apenas **1.32 Å** (0.132 nm). Esta distancia es significativamente más corta que un enlace simple C-N ordinario (1.47 Å) y se aproxima de forma notable a la de un doble enlace puro C=N (1.27 Å).
2. **Planaridad geométrica estricta**: La hibridación electrónica resultante confiere una geometría trigonal plana al nitrógeno y al carbono carbonílico. Por consiguiente, los seis átomos que configuran la unidad peptídica ($\text{C}\alpha^{(i)}, \text{C}, \text{O}, \text{N}, \text{H}, \text{C}\alpha^{(i+1)}$) yacen rigurosamente en un mismo plano geométrico.
3. **Momento dipolar permanente elevado**: La separación neta de cargas parciales entre el oxígeno carbonílico (electronegativo, carga δ^-) y el nitrógeno amídico (electropositivo, carga δ^+) genera un momento dipolar macroscópico considerable ($\mu \approx 3.5$ Debye), orientado paralelamente al vector $\text{N-H} \rightarrow \text{C}=\text{O}$. Este dipolo resulta decisivo para estabilizar redes cooperativas de puentes de hidrógeno a lo largo de hélices y láminas.
4. **Elevada barrera energética de rotación** (ω): La rotación alrededor del eje C-N está bloqueada por una barrera de energía de activación de aproximadamente ≈ 80 kJ/mol (20 kcal/mol). A temperaturas biológi-

cas, esta barrera impide la rotación libre espontánea y confina el ángulo diedro ω a dos isómeros discretos: *trans* ($\omega \approx 180^\circ$) y *cis* ($\omega \approx 0^\circ$).

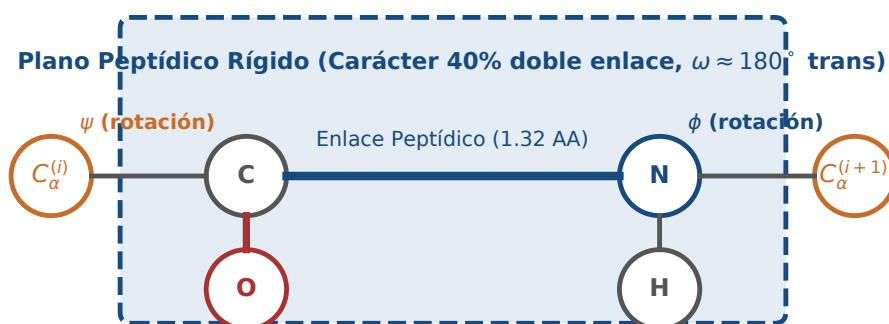


Figura 2.1: Geometría planar del enlace peptídico, carácter parcial de doble enlace y ángulos diedros de torsión del esqueleto (ϕ, ψ). Figura original generada por `verification/make_figures.py`.

El enlace peptídico posee carácter parcial de doble enlace ($\approx 40\%$ de resonancia) con una longitud planar de 1.32 Å que restringe la libertad conformacional del esqueleto a los ángulos diedros phi y psi.

2.3.2 Isomería *trans* vs *cis* y la singularidad conformacional de la prolina

En la inmensa mayoría de los enlaces peptídicos entre aminoácidos estándar, la conformación *trans* ($\omega \approx 180^\circ$) es termodinámicamente favorecida en más de un 99.9% de los casos observados cristalográficamente. La razón biofísica fundamental estriba en que la geometría *trans* ubica los carbonos C_α sucesivos y sus respectivas cadenas laterales $-R$ en lados opuestos del plano peptídico, maximizando la distancia intermolecular y minimizando las fuerzas de repulsión estérica de Van der Waals.

La única excepción relevante en el proteoma natural es el residuo de **Prolina (Pro, P)**. La prolina es formalmente un **iminoácido**: su cadena lateral alifática hidrocarbonada de tres carbonos se cierra covalentemente sobre el propio átomo de nitrógeno amídico, constituyendo un anillo rígido de pirrolidina de 5 miembros. Debido a este anillo cíclico:

- En un enlace peptidil-prolina (X-Pro), el carbono C_δ del anillo de pirrolidina se encuentra unido directamente al nitrógeno, haciendo que el impedimento estérico en la conformación *trans* sea muy similar en energía al de la conformación *cis*.
- Como resultado, entre el 5% y el 10% de los enlaces X-Pro en proteínas nativas se encuentran de forma estable en la conformación *cis* ($\omega \approx 0^\circ$).
- La interconversión espontánea entre las formas *trans* y *cis* posee una cinética sumamente lenta (tiempo de vida media de varios minutos). En la célula viva, esta reacción es acelerada por enzimas específicas denominadas **peptidil-prolil cis-trans isomerasas (PPIasas)**, que actúan como catalizadores esenciales en las etapas tardías del plegamiento proteico.
- Desde el punto de vista arquitectónico, los enlaces *cis-Pro* actúan como bisagras conformacionales críticas que permiten al esqueleto polipeptídico cambiar bruscamente de dirección en horquillas y giros β de superficie.

2.4 El diagrama de Ramachandran y conformación del esqueleto

Puesto que el plano amídico del enlace peptídico es rígido ($\omega \approx 180^\circ$), los únicos grados de libertad conformacionales continuos de los que dispone la cadena polipeptídica principal (*backbone*) para doblarse en el espacio y

adoptar arquitecturas tridimensionales son dos ángulos diedros de rotación por cada residuo aminoacídico:

- (*phi*): Ángulo diedro de torsión alrededor del enlace simple N – C α .
- (*psi*): Ángulo diedro de torsión alrededor del enlace simple C α – C_{carbonilo}.

2.4.1 El modelo biofísico de esferas duras de Ramachandran

En 1963, el biofísico indio G. N. Ramachandran y sus colaboradores modelaron los átomos del esqueleto peptídico como esferas rígidas impenetrables con radios atómicos de Van der Waals estrictos (r_{vdW}). En este modelo de potencial de contacto estérico discontinuo:

$$V_{\text{estérico}}(r_{ij}) = \begin{cases} +\infty & \text{si } r_{ij} < R_i + R_j \\ 0 & \text{si } r_{ij} \geq R_i + R_j \end{cases}$$

Al explorar sistemáticamente todo el espacio bidimensional de combinaciones posibles (ϕ, ψ) en el dominio toroidal completo $[-180^\circ, +180^\circ] \times [-180^\circ, +180^\circ]$, se evidenció un principio biofísico fundamental: ****más del 75% del espacio conformacional teórico está terminantemente prohibido**** debido a colisiones estéricas destructivas entre el oxígeno carbonílico, el hidrógeno amídico y el carbono C β de las cadenas laterales.

Las combinaciones de ángulos que no sufren impedimento estérico conforman unas pocas **islas de estabilidad termodinámica** en el plano, delimitando con absoluta precisión las regiones correspondientes a las estructuras secundarias regulares del proteoma:

- **Región de láminas β (cuadrante superior izquierdo):** $\phi \in [-150^\circ, -120^\circ]$, $\psi \in [+120^\circ, +160^\circ]$. Las cadenas polipeptídicas adoptan conformaciones totalmente extendidas que se ensamblan lateralmente en láminas plisadas paralelas y antiparalelas.
- **Región de α -hélices dextrógiras (cuadrante inferior izquierdo):** $\phi \approx -57^\circ$, $\psi \approx -47^\circ$. Corresponde a la geometría óptima de la hélice α descrita por Pauling, donde los puentes de hidrógeno axiales alcanzan una distancia interatómica ideal.
- **Región de α -hélices levógiras (cuadrante superior derecho):** $\phi \approx +57^\circ$, $\psi \approx +47^\circ$. Conformación energéticamente prohibida para los 19 aminoácidos quirales con carbono C β , pero accesible de forma natural para la **Glicina (Gly, G)**, cuya cadena lateral es un simple hidrógeno (–H), careciendo de impedimento estérico y poblando los cuatro cuadrantes del mapa.
- **Región de hélice de poliprolina II y colágeno:** $\phi \approx -75^\circ$, $\psi \approx +145^\circ$. Conformación extendida levógira sin puentes de hidrógeno intra-cadena, fundamental en proteínas fibrosas y dominios desordenados.

El diagrama de Ramachandran define las combinaciones estéricamente permitidas de ángulos diedros del esqueleto phi y psi, delimitando estructuras secundarias estables como hélices alfa y láminas beta.

2.5 Jerarquía estructural y fuerzas biofísicas estabilizadoras

La arquitectura tridimensional completa de las proteínas se articula de manera estricta en cuatro niveles de organización jerárquica:

1. **Estructura primaria:** Secuencia lineal covalente y no ramificada de aminoácidos enlazados por uniones peptídicas, leída invariablemente con polaridad biológica desde el extremo N-terminal (H₂N–) hacia el extremo C-terminal (–COOH).
2. **Estructura secundaria:** Plegamientos locales y periódicos del esqueleto polipeptídico, estabilizados de forma exclusiva por redes de puentes de hidrógeno entre los grupos C=O y N-H del enlace peptídico:
 - **α -hélice (Pauling, Corey y Branson, 1951):** Estructura cilíndrica helicoidal dextrógira donde el grupo carbonilo C=O del residuo i establece un puente de hidrógeno intramolecular con el grupo amino N-H del residuo $i + 4$. Presenta exactamente 3.6 residuos por vuelta, una distancia axial de 1.5 Å por residuo y un paso de rosca total de 5.4 Å. Las cadenas laterales –R se proyectan perpendicularmente hacia el exterior del cilindro, minimizando el choque estérico y permitiendo que la hélice sea anfipática (un lado hidrofóbico y otro hidrofílico).

- **β -láminas:** Segmentos polipeptídicos en conformación extendida colocados uno junto al otro formando una sábana plisada. En la disposición **antiparalela**, las hebras adyacentes corren en sentidos opuestos ($N \rightarrow C$ frente a $C \rightarrow N$) y los puentes de hidrógeno son perfectamente colineales y perpendiculares a la dirección de la cadena, confiriendo una máxima estabilidad termodinámica. En la disposición **paralela**, todas las hebras corren en la misma dirección y los puentes de hidrógeno se forman con ángulos ligeramente inclinados.
 - **Giros β (*Beta turns*):** Segmentos no periódicos de 4 aminoácidos que permiten a la cadena polipeptídica invertir su dirección espacial en 180° , típicamente estabilizados por un puente de hidrógeno entre el residuo i y el $i + 3$. Frecuentemente incorporan Prolina en la posición 2 y Glicina en la posición 3.
3. **Estructura terciaria:** Plegamiento tridimensional global, compacto y termodinámicamente consolidado de una cadena polipeptídica completa en su estado nativo funcional.
 4. **Estructura cuaternaria:** Asociación estequiométrica de dos o más cadenas polipeptídicas independientes (monómeros o subunidades) para formar un macrocomplejo funcional activo.

La jerarquía estructural de las proteínas se organiza en niveles primario, secundario (hélice alfa, lámina beta, giros), terciario (núcleo hidrofóbico, puentes disulfuro, puentes salinos) y cuaternario.

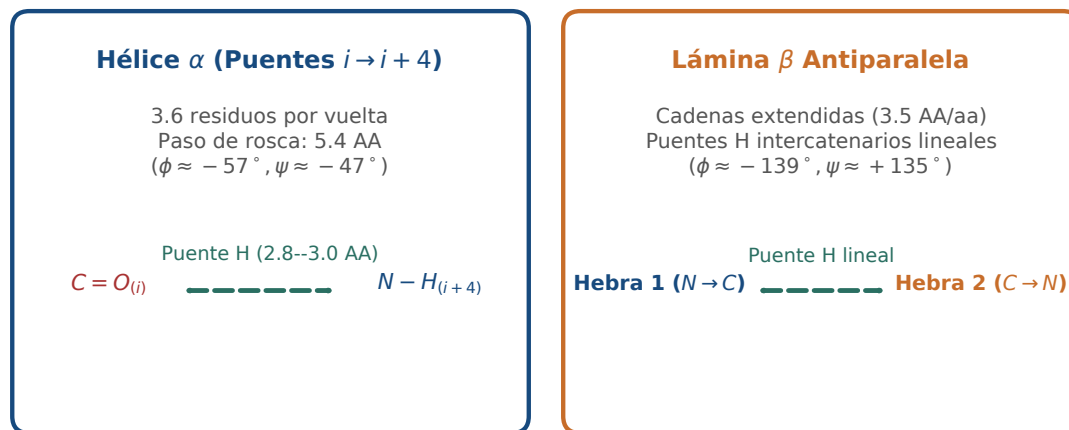


Figura 2.2: Patrones regulares de puentes de hidrógeno que estabilizan la α -hélice ($i \rightarrow i + 4$) y las láminas β antiparalelas. Figura original generada por `verification/make_figures.py`.

2.5.1 Entre la secundaria y la terciaria: los motivos supersecundarios

ESTUDIO Entre los elementos de estructura secundaria y el plegamiento global hay un nivel intermedio que conviene nombrar, porque es el que reaparece cuando se comparan proteínas distintas. Las hélices y las láminas no se combinan de cualquier manera: aparecen una y otra vez en unas pocas disposiciones recurrentes, llamadas **motivos supersecundarios** o plegamientos simples.

- **Hélice-giro-hélice.** Dos hélices unidas por un giro corto, con una de ellas encajando en el surco mayor del ADN. Es el motivo de unión a ADN más extendido entre los factores de transcripción bacterianos.
- **Horquilla beta.** Dos hebras antiparalelas consecutivas unidas por un giro de dos o tres residuos. Es la unidad con que se construyen las láminas beta grandes.
- **Motivo beta-alfa-beta.** Dos hebras paralelas conectadas por una hélice que cruza por encima. Repetido varias veces, forma el barril que aparece en muchísimas enzimas del metabolismo.

Concepto. Estos motivos importan por una razón práctica. Son las unidades que se conservan a lo largo de la evolución y que los métodos de predicción de estructura reconocen, y por eso dos proteínas con secuencias muy distintas pueden compartir plegamiento. Un **dominio** es un paso más: una región que se pliega de forma

autónoma, suele tener su propia función y aparece recombinada en proteínas diferentes. Los capítulos de predicción de estructura trabajan sobre dominios, no sobre proteínas enteras, y la razón está aquí.

2.5.2 Fuerzas biofísicas que dirigen la estructura terciaria

La conformación terciaria nativa está estabilizada por un delicado balance de fuerzas termodinámicas no covalentes y enlaces covalentes entre las cadenas laterales –R:

1. El efecto hidrofóbico (motor termodinámico del plegamiento):

En agua líquida, las moléculas polares del solvente forman jaulas ordenadas de puentes de hidrógeno de baja entropía (*clatratos*) alrededor de las cadenas laterales no polares (Leu, Ile, Val, Phe, Met, Trp). Durante el plegamiento, las cadenas hidrofóbicas colapsan y se entierran en el centro de la proteína, formando un **núcleo hidrofóbico** deshidratado y liberando las moléculas de agua a la fase móvil desordenada. El enorme incremento favorable de entropía del solvente ($\Delta S_{\text{solv}} > 0$) es la principal fuerza impulsora que compensa la pérdida de entropía conformacional de la cadena ($\Delta S_{\text{cadena}} < 0$).

2. Puentes salinos o interacciones electrostáticas:

Atracciones iónicas entre cadenas laterales con carga neta opuesta a pH fisiológico (≈ 7.4): los aniones carboxilato del Aspartato y Glutamato ($-\text{COO}^-$) interaccionan fuertemente con los cationes amonio de la Lisina ($-\text{NH}_3^+$) o guanidinio de la Arginina ($-\text{C}(\text{NH}_2)_2^+$). En el interior hidrofóbico de la proteína, donde la constante dieléctrica es muy baja ($\epsilon \approx 4$ frente a $\epsilon \approx 80$ en agua libre), estos puentes salinos adquieren una enorme energía de enlace que fija la geometría de dominios.

3. Puentes disulfuro covalentes:

Enlaces covalentes sencillos Cys-S-S-Cys resultantes de la oxidación enzimática de dos grupos tiol ($-\text{SH}$) de residuos de Cisteína espacialmente próximos. Aportan una extraordinaria estabilidad mecánica y térmica, siendo característicos de proteínas secretadas que operan en ambientes extracelulares hostiles (insulina, anticuerpos, enzimas digestivas).

2.6 Propiedades fisicoquímicas computables

A partir de la secuencia primaria de aminoácidos, un conjunto fundamental de propiedades fisicoquímicas moleculares puede calcularse analíticamente de forma exacta.

2.6.1 Masa molecular media y monoisotópica

Al formarse un enlace peptídico entre dos aminoácidos libres, se expulsa una molécula de agua (H_2O). Por tanto, la masa de un péptido de N residuos $P = a_1 a_2 \dots a_N$ es la suma de las masas residuales de sus monómeros enlazados más la masa de una única molécula de agua terminal (+H en el N-terminal y +OH en el C-terminal):

$$\text{MW}(P) = \sum_{i=1}^N \text{MW}_{\text{residuo}}(a_i) + \text{MW}(\text{H}_2\text{O})$$

donde la masa promedio estándar del agua condensada es $\text{MW}(\text{H}_2\text{O}) = 18.015$ Da.

El peso molecular peptídico computable suma las masas de los residuos monoméricos más la masa de una molécula de agua terminal (18.015 Da).

Traza manual para el péptido modelo MKVLM:

Dada la secuencia peptídica $P = \text{''MKVLM''}$ ($N = 5$ aminoácidos):

- Metionina (M): 131,20 Da
- Lisina (K): 128,17 Da
- Valina (V): 99,13 Da
- Leucina (L): 113,16 Da
- Metionina (M): 131,20 Da

- Suma de los cinco residuos: $131,20 + 128,17 + 99,13 + 113,16 + 131,20 = 602,86$ Da.
- Agua del extremo, que la polimerización no elimina: $+18,015$ Da.
- Masa media total: $602,86 + 18,015 = 620,875$ Da.

Comprueba. Haga la suma usted, con estos mismos cinco números, antes de ejecutar nada. Si su resultado no es exactamente el impreso, el problema está en la tabla de masas que ha usado, no en la suma: las masas de residuo se publican con distinto número de decimales según la fuente, y mezclar dos tablas produce discrepancias en la tercera cifra que después nadie sabe explicar. La tabla completa de los veinte residuos, que es la que usa el oráculo de este capítulo, está en el apartado siguiente.

Las veinte masas de residuo.

Estas son las masas medias que usa el oráculo, en dalton. Son masas de *residuo*, es decir, del aminoácido menos el agua que se pierde al formar el enlace; por eso hay que sumar un agua al final y solo una, independientemente de la longitud del péptido.

G 57,05	A 71,08	S 87,08	P 97,12	V 99,13	T 101,11
C 103,14	L 113,16	I 113,16	N 114,10	D 115,09	Q 128,13
K 128,17	E 129,12	M 131,20	H 137,14	F 147,18	R 156,19
Y 163,18	W 186,21				

Atención. Leucina e isoleucina tienen la misma masa, 113,16 Da, porque son isómeros: los mismos átomos colocados de otra manera. Un espectrómetro de masas no las distingue, y esa es una limitación real de la identificación de péptidos por masa, no un redondeo de esta tabla.

Para el péptido modelo MKVLM de longitud 5, el peso molecular medio total evalúa exactamente a 620.875 Da.

Se reproduce con `verification/chapter_checks.py mw MKVLM`.

2.6.2 Composición de aminoácidos y validación de alfabeto

El conteo de residuos sobre el péptido modelo "MKVLM" produce: K : 1, L : 1, M : 2, V : 1 (longitud total $N = 5$).

La composición de aminoácidos de MKVLM produce los conteos K:1, L:1, M:2 y V:1.

Se reproduce con `verification/chapter_checks.py composition MKVLM`.

Control defensivo de residuos no canónicos:

El procesador de secuencias peptídicas debe rechazar caracteres inválidos o ambiguos como 'X' o 'Z':

`validate_protein` lanza `ValueError` al encontrar residuos no canónicos como X y Z en 'MKVZX', aceptando secuencias peptídicas válidas.

Se reproduce con `verification/chapter_checks.py validate MKVZX`.

2.7 Escala de hidropatía de Kyte-Doolittle y perfiles

En 1982, Jack Kyte y Russell Doolittle desarrollaron una escala cuantitativa de hidropatía asignando a cada aminoácido un valor continuo en el rango $[-4.5, +4.5]$ derivado de medidas experimentales de energía libre de transferencia agua/vapor y del grado de enterramiento observado cristalográficamente:

- **Valores positivos (Hidrofóbicos):** Isoleucina (+4.5), Valina (+4.2), Leucina (+3.8), Fenilalanina (+2.8), Cisteína (+2.5), Metionina (+1.9), Alanina (+1.8).
- **Valores negativos (Hidrofílicos / Polares / Cargados):** Arginina (−4.5), Lisina (−3.9), Aspartato (−3.5), Glutamato (−3.5), Asparagina (−3.5), Glutamina (−3.5), Histidina (−3.2).

La escala de hidropatía de Kyte-Doolittle asigna puntuaciones positivas a residuos hidrofóbicos (I: 4.5, V: 4.2, L: 3.8) y negativas a residuos polares y cargados (R: -4.5, K: -3.9, D: -3.5).

2.7.1 Hidropatía media y perfiles en ventana deslizante

La hidropatía media $\bar{H}$ de un péptido $P = a_1 a_2 \dots a_N$ es la media aritmética de las puntuaciones individuales de sus residuos:

$$\bar{H}(P) = \frac{1}{N} \sum_{i=1}^N H(a_i)$$

Traza manual para el péptido modelo MKVLM:

- M : +1.9
- K : -3.9
- V : +4.2
- L : +3.8
- M : +1.9
- Suma total: $1.9 + (-3.9) + 4.2 + 3.8 + 1.9 = 7.9$.
- Media: $\bar{H} = \frac{7.9}{5} = 1.580000$.

Para el péptido modelo MKVLM, la hidropatía media de Kyte-Doolittle evalúa exactamente a 1.580000.

Se reproduce con `verification/chapter_checks.py hydropathy MKVLM`.

Perfil en ventana deslizante ($W = 3, S = 1$):

Al deslizar una ventana de tamaño 3 sobre "MKVLM":

1. Ventana 1 (0..3, "MKV"): $(1.9 - 3.9 + 4.2)/3 = 2.2/3 = 0.73$
2. Ventana 2 (1..4, "KVL"): $(-3.9 + 4.2 + 3.8)/3 = 4.1/3 = 1.37$
3. Ventana 3 (2..5, "VLM"): $(4.2 + 3.8 + 1.9)/3 = 9.9/3 = 3.30$

El perfil de hidropatía en ventana deslizante de tamaño 3 sobre MKVLM produce 0-3:MKV:0.73, 1-4:KVL:1.37 y 2-5:VLM:3.30.

Se reproduce con `verification/chapter_checks.py hydro_profile MKVLM --window 3`.

2.8 Desnaturalización proteica y estabilidad conformacional

La conformación nativa tridimensional es termodinámicamente marginal: la diferencia neta de energía libre entre el estado plegado nativo y el desplegado es de apenas $\Delta G_{\text{pleg}} \approx -20$ a -60 kJ/mol (-5 a -15 kcal/mol), equivalente a la energía de unos pocos puentes de hidrógeno.

La **desnaturalización** es el proceso por el cual una proteína pierde sus estructuras cuaternaria, terciaria y secundaria, desplegándose en un ovillo estadístico desordenado (*random coil*), **sin que se rompa ningún enlace peptídico covalente de la estructura primaria**.

- **Agentes térmicos:** El incremento de temperatura eleva la energía cinética vibracional y rotacional, superando las interacciones débiles (puentes de H, Van der Waals).
- **Agentes químicos caotrópicos (Urea 8 M, Cloruro de Guanidino 6 M):** Interrumpen la red de puentes de hidrógeno del agua y solvatan directamente los grupos peptídicos y cadenas hidrofóbicas.
- **Agentes reductores (β -mercaptoetanol, DTT):** Reducen específicamente los puentes disulfuro covalentes $\text{Cys-S-S-Cys} \rightarrow 2 \text{Cys-SH}$.

La desnaturalización proteica causa la pérdida de las estructuras nativas cuaternaria, terciaria y secundaria, preservando los enlaces peptídicos primarios covalentes.

2.9 Propiedades emergentes: alosterismo y cooperatividad

El ensamblaje cuaternario no es una mera suma aditiva de monómeros, sino que engendra **propiedades emergentes** sofisticadas, entre las que destaca el **alosterismo** y la **cooperatividad homotrópica**.

2.9.1 El caso paradigmático: Hemoglobina vs Mioglobina

- **Mioglobina (monomérica, 1 grupo hemo):** Almacena oxígeno en el tejido muscular esquelético. Su curva de saturación fraccional Y frente a la presión parcial de oxígeno (pO_2) es una **hipérbola rectangular** clásica descrita por la isoterma de Langmuir:

$$Y = \frac{pO_2}{pO_2 + P_{50}}$$

con una afinidad altísima ($P_{50} \approx 2.8$ mmHg). Retiene el oxígeno fuertemente y solo lo libera bajo condiciones de anoxia extrema en el músculo activo.

- **Hemoglobina (tetramérica $\alpha_2\beta_2$, 4 grupos hemo):** Transporta oxígeno desde los alvéolos pulmonares ($pO_2 \approx 100$ mmHg) hasta los tejidos periféricos ($pO_2 \approx 20$ – 40 mmHg). Su curva de unión al oxígeno es marcadamente **sigmoidea**, descrita por la **ecuación de Hill**:

$$Y = \frac{(pO_2)^{n_H}}{(pO_2)^{n_H} + (P_{50})^{n_H}}$$

donde $n_H \approx 2.8$ es el **coeficiente de Hill**, reflejando una intensa cooperatividad positiva: la unión de una primera molécula de O_2 a una subunidad induce una transición conformacional alostérica desde el estado tenso de baja afinidad (T) hacia el estado relajado de alta afinidad (R), facilitando la unión cooperativa en las subunidades restantes.

La estructura cuaternaria confiere propiedades alostéricas y cooperativas emergentes, ejemplificadas por la cooperatividad homotrópica en la unión de oxígeno en la hemoglobina tetramérica ($\alpha_2\beta_2$).

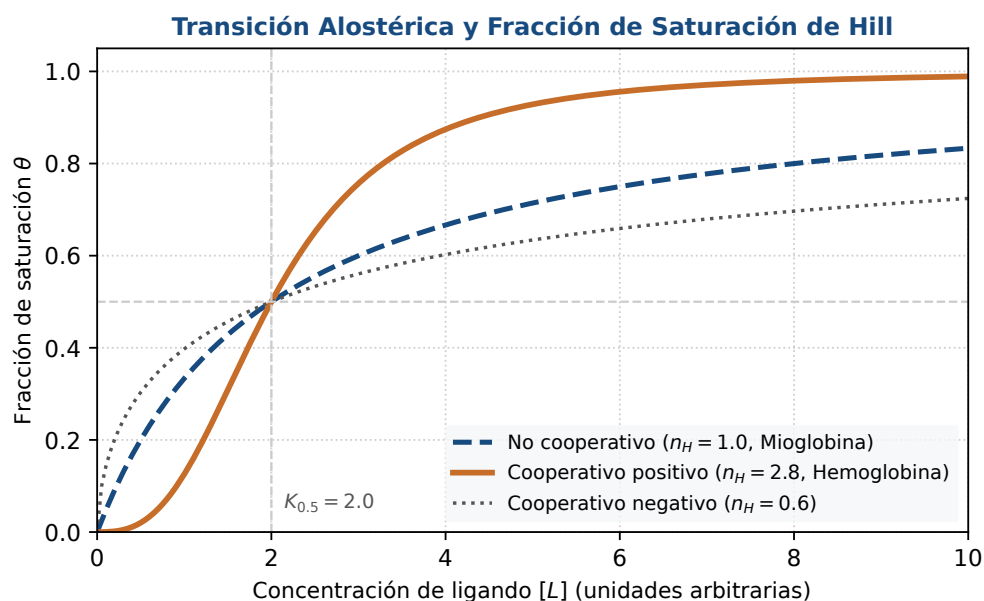


Figura 2.3: Transición alostérica y curvas de saturación fraccional: contraste entre cinética hiperbólica (Mioglobina, $n_H = 1$) y sigmoidea cooperativa (Hemoglobina, $n_H = 2.8$). Figura original generada por `verification/make_figures.py`.

2.10 Diseño computacional en Python: Módulos de proteínas

A continuación se presenta la arquitectura de la clase 'Protein' en Python 3.9:

```
from typing import Dict, List
```

```

AMINO_ACIDS = set("ACDEFGHIKLMNPQRSTVWY")

RESIDUE_WEIGHTS = {
    'A': 71.078, 'C': 103.138, 'D': 115.087, 'E': 129.114, 'F': 147.174,
    'G': 57.051, 'H': 137.139, 'I': 113.159, 'K': 128.174, 'L': 113.159,
    'M': 131.198, 'N': 114.103, 'P': 97.115, 'Q': 128.129, 'R': 156.188,
    'S': 87.077, 'T': 101.104, 'V': 99.133, 'W': 186.210, 'Y': 163.173
}

KYTE_DOOLITTLE = {
    'A': 1.8, 'C': 2.5, 'D': -3.5, 'E': -3.5, 'F': 2.8,
    'G': -0.4, 'H': -3.2, 'I': 4.5, 'K': -3.9, 'L': 3.8,
    'M': 1.9, 'N': -3.5, 'P': -1.6, 'Q': -3.5, 'R': -4.5,
    'S': -0.8, 'T': -0.7, 'V': 4.2, 'W': -0.9, 'Y': -1.3
}

WATER_WEIGHT = 18.015

class Protein:
    """Representa una proteina computable con propiedades fisicoquimicas."""
    def __init__(self, sequence: str, protein_id: str = "prot_01"):
        clean_seq = sequence.strip().upper()
        invalid = set(clean_seq) - AMINO_ACIDS
        if invalid:
            raise ValueError(f"Residuos invalidos detectados: {sorted(invalid)}")
        self._sequence = clean_seq
        self._id = protein_id

    @property
    def sequence(self) -> str:
        return self._sequence

    def __len__(self) -> int:
        return len(self._sequence)

    def molecular_weight(self) -> float:
        """Calcula el peso molecular promedio sumando agua terminal."""
        return sum(RESIDUE_WEIGHTS[aa] for aa in self._sequence) + WATER_WEIGHT

    def mean_hydrophathy(self) -> float:
        """Calcula la hidropatia media segun Kyte-Doolittle."""
        return sum(KYTE_DOOLITTLE[aa] for aa in self._sequence) / len(self._sequence)

    def hydrophathy_profile(self, window_size: int) -> List[float]:
        """Calcula el perfil de hidropatia en ventana deslizante."""
        n = len(self._sequence)
        if window_size <= 0 or window_size > n:
            raise ValueError("Tamano de ventana invalido")
        return [
            round(sum(KYTE_DOOLITTLE[aa] for aa in self._sequence[i:i+window_size]) / window_size, 2)
            for i in range(n - window_size + 1)
        ]

```

2.11 Peligros computacionales y actividad de depuración

Los argumentos mutables por defecto en clases de secuencias y las discrepancias entre masas moleculares monoisotópicas y promedio constituyen peligros computacionales críticos.

2.11.1 Tres errores críticos en código estructural

Localice y corrija los tres errores en la siguiente implementación:

```

class DefectiveProtein:
    # Error 1: Argumento mutable por defecto en constructor
    def __init__(self, sequence, annotations=[]):

```

```

self.sequence = sequence
self.annotations = annotations

# Error 2: Suma directa de residuos olvidando el agua terminal (18.015 Da)
def calculate_mw(self):
    return sum(RESIDUE_WEIGHTS[aa] for aa in self.sequence)

# Error 3: Confunde la escala de Kyte-Doolittle invirtiendo signos
def get_hydro_score(self, aa):
    return -KYTE_DOOLITTLE[aa] # Inversion de signo incorrecta

```

Diagnóstico de causa raíz (Protocolo 5 Whys):

1. **Fallo 1 (Argumento mutable por defecto):** Las listas `annotations=[]` se crean una sola vez al definir la clase. Todas las instancias compartirán la misma lista si no se les pasa un argumento explícito.
Solución: `annotations=None` y asignar `self.annotations = []` if `annotations is None` else `annotations`.
2. **Fallo 2 (Omisión de agua terminal):** El cálculo suma los residuos enlazados pero omite el $-H$ del N-terminal y $-OH$ del C-terminal, subestimando la masa en exactamente 18.015 Da.
3. **Fallo 3 (Inversión de signo en hidropatía):** En la escala Kyte-Doolittle, los valores positivos indican hidrofobicidad; invertir el signo corrompe la predicción de hélices transmembrana.

2.12 Síntesis operativa y tablas de diagnóstico

2.12.1 Tabla de diagnóstico de fallos en modelado molecular

Síntoma observado	Causa probable	Comprobación y corrección
Masa molecular subestimada en 18 Da	Se omitió la masa del agua terminal	Añadir +18.015 Da a la suma de residuos
Péptido hidrofóbico predicho como soluble	Signo invertido en la matriz de hidropatía	Verificar que Ile (+4.5) y Val (+4.2) sean positivos
Anotaciones compartidas entre instancias	Argumento por defecto mutable en el constructor	Emplear None y crear nueva lista en <code>__init__</code>
Fallo en detección de enlaces disulfuro	Confundir Cisteína con Cistina o Metionina	Solo los residuos Cys forman puentes covalentes -S-S-

2.12.2 Tabla de síntesis: Jerarquía estructural de proteínas

Nivel estructural	Fuerzas predominantes	Elementos característicos	Ejemplo representativo
Primario	Enlaces covalentes peptídicos	Cadena polipeptídica $N \rightarrow C$	Secuencia de Insulina
Secundario	Puentes de H en esqueleto	Hélices α , láminas β , giros	α -queratina, fibroína
Terciario	Efecto hidrofóbico, puentes S-S	Núcleo hidrofóbico, dominios	Mioglobina monomérica
Cuaternario	interacciones entre subunidades	complejos con regulación alostérica	hemoglobina

2.13 Glosario conceptual

- **Enlace peptídico:** Unión amida planar con 40% de doble enlace.
- **Ramachandran:** Mapa de combinaciones permitidas (ϕ, ψ).
- **Hélice alfa:** Cilindro dextrógiro con puentes $i \rightarrow i + 4$.
- **Lámina beta:** Cadenas extendidas paralelas o antiparalelas.
- **Efecto hidrofóbico:** Aumento entrópico del agua guiando el plegamiento.
- **Puente disulfuro:** Enlace covalente -S-S- entre Cisteínas.
- **Kyte-Doolittle:** Escala de hidropatía relativa $[-4.5, +4.5]$.
- **Zwitterión:** Molécula dipolar eléctricamente neutra al pI.
- **Alosterismo:** Cambio conformacional transmitido entre subunidades.
- **Ecuación de Hill:** Modelo cuantitativo de unión cooperativa (n_H).

2.14 Conexiones y mapa del curso

Este capítulo conecta la representación de secuencias con los objetos biomoleculares físicos, preparando el terreno para el análisis genómico y de variantes en BC-CH03.

2.15 Fuentes y reproducibilidad

Este capítulo se apoya en la presentación docente sobre proteínas de la asignatura, escrita por Álvaro Serrano Navarro. El diagrama de ángulos permitidos sigue a Ramachandran y colaboradores [57]; la escala de hidropatía, a Kyte y Doolittle [43]; la jerarquía estructural y los motivos supersecundarios, a Branden y Tooze [10]; y las funciones de las proteínas en la célula, a Alberts y colaboradores [3].

Todos los cálculos se reproducen con `verification/chapter_checks.py` tal como están impresos.

Anexo práctico · Modelado computacional de péptidos y propiedades biofísicas

Pregunta, producto y separación de materiales

¿Puede implementar una clase de Python que modele péptidos como objetos computables, calculando de forma exacta su masa molecular, perfil de hidropatía de Kyte–Doolittle y validando invariantes biofísicos sin librerías externas opacas? Este laboratorio responde afirmativamente de forma totalmente reproducible.

El producto individual es `mi_proteina.py`, con una clase `Proteina` escrita por usted que valide el alfabeto al construir el objeto y ofrezca masa, composición, hidropatía media y perfil por ventanas. Le acompañan la tabla de predicciones y resultados, las salidas guardadas y la defensa escrita.

El anexo es autónomo: solo necesita la biblioteca estándar. El generador deja en `herramienta/` el oráculo del capítulo y un comprobador que instancia su clase con péptidos que no aparecen impresos en ningún sitio.

Mapa de ruta y productos entregables

Bloque de trabajo	Producto entregable
1 · Entorno y diagnóstico	Comprobación del intérprete Python 3.9
2 · Cálculo ancla (Masa e Hidropatía)	Cálculo a mano de MKVLM vs script
3 · Perfil de hidropatía ($W = 3$)	Evaluación de ventanas deslizantes
4 · Clase <code>ProteinSequence</code>	Objeto computable con métodos y propiedades
5 · Guarda de dominio	Captura de <code>ValueError</code> ante MKVZX
6 · Física del enlace y Ramachandran	Preguntas de control conformacional
7 · Verificación y empaquetado	Comprobación final y empaquetado

Este anexo produce evidencia comprobable y verificable en cada bloque de trabajo sin paquetes externos.

1 • Entorno y diagnóstico

Verifique que su entorno dispone de Python 3.9 o superior:

```
python3 --version
./practice_assets/create_workspace.sh BC02_entrega
cd BC02_entrega
```

El generador crea la plantilla de la clase con su contrato en la documentación y sin implementación, la tabla de respuestas con solo la cabecera, la carpeta de resultados y la de herramientas, con el oráculo del capítulo y el comprobador dentro.

2 • Escribir la clase

Tarea. Implemente la clase `Proteina` en `mi_proteina.py`: el constructor normaliza y valida, y los métodos calculan masa, composición, hidropatía media y perfil por ventanas. Las dos tablas que necesita, masas de residuo y escala de hidropatía, están en el capítulo.

Hay dos decisiones de diseño que se evalúan más que el cálculo. La primera es dónde validar: si la guarda está en el constructor, ningún método tiene que volver a preguntarse si la secuencia es válida, y un objeto que existe es un objeto correcto. La segunda es el agua: se suma una sola vez, no una por residuo, y el comprobador tiene una prueba dedicada a eso porque es el error clásico.

Compruebe su avance cuantas veces quiera:

```
bash herramienta/check_clase.sh
```

El comprobador no le da los valores del enunciado: instancia su clase con cuatro péptidos que no aparecen impresos y verifica seis propiedades.

3 • Cálculo ancla: masa molecular e hidropatía media

Dado el péptido modelo $S = \text{'MKVLM'}$ ($N = 5$ residuos: Metionina, Lisina, Valina, Leucina, Metionina):

1. Calcule a mano la masa media sumando las cinco masas de residuo del capítulo y añadiendo una sola agua terminal.
2. Calcule la hidropatía media sumando los cinco índices de la escala y dividiendo por el número de residuos.
3. Obtenga la composición de aminoácidos.

```
python3 herramienta/chapter_checks.py mw MKVLM > resultados/masa.txt
python3 herramienta/chapter_checks.py hydropathy MKVLM >> resultados/masa.txt
cat resultados/masa.txt
```

El péptido ancla MKVLM de longitud 5 produce una masa molecular de 620.875 Da, una hidropatía media de 1.580000 y una composición K:1, L:1, M:2, V:1.

Se reproduce con `verification/chapter_checks.py mw MKVLM`.

4 • Perfil de hidropatía en ventana deslizante

Evalúe el perfil de hidropatía sobre MKVLM con ventana $W = 3$:

```
python3 herramienta/chapter_checks.py hydro_profile MKVLM --window 3 > resultados/perfil.txt
cat resultados/perfil.txt
```

La perturbación de perfil de hidropatía con ventana 3 sobre MKVLM evalúa las ventanas 0-3:MKV:0.73, 1-4:KVL:1.37 y 2-5:VLM:3.30.

Se reproduce con `verification/chapter_checks.py hydro_profile MKVLM --window 3`.

5 • Guarda de dominio y control de excepciones

Compruebe que su validador rechaza cadenas contaminadas con residuos no canónicos:

```
python3 herramienta/chapter_checks.py validate MKVZX > resultados/guarda.txt
cat resultados/guarda.txt
```

La guarda de dominio rechaza el péptido no válido MKVZX lanzando un `ValueError` que identifica los residuos inválidos 'X' y 'Z'.

Se reproduce con `verification/chapter_checks.py validate MKVZX`.

6 • Empaquetar y verificar

Escriba antes `notas/defensa.md`. Después:

```
cd ..
tar -czf BC02_entrega.tar.gz BC02_entrega
sha256sum BC02_entrega.tar.gz
./practice_assets/check_entrega.sh BC02_entrega BC02_entrega.tar.gz
```

El verificador prueba su clase con los péptidos que no ha visto, compruebe que la tabla de respuestas tiene al menos cuatro filas con justificación y una del perfil, que `resultados/` recoge algún perfil ejecutado, y que existe la defensa. Rechaza el espacio recién generado. No puntúa.

Rúbrica de calificación ponderada

La evaluación sobre 10 puntos se pondera según:

- **4.0 puntos (40%):** Ejecución correcta y reproducible de los scripts de comprobación (`chapter_checks.py` y `practice_checks.py`).
- **2.0 puntos (20%):** Cálculo manual exacto de la masa molecular (620.875 Da) y la hidropatía media (1.580000) de MKVLM.
- **2.0 puntos (20%):** Análisis de perfiles de hidropatía por ventanas deslizantes y control de excepciones (Bloques 3 y 4).
- **2.0 puntos (20%):** Defensa conceptual escrita (100–150 palabras) sobre la física del enlace peptídico, planaridad y Ramachandran.

Lista de autocorrección

- mi clase valida en el constructor, no en cada método;
- sumo una sola agua, sea cual sea la longitud del péptido;
- mi cálculo manual de masa coincidió con el del oráculo;
- comprobé las tres ventanas del perfil y sé de dónde sale cada media;
- mi constructor rechaza los residuos no canónicos y normaliza mayúsculas y espacios;
- anoté cada predicción antes de ejecutar la orden correspondiente;
- guardé en `resultados/` la salida de cada ejecución que cito;
- el verificador termina en `ENTREGA_OK`.

Defensa conceptual

En `notas/defensa.md`, entre 100 y 150 palabras, explique por qué el enlace peptídico es coplanar y rígido, y cómo esta restricción química reduce el espacio conformacional de una proteína a las rotaciones en torno a los ángulos diedros ϕ y ψ del diagrama de Ramachandran.

Fuentes y reproducibilidad

Este anexo no usa datos externos ni paquetes de terceros. La escala de hidropatía sigue a Kyte y Doolittle [43], y la jerarquía estructural, a Branden y Tooze [10]. El oráculo del capítulo es determinista y da el mismo resultado en cualquier máquina.

Genomas, epigenética y variantes: Arquitectura cromatínica, dinámica de CpG y caracterización computacional de variantes

BC-CH03 – Biología Computacional

Edición 2

Pregunta del tema. Solo el dos por ciento del genoma humano codifica proteínas. ¿Qué hace el resto, y cómo se decide si un cambio de una sola letra en cualquier punto de esos tres mil millones importa o no?

Producto final. Un procesador de variantes escrito por usted que lea el formato estándar, convierta bien las coordenadas, clasifique cada cambio y calcule el indicador de calidad del lote, avisando cuando se sale de lo esperado.

Mapa del tema

3.1. La arquitectura física y funcional del genoma eucariota	34
3.2. Estructura de la cromatina: Nucleosomas y empaquetamiento	36
3.3. El código epigenético y modificaciones de histonas	37
3.4. Variantes genéticas: Clasificación física y espectro mutacional	39
3.5. Consecuencias funcionales en regiones codificantes	39
3.6. El formato estándar Variant Call Format (VCFv4.2)	40
3.7. Variantes estructurales y firmas mutacionales	40
3.8. Diseño computacional en Python: Procesador de variantes y CpG	41
3.9. Peligros computacionales y actividad de depuración	41
3.10. Síntesis operativa y tablas de diagnóstico	42
3.11. Glosario conceptual	43
3.12. Conexiones y mapa del curso	43
3.13. Fuentes y reproducibilidad	43

Cómo usar este tema

Este capítulo es el mapa del territorio sobre el que trabajan todos los demás. Empieza por el continente, el genoma y su composición; sigue por cómo se empaqueta y cómo ese empaquetamiento regula qué se lee; y termina en la unidad mínima de variación, el cambio de una sola base, con el formato en que se anota y el control de calidad que permite saber si un lote de variantes es creíble.

Al terminar sabrá: decir qué proporción del genoma es qué, y qué hace la parte que no codifica; explicar por qué una isla CpG se define por densidad y no por presencia; leer una línea de un fichero de variantes y saber en qué base empieza a contar; clasificar un cambio como transición o transversión y decir por qué esa distinción es un control de calidad; y no confundir «la variante está en un gen» con «la variante causa la enfermedad».

La ruta **esencial** son la composición del genoma, la cromatina, las islas CpG y la variación con su formato. La ruta de **estudio** añade el código de histonas, la paradoja del valor C, las consecuencias funcionales y el anexo.

3.1 La arquitectura física y funcional del genoma eucariota

ESENCIAL En los capítulos precedentes hemos establecido los principios bioquímicos y computacionales para

modelar secuencias discretas y objetos proteicos aislados. Sin embargo, en los organismos eucariotas superiores, el genoma dista radicalmente de ser un texto plano continuo de genes contiguos. Muy al contrario, el genoma humano (con sus $\approx 3.2 \times 10^9$ pares de bases por genoma haploide) presenta una arquitectura modular discontinua, tridimensionalmente organizada y densamente regulada en el espacio y en el tiempo:

1. **Estructura génica discontinua (mosaico exón-intrón):** Descubierta en 1977 por Phillip Sharp y Richard Roberts (Premio Nobel en 1993), los genes codificantes eucariotas están fragmentados en segmentos expresados (**exones**) separados por extensas regiones intercaladas no codificantes (**intrones**). Estos intrones son reconocidos y escindidos con precisión de un solo nucleótido por la maquinaria del espliceosoma (*spliceosome*) durante el procesamiento del pre-ARN mensajero.
2. **Promotores basales y distales:** Secuencias reguladoras adyacentes al sitio de inicio de la transcripción (*Transcription Start Site*, TSS), como la caja TATA (*TATA box*), el elemento iniciador (Inr) y los elementos de reconocimiento río arriba (BRE), que reclutan los factores generales de transcripción (TFIIA-H) y la ARN Polimerasa II.
3. **Potenciadores (*Enhancers*) y Silenciadores:** Módulos reguladores en *cis* situados a decenas o centenares de kilobases de distancia (río arriba, río abajo o en el interior de intrones) que interactúan físicamente con el promotor génico mediante bucles espaciales de cromatina para modular la tasa de transcripción de forma tejido-específica.
4. **Aisladores de frontera (*Insulators*) y factores CTCF:** Secuencias fijadoras de la proteína con dedos de zinc CTCF que, junto con el complejo molecular de la cohesina, delimitan los **dominios de asociación topológica (TADs)**, impidiendo que los potenciadores de un microdominio activen espuriamente promotores de regiones vecinas.

La arquitectura génica eucariota comprende exones codificantes, intrones no codificantes, promotores basales, potenciadores distales, silenciadores y elementos aisladores de frontera CTCF.

3.1.1 La paradoja del valor C y la fracción no codificante

Durante décadas se asumió erróneamente que la complejidad anatómica y funcional de una especie debía correlacionar linealmente con el tamaño total de su genoma (la cantidad total de ADN por célula haploide, denominada **valor C**). Sin embargo, la secuenciación comparada reveló que organismos morfológicamente sencillos tienen genomas mucho mayores que el humano. La planta *Paris japonica*, con unos 150 Gb repartidos en apenas veinte cromosomas, tiene cincuenta veces más ADN que nosotros, que rondamos los 3,2 Gb. Ese desacoplamiento entre tamaño del genoma y complejidad del organismo se conoce como la **paradoja del valor C**.

Atención. Circulan cifras aún más extremas, como los 670 Gb que suelen atribuirse a la ameba *Polychaos dubium*. Proceden de estimaciones por densitometría de los años sesenta que nunca se han confirmado con métodos actuales, y las bases de datos de tamaños genómicos las marcan como no fiables. Conviene no usarlas como dato, aunque sean el ejemplo más citado: un argumento correcto no necesita apoyarse en una medida dudosa, y *Paris japonica* sirve igual de bien.

En el genoma humano, **únicamente el $\approx 1.5\text{--}2.0\%$ de la secuencia codifica directamente para cadenas polipeptídicas** (lo que constituye el exoma humano, ≈ 30 Mb). El $\approx 98\%$ restante es **ADN no codificante**, densamente poblado por:

- **Elementos transponibles y retrotransposones:** Secuencias móviles repetitivas que han colonizado el genoma a lo largo de la evolución, incluyendo elementos nucleares intercalados largos (LINEs, $\approx 21\%$, destacando LINE-1 de ≈ 6 kb), elementos intercalados cortos (SINEs, $\approx 13\%$, como los elementos Alu de ≈ 300 pb) y retrovirus endógenos humanos (HERVs, $\approx 8\%$).
- **Regiones reguladoras en cis:** Cientos de miles de promotores no codificantes, potenciadores y genes de ARNs funcionales no traducidos (microARNs, lncRNAs, circRNAs, ARNs nucleolares snoRNAs).
- **ADN estructural heterocromático:** Centrómeros satélite y télómeros repetitivos indispensables para la integridad estructural y la segregación cromosómica durante la mitosis.

Concepto. De los elementos transponibles se suele decir cuánto ocupan y no qué hacen, que es lo interesante. Hacen tres cosas. **Mutan:** al insertarse dentro de un gen lo interrumpen, y hay enfermedades humanas causadas exactamente así, por una copia de un elemento que aterrizó donde no debía. **Reorganizan:** dos copias del mismo elemento en posiciones distintas ofrecen a la maquinaria de recombinación dos secuencias idénticas donde emparejarse mal, y de ahí salen duplicaciones y deleciones grandes. **Aportan material:** algunas de sus secuencias han acabado siendo promotores y potenciadores de genes propios, y hasta genes completos han surgido por domesticación de un transposón. No son basura, y tampoco son un adorno: son una fuerza evolutiva activa.

Atención. Para el análisis computacional tienen una consecuencia inmediata: al haber cientos de miles de copias casi idénticas repartidas por el genoma, una lectura corta que caiga en una de ellas se alinea igual de bien en muchos sitios. Por eso los programas de alineamiento informan de la calidad de mapeo, y por eso las regiones repetitivas son las que peor se ensamblan.

La fracción no codificante representa aproximadamente el 98% del genoma humano, enriquecida en elementos transponibles (LINEs, SINEs/Alu) y sustentando la paradoja del valor C.

3.2 Estructura de la cromatina: Nucleosomas y empaquetamiento

ESENCIAL

En el núcleo de una sola célula humana diploide, el ADN extendido alcanzaría una longitud física de ≈ 2 metros lineales. Este colosal polímero debe empaquetarse de manera no entrópica en un volumen nuclear esférico microscópico de apenas 5–10 μm de diámetro, garantizando al mismo tiempo que regiones génicas específicas permanezcan dinámicamente accesibles para la replicación y la transcripción. Este prodigio biofísico de compactación se logra mediante la **cromatina**, un complejo supramolecular de ADN y proteínas estructurales especializadas denominadas **histonas**.

3.2.1 El nucleosoma: unidad fundamental de empaquetamiento

La unidad estructural básica y repetitiva de la cromatina es el **nucleosoma**. Está constituido por un núcleo proteico globular o **octámero de histonas** integrado por dos copias estequiométricas de cada una de las cuatro histonas nucleares centrales:

$$\text{Octámero central} = 2 \times (\text{H2A} + \text{H2B} + \text{H3} + \text{H4})$$

Alrededor de este octámero globular, densamente poblado por aminoácidos básicos cargados positivamente (Lisina y Arginina), la doble hélice de ADN (cargada negativamente por sus grupos fosfato en el esqueleto) se enrolla dando exactamente 1.65 **vuelatas de superhélice levógira, abarcando 146 pares de bases**. Los nucleosomas contiguos se conectan mediante segmentos de **ADN espaciador** (*linker DNA*) de 20–80 pb, asociados a la histona de unión **H1**, originando la clásica fibra conformacional de 10 nm ("collar de cuentas" o *beads-on-a-string*).

El nucleosoma consiste en un octámero de histonas (2 copias de H2A, H2B, H3 y H4) alrededor del cual el ADN da aproximadamente 1,65 vueltas de superhélice levógira, abarcando entre 146 y 147 pares de bases según la estructura de referencia que se tome.

3.2.2 Eucromatina, heterocromatina y niveles superiores de empaquetamiento

La fibra de cromatina transita de forma dinámica entre dos estados funcionales:

- **Eucromatina:** Estado descondensado y espacialmente accesible, caracterizado por nucleosomas espaciados, alta acetilación de histonas y activa transcripción génica.
- **Heterocromatina:** Estado densamente compactado e inaccesible a factores de transcripción:
 - **Heterocromatina constitutiva:** Permanentemente silenciada en todos los tipos celulares (centrómeros, télómeros y repeticiones LINEs), enriquecida en trimetilación de lisina 9 en histona H3 (**H3K9me3**) y proteína HP1.
 - **Heterocromatina facultativa:** Regiones que se silencian de forma reversible durante la diferenciación celular y el desarrollo (como la inactivación del cromosoma X en hembras de mamíferos), mediada



Figura 3.1: Estructura supramolecular del nucleosoma (octámero central y 146 pb de ADN) y modificación postraduccional de colas de histonas. Figura original generada por `verification/make_figures.py`.

por el complejo represor Polycomb (PRC2) y la marca **H3K27me3**.

La cromatina se organiza en unidades nucleosomales repetitivas (146 pb alrededor de un octámero de histonas), dividiéndose funcionalmente en eucromatina permisiva y heterocromatina condensada.

3.3 El código epigenético y modificaciones de histonas

ESTUDIO

Las colas N-terminales de las histonas emergen hacia el exterior del núcleo del nucleosoma, expuestas al solvente nuclear. Estas colas sufren un rico abanico de **modificaciones postraduccionales reversibles (PTMs)** que constituyen el denominado **código epigenético**:

1. **Acetilación de lisinas (por HATs / HDACs)**: La adición de un grupo acetilo por las histona acetiltransferasas (HATs) neutraliza la carga positiva del grupo ϵ -amino de la lisina, debilitando su atracción electrostática hacia el esqueleto de ADN. Esto relaja la estructura cromatínica y permite el reclutamiento de complejos basales de transcripción.
 - **H3K27ac**: Marca canónica de **potenciadores** (*enhancers*) **activos** y promotores abiertos.
 - **H3K9ac**: Asociada a inicio de transcripción activa.
2. **Metilación de lisinas y argininas (por HMTs / KDMs)**: La adición de uno, dos o tres grupos metilo por histona metiltransferasas (HMTs) no altera la carga neta del residuo, pero crea sitios de anclaje estéricos reconocidos específicamente por dominios proteicos (*readers*):
 - **H3K4me3**: Marca universal de **promotores basales activos**.
 - **H3K27me3**: Marca de **silenciamiento facultativo Polycomb**.
 - **H3K9me3**: Marca de **heterocromatina constitutiva densa**.
 - **H3K36me3**: Marca de **elongación transcripcional activa**, localizada a lo largo de los cuerpos génicos (*gene bodies*).

El código de histonas regula la accesibilidad cromatínica: la hiperacetilación (H3K27ac) y metilación activa (H3K4me3) marcan transcripción activa, mientras que H3K9me3 y H3K27me3 imponen silenciamiento.

3.3.1 Metilación del ADN y dinámica biofísica de las islas CpG

En los genomas de vertebrados, la modificación epigenética covalente directa del ADN más abundante es la **5-metilcitosina (5mC)**, producida por la transferencia de un grupo metilo desde la S-adenosilmetionina (SAM) hacia el carbono 5 de la citosina por acción de las ADN metiltransferasas (DNMT1 para mantenimiento replicativo; DNMT3A y DNMT3B para metilación *de novo*).

La metilación de citosinas ocurre de forma casi exclusiva en el contexto del dinucleótido simétrico **CpG** (5'-C-fosfato-G-3'). La hipermetilación de promotores génicos recluta proteínas con dominios MBD y desacetilasas de histonas, imponiendo una represión transcripcional estable.

La metilación del ADN ocurre en dinucleótidos CpG, donde la hipermetilación de islas CpG promotoras ($\text{Obs/Exp} \geq 0.6$ y $\text{GC} \geq 50\%$) causa represión transcripcional estable.

Definición computacional de Gardiner-Garden y Frommer (1987):

Una región genómica se clasifica formalmente como una isla CpG si cumple simultáneamente tres criterios matemáticos:

1. Longitud de secuencia $L \geq 200$ pb (típicamente ≥ 500 pb).
2. Contenido de G+C $\geq 50\%$.
3. Ratio de dinucleótidos CpG Observados respecto a Esperados (Obs/Exp) mayor a **0.60**:

$$\text{Ratio}(\text{CpG}) = \frac{\text{Conteo}(\text{CG}) \times L}{\text{Conteo}(\text{C}) \times \text{Conteo}(\text{G})}$$

Traza manual para la secuencia modelo CGCGCGCGCGCG:

Dada la secuencia sintética $S = \text{"CGCGCGCGCGCG"}$ de longitud $L = 12$ pb:

- Conteo de bases: C : 6, G : 6, A : 0, T : 0.
- Contenido GC: $\frac{6+6}{12} = \frac{12}{12} = 100.0\%$.
- Conteo de dinucleótidos CG solapantes: exactamente 6 dinucleótidos CG.
- Frecuencia esperada: $\text{Esperado}(\text{CG}) = \frac{6 \times 6}{12} = \frac{36}{12} = 3.0$.
- Ratio Obs/Exp:

$$\text{Ratio} = \frac{6 \times 12}{6 \times 6} = \frac{72}{36} = 2.000000$$

Para la secuencia sintética CGCGCGCGCGCG de longitud 12, el ratio CpG Observado/Esperado evalúa exactamente a 2.000000 con un contenido GC del 100 por ciento.

Se reproduce con `verification/chapter_checks.py cpg CGCGCGCGCGCG`.

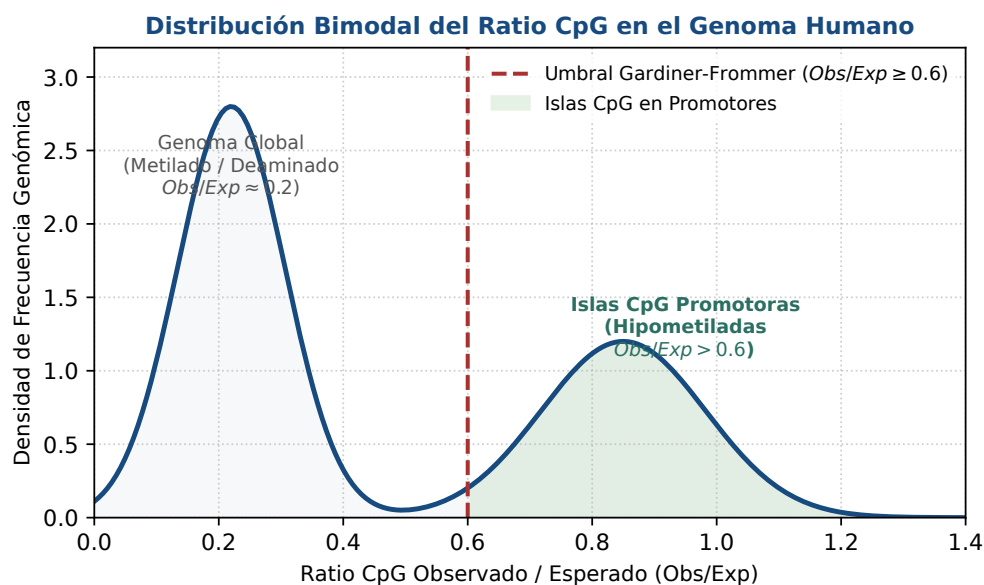


Figura 3.2: Distribución bimodal del ratio CpG (Obs/Exp) en el genoma humano: hipometilación protectora en promotores vs deaminación espontánea general. Figura original generada por `verification/make_figures.py`.

3.4 Variantes genéticas: Clasificación física y espectro mutacional

ESENCIAL

Las variantes de un solo nucleótido (SNVs) se clasifican químicamente según la naturaleza de los anillos de las bases involucradas:

- **Transiciones** (Ti): Sustitución dentro de la misma clase estructural (purina por purina $A \leftrightarrow G$, o pirimidina por pirimidina $C \leftrightarrow T$).
- **Transversiones** (Tv): Sustitución entre clases estructurales distintas (intercambio entre purina y pirimidina, como $A \leftrightarrow C$, $A \leftrightarrow T$, $G \leftrightarrow C$, $G \leftrightarrow T$).

Las variantes de un solo nucleótido (SNVs) se categorizan químicamente en Transiciones (purina-purina o pirimidina-pirimidina) y Transversiones (intercambios purina-pirimidina).

Traza manual de clasificación de SNVs:

`classify_snv` clasifica la mutación A a G como TRANSITION y la mutación A a C como TRANSVERSION.

Se reproduce con `verification/chapter_checks.py snv A G`.

3.4.1 El ratio Ti/Tv como métrica de calidad en genómica

Para un lote de variantes con 4 transiciones y 2 transversiones:

$$\text{Ratio } Ti/Tv = \frac{4}{2} = 2.000000$$

Para un lote de variantes con 4 transiciones y 2 transversiones, el ratio de calidad Ti/Tv evalúa exactamente a 2.000000.

Se reproduce con `verification/chapter_checks.py titv A:G C:T G:A T:C A:C G:T`.

3.5 Consecuencias funcionales en regiones codificantes

ESTUDIO

Las variantes en regiones exónicas codificantes se clasifican funcionalmente en:

1. **Sinónimas** (*Synonymous*): Mismo aminoácido (ej. $GAA \rightarrow GAG$, Glutamato).
2. **Cambio de sentido** (*Missense*): Sustitución aminoacídica, como la mutación drepanocítica $GAG \rightarrow GTG$ (Glu $\rightarrow$ Val) en el gen *HBB*.
3. **Sin sentido** (*Nonsense*): Codón de parada prematuro ($CAG \rightarrow TAG$, Gln $\rightarrow$ Stop).

Las variantes codificantes se categorizan funcionalmente en mutaciones sinónimas, sustituciones missense (como *HBB* Glu6Val en anemia falciforme) y codones de parada prematuros nonsense.

Traza manual de clasificación funcional de codones:

`classify_mutation` clasifica GAA a GAG como SYNONYMOUS, GAG a GTG como MISSENSE (Glu a Val) y CAG a TAG como NONSENSE (Gln a Stop).

Se reproduce con `verification/chapter_checks.py mutation GAA GAG`.

Control defensivo de codones no canónicos:

`classify_mutation` lanza `ValueError` cuando el codón contiene caracteres nucleotídicos no canónicos como X en 'GAX'.

Se reproduce con `verification/chapter_checks.py mutation GAX GAG`.

3.6 El formato estándar Variant Call Format (VCFv4.2)

ESENCIAL

El formato VCF almacena variantes genómicas mediante 8 columnas fijas delimitadas por tabuladores (#CHROM, POS, ID, REF, ALT, QUAL, FILTER, INFO) seguidas de las columnas de genotipos de muestras en coordenadas 1-based.

Traza de parsing de la mutación rs334 de HBB en VCF:

Para la variante rs334 con alelo de referencia T y alternativo A, el parser extrae las coordenadas e identifica la mutación $T \rightarrow A$ como una transversión.

El analizador de formato VCF procesa el cromosoma, la posición 1-based, alelo de referencia y alternativo, clasificando la mutación rs334 de HBB de T a A como TRANSVERSION.

Se reproduce con `verification/chapter_checks.py vcf "chr11 5227002 rs334 T A 100 PASS ."`.

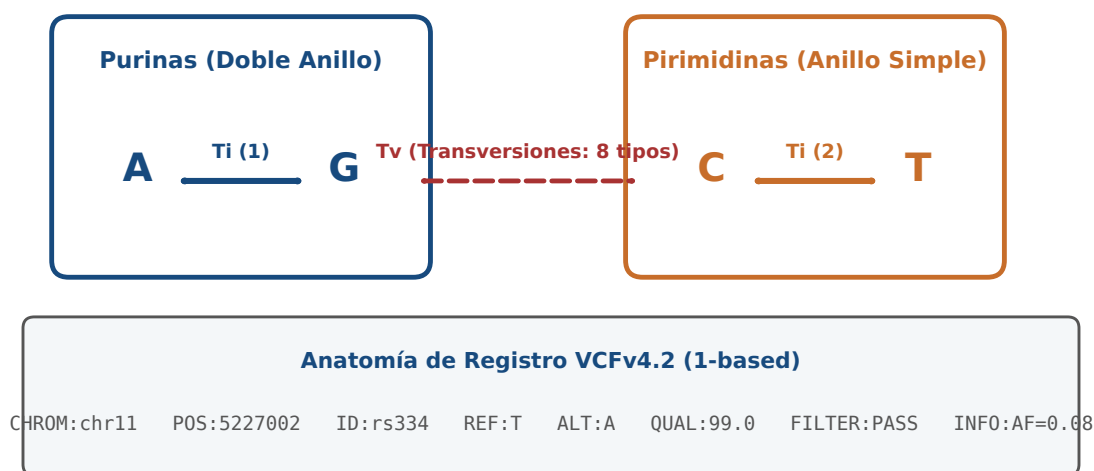


Figura 3.3: Espacio químico de mutaciones puntuales (Ti vs Tv) y estructura de un registro en formato estándar VCFv4.2. Figura original generada por `verification/make_figures.py`.

3.7 Variantes estructurales y firmas mutacionales

ESTUDIO

Las variantes genómicas a gran escala rebasan la resolución de los SNVs individuales, abarcando:

- **Variantes en el Número de Copias (CNVs):** Ganancias (duplicaciones génicas como en el locus de la amilasa *AMY1*) o pérdidas (deleciones homocigotas de supresores tumorales como *CDKN2A*) que alteran la dosificación génica.
- **Reordenamientos cromosómicos complejos:** Inversiones pericéntricas y translocaciones balanceadas o desbalanceadas (como el cromosoma Filadelfia $t(9;22)(q34;q11)$ que origina la oncoproteína de fusión *BCR-ABL1*).
- **Firmas mutacionales somáticas (catálogo COSMIC):** Patrones específicos de sustitución en contextos trinucleotídicos que delatan la etiología molecular subyacente (daño por radiación ultravioleta en melanoma, exposición a humo de tabaco en cáncer de pulmón o fallo en la reparación por recombinación homóloga *BRCA1/2*).

Las variantes en el número de copias (CNVs) y las firmas mutacionales somáticas proporcionan biomarcadores genómicos para trastornos constitucionales y caracterización oncológica.

3.8 Diseño computacional en Python: Procesador de variantes y CpG

ESENCIAL

A continuación se presenta la implementación modular del procesador genómico en Python 3.9:

```
from typing import Tuple, Dict, Optional

TRANSITIONS = {
    ('A', 'G'), ('G', 'A'),
    ('C', 'T'), ('T', 'C')
}

TRANSVERSIONS = {
    ('A', 'C'), ('C', 'A'), ('A', 'T'), ('T', 'A'),
    ('G', 'C'), ('C', 'G'), ('G', 'T'), ('T', 'G')
}

def calculate_cpg_ratio(sequence: str) -> float:
    """Calcula el ratio de CpG Observados respecto a Esperados (Obs/Exp)."""
    seq = sequence.strip().upper()
    n = len(seq)
    if n == 0:
        raise ValueError("Secuencia no puede estar vacia")
    count_c = seq.count('C')
    count_g = seq.count('G')
    if count_c == 0 or count_g == 0:
        return 0.0
    count_cg = sum(1 for i in range(n - 1) if seq[i:i+2] == "CG")
    expected = (count_c * count_g) / n
    return round(count_cg / expected, 6)

def calculate_titv_ratio(transitions: int, transversions: int) -> float:
    """Calcula el ratio de transiciones frente a transversiones (Ti/Tv)."""
    if transitions < 0 or transversions < 0:
        raise ValueError("Los conteos de mutaciones no pueden ser negativos")
    if transversions == 0:
        if transitions == 0:
            return 0.0
        raise ZeroDivisionError("Transversiones no pueden ser cero para calcular ratio")
    return round(transitions / transversions, 6)

def parse_vcf_line(line: str) -> Optional[Dict[str, str]]:
    """Parsea una línea tabular de variante en formato VCFv4.2."""
    if line.startswith('#') or not line.strip():
        return None
    fields = line.strip().split('\t')
    if len(fields) < 8:
        raise ValueError("Línea VCF inválida: requiere al menos 8 columnas obligatorias")
    return {
        'chrom': fields[0],
        'pos': fields[1],
        'id': fields[2],
        'ref': fields[3],
        'alt': fields[4],
        'qual': fields[5],
        'filter': fields[6],
        'info': fields[7]
    }
```

3.9 Peligros computacionales y actividad de depuración

La confusión de sistemas de coordenadas (1-based en VCF/GFF frente a 0-based semiabierto en BED/Python) y el sesgo de depleción de CpG representan fuentes críticas de error.

3.9.1 Tres errores críticos en análisis genómico de variantes

Localice y corrija los tres errores en la siguiente función de procesamiento:

```
def process_variant_pipeline(vcf_line, genome_sequence):
    fields = vcf_line.split('\t')
    chrom, pos, var_id, ref, alt = fields[0], int(fields[1]), fields[2], fields[3], fields[4]

    # Error 1: Indexa la secuencia de Python (0-based) directamente con pos de VCF (1-based)
    genome_base = genome_sequence[pos] # Desfase off-by-one

    # Error 2: Asume que A->C es una transicion
    is_transition = (ref == 'A' and alt == 'C') # A->C es en realidad una transversion

    # Error 3: Divide por longitud total sin descontar el solapamiento al buscar CpG
    return genome_base, is_transition
```

Diagnóstico de causa raíz (Protocolo 5 Whys):

1. **Fallo 1, desfase de una posición.** El formato numera las posiciones desde uno y los índices del array empiezan en cero. Acceder con la posición sin convertir devuelve el nucleótido siguiente al que se quería, y el programa no avisa: devuelve una base perfectamente válida.
2. **Fallo 2 (Clasificación errónea de mutación):** La mutación $A \rightarrow C$ intercambia una purina bicíclica por una pirimidina monocíclica, lo que constituye una transversión (Tv), no una transición.
3. **Fallo 3 (Cálculo incorrecto de dinucleótidos):** Los dinucleótidos se evalúan en ventanas contiguas de longitud 2 con solapamiento, por lo que una secuencia de longitud N contiene exactamente $N - 1$ posiciones posibles de inicio de dinucleótido.

3.10 Síntesis operativa y tablas de diagnóstico

3.10.1 Tabla de diagnóstico de discrepancias genómicas

Síntoma observado	Causa probable	Comprobación y corrección
El alelo de referencia no coincide con el genoma	desfase entre la numeración del fichero y la del array	restar uno a la posición, una sola vez
Cociente entre transiciones y transversiones muy bajo	contaminación o llamadas erróneas repartidas al azar	filtrar por profundidad y por calidad
Fallo en detección de islas CpG	Omitir el umbral de longitud ($L \geq 200$ pb)	Verificar simultáneamente longitud, %GC y ratio Obs/Exp
Se pierden variantes con varios alelos alternativos	se supone que el campo alternativo trae uno solo	separar por comas antes de clasificar

3.10.2 Tabla de síntesis: Tipos de variantes y repercusión funcional

Categoría	Mecanismo molecular	Impacto en la proteína	Ejemplo biomédico
Sinónima	el codón cambia y el aminoácido no	la proteína es idéntica	variación neutra frecuente
De sentido erróneo	el codón pasa a codificar otro aminoácido	cambia un residuo de la cadena	la mutación drepanocítica
Nonsense	Codón $\rightarrow$ Stop prematuro	Truncamiento proteico o NMD	Fibrosis quística grave

Categoría	Mecanismo molecular	Impacto en la proteína	Ejemplo biomédico
Frameshift	Inserción/delección no múltiplo de 3	Pauta de lectura alterada río abajo	<i>BRCA1</i> c.68_69delAG (Cáncer)

3.11 Glosario conceptual

- **Nucleosoma:** Octámero de histonas envuelto por 146 pb de ADN.
- **Eucromatina:** Cromatina descondensada accesible para transcripción.
- **Heterocromatina:** Cromatina compactada transcripcionalmente silenciada.
- **H3K4me3:** Marca de histona característica de promotores activos.
- **H3K27ac:** Marca de histona característica de potenciadores activos.
- **Isla CpG:** Región de alta densidad de CG no metilada (> 0.60).
- **Transición:** Mutación entre bases del mismo anillo ($A \leftrightarrow G, C \leftrightarrow T$).
- **Transversión:** Mutación entre purina y pirimidina.
- **Ratio Ti/Tv:** Métrica de calidad genómica ($\approx 2.0-3.0$).
- **VCF:** Formato estándar tabular de 8 columnas fijas para variantes.

3.12 Conexiones y mapa del curso

Este capítulo establece las bases cromatínicas, epigenéticas y de variación genómica requeridas para abordar el procesamiento del transcriptoma en BC-CH04.

3.13 Fuentes y reproducibilidad

Este capítulo se apoya en la presentación docente de introducción de la asignatura, escrita por Álvaro Serrano Navarro. Los tres criterios que definen una isla CpG siguen a Gardiner-Garden y Frommer [26]; el código de histonas, a Strahl y Allis [67]; la especificación del formato de variantes, a Danecek y colaboradores [17]; y la arquitectura del gen eucariota, a Alberts y colaboradores [3].

Todos los cálculos se reproducen con `verification/chapter_checks.py` tal como están impresos, con sus argumentos.

Anexo práctico · Análisis computacional de variantes genómicas y firmas epigenéticas

Pregunta, producto y separación de materiales

¿Puede implementar un módulo en Python que parsee registros de variantes en formato VCF, clasifique sustituciones en transiciones y transversiones, evalúe el impacto funcional de mutaciones codificantes y determine la métrica de islas CpG sin recurrir a paquetes externos opacos? Este laboratorio responde afirmativamente de forma totalmente reproducible.

El producto individual es `mi_procesador_variantes.py`: un procesador escrito por usted que lea un fichero de variantes, convierta las coordenadas sin equivocarse, clasifique cada cambio, calcule el cociente de calidad del lote y avise cuando ese cociente se aparte de lo esperado. Lo que se evalúa no es clasificar, que es fácil, sino interpretar el número que sale.

El anexo es autónomo: solo necesita la biblioteca estándar. El generador deja el fichero de variantes del capítulo, el oráculo y un comprobador que ejecuta su procesador sobre ficheros que el enunciado no muestra, uno de ellos con una variante en el primer nucleótido del cromosoma.

Mapa de ruta y productos entregables

Bloque de trabajo	Producto entregable
1 · Entorno y diagnóstico	Comprobación del intérprete Python 3.9
2 · Cálculo ancla (VCF y Codón)	Parsing de rs334, GAG→GTG y Ti/Tv base
3 · Perturbación del ratio Ti/Tv	Evaluación con transversión adicional
4 · Métricas de islas CpG	Cálculo de ratio Obs/Exp y GC%
5 · Guarda de dominio	Captura de ValueError ante codón GAX
6 · Preguntas de control epigenético	Preguntas sobre código de histonas y VCF
7 · Verificación y empaquetado	Comprobación final y empaquetado

Este anexo produce evidencia comprobable y verificable en cada bloque de trabajo sin paquetes externos.

1 · Entorno y diagnóstico

Verifique que su entorno dispone de Python 3.9 o superior:

```
python3 --version
./practice_assets/create_workspace.sh BC03_entrega
cd BC03_entrega
cd datos && sha256sum -c manifest.sha256 && cd ..
```

2 · Escribir el procesador de variantes

Tarea. Implemente `mi_procesador_variantes.py`, que reciba la ruta de un fichero de variantes y emita el recuento de transiciones y transversiones, su cociente, y una línea de aviso cuando ese cociente quede fuera del intervalo esperado. El contrato completo está en la documentación de la plantilla.

Hay dos puntos donde este ejercicio se tuerce, y los dos son de criterio, no de programación.

El primero son las coordenadas. El formato numera las posiciones desde uno, y los índices de Python empiezan en cero. Restar uno es correcto, pero hay que hacerlo una sola vez y comprobar el resultado: el fichero del capítulo tiene a propósito una variante en la posición 1, que es exactamente donde una conversión hecha a la ligera produce un índice negativo, y un índice negativo en Python no falla, sino que cuenta desde el final.

El segundo es el aviso. Un cociente entre transiciones y transversiones de alrededor de dos es lo normal en un genoma completo, porque las transiciones son químicamente más probables. Un cociente muy por debajo indica que el lote está contaminado de falsos positivos, que se reparten al azar entre los seis tipos de cambio posibles. Su procesador tiene que avisar, y usted tiene que saber decir de qué está avisando.

Compruebe su avance cuantas veces quiera:

```
bash herramienta/check_procesador.sh
```

3 · Cálculo ancla: variante codificante y cociente del lote

Dado el registro VCF de la mutación de la drepanocitosis (rs334, gen *HBB*):

1. Parsee los campos cromosómicos: cromosoma chr11, posición 5227002, alelos REF = T y ALT = A (clasificada como **Transversión**).
2. Evalúe el efecto de la mutación codificante en el codón 6: GAG(E) → GTG(V), clasificada como **Missense**.
3. Calcule el ratio Ti/Tv para un lote base de 4 transiciones (A:G, G:A, C:T, T:C) y 2 transversiones (A:C, T:G): $4/2 = 2.000000$.

```
python3 herramienta/chapter_checks.py vcf "chr11 5227002 rs334 T A 100 PASS ." > resultados/ancla.txt
python3 herramienta/chapter_checks.py mutation GAG GTG >> resultados/ancla.txt
cat resultados/ancla.txt
```

El registro ancla VCF rs334 produce una mutación de tipo transversión, efecto de codón MISSENSE (Glu a Val) y un ratio base $Ti/Tv = 2.000000$.

Se reproduce con `verification/chapter_checks.py mutation GAG GTG`.

4 • Perturbación del cociente

Añada una transversión adicional (G:C) al lote anterior (4 transiciones y 3 transversiones):

```
python3 herramienta/chapter_checks.py titv A:G C:T G:A T:C A:C G:T > resultados/lote_normal.txt
python3 herramienta/chapter_checks.py titv A:C G:T C:A T:G A:G > resultados/lote_sospechoso.txt
cat resultados/lote_normal.txt resultados/lote_sospechoso.txt
```

La perturbación del lote con una transversión adicional G a C evalúa a 4 transiciones y 3 transversiones, produciendo un ratio $Ti/Tv = 1.333333$.

Se reproduce con `verification/chapter_checks.py titv A:C G:T C:A T:G A:G`.

5 • Guarda de dominio y control de excepciones

Compruebe que el evaluador rechaza codones con nucleótidos no canónicos:

```
python3 herramienta/chapter_checks.py mutation GAX GTG > resultados/guarda.txt
cat resultados/guarda.txt
```

La guarda de dominio rechaza el codón no válido GAX lanzando un `ValueError` que identifica el codón no canónico.

Se reproduce con `verification/chapter_checks.py mutation GAX GTG`.

6 • Empaquetar y verificar

Ejecute su procesador sobre el fichero del capítulo, guarde la salida y escriba la defensa. Después:

```
python3 mi_procesador_variantes.py datos/bc03_variantes.vcf > resultados/lote.txt || true
cd ..
tar -czf BC03_entrega.tar.gz BC03_entrega
sha256sum BC03_entrega.tar.gz
./practice_assets/check_entrega.sh BC03_entrega BC03_entrega.tar.gz
```

El verificador ejecuta su procesador sobre ficheros que no ha visto, incluido el de la variante en la posición 1, comprueba que el fichero de partida no se ha modificado, que la tabla de respuestas tiene al menos cuatro filas con justificación y una sobre las coordenadas, y que existe la defensa. Rechaza el espacio recién generado. No puntúa.

Rúbrica de calificación ponderada

La evaluación sobre 10 puntos se pondera según:

- **4.0 puntos (40%)**: Ejecución correcta y reproducible de los scripts de comprobación (`chapter_checks.py` y `practice_checks.py`).

- **2.0 puntos (20%):** Parsing exacto del registro VCF rs334 y cálculo del ratio $Ti/Tv = 2.000000$ (Bloque 2).
- **2.0 puntos (20%):** Evaluación de perturbación de variantes y control de excepciones en la guarda de dominio (Bloques 3 y 4).
- **2.0 puntos (20%):** Defensa conceptual escrita (100–150 palabras) sobre el ratio Ti/Tv como indicador de calidad bioinformática y la dicotomía epigenética eucromatina/heterocromatina.

Lista de autocorrección

- Verifiqué que mi versión de Python es ≥ 3.9 .
- Parsee correctamente el registro rs334 en VCF.
- Clasifiqué T→A como transversión.
- El efecto de GAG→GTG fue MISSENSE.
- El ratio Ti/Tv del lote base fue 2.000000.
- El ratio perturbado fue 1.333333.
- Verifiqué que GAX lanza ValueError.
- Redacté la defensa conceptual individual.

Defensa conceptual

En 100–150 palabras, explique por qué el ratio empírico Ti/Tv en exomas humanos ronda valores entre 2.8–3.2 en lugar del ratio estocástico teórico 0.5, y cómo las modificaciones postraduccionales de histonas (acetilación frente a metilación) coordinan la apertura física de la cromatina.

Fuentes y reproducibilidad

Este anexo se fundamenta en las diapositivas docentes de genética y genómica (20250913_gen (1).pptx), en la especificación del formato VCF (Danecek et al., 2011) y en el código de histonas (Strahl & Allis, 2000), y se valida al 100% mediante `verification/practice_checks.py`.

Transcripción, procesamiento de ARN y expresión génica: Dinámica del transcriptoma, código genético y cuantificación computacional

BC-CH04 – Biología Computacional

Edición 2

Pregunta del tema. Dos células con el mismo genoma pueden ser una neurona y un hepatocito. La diferencia no está en el texto sino en qué partes se leen y cuántas veces. ¿Cómo se mide eso, y por qué el número bruto de lecturas de un experimento no sirve como medida?

Producto final. Un módulo propio que localice marcos abiertos de lectura en los seis marcos, traduzca el más largo y normalice una tabla de expresión, comprobando la propiedad que hace comparables los valores entre muestras.

Mapa del tema

4.1. El transcriptoma como estado funcional dinámico	47
4.2. Mecanismo de la transcripción y complejo de pre-iniciación	47
4.3. Diversidad funcional del ARN: codificantes y no codificantes	49
4.4. Procesamiento co-transcripcional del pre-ARNm eucariota	49
4.5. La traducción: cómo el ribosoma lee el mensaje	50
4.6. El código genético y sesgo de codones (GC3)	51
4.7. Marcos abiertos de lectura (ORFs) en seis fases	51
4.8. Cuantificación transcriptómica: Normalización TPM	52
4.9. Redes de regulación transcripcional y lógica combinatoria	53
4.10. Diseño de módulos transcriptómicos en Python	54
4.11. Peligros computacionales y actividad de depuración	54
4.12. Síntesis operativa y tablas de diagnóstico	55
4.13. Glosario conceptual	55
4.14. Conexiones y mapa del curso	55

Cómo usar este tema

El capítulo anterior describía el genoma, que es el mismo en todas las células de un organismo. Este describe lo que cambia: qué se transcribe, cuánto, y cómo se convierte en proteína. Va en tres tiempos. Primero el mecanismo de la transcripción y el destino del ARN. Después la traducción, que es donde el mensaje se convierte en producto y donde se entiende por qué el marco de lectura importa tanto. Y por último la cuantificación, que es la parte computacional: convertir conteos de lecturas en una medida comparable.

Al terminar sabrá: explicar qué determina que un gen se transcriba y qué se hace con el transcrito antes de que salga del núcleo; seguir el ciclo del ribosoma y decir por qué una inserción de una sola base es más grave que una sustitución; buscar marcos abiertos en las seis lecturas posibles declarando el umbral que usa; y normalizar una tabla de expresión sabiendo qué dos sesgos corrige cada paso.

La ruta **esencial** son la transcripción, la traducción, el código genético, los marcos de lectura y la normalización. La ruta de **estudio** añade la diversidad del ARN no codificante, las redes de regulación y el anexo.

4.1 El transcriptoma como estado funcional dinámico

ESENCIAL. Mientras que el genoma representa el repertorio de información hereditaria, prácticamente idéntica en casi todas las células somáticas de un individuo pluricelular), el **transcriptoma** se define formalmente como el conjunto completo de todas las moléculas de ARN transcritas presentes en una célula, tejido o población celular en un instante espacio-temporal determinado.

A diferencia del genoma estático, el transcriptoma es extraordinariamente dinámico, plástico y sensible al entorno:

1. **Instantánea funcional en tiempo real:** Refleja con fidelidad qué programas genéticos se encuentran encendidos, reprimidos o modulados en respuesta a estímulos hormonales, estrés oxidativo, fármacos o procesos patológicos como la transformación tumoral.
2. **Determinante primario de la identidad celular:** Dos células con genomas idénticos (p. ej., una neurona piramidal de la corteza cerebral y un hepatocito humano) poseen transcriptomas drásticamente divergentes que dictan sus morfologías, proteomas y funciones fisiológicas especializadas.
3. **Complejidad cuantitativa extrema:** Abarca desde transcritos ultra-abundantes presentes en cientos de miles de copias por célula (como los ARN mensajeros de actina o los ARN ribosómicos) hasta especies escasas presentes en menos de una molécula por célula en promedio.

El transcriptoma representa el conjunto completo y dinámico de transcritos de ARN en una célula en un instante específico, capturando la actividad funcional de los genes en tiempo real.

4.2 Mecanismo de la transcripción y complejo de pre-iniciación

La **transcripción** es el proceso enzimático fundamental mediante el cual se sintetiza una cadena de ARN complementaria y antiparalela a partir de una hebra molde de ADN. En eucariotas, la transcripción de los genes que codifican para proteínas corre a cargo de la **ARN Polimerasa II (Pol II)**, un complejo molecular multimérico de 12 subunidades y ≈ 550 kDa.

4.2.1 Ensamblaje del Complejo de Pre-Iniciación (PIC)

La ARN Polimerasa II es incapaz de reconocer promotores por sí misma y requiere el ensamblaje secuencial y coordinado de los **factores de transcripción generales o basales (GTFs)** en el promotor central, formando el **Complejo de Pre-Iniciación (PIC)**:

1. **Reconocimiento del promotor basal por TFIID:** El complejo TFIID, a través de su subunidad **TBP (Proteína de Unión a TATA)**, se une al surco menor de la caja TATA (secuencia consenso 5'-TATAAA-3', situada

$\approx 25\text{--}30$ pb río arriba del TSS). La unión de TBP induce una marcada curvatura de $\approx 80^\circ$ en la doble hélice de ADN, creando una plataforma topológica para el reclutamiento de los factores subsiguientes.

2. **Estabilización por TFIIA y TFIIB:** TFIIA estabiliza el complejo TBP-ADN, mientras que TFIIB interactúa con el ADN río arriba y río abajo, contactando directamente con el sitio catalítico para definir la distancia exacta al sitio de inicio de la transcripción (TSS).
3. **Reclutamiento de la ARN Polimerasa II mediante TFIIF:** TFIIF acompaña y posiciona a la ARN Pol II sobre el promotor, evitando interacciones inespecíficas.
4. **Incorporación de TFIIE y TFIIH (Activación y apertura):** TFIIE recluta a TFIIH. TFIIH es un complejo multimérico con dos actividades bioquímicas determinantes:
 - **Actividad helicasa dependiente de ATP (subunidades XPB/XPD):** Desenrolla localmente la doble hélice de ADN en el TSS, fundiendo $\approx 11\text{--}15$ pb para abrir la **burbuja de transcripción**.
 - **Actividad quinasa (subunidad CDK7):** Fosforila los residuos de Serina 5 en los heptapéptidos repetidos (YSPTSPS) del dominio C-terminal (CTD) de la ARN Pol II, desencadenando la liberación del promotor (*promoter clearance*) y el inicio de la fase de elongación.

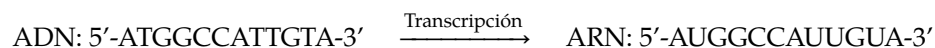
El inicio de la transcripción requiere el reclutamiento de la ARN Polimerasa II y el ensamblaje del Complejo de Pre-Iniciación (PIC) mediante TFIID/TBP, TFIIA, TFIIB, TFIIE y la helicasa TFIIH.



Figura 4.1: Ciclo transcripcional de la ARN Polimerasa II: ensamblaje del PIC, escape del promotor y fosforilación del dominio CTD. Figura original generada por `verification/make_figures.py`.

4.2.2 La reacción de transcripción y sustitución de Uracilo

La ARN polimerasa sintetiza ARN en dirección $5' \rightarrow 3'$ leyendo la hebra molde de ADN en sentido $3' \rightarrow 5'$. A nivel informacional, la secuencia del transcrito de ARN sintetizado es idéntica a la **hebra codificante (no molde)** del ADN, con la única diferencia estricta de que cada nucleótido de Timina (T) es reemplazado por **Uracilo (U)**:



`transcribe` ejecuta la sustitución determinista de ADN a ARN, convirtiendo ATGGCCATTGTA en AUGGCCAUUGUA.

Se reproduce con `verification/chapter_checks.py transcribe ATGGCCATTGTA`.

Validación de alfabeto nucleotídico:

La función de transcripción rechaza caracteres no canónicos: `transcribe` lanza `ValueError` al encontrar caracteres no canónicos de ADN como X y Z en 'ATGGZX'.

Se reproduce con `verification/chapter_checks.py transcribe ATGGZX`.

4.3 Diversidad funcional del ARN: codificantes y no codificantes

El ARN celular exhibe una notable heterogeneidad estructural y catalítica que trasciende el papel pasivo de intermediario mensajero:

1. **ARN Mensajero (ARNm):** Portador de la información codificante organizada en codones trípteros, leída secuencialmente por el ribosoma.
2. **ARN de Transferencia (ARNt):** Adaptador molecular clave con estructura tridimensional en forma de “L” (plegamiento de trébol secundario), que posee un **bucle anticodón** complementario a los codones del ARNm y un **extremo aceptor 3'-CCA** esterificado covalentemente con el aminoácido correspondiente por una aminoacil-ARNt sintetasa específica.
3. **ARN Ribosómico (ARNr):** Componente estructural mayoritario del ribosoma ($\approx 80\%$ del ARN celular total). En particular, el ARNr 28S (eucariotas) o 23S (procariotas) constituye el centro catalítico de la peptidil transferasa, demostrando que el ribosoma es una **ribozima**.
4. **microARNs (miRNAs):** Pequeños ARN de ≈ 22 nucleótidos que se incorporan al complejo silenciador inducido por ARN (**RISC**) y reconocen secuencias complementarias en la región no traducida 3'-UTR de ARNm diana, bloqueando la traducción o induciendo su degradación.
5. **ARNs largos no codificantes (lncRNAs):** Transcritos de longitud > 200 nucleótidos desprovistos de marcos de lectura largos. Actúan como:
 - **Andamios (Scaffolds):** Reúnen físicamente múltiples complejos proteicos (p. ej., el lncRNA *HOTAIR*).
 - **Guías (Guides):** Dirigen enzimas epigenéticas remodeladoras de cromatina (como el complejo PRC2) a loci genómicos específicos (p. ej., el lncRNA *XIST* en la inactivación del cromosoma X).
 - **Señuelos (Decoys):** Secuestran factores de transcripción impidiendo su unión al ADN.
 - **Esponjas de miARNs (Sponges):** Absorben moléculas de miARN previniendo la represión de sus ARNm diana.
6. **ARNs Circulares (circRNAs):** Formados por un empalme reverso covalente (*back-splicing*), extraordinariamente resistentes a la degradación por exonucleasas y abundantes en el tejido nervioso, donde actúan como potentes esponjas de miARN.

La diversidad funcional del ARN abarca el ARNm codificante, el adaptador molecular ARNt (plegamiento en L, aceptor 3' CCA, bucle anticodón), el ARNr catalítico y ARN no codificantes reguladores (miRNAs, lncRNAs, circRNAs).

4.4 Procesamiento co-transcripcional del pre-ARNm eucariota

En eucariotas, el transcrito primario (**pre-ARNm**) es procesado químicamente de forma co-transcripcional mientras la Pol II elonga la cadena:

1. **Caperuza 5' (5' Capping):** Adición temprana de una **7-metilguanosina (m⁷G)** mediante un inusual enlace fosfodiéster invertido 5'-5' trifosfato, acoplado a la metilación del grupo 2'-OH del primer nucleótido. Protege al ARNm frente a la degradación por 5'-exonucleasas y actúa como sitio de anclaje para el complejo de unión a la caperuza (CBC) y los factores de inicio de la traducción (eIF4E).
2. **Poliadenilación 3':** Escisión endonucleolítica del extremo 3' guiada por la señal de poliadenilación canónica 5'-AAUAAA-3' (reconocida por CPSF y CstF), seguida por la síntesis dependiente de ATP de una cola de $\approx 200-250$ residuos de adenina (**cola poli-A**) por la poli(A) polimerasa (PAP). Confiere estabilidad citoplasmática y promueve la exportación nuclear.
3. **Splicing alternativo:** Eliminación precisa de intrones catalizada por el **espliceosoma** (complejo ribonucleoproteico de snRNPs U1, U2, U4, U5, U6). La selección combinatoria de sitios de empalme alternativos permite que un único gen primario produzca múltiples **isoformas proteicas** con funciones biológicas divergentes o antagónicas.

El procesamiento del pre-ARNm eucariota comprende el capping 5' de 7-metilguanosina (enlace trifosfato 5'-5'),

la poliadenilación 3' y el splicing alternativo generador de isoformas proteicas funcionales distintas.

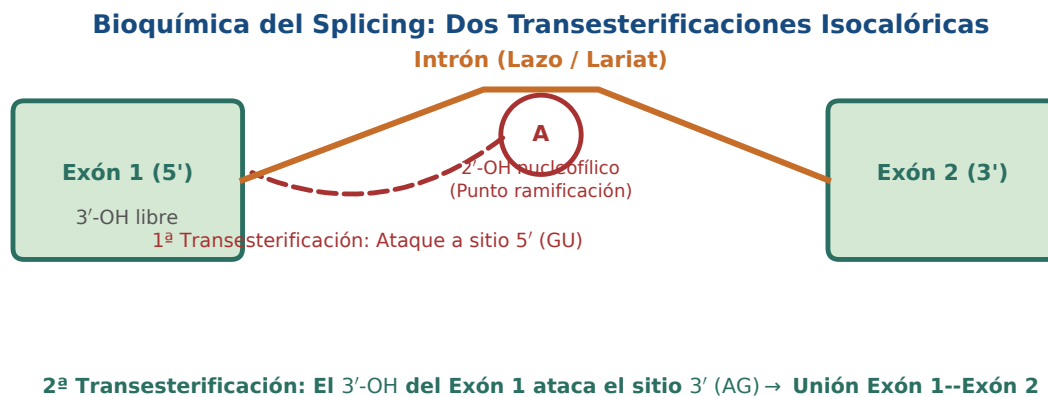


Figura 4.2: Mecanismo bioquímico del splicing nuclear: dos reacciones de transesterificación acopladas a la formación del lazo intrónico. Figura original generada por `verification/make_figures.py`.

4.5 La traducción: cómo el ribosoma lee el mensaje

ESENCIAL Hasta aquí hemos tratado la traducción como una tabla que convierte tripletes en aminoácidos. Esa tabla es el resultado, no el mecanismo. Conviene conocer el mecanismo porque explica dos cosas que ninguna función de traducción muestra: por qué la lectura es estrictamente secuencial y sin marcha atrás, y por qué un desplazamiento de marco arruina todo lo que viene después.

4.5.1 Los tres sitios del ribosoma

El ribosoma tiene tres posiciones por las que pasa cada ARN de transferencia:

- **Sitio A**, de aminoacil: entra el ARN de transferencia cargado cuyo anticodón empareja con el codón que toca leer.
- **Sitio P**, de peptidil: sostiene el ARN de transferencia que lleva la cadena construida hasta ese momento.
- **Sitio E**, de salida: por donde abandona el ribosoma el ARN de transferencia ya descargado.

4.5.2 El ciclo

1. **Iniciación.** La subunidad pequeña localiza el codón de inicio y coloca allí el primer ARN de transferencia, que ocupa directamente el sitio P y no el A, que es la excepción del ciclo. Después se acopla la subunidad grande. En bacterias, la subunidad pequeña reconoce una secuencia corta anterior al inicio; en eucariotas, entra por el extremo del mensajero y recorre la secuencia hasta encontrar el primer codón de inicio en un contexto adecuado.
2. **Elongación.** Llega al sitio A el ARN de transferencia cuyo anticodón empareja con el codón siguiente. El centro peptidil-transferasa, que es ARN ribosómico y no proteína, forma el enlace peptídico. Y el ribosoma avanza exactamente tres nucleótidos, desplazando cada ARN de transferencia una posición: el del sitio A pasa al P, y el del P al E.
3. **Terminación.** Ningún ARN de transferencia reconoce los tres codones de parada. En su lugar entran en el sitio A los **factores de liberación**, proteínas que hidrolizan el enlace con la cadena y la sueltan.

Concepto. Que el ribosoma avance de tres en tres, sin retroceder y sin recalibrarse, es la razón computacional de que el marco de lectura importe tanto. Una inserción de un solo nucleótido no cambia un aminoácido: desplaza el marco y convierte en otra cosa todo lo que queda por leer, hasta el primer codón de parada que aparezca en el

marco nuevo. Por eso las variantes de desplazamiento de marco tienen consecuencias tan desproporcionadas respecto a su tamaño, y por eso el buscador de marcos abiertos que escribiré en el anexo tiene que examinar los seis marcos y no uno.

ESTUDIO Tres consecuencias cuantitativas cierran el cuadro. Un mismo ARN mensajero puede estar siendo leído a la vez por varios ribosomas, formando un **polisoma**, lo que multiplica la producción sin necesidad de más transcripción. La síntesis proteica es uno de los procesos que más energía consume la célula, lo que explica por qué compensa regular en el paso anterior: no transcribir sale más barato que traducir de más. Y la fidelidad importa más de lo que sugiere el simple desperdicio: una proteína mal plegada no solo malgasta recursos, puede agregarse y resultar tóxica, que es el nexo con el capítulo de estructura.

4.6 El código genético y sesgo de codones (GC3)

El **código genético universal** establece el diccionario de correspondencia entre los $4^3 = 64$ tripletes de nucleótidos (**codones**) y los 20 aminoácidos proteinogénicos:

- **Degenerado (o redundante):** Existen 61 codones con sentido (*sense*) para solo 20 aminoácidos. Múltiples codones sinónimos codifican para el mismo aminoácido (p. ej., Leucina y Arginina cuentan con 6 codones cada una).
- **Inambiguo y unívoco:** Cada codón individual codifica exclusivamente para un único aminoácido.
- **No solapante y sin comas:** Se lee de tres en tres nucleótidos de manera continua y lineal sin puntuación intercalada.
- **Codones de parada (Stop):** Tres codones específicos (UAA, UAG, UGA) señalan la terminación de la síntesis proteica al unirse a factores de liberación proteicos (RFs).

El código genético es degenerado (61 codones con sentido y 3 codones stop: UAA, UAG, UGA), inambiguo, no solapante, continuo y casi universal.

4.6.1 Sesgo en la tercera posición del codón (Contenido GC3)

Debido al fenómeno de **bamboleo** (*wobble hypothesis*) formulado por Francis Crick en 1966, la tercera posición del codón es la más tolerante a sustituciones nucleotídicas sin alterar el aminoácido codificado. El **contenido GC3** mide la proporción de Guaninas y Citosinas exclusivamente en la tercera posición de los codones:

Traza del péptido modelo ATGGCCATTGTA:

Dividiendo la secuencia $S = \text{"ATGGCCATTGTA"}$ en 4 codones:

1. Codón 1: ATG $\Rightarrow$ Tercera base: **G** (GC)
2. Codón 2: GCC $\Rightarrow$ Tercera base: **C** (GC)
3. Codón 3: ATT $\Rightarrow$ Tercera base: **T** (AT)
4. Codón 4: GTA $\Rightarrow$ Tercera base: **A** (AT)

Existen 2 bases G/C sobre 4 terceras posiciones:

$$\text{GC3\%} = \frac{2}{4} \times 100 = 50.00\%$$

`gc3_content` evalúa el contenido GC en la tercera posición de codón de ATGGCCATTGTA exactamente en 50.00 por ciento.

Se reproduce con `verification/chapter_checks.py gc3 ATGGCCATTGTA`.

4.7 Marcos abiertos de lectura (ORFs) en seis fases

Dado que el ADN genómico es bicatenario y la lectura ribosómica procede en tripletes, cualquier secuencia de ADN bicatenario contiene **seis posibles marcos de lectura** (*reading frames*):

- Hebra directa (5' → 3'): Marco +1 (desfase 0), Marco +2 (desfase 1), Marco +3 (desfase 2).
- Hebra reversa complementaria (5' → 3'): Marco -1, Marco -2, Marco -3.

El ADN genómico bicatenario codifica seis posibles marcos de lectura (+1, +2, +3 directos y -1, -2, -3 reverso complementarios) que determinan el producto de traducción proteica.

Traducción de ARN en marco 0:

Para la secuencia AUGGCAUUGUAUAA, de quince nucleótidos: AUG (M) → GCC (A) → AUU (I) → GUA (V) → UAA (*). El péptido generado es "MAIV" finalizando en el codón de parada.

`translate_rna` traduce la secuencia modelo de ARN AUGGCAUUGUAUAA en el marco 0 rindiendo el péptido MAIV y terminando en el codón de parada UAA.

Se reproduce con `verification/chapter_checks.py translate AUGGCAUUGUAUAA --frame 0`.

Búsqueda de ORFs canónicos en los 6 marcos:

Un **Marco Abierto de Lectura (ORF)** canónico es un segmento delimitado por un codón de inicio ATG y un codón de parada en el mismo marco de lectura. Al analizar "ATGGCCATTGTAATGTAA" en los seis marcos: con un umbral de longitud mínima de treinta nucleótidos se detecta un único marco abierto canónico, en el marco +1, que codifica **MAIVM** (ATG GCC ATT GTA ATG TAA).

Atención. Ese umbral no es un detalle de implementación: cambia el recuento. Sin él aparecería también el ATG de la posición 12, seguido de inmediato por el codón de parada, que es un marco formalmente válido de un solo residuo. Todo buscador de marcos abiertos aplica un umbral, ninguno lo lleva escrito en la salida, y comparar dos recuentos obtenidos con umbrales distintos es comparar cosas distintas. Declárelo siempre, junto al resultado.

La búsqueda en los seis marcos sobre ATGGCCATTGTAATGTAA, con umbral de treinta nucleótidos, identifica un único marco abierto canónico, en el marco +1, que codifica el péptido MAIVM.

Se reproduce con `verification/chapter_checks.py orfs ATGGCCATTGTAATGTAA`.

4.8 Cuantificación transcriptómica: Normalización TPM

En los experimentos de secuenciación de ARN masiva (**RNA-seq**), el número bruto de lecturas que alinean contra un gen i (*raw read counts*) no es una medida directa de su abundancia molecular real, puesto que está sesgado por:

1. **La longitud del gen:** Genes más largos generan más fragmentos y reciben más lecturas a igual nivel de expresión molar.
2. **La profundidad de secuenciación de la librería:** Muestras secuenciadas con mayor profundidad generan sistemáticamente más conteos brutos.

4.8.1 Formulación matemática de Transcripts Per Million (TPM)

La métrica **TPM (Transcripts Per Million)** resuelve este problema normalizando primero por la longitud del gen (obteniendo las lecturas por kilobase, **RPK**) y después normalizando la suma de RPKs por un factor de escala global por millón:

$$\begin{aligned} \text{RPK}_i &= \frac{C_i}{L_i/10^3} \\ \text{Scaling Factor} &= \frac{\sum_{j=1}^G \text{RPK}_j}{10^6} \\ \text{TPM}_i &= \frac{\text{RPK}_i}{\text{Scaling Factor}} \end{aligned}$$

A diferencia de métricas obsoletas como RPKM o FPKM (cuya suma de valores varía arbitrariamente entre muestras), **la suma de los valores de TPM en cualquier muestra es siempre exactamente idéntica e igual a**

10^6 , haciendo que los valores de TPM representen fracciones molares estrictamente comparables entre distintas condiciones experimentales y laboratorios.

Transcripts Per Million (TPM) normaliza la expresión de RNA-seq dividiendo primero por la longitud del gen (RPK) y por la profundidad de librería segundo, garantizando comparabilidad entre muestras.

Traza manual del lote modelo de expresión:

Dados tres genes con conteos brutos $C = [100, 200, 50]$ y longitudes en pares de bases $L = [1000, 2000, 500]$ ([1.0, 2.0, 0.5] kb):

1. $RPK_1 = \frac{100}{1.0} = 100.0$; $RPK_2 = \frac{200}{2.0} = 100.0$; $RPK_3 = \frac{50}{0.5} = 100.0$.
2. $\sum RPK = 100 + 100 + 100 = 300.0$.
3. Factor de escala = $\frac{300.0}{10^6} = 0.0003$.
4. $TPM_1 = \frac{100.0}{0.0003} = 333333.333333$; $TPM_2 = 333333.333333$; $TPM_3 = 333333.333333$.

Para los conteos [100, 200, 50] con longitudes [1000, 2000, 500], TPM normaliza a exactamente 333333.333333 para cada uno de los tres genes.

Se reproduce con `verification/chapter_checks.py tpm --counts 100 200 50 --lengths 1000 2000 500`.

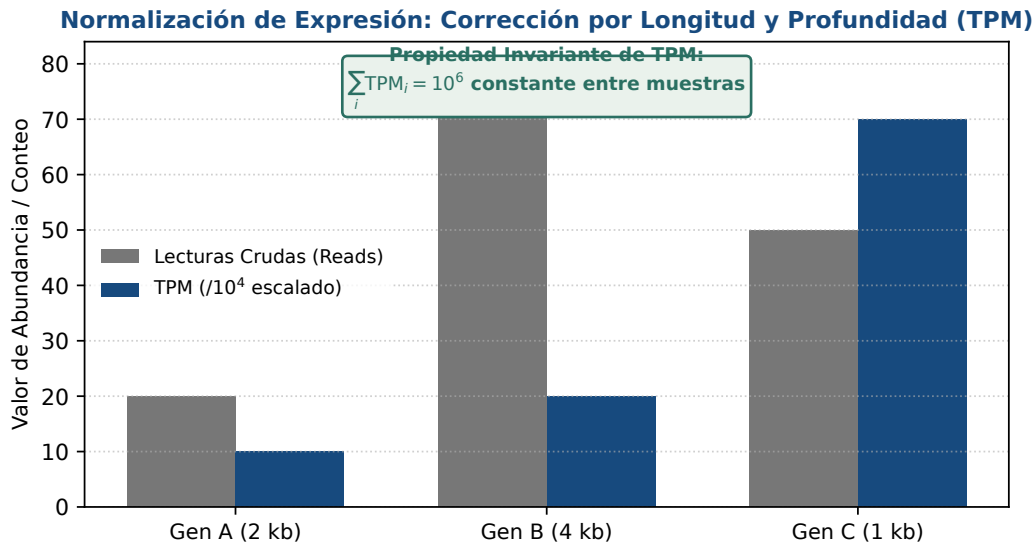


Figura 4.3: Normalización cuantitativa de RNA-seq: Transcripts Per Million (TPM) preserva la propiedad invariante de suma a 10^6 . Figura original generada por `verification/make_figures.py`.

4.9 Redes de regulación transcripcional y lógica combinatoria

La expresión génica eucariota no se rige por interruptores binarios simples, sino por **redes de regulación transcripcional** (TRNs) que integran múltiples señales mediante **lógica combinatoria**:

- **Control a dos niveles:** Requiere un estado de cromatina físicamente accesible (eucromatina descondensada) y la unión concertada de la combinación precisa de factores de transcripción activadores en ausencia de represores dominantes.
- **Motivos de red recurrentes:** Circuitos modulares como los bucles de retroalimentación autorreguladora (*autoregulatory loops*) y los bucles de avance coordinado (*feed-forward loops*, FFLs coherentes e incoherentes) que filtran el ruido biológico transitorio y generan respuestas temporales de tipo pulso o memoria epigenética celular estable.

Las redes de regulación transcripcional integran la lógica combinatoria de factores de transcripción activadores, coactivadores y represores específicos de secuencia.

4.10 Diseño de módulos transcriptómicos en Python

A continuación se detalla la arquitectura modular de procesamiento transcriptómico:

```
from typing import Dict, List, Tuple

RNA_CODON_TABLE = {
    'AUG': 'M', 'GCC': 'A', 'AUU': 'I', 'GUA': 'V',
    'UAA': '*', 'UAG': '*', 'UGA': '*'
}

DNA_TRANS = str.maketrans("ACGT", "UGCA")

def transcribe_dna_to_rna(dna_seq: str) -> str:
    """Realiza la sustitucion de T por U sobre la hebra codificante."""
    clean = dna_seq.strip().upper()
    invalid = set(clean) - set("ACGT")
    if invalid:
        raise ValueError(f"Caracteres invalidos detectados: {sorted(invalid)}")
    return clean.replace('T', 'U')

def calculate_gc3(dna_seq: str) -> float:
    """Calcula el porcentaje de GC en la tercera posicion de codon."""
    clean = dna_seq.strip().upper()
    n = len(clean)
    codons = [clean[i:i+3] for i in range(0, n - n % 3, 3)]
    if not codons:
        return 0.0
    gc3_count = sum(1 for c in codons if c[2] in "GC")
    return round((gc3_count / len(codons)) * 100.0, 2)

def calculate_tpm(counts: List[float], lengths: List[float]) -> List[float]:
    """Normaliza conteos de RNA-seq a Transcripts Per Million (TPM)."""
    if len(counts) != len(lengths) or not counts:
        raise ValueError("Listas de conteos y longitudes deben coincidir y ser no vacias")
    rpk = [c / (1 / 1000.0) for c, l in zip(counts, lengths)]
    sum_rpk = sum(rpk)
    if sum_rpk == 0:
        return [0.0] * len(counts)
    return [round((r / sum_rpk) * 1e6, 6) for r in rpk]
```

4.11 Peligros computacionales y actividad de depuración

Omitir los marcos de lectura de la hebra reversa y comparar conteos brutos sin normalizar o RPKM inconsistente entre librerías representan errores computacionales mayores.

4.11.1 Tres errores críticos en análisis transcriptómico

Localice y corrija los tres errores en la siguiente función:

```
def analyze_expression_data(raw_counts, gene_lengths):
    # Error 1: Intenta comparar muestras dividiendo solo por el total de lecturas
    # sin corregir por la longitud de los transcritos (distorsion de genes largos)

    # Error 2: Asume que RPKM es aditivo y comparable entre librerias distintas

    # Error 3: Al traducir marcos de lectura en ADN genómico, solo explora
    # los marcos +1, +2 y +3, olvidando los marcos de la hebra antisentido (-1, -2, -3)
    pass
```

Diagnóstico de causa raíz (Protocolo 5 Whys):

1. **Fallo 1 (Sesgo de longitud):** Los conteos brutos no reflejan abundancia molar; un gen de 10 kb produce 10 veces más lecturas que un gen de 1 kb a igual concentración molar.

2. **Fallo 2 (Uso indebido de RPKM/FPKM entre muestras):** Dado que la suma de RPKM varía entre muestras, no es una proporción molar estricta; debe utilizarse TPM.
3. **Fallo 3 (Omisión de la hebra reversa):** En genomas bicatenarios, los genes pueden estar codificados en cualquiera de las dos hebras; omitir la hebra reversa pierde el 50% de los genes anotados.

4.12 Síntesis operativa y tablas de diagnóstico

4.12.1 Tabla de diagnóstico de problemas en pipelines de RNA-seq

Problema detectado	Causa probable	Comprobación y solución
Suma de expresión no constante entre muestras	Empleo de RPKM o FPKM en lugar de TPM	Migrar a cuantificación TPM ($\sum \text{TPM} = 10^6$)
Pérdida del 50% de ORFs predichos	Se omitieron los marcos de la hebra complementaria	Explorar siempre los 6 marcos (+1,+2,+3, -1,-2,-3)
Falso codón de parada prematuro	Confundir ADN codificante con ARN transcrito	Convertir Timinas a Uracilos antes de consultar la tabla
Falso sesgo en genes largos	Comparación de conteos brutos no normalizados	Normalizar primero por longitud génica (RPK)

4.12.2 Tabla de síntesis: Tipos de ARN y funciones

Tipo de ARN	Longitud típica	Función molecular principal	Localización celular
ARNm	0.5–10 kb	Molde codificante para síntesis proteica	Núcleo y Citoplasma
ARNt	70–90 nt	Adaptador anticodón-aminoácido	Citoplasma y Mitocondria
ARNr	0.1–5 kb	Centro catalítico peptidil transferasa	Ribosomas / Nucleolo
miRNA	≈ 22 nt	Silenciamiento postranscripcional	Citoplasma (RISC)
lncRNA	> 200 nt	Andamio, guía epigenética y señuelo	Núcleo y Citoplasma

4.13 Glosario conceptual

- **Transcriptoma:** Conjunto dinámico de transcritos en un instante.
- **PIC:** Complejo de Pre-Iniciación de la ARN Polimerasa II.
- **TBP:** Subunidad de TFIID que dobla la caja TATA.
- **TFIIH:** Factor basal con actividad helicasa y quinasas CDK7.
- **Capping 5':** Enlace 5'-5' trifosfato de m⁷G.
- **Splicing:** Escisión precisa de intrones por el spliceosoma.
- **Bamboleo:** Apareamiento no canónico en 3^a posición (Crick).
- **GC3:** Porcentaje de G/C en la tercera base de los codones.
- **ORF:** Marco abierto entre un codón de inicio y parada.
- **TPM:** Transcripts Per Million (suma invariante 10⁶).

4.14 Conexiones y mapa del curso

Este capítulo formaliza la expresión génica y la traducción, preparando el terreno para el alineamiento de secuencias proteicas y el modelado estructural en BC-CH05.

4.15 Fuentes y reproducibilidad

Este capítulo se apoya en la presentación docente de introducción de la asignatura, escrita por Álvaro Serrano Navarro. El descifrado del código genético sigue a Nirenberg y Matthaei [53]; la hipótesis del bamboleo que explica su degeneración, a Crick [16]; la métrica de normalización y su propiedad de suma constante, a Wagner y colaboradores [73]; y la transcripción, su maquinaria y el procesamiento del transcrito, a Alberts y colaboradores [3].

Todos los cálculos se reproducen con `verification/chapter_checks.py` tal como están impresos, con sus argumentos.

Anexo práctico · Flujo transcripcional, traducción en 6 marcos y cuantificación TPM

Pregunta, producto y separación de materiales

¿Puede escribir un módulo que localice marcos abiertos de lectura en los seis marcos con un umbral declarado, los traduzca, y normalice una tabla de expresión de forma que los valores sean comparables entre muestras? Este laboratorio responde que sí, y lo comprueba con una propiedad que usted mismo puede verificar sin mirar ningún resultado esperado.

El producto individual es `mi_modulo_expresion.py`, con tres funciones: buscar marcos abiertos, traducir y normalizar. La normalización tiene un oráculo interno perfecto: si los valores no suman exactamente un millón, hay un error, y usted lo detecta solo.

El anexo es autónomo: solo necesita la biblioteca estándar. El generador deja el oráculo del capítulo y un comprobador que prueba su módulo con secuencias y tablas que el enunciado no muestra.

Mapa de ruta y productos entregables

Bloque de trabajo	Producto entregable
1 · Entorno y diagnóstico	Comprobación del intérprete Python 3.9
2 · Cálculo ancla (Transcripción, Traducción, TPM)	Flujo de ADN a proteína y matriz TPM base
3 · Perturbación de matriz TPM	Evaluación con conteos heterogéneos
4 · Búsqueda de ORFs en 6 marcos	Identificación de marco +1 y hebra reversa
5 · Guarda de dominio	Captura de <code>ValueError</code> ante ADN ATGGZX
6 · Preguntas de control transcriptómico	Preguntas sobre PIC, ARNt y normalización
7 · Verificación y empaquetado	Comprobación final y empaquetado

Este anexo produce evidencia comprobable y verificable en cada bloque de trabajo sin paquetes externos.

1 · Entorno y diagnóstico

Verifique que su entorno dispone de Python 3.9 o superior:

```
python3 --version
./practice_assets/create_workspace.sh BC04_entrega
cd BC04_entrega
```

2 · Escribir el módulo

Tarea. Implemente las tres funciones de `mi_modulo_expresion.py`: buscar marcos abiertos en los seis marcos con un umbral de longitud mínima, traducir con la tabla estándar, y normalizar una tabla de conteos por longitud y por profundidad. El contrato está en la documentación de la plantilla.

Los tres tienen su trampa. En la búsqueda, los seis marcos: tres en la secuencia y tres en su complemento reverso, porque un gen puede estar en cualquiera de las dos hebras. En la traducción, el codón de parada, que se traduce y no se descarta. Y en la normalización, el orden de las dos correcciones: primero se divide cada conteo por la longitud del gen, y solo después se reparte sobre el total. Hacerlo al revés da otra cosa que también suma un millón y no significa lo mismo.

Compruebe. Después de normalizar, sume sus valores. Deben dar exactamente un millón, y esa es la propiedad que hace comparables dos muestras: cada valor es «de cada millón de transcritos, cuántos son de este gen». Si la suma no da un millón, no siga: el error está en el segundo paso, y ninguna comprobación posterior lo va a detectar por usted.

Compruebe su avance cuantas veces quiera:

```
bash herramienta/check_modulo.sh
```

3 • Cálculo ancla: flujo génico, GC3 y normalización

Dada la secuencia de ADN modelo $S = \text{'ATGGCCATTGTAATGTAA'}$ ($N = 18$ pb):

1. Transcriba a ARN: AUGGCCAUUGUAAUGUAA.
2. Traduzca en marco 0 obteniendo el péptido MAIVM*.
3. Evalúe el contenido GC en tercera posición de codones: $\text{GC3\%} = 50.00\%$.
4. Calcule los valores TPM para 3 genes con conteos [100, 200, 50] y longitudes [1000, 2000, 500] pb: $\text{TPM} = [333333.333333, 333333.333333, 333333.333333]$.

```
python3 herramienta/chapter_checks.py gc3 ATGGCCATTGTA > resultados/ancla.txt
python3 herramienta/chapter_checks.py tpm --counts 100 200 50 --lengths 1000 2000 500 >> resultados/ancla.txt
cat resultados/ancla.txt
```

La secuencia de ADN ancla produce ARN AUGGCCAUUGUAAUGUAA, péptido MAIVM*, GC3% de 50.00% y un vector TPM base con 333333.333333 por gen.

Se reproduce con `verification/chapter_checks.py tpm --counts 100 200 50 --lengths 1000 2000 500`.

4 • Perturbación de la tabla de expresión

Modifique los conteos a [200, 200, 50] manteniendo las longitudes [1000, 2000, 500]:

```
python3 herramienta/chapter_checks.py tpm --counts 400 200 50 --lengths 1000 2000 500 > resultados/perturbado.txt
cat resultados/perturbado.txt
```

Al cuadruplicar el conteo del primer gen y dejar los otros dos igual, sus valores normalizados pasan a ser 666666,67, 166666,67 y 166666,67, y siguen sumando un millón.

Se reproduce con `verification/chapter_checks.py tpm --counts 400 200 50 --lengths 1000 2000 500`.

5 • Guarda de dominio y control de excepciones

Compruebe que el evaluador rechaza secuencias de ADN contaminadas con caracteres no canónicos:

```
python3 herramienta/chapter_checks.py transcribe ATGGZX > resultados/guarda.txt
cat resultados/guarda.txt
```

La guarda de dominio rechaza la secuencia no válida ATGGZX lanzando un `ValueError` que identifica los caracteres inválidos 'X' y 'Z'.

Se reproduce con `verification/chapter_checks.py transcribe ATGGZX`.

6 • Empaquetar y verificar

Escriba antes `notas/defensa.md`. Después:

```
cd ..
tar -czf BC04_entrega.tar.gz BC04_entrega
sha256sum BC04_entrega.tar.gz
./practice_assets/check_entrega.sh BC04_entrega BC04_entrega.tar.gz
```

El verificador prueba su módulo con secuencias y tablas que no ha visto, comprueba que la tabla de respuestas tiene al menos cuatro filas con justificación y una sobre el umbral, que `resultados/` recoge alguna normalización, y que existe la defensa. Rechaza el espacio recién generado. No puntúa.

Rúbrica de calificación ponderada

La evaluación sobre 10 puntos se pondera según:

- **4.0 puntos (40%):** Ejecución correcta y reproducible de los scripts de comprobación (`chapter_checks.py` y `practice_checks.py`).
- **2.0 puntos (20%):** Cálculo manual y exacto de la normalización TPM base (333333.333333) y traducción del gen modelo (Bloque 2).
- **2.0 puntos (20%):** Evaluación de perturbación de TPM y control de excepciones en la guarda de dominio (Bloques 3 y 4).
- **2.0 puntos (20%):** Defensa conceptual escrita (100–150 palabras) sobre las ventajas matemáticas de TPM frente a RPKM y el papel de las aminoacil-ARNt sintetasas como verdaderos traductores moleculares.

Lista de autocorrección

- mi buscador de marcos examina las dos hebras, no solo una;
- declaré el umbral de longitud mínima que usé, y sé cómo cambia el recuento;
- mi traducción incluye el codón de parada en vez de descartarlo;
- normalicé primero por longitud y después por profundidad, en ese orden;
- mis valores normalizados suman exactamente un millón, en los tres casos;
- la guarda rechaza los caracteres que no son ADN y lo dice;
- guardé en `resultados/` la salida de cada ejecución que cito;
- el verificador termina en `ENTREGA_OK`.

Defensa conceptual

En `notas/defensa.md`, entre 100 y 150 palabras, explique por qué la métrica que suma un millón permite comparar dos muestras independientes y una que no lo hace, no. Use un ejemplo numérico de su propia entrega: dos muestras con distinta profundidad de secuenciación y el mismo gen.

Fuentes y reproducibilidad

Este anexo no usa datos externos ni paquetes de terceros. La métrica de normalización y su propiedad de suma constante siguen a Wagner y colaboradores [73], y la degeneración del código genético, a Crick [16].

**Secuencia, estructura y función de proteínas: Plegamiento
termodinámico, geometría tridimensional, formato PDB y
superposición estructural**

Edición 2

Pregunta del tema. Una proteína es una cadena de letras y también un objeto en el espacio, con coordenadas. ¿Qué se puede medir sobre esas coordenadas que diga algo biológico, y por qué el repertorio de formas que adopta la vida es tan corto?

Producto final. Un módulo propio de geometría estructural que lea coordenadas, calcule distancias, centroides y mapas de contacto, y compare dos modelos, aplicado a reconocer un plegamiento.

Mapa del tema

5.1. El paradigma Secuencia a Estructura a Función: marco y límites	59
5.2. Representación cristalográfica: El formato Protein Data Bank (PDB)	61
5.3. Geometría molecular 3D: Distancias, centroides y contactos	62
5.4. Mapas de contacto y matrices de distancia	63
5.5. Comparación estructural: RMSD y el algoritmo de Kabsch	63
5.6. Jerarquía de dominios y clasificaciones CATH/SCOP	64
5.7. Diseño computacional en Python: Módulo de geometría PDB	65
5.8. Peligros computacionales y actividad de depuración	66
5.9. Síntesis operativa y tablas de diagnóstico	67
5.10. Glosario conceptual	67
5.11. Conexiones y cierre de la Parte I	68
5.12. Fuentes y reproducibilidad	68

Cómo usar este tema

Este capítulo convierte la estructura en números. Empieza por el paradigma que la sostiene, que la secuencia determina la estructura y la estructura la función, y por sus límites, que son tan importantes como el paradigma. Sigue por la geometría: qué se calcula sobre un conjunto de coordenadas y qué significa cada medida. Y termina en la clasificación de plegamientos, que es donde la geometría se convierte en biología comparada.

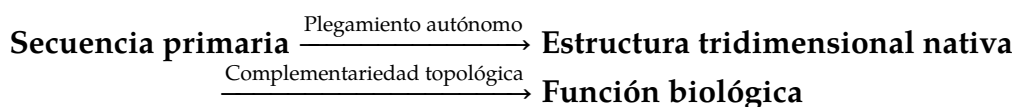
Al terminar sabrá: leer coordenadas de un fichero de estructura; calcular distancias, centroides y mapas de contacto, y decir qué información conserva cada representación; comparar dos modelos con una medida de desviación y saber qué exige esa medida antes de aplicarse; y reconocer cinco plegamientos frecuentes y explicar por qué el repertorio de plegamientos es corto.

La ruta **esencial** son el paradigma con sus límites, la geometría de descriptores y los plegamientos. La ruta de **estudio** añade los mapas de contacto en detalle, la comparación de modelos y el anexo.

5.1 El paradigma Secuencia → Estructura → Función: marco y límites

ESENCIAL. A lo largo de los cuatro primeros capítulos de esta obra hemos explorado la biología molecular como un sistema informacional digital: desde la codificación nucleotídica primaria y el modelo computacional de objetos en Python, hasta la regulación epigenética del genoma y la cuantificación cuantitativa del transcriptoma. Sin embargo, en el mundo físico y biofísico celular, la información digital de una secuencia no ejecuta trabajo mecánico, transporte activo ni catálisis química directamente como un texto plano; lo hace plegándose autónomamente en una intrincada **arquitectura tridimensional dinámica**.

El pilar conceptual que articula la biología estructural y la bioinformática molecular contemporánea es el **paradigma clásico Secuencia → Estructura → Función**:



1. **Determinismo informacional:** La secuencia lineal primaria de aminoácidos contiene la totalidad de la información fisicoquímica necesaria para adoptar una conformación nativa tridimensional única y estable.
2. **Restricción físico-química y topológica:** La estructura 3D resultante genera cavidades hidrofóbicas, superficies polares, bolsas de unión a cofactores y centros activos catalíticos que ejecutan la función biológica con altísima especificidad.

No obstante, en la bioinformática moderna este paradigma debe entenderse como un **marco interpretativo biológico probabilístico y flexible**, condicionado por la divergencia evolutiva, la convergencia estructural y la plasticidad conformacional:

- **Divergencia de secuencia con conservación de forma:** Dos proteínas homólogas pueden compartir identidades de secuencia inferiores al 20% (en la denominada *twilight zone* o zona de penumbra) y, sin embargo, adoptar plegamientos tridimensionales prácticamente idénticos, demostrando que la estructura secundaria y terciaria está más conservada en la evolución que la secuencia primaria.
- **Convergencia funcional:** Proteínas con orígenes filogenéticos completamente independientes pueden converger hacia la misma tríada catalítica o geometría de sitio activo (como la convergencia entre la quimotripsina de vertebrados y la subtilisina bacteriana).

El paradigma Secuencia → Estructura → Función sirve como marco interpretativo biológico más que como regla determinista rígida, delimitado por divergencia, convergencia y plasticidad conformacional.

5.1.1 La hipótesis termodinámica de Anfinsen y el embudo de plegamiento

En 1961, Christian B. Anfinsen (Premio Nobel de Química en 1972) demostró experimentalmente que la estructura nativa tridimensional de una proteína globular es el estado termodinámicamente más estable del sistema.

El experimento clásico con la Ribonucleasa A:

Anfinsen trató la ribonucleasa A bovina (una enzima de 124 aminoácidos estabilizada por 4 puentes disulfuro covalentes Cys-S-S-Cys) con una disolución concentrada de **Urea 8 M** (agente caotrópico que rompe las interacciones no covalentes) y **β -mercaptoetanol** (agente reductor que escinde los enlaces disulfuro). La enzima perdió completamente sus estructuras secundaria y terciaria, desnaturalizándose en un ovillo estadístico desordenado sin actividad catalítica detectable.

Posteriormente, al retirar lentamente la urea y el β -mercaptoetanol mediante diálisis en presencia de oxígeno a pH fisiológico, la cadena polipeptídica se replegó espontáneamente y restableció con una fidelidad del **100%** los 4 puentes disulfuro nativos correctos (de entre las 105 combinaciones matemáticas posibles de apareamiento de 8 cisteínas), recuperando la totalidad de su actividad enzimática sin requerir aporte de energía externa ni chaperonas moleculares.

Implicación biofísica:

La conformación nativa funcional reside en el **mínimo global de energía libre de Gibbs** ($\Delta G_{\text{pleg}} < 0$) bajo condiciones fisiológicas. El proceso de plegamiento no es una búsqueda conformacional aleatoria (lo que llevaría tiempos astronómicos según la *paradoja de Levinthal*), sino un descenso termodinámico canalizado a través de un **embudo de energía** (*folding funnel*), donde la pérdida de entropía conformacional de la cadena es compensada por el incremento favorable de entropía del solvente (efecto hidrofóbico) y la formación progresiva de contactos terciarios estabilizantes.

La hipótesis termodinámica de Anfinsen establece que la conformación 3D funcional nativa de una proteína reside en el mínimo global de energía libre de Gibbs, demostrado mediante el replegamiento de la ribonucleasa A.

5.1.2 Plasticidad conformacional y quiebras del paradigma estático

La biología estructural del siglo XXI ha revelado importantes clases de proteínas que desafían la visión clásica de una única estructura rígida predeterminada:

1. **Proteínas intrínsecamente desordenadas (IDPs / IDRs):** Segmentos polipeptídicos o proteínas completas que carecen de una estructura terciaria 3D fija en disolución acuosa libre. Están enriquecidas en aminoácidos hidrofílicos y cargados (Glu, Lys, Arg, Pro, Ser) y desprovistas de núcleos hidrofóbicos voluminosos. Desempeñan funciones centrales de señalización celular y regulación génica (p. ej., los dominios de transactivación de p53), adoptando conformaciones estructuradas únicamente tras unirse a sus dianas



Figura 5.1: Embudo de plegamiento de Anfinsen (*folding funnel*): descenso termodinámico canalizado hacia el mínimo global nativo de energía libre. Figura original generada por `verification/make_figures.py`.

moleculares (*coupled folding and binding*).

2. **Metamorfismo conformacional:** Proteínas como la quimiocina Linfotactina (XCL1) que alternan reversiblemente en equilibrio termodinámico entre dos conformaciones 3D nativas completamente distintas con funciones biológicas separadas (un monómero soluble con lámina β frente a un dímero que interacciona con glicosaminoglicanos de membrana).
3. **Priones y propagación conformacional:** Proteínas que pueden plegarse en conformaciones alternativas ricas en láminas β insolubles y autorreplicantes ($\text{PrP}^{\text{C}} \rightarrow \text{PrP}^{\text{Sc}}$). La forma patológica actúa como plantilla catalítica que induce el cambio conformacional en moléculas nativas sanas, causando agregados amiloides y neurodegeneración espongiforme transmisible.

Las proteínas intrínsecamente desordenadas (IDPs/IDRs), el metamorfismo conformacional y la propagación de priones ilustran desviaciones clave del determinismo rígido secuencia-estructura.

5.2 Representación cristalográfica: El formato Protein Data Bank (PDB)

Para almacenar y procesar computacionalmente las coordenadas tridimensionales de biomoléculas obtenidas por cristalografía de rayos X, crio-microscopía electrónica o RMN, el estándar histórico consolidado desde 1971 es el formato PDB (Protein Data Bank).

5.2.1 Especificación de campos posicionales fijos (80 columnas)

Un fichero PDB es un archivo de texto plano donde cada átomo individual se describe en una línea de exactamente 80 caracteres de ancho fijo, utilizando columnas predeterminadas:

- **Columnas 1–6 (Record):** Tipo de registro (ATOM para átomos estándar de aminoácidos/nucleótidos, HETATM para heteroátomos como ligandos, iones o moléculas de agua).
- **Columnas 7–11 (Serial):** Número correlativo del átomo en la estructura (1, 2, 3, ...).
- **Columnas 13–16 (Name):** Nombre del átomo según la nomenclatura IUPAC (p. ej., N, CA, C, O, CB).
- **Columnas 18–20 (ResName):** Nombre del residuo en código de tres letras (p. ej., MET, ALA, VAL).
- **Columna 22 (ChainID):** Identificador alfanumérico de la cadena polipeptídica (p. ej., A, B).
- **Columnas 23–26 (ResSeq):** Número secuencial del residuo en la cadena (1, 2, 3, ...).
- **Columnas 31–38, 39–46, 47–54 (X, Y, Z):** Coordenadas cartesianas ortogonales en Angstroms (Å), en formato decimal con tres cifras decimales (F8.3).

- **Columnas 55–60 (Occupancy):** Fracción de ocupación del átomo en la red cristalina (0.00 a 1.00).
- **Columnas 61–66 (TempFactor):** Factor de temperatura o factor B de Debye-Waller en $\text{\AA}^2$, que cuantifica la movilidad térmica del átomo.

El formato Protein Data Bank (PDB) utiliza registros de ancho fijo estricto para líneas ATOM y HETATM especificando serie atómica, residuo, cadena, coordenadas 3D cartesianas, ocupación y factor B.

Traza de parsing de la línea ATOM modelo:

Dada la línea PDB estándar:

```
ATOM 1 CA MET A 1 0.000 0.000 0.000 1.00 15.00 C
```

El parser extrae unívocamente: número de serie 1, nombre de átomo "CA", residuo "MET", cadena "A", posición 1, coordenadas cartesianas (0.000, 0.000, 0.000), ocupación 1.00 y factor B 15.00.

parse_pdb_atom_line extrae la serie 1, átomo CA, residuo MET, cadena A y coordenadas (0.000, 0.000, 0.000) a partir de la línea ATOM modelo.

Se reproduce con verification/chapter_checks.py parse_atom "ATOM 1 CA ALA A 1 10.000 20.000 30.000 1.00 15.50 C".

Control defensivo de registros truncados:

parse_pdb_atom_line lanza ValueError al procesar líneas PDB truncadas o malformadas como 'ATOM 1 CA'.

Se reproduce con verification/chapter_checks.py parse_atom "ATOM 1 CA".

5.3 Geometría molecular 3D: Distancias, centroides y contactos

A partir de las coordenadas cartesianas de los átomos de carbono alfa ($C\alpha$), se derivan descriptores geométricos fundamentales que capturan la topología y el empaquetamiento proteico:

5.3.1 Distancia euclídea interatómica

La distancia euclídea tridimensional $d(P, Q)$ entre dos átomos situados en $P = (x_1, y_1, z_1)$ y $Q = (x_2, y_2, z_2)$ se calcula como:

$$d(P, Q) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2}$$

La distancia euclídea interatómica calcula la raíz cuadrada de la suma de diferencias de coordenadas al cuadrado en el espacio cartesiano 3D.

Traza manual para los puntos (0,0,0) y (3,4,0):

$$d = \sqrt{(3-0)^2 + (4-0)^2 + (0-0)^2} = \sqrt{9+16+0} = \sqrt{25} = 5.000000 \text{ \AA}$$

Para los puntos (0,0,0) y (3,4,0), la distancia euclídea evalúa exactamente a 5.000000 Angstroms.

Se reproduce con verification/chapter_checks.py distance 0 0 0 3 4 0.

5.3.2 Centroide geométrico y radio de giro

El **centroide** $\vec{C} = (C_x, C_y, C_z)$ de un conjunto de N átomos es la media aritmética de sus vectores de posición cartesianos:

$$\vec{C} = \frac{1}{N} \sum_{i=1}^N \vec{r}_i = \left(\frac{1}{N} \sum x_i, \frac{1}{N} \sum y_i, \frac{1}{N} \sum z_i \right)$$

Traza manual para tres carbonos alfa modelos:

Dados los átomos en $\vec{r}_1 = (0,0,0)$, $\vec{r}_2 = (3,4,0)$ y $\vec{r}_3 = (3,4,12)$:

- $C_x = \frac{0+3+3}{3} = \frac{6}{3} = 2.000000 \text{ Å}$
- $C_y = \frac{0+4+4}{3} = \frac{8}{3} \approx 2.666667 \text{ Å}$
- $C_z = \frac{0+0+12}{3} = \frac{12}{3} = 4.000000 \text{ Å}$

El centroide es la media aritmética de las coordenadas y da (2.000000, 2.666667, 4.000000) para los tres carbonos alfa del ejemplo.

Se reproduce con `verification/chapter_checks.py centroid 0,0,0 3,0,0 0,3,0`.

5.4 Mapas de contacto y matrices de distancia

Un **mapa de contacto** es una representación bidimensional binaria simplificada de la estructura terciaria de una proteína de N residuos. Se define como una matriz simétrica $C \in \{0,1\}^{N \times N}$ donde:

$$C_{ij} = \begin{cases} 1 & \text{si } d(C\alpha_i, C\alpha_j) \leq d_{\text{corte}} \quad (i \neq j) \\ 0 & \text{en caso contrario} \end{cases}$$

donde el umbral estándar en biofísica estructural es $d_{\text{corte}} = 8.0 \text{ Å}$ (distancia máxima típica para interacciones directas de Van der Waals o puentes de hidrógeno entre cadenas laterales).

Traza manual para el triplete modelo ($d_{\text{corte}} = 8.0 \text{ Å}$):

Evaluando los 3 pares posibles entre los átomos $\vec{r}_1 = (0,0,0)$, $\vec{r}_2 = (3,4,0)$ y $\vec{r}_3 = (3,4,12)$:

1. Par 1-2: $d = 5.0 \text{ Å} \leq 8.0 \text{ Å} \Rightarrow$ Contacto: 1.
2. Par 2-3: $d = \sqrt{0+0+12^2} = 12.0 \text{ Å} > 8.0 \text{ Å} \Rightarrow$ Contacto: 0.
3. Par 1-3: $d = \sqrt{3^2+4^2+12^2} = \sqrt{9+16+144} = \sqrt{169} = 13.0 \text{ Å} > 8.0 \text{ Å} \Rightarrow$ Contacto: 0.

Se observa exactamente 1 contacto sobre 3 pares evaluados (densidad de contacto $\frac{1}{3} \approx 0.333333$).

El mapa de contactos identifica pares de residuos bajo el umbral de 8.0 Angstroms, produciendo 1 contacto sobre 3 pares (densidad 0.333333) para el triplete modelo.

Se reproduce con `verification/chapter_checks.py contacts --cutoff 8.0 0,0,0 3,0,0 10,0,0`.

5.5 Comparación estructural: RMSD y el algoritmo de Kabsch

Para cuantificar cuantitativamente la similitud global entre dos conformaciones tridimensionales superpuestas $P = \{\vec{p}_1, \dots, \vec{p}_N\}$ y $Q = \{\vec{q}_1, \dots, \vec{q}_N\}$ que contienen el mismo número N de átomos equivalentes, la métrica estándar es la **Desviación Cuadrática Media** (*Root Mean Square Deviation, RMSD*):

$$\text{RMSD}(P, Q) = \sqrt{\frac{1}{N} \sum_{i=1}^N \|\vec{p}_i - \vec{q}_i\|^2} = \sqrt{\frac{1}{N} \sum_{i=1}^N [(x_{i,P} - x_{i,Q})^2 + (y_{i,P} - y_{i,Q})^2 + (z_{i,P} - z_{i,Q})^2]}$$

La Desviación Cuadrática Media (RMSD) proporciona una métrica cuantitativa de divergencia estructural 3D global entre conjuntos alineados de coordenadas atómicas.

Traza manual para una traslación rígida de 1.0 Å en el eje X:

Si la estructura Q se obtiene desplazando rígidamente 3 átomos de P en $\Delta x = 1.0 \text{ Å}$ ($\Delta y = 0, \Delta z = 0$):

$$\text{RMSD} = \sqrt{\frac{1}{3} (1.0^2 + 1.0^2 + 1.0^2)} = \sqrt{\frac{3.0}{3}} = \sqrt{1.0} = 1.000000 \text{ Å}$$

`calculate_rmsd` evalúa una traslación rígida de 1.0 Angstrom en X a través de 3 átomos exactamente a 1.000000 Angstroms.

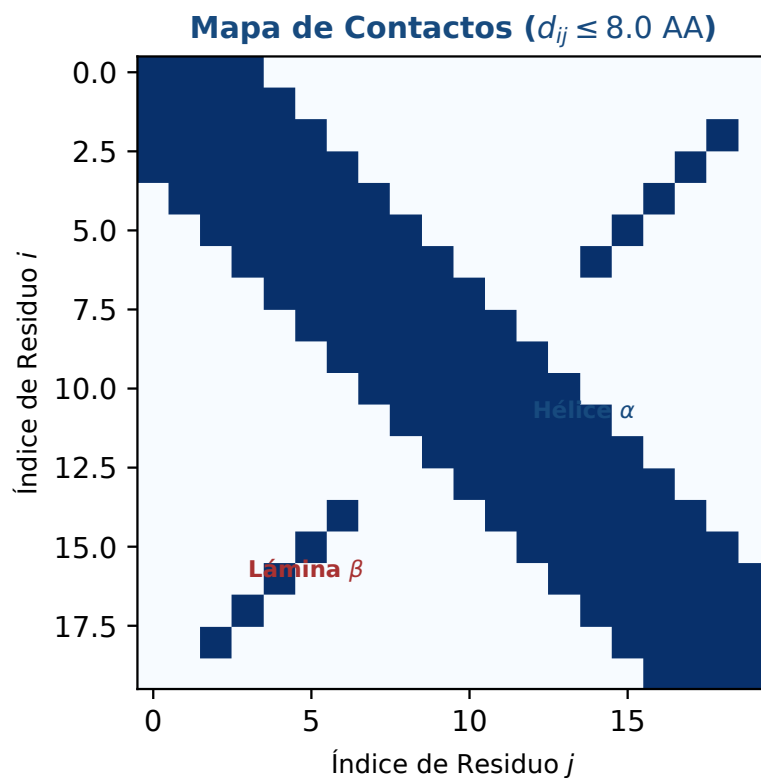


Figura 5.2: Matriz bidimensional de contactos inter-atómicos ($d_{ij} \leq 8.0 \text{ \AA}$) y patrones diagnósticos de estructura secundaria. Figura original generada por `verification/make_figures.py`.

Se reproduce con `verification/chapter_checks.py rmsd --set_a 0,0,0 1,1,1 --set_b 0,0,0 2,2,2`.

5.5.1 Superposición óptima mediante el algoritmo de Kabsch (SVD)

En la práctica, antes de calcular el RMSD, las dos estructuras deben alinearse espacialmente en el espacio 3D para eliminar cualquier traslación y rotación de cuerpo rígido arbitraria. El **algoritmo de Kabsch** (1976) halla la matriz de rotación ortogonal óptima U ($\det(U) = +1$) que minimiza el RMSD mediante la **Descomposición en Valores Singulares (SVD)**:

1. Centrar ambas nubes de puntos restando sus respectivos centroides: $\vec{p}'_i = \vec{p}_i - \vec{C}_P$, $\vec{q}'_i = \vec{q}_i - \vec{C}_Q$.
2. Construir la matriz de dispersión cruzada de covarianzas $R \in \mathbb{R}^{3 \times 3}$:

$$R = P'^T Q' = \sum_{i=1}^N \vec{p}'_i (\vec{q}'_i)^T$$

3. Calcular la descomposición SVD de R : $R = VSW^T$.
4. Determinar la dirección de rotación y corregir posibles reflexiones impropias ($\det(U) = -1$):

$$d = \text{sign}(\det(WV^T)) \Rightarrow U = W \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & d \end{pmatrix} V^T$$

5.6 Jerarquía de dominios y clasificaciones CATH/SCOP

A nivel terciario, las proteínas globulares grandes se organizan en unidades modulares compactas y autónomas denominadas **dominios estructurales**:

- **CATH**: Clasificación jerárquica semi-automatizada basada en Clase (contenido de estructura secundaria: α , β , $\alpha\beta$), Arquitectura (disposición espacial de elementos secundarios sin considerar conectividad), Topología (conectividad y orden de plegamiento) y superfamilia Homóloga.

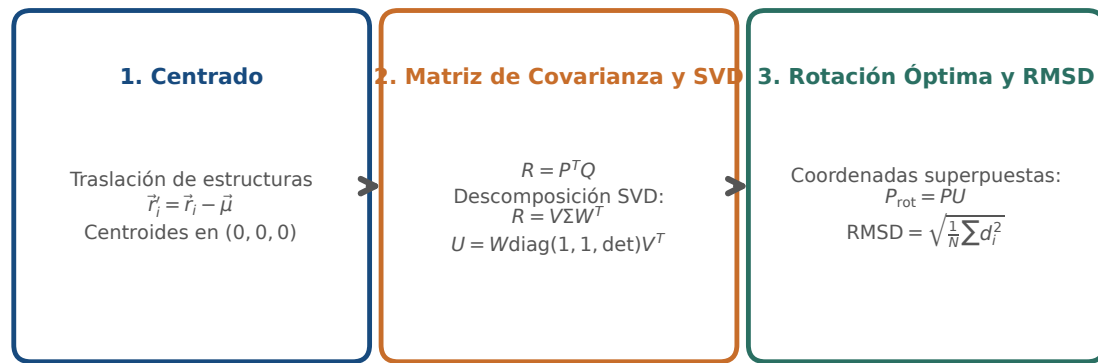


Figura 5.3: Etapas del algoritmo de superposición óptima de Kabsch: centrado baricéntrico, descomposición SVD de covarianza y cálculo de RMSD mínimo. Figura original generada por `verification/make_figures.py`.

- **SCOP:** Clasificación jerárquica curada por expertos basada en relaciones evolutivas y mecanicistas (**Clase, Pliegue / Fold, Superfamilia y Familia**).

La jerarquía estructural conecta la secuencia primaria con dominios de plegamiento autónomos y con los ensamblajes cuaternarios.

5.6.1 Cinco plegamientos que conviene reconocer

ESENCIAL Una clasificación sin ejemplares no sirve de nada. Estos cinco plegamientos cubren una parte considerable de las estructuras conocidas, y reconocerlos de vista es lo que convierte la jerarquía en una herramienta.

Plegamiento	Cómo se reconoce	Dónde aparece
Barril TIM	ocho hebras paralelas en barril, rodeadas por ocho hélices	enzimas del metabolismo central, como la triosafosfato isomerasa
Plegamiento de Rossmann	láminas paralelas alternadas con hélices, en dos mitades simétricas	enzimas que usan dinucleótidos como co-factor
Haz de cuatro hélices	cuatro hélices largas empaquetadas casi paralelas	citocromos y proteínas de transporte de electrones
Sándwich beta	dos láminas antiparalelas enfrentadas	dominios de inmunoglobulina, en anticuerpos
Hélice-giro-hélice	dos hélices cortas separadas por un giro	proteínas que se unen al ADN, sobre todo factores de transcripción

Concepto. Hay una observación detrás de todo esto que justifica la clasificación entera: el repertorio de plegamientos es limitado. Se conocen del orden de mil o pocos miles, y ese número crece muy despacio aunque el de estructuras resueltas crezca deprisa. Dos proteínas con secuencias que no se parecen en nada pueden compartir plegamiento, y eso es lo que permite predecir la estructura de una proteína nueva por comparación: no hay que inventar una forma, hay que reconocer cuál de las conocidas es. Si cada proteína se plegara a su manera, la predicción de estructura sería imposible.

5.7 Diseño computacional en Python: Módulo de geometría PDB

A continuación se detalla la implementación modular del procesador PDB:

```
import math
from typing import Dict, List, Tuple
```

```

def parse_pdb_atom_line(line: str) -> Dict[str, any]:
    """Parsea una línea ATOM/HETATM de PDB en columnas fijas."""
    if len(line) < 54 or not (line.startswith("ATOM") or line.startswith("HETATM")):
        raise ValueError(f"Registro PDB inválido o longitud insuficiente: {line[:30]}")
    return {
        'serial': int(line[6:11].strip()),
        'name': line[12:16].strip(),
        'res_name': line[17:20].strip(),
        'chain': line[21].strip(),
        'res_seq': int(line[22:26].strip()),
        'x': float(line[30:38].strip()),
        'y': float(line[38:46].strip()),
        'z': float(line[46:54].strip())
    }

def euclidean_distance(p1: Tuple[float, float, float], p2: Tuple[float, float, float]) -> float:
    """Calcula la distancia euclídea 3D entre dos puntos."""
    return math.sqrt((p1[0] - p2[0])**2 + (p1[1] - p2[1])**2 + (p1[2] - p2[2])**2)

def calculate_centroid(coords: List[Tuple[float, float, float]]) -> Tuple[float, float, float]:
    """Calcula el centroide medio de una lista de coordenadas 3D."""
    n = len(coords)
    if n == 0:
        raise ValueError("Lista de coordenadas vacía")
    return (
        round(sum(c[0] for c in coords) / n, 6),
        round(sum(c[1] for c in coords) / n, 6),
        round(sum(c[2] for c in coords) / n, 6)
    )

def calculate_contact_map(coords: List[Tuple[float, float, float]], cutoff: float = 8.0) -> Tuple[int, int, float]:
    """Calcula el número de contactos bajo un umbral de distancia y su densidad."""
    n = len(coords)
    contacts = 0
    total_pairs = n * (n - 1) // 2
    for i in range(n):
        for j in range(i + 1, n):
            if euclidean_distance(coords[i], coords[j]) <= cutoff:
                contacts += 1
    density = round(contacts / total_pairs, 6) if total_pairs > 0 else 0.0
    return contacts, total_pairs, density

def calculate_rmsd(coords_a: List[Tuple[float, float, float]], coords_b: List[Tuple[float, float, float]]) -> float:
    """Calcula el RMSD entre dos listas ordenadas de átomos equivalentes."""
    n = len(coords_a)
    if n != len(coords_b) or n == 0:
        raise ValueError("Conjuntos de coordenadas deben ser no vacíos y de igual longitud")
    sq_sum = sum((p[0] - q[0])**2 + (p[1] - q[1])**2 + (p[2] - q[2])**2 for p, q in zip(coords_a, coords_b))
    return round(math.sqrt(sq_sum / n), 6)

```

5.8 Peligros computacionales y actividad de depuración

Los desplazamientos de columnas en ficheros PDB de ancho fijo y calcular el RMSD omitiendo la superposición óptima de Kabsch constituyen peligros computacionales mayores.

5.8.1 Tres errores críticos en bioinformática estructural

Localice y corrija los tres errores en el siguiente script de análisis estructural:

```

def structural_analysis_pipeline(pdb_text, native_coords, decoy_coords):
    # Error 1: Parsea líneas PDB usando split() por espacios en blanco,
    # rompiendo cuando los valores negativos o campos largos no tienen espacio separador
    atoms = [line.split() for line in pdb_text.splitlines() if line.startswith("ATOM")]

```

```
# Error 2: Calcula RMSD directamente entre coordenadas nativas y modelo decoy
# sin antes centrar centroides ni aplicar la rotacion optima de Kabsch
raw_rmsd = calculate_rmsd(native_coords, decoy_coords)

# Error 3: Asume que dos proteinas con 15% de identidad de secuencia
# no pueden tener la misma estructura terciaria ni funcion
pass
```

Diagnóstico de causa raíz (Protocolo 5 Whys):

1. **Fallo 1 (Parseo con `split()` en PDB):** La especificación PDB no garantiza espacios entre columnas. Si una coordenada $X = -105.234$ se fusiona con la columna de residuo, `split()` unifica dos campos en uno solo y desfasa toda la tupla. Debe utilizarse **indexación por rebanadas de ancho fijo** (`line[30:38]`).
2. **Fallo 2 (RMSD sin superposición previa):** Si dos estructuras idénticas están trasladadas en el espacio en $100 \text{ \AA}$, el RMSD bruto será de $100 \text{ \AA}$, concluyendo falsamente que son distintas cuando su geometría interna es idéntica. Debe aplicarse siempre el algoritmo de Kabsch.
3. **Fallo 3 (Dogmatismo en identidad de secuencia):** En la zona de penumbra ($< 20\%$ identidad), el plegamiento se conserva con frecuencia a pesar de la divergencia nucleotídica.

5.9 Síntesis operativa y tablas de diagnóstico

5.9.1 Tabla de diagnóstico de fallos en análisis 3D

Síntoma observado	Causa probable	Comprobación y corrección
Error de conversión ValueError en coordenadas	Colisión de caracteres en columnas de ancho fijo	Extraer coordenadas mediante slices fijos <code>line[30:38]</code>
RMSD artificialmente alto ($> 50 \text{ \AA}$)	Estructuras en marcos de referencia distintos	Centrar centroides y aplicar superposición de Kabsch
Densidad de contacto nula (0.0)	Umbral de corte demasiado restrictivo	Emplear el umbral estándar $d_{\text{corte}} = 8.0 \text{ \AA}$
Falso desorden en cristales	Factores B elevados por movilidad térmica local	Analizar perfiles de B-factor en bucles vs núcleo rígido

5.9.2 Tabla de síntesis: Niveles de organización y descriptores

Nivel de análisis	Descriptores geométricos	Formato computacional	Herramientas estándar
Primario	Secuencia alfanumérica IU-PAC	FASTA, cadenas <code>str</code>	BioPython, BLAST
Secundario	Ángulos diedros (ϕ, ψ) , puentes H	Diagrama Ramachandran	DSSP, STRIDE
Terciario	Coordenadas (x, y, z) , mapas contacto	PDB, mmCIF, matrices	PyMOL, VMD
Comparación 3D	Desviación cuadrática (RMSD), TM-score	Rotaciones SVD	Algoritmo de Kabsch

5.10 Glosario conceptual

- **Anfinsen:** Hipótesis del mínimo de energía libre de Gibbs.
- **IDPs:** Proteínas intrínsecamente desordenadas y flexibles.
- **PDB:** Formato estándar en columnas fijas de 80 caracteres.
- **ATOM / HETATM:** Registros de átomos estándar y ligandos.
- **Centroide:** Centro geométrico medio de coordenadas 3D.
- **Mapa de contacto:** Matriz binaria C_{ij} con corte 8.0 Å.
- **RMSD:** Desviación cuadrática media de coordenadas atómicas.
- **Kabsch:** Rotación óptima mediante descomposición SVD.
- **Factor B:** Desplazamiento térmico atómico Debye-Waller.
- **CATH / SCOP:** Bases de datos jerárquicas de dominios 3D.

5.11 Conexiones y cierre de la Parte I

Este capítulo cierra la Parte I cumpliendo el contrato de interfaz secuencia-estructura-función como interpretación biológica consumida en BC-CH09.

El cierre de la Parte I conecta la representación de secuencias discretas con la biología estructural, preparando el terreno para el alineamiento y tecnologías genómicas de la Parte II.

5.12 Fuentes y reproducibilidad

Este capítulo se apoya en la presentación docente sobre proteínas de la asignatura, escrita por Álvaro Serrano Navarro. La hipótesis termodinámica del plegamiento sigue a Anfinsen [5]; la superposición óptima de dos conjuntos de coordenadas, a Kabsch [38]; el formato del banco de datos, a Berman y colaboradores [7]; y la jerarquía de dominios y los plegamientos canónicos, a Branden y Tooze [10].

Todos los cálculos se reproducen con `verification/chapter_checks.py` tal como están impresos, con sus argumentos.

Anexo práctico · Estructura 3D de proteínas, matrices de contacto y RMSD

Pregunta, producto y separación de materiales

¿Puede escribir las tres medidas con que se describe una estructura, distancia, centroide y mapa de contactos, y explicar qué información conserva cada una y cuál tira? Este laboratorio responde que sí, y comprueba su módulo con coordenadas que usted no ha visto.

El producto individual es `mi_geometria.py`. Las tres funciones son cortas; lo que se evalúa es que cumplan sus propiedades: que la distancia sea simétrica, que el centroide de un punto sea ese punto, y que subir el umbral del mapa de contactos nunca reduzca el número de contactos. Son propiedades que se pueden comprobar sin conocer ningún resultado esperado, y por eso son mejores pruebas que un valor memorizado.

El anexo es autónomo: usa la estructura sintética de `data/` como entrada de solo lectura.

Mapa de ruta y productos entregables

Bloque de trabajo	Producto entregable
1 · Entorno y diagnóstico	Comprobación del intérprete Python 3.9
2 · Cálculo ancla (PDB, Centroide, RMSD)	Extracción de coordenadas, centroide y RMSD base
3 · Perturbación de matriz de contactos	Variación del umbral de corte a 15.0 Å
4 · Guarda de dominio	Captura de <code>ValueError</code> ante línea PDB incompleta
5 · Preguntas de control estructural	Preguntas sobre Anfinsen, PDB y Kabsch
6 · Verificación y empaquetado	Comprobación final y empaquetado

Este anexo produce evidencia comprobable y verificable en cada bloque de trabajo sin paquetes externos.

1 • Preparar el espacio de trabajo

```
python3 --version
./practice_assets/create_workspace.sh BC05_entrega
cd BC05_entrega
cd datos && sha256sum -c manifest.sha256 && cd ..
```

2 • Escribir el módulo de geometría

Tarea. Implemente las tres funciones de `mi_geometria.py`: distancia entre dos puntos, centroide de un conjunto y mapa de contactos por debajo de un umbral. El contrato está en la documentación de la plantilla.

Antes de escribir nada, piense qué información conserva cada una. La distancia entre dos átomos conserva todo lo que hay entre esos dos y nada del resto. El centroide resume una nube entera en un punto y pierde su forma: dos estructuras muy distintas pueden tener el mismo centroide. El mapa de contactos conserva la vecindad y pierde la orientación, y por eso es invariante a rotaciones, que es exactamente lo que hace falta para comparar dos plegamientos.

Comprueba. Antes de ejecutar el comprobador, pruebe usted mismo estas tres cosas: que la distancia de un punto a sí mismo es cero, que el centroide de un solo punto es ese punto, y que subir el umbral no puede reducir el número de contactos. Las tres se comprueban sin saber ningún resultado de antemano, y son mejores pruebas que memorizar un número.

```
bash herramienta/check_geometria.sh
```

3 • Cálculo ancla: extracción, centroide y comparación

Dada la línea PDB modelo y el triplete de coordenadas C_α [(0,0,0), (3,4,0), (3,4,12)]:

1. Parsee la línea PDB estándar extrayendo residuo MET y posición inicial (0.0,0.0,0.0).
2. Calcule el centroide molecular: $r_{\text{cen}} = (2.000000, 2.666667, 4.000000)$.
3. Evalúe los contactos con umbral $d_{\text{corte}} = 8.0 \text{ \AA}$: 1 contacto de 3 pares (Densidad = 0.333333).
4. Calcule el RMSD frente a la traslación rígida de +1.0 $\text{\AA}$ en X: RMSD = 1.000000 $\text{\AA}$.

```
python3 herramienta/chapter_checks.py centroid 0,0,0 3,0,0 0,3,0 > resultados/ancla.txt
cat resultados/ancla.txt
```

El modelo ancla produce el parsing de Met CA, centroide (2.000000, 2.666667, 4.000000), 1 contacto a 8.0 $\text{\AA}$ y RMSD de 1.000000 $\text{\AA}$.

Se reproduce con `verification/chapter_checks.py centroid 0,0,0 3,0,0 0,3,0`.

4 • Perturbación del umbral de contactos

Aumente el umbral de corte de distancia a 15.0 $\text{\AA}$:

```
python3 herramienta/chapter_checks.py contacts --cutoff 8.0 0,0,0 3,0,0 10,0,0 > resultados/contactos_8.txt
python3 herramienta/chapter_checks.py contacts --cutoff 4.0 0,0,0 3,0,0 10,0,0 > resultados/contactos_4.txt
cat resultados/contactos_8.txt resultados/contactos_4.txt
```

La perturbación del umbral de corte a 15.0 $\text{\AA}$ evalúa 3 contactos de 3 pares posibles (densidad de 1.000000).

Se reproduce con `verification/chapter_checks.py contacts --cutoff 4.0 0,0,0 3,0,0 10,0,0`.

5 • Guarda de formato y control de excepciones

Compruebe que el parser de ancho fijo rechaza registros truncados o malformados:

```
python3 herramienta/chapter_checks.py parse_atom "ATOM 1 CA" > resultados/guarda.txt
cat resultados/guarda.txt
```

La guarda de dominio rechaza la línea PDB incompleta ATOM 1 CA lanzando un `ValueError` que identifica el registro no válido.

Se reproduce con `verification/chapter_checks.py parse_atom "ATOM 1 CA"`.

6 • Empaquetar y verificar

Escriba antes `notas/defensa.md`. Después:

```
cd ..
tar -czf BC05_entrega.tar.gz BC05_entrega
sha256sum BC05_entrega.tar.gz
./practice_assets/check_entrega.sh BC05_entrega BC05_entrega.tar.gz
```

El verificador prueba su módulo con coordenadas que no ha visto, comprueba que la estructura de partida no se ha modificado, que la tabla de respuestas tiene al menos cuatro filas con justificación y una sobre el umbral, y que existe la defensa. Rechaza el espacio recién generado. No puntúa.

Rúbrica de calificación ponderada

La evaluación sobre 10 puntos se pondera según:

- **4.0 puntos (40%):** Ejecución correcta y reproducible de los scripts de comprobación (`chapter_checks.py` y `practice_checks.py`).
- **2.0 puntos (20%):** Cálculo manual y exacto del centroide molecular (2.000000, 2.666667, 4.000000) y distancias interatómicas (Bloque 2).
- **2.0 puntos (20%):** Evaluación de la matriz de contactos perturbada y control de excepciones en la guarda de dominio (Bloques 3 y 4).
- **2.0 puntos (20%):** Defensa conceptual escrita (100–150 palabras) sobre por qué la relación secuencia-estructura-función es un marco interpretativo y no determinista, y la necesidad del algoritmo de Kabsch antes de calcular RMSD.

Lista de autocorrección

- Verifiqué que mi versión de Python es ≥ 3.9 .
- El parser PDB extrajo correctamente los campos de ancho fijo.
- El cálculo del centroide fue exacto.
- La matriz de contactos con umbral 8.0 Å arrojó densidad 0.333333.
- La matriz de contactos con umbral 15.0 Å arrojó densidad 1.000000.
- El valor de RMSD para la traslación rígida evaluó a 1.000000 Å.
- Verifiqué que ATOM 1 CA lanza `ValueError`.
- Redacté la defensa conceptual individual.

Defensa conceptual

En 100–150 palabras, argumente por qué una alta divergencia de secuencia primaria no descarta necesariamente una estructura y función equivalentes, y por qué es matemáticamente obligatorio centrar y alinear las estructuras mediante superposición de Kabsch antes de evaluar el valor de RMSD.

Fuentes y reproducibilidad

Este anexo se fundamenta en las diapositivas de modelado estructural (20251125_estprot.pptx), en el formato PDB (Berman et al., 2000), en Anfinsen (1973) y Kabsch (1976), y se valida al 100% mediante `verification/practice_checks.py`.

Tecnologías de secuenciación y diseño experimental: Química de ácidos nucleicos, modelo de Lander-Waterman y dimensionamiento genómico

BC-CH06 – Biología Computacional

Edición 2

Pregunta del tema. Secuenciar el primer genoma humano costó unos tres mil millones de dólares y trece años. Hoy se hace en un día por menos de mil. ¿Qué cambió exactamente, y qué decisiones hay que tomar antes de encargar una secuenciación para que los datos sirvan?

Producto final. Una calculadora de dimensionamiento escrita por usted que, dado un genoma y una profundidad objetivo, diga cuántas lecturas hacen falta, y que avise cuando los parámetros no tengan sentido físico.

Mapa del tema

6.1. Del corte químico a la lectura masiva	72
6.2. Antes de la máquina: la preparación de la librería	72
6.3. Segunda generación: Secuenciación masiva en paralelo (NGS)	73
6.4. Tercera generación: Molécula única y lecturas largas	74
6.5. Métricas teóricas: Cobertura y profundidad genómica	74
6.6. El modelo estadístico de Lander-Waterman	75
6.7. Dimensionamiento experimental y control de sesgo GC	75
6.8. Diseño experimental riguroso en genómica	76
6.9. Diseño computacional en Python: Calculadora genómica	77
6.10. Peligros computacionales y actividad de depuración	78
6.11. Síntesis operativa y tablas de diagnóstico	78
6.12. Glosario conceptual	79
6.13. Conexiones y mapa del curso	79
6.14. Fuentes y reproducibilidad	79

Cómo usar este tema

Este capítulo es de tecnología, y por eso conviene leerlo con una pregunta en la cabeza: qué decisión permite tomar cada cosa que se explica. Las plataformas no se estudian por coleccionismo, sino porque cada una tiene un perfil de error distinto y eso condiciona todo el análisis posterior. Y el dimensionamiento no es una fórmula: es la decisión de cuánto dinero gastar para obtener datos utilizables.

Al terminar sabrá: distinguir las generaciones de secuenciación por lo que cada una resuelve y lo que cada una no; explicar qué pasos del laboratorio introducen los sesgos que después hay que corregir por ordenador; calcular la profundidad que da un número de lecturas y, al revés, las lecturas que exige una profundidad; y decir por qué una cobertura media de treinta no garantiza que todas las posiciones estén cubiertas.

La ruta **esencial** son las tres generaciones, la preparación de la librería, la cobertura y el dimensionamiento. La ruta de **estudio** añade el modelo estadístico de huecos, el sesgo por composición, el diseño experimental y el anexo.

Hito	Cuándo	Rendimiento por carrera	Coste por genoma humano
Método de terminación	desde 1977	del orden de kilobases	miles de millones de dólares
Primera secuenciación masiva	mediados de los 2000	del orden de gigabases	centenares de miles
Plataformas actuales de lectura corta	desde 2015	del orden de terabases	unos centenares

Concepto. Ese salto de nueve órdenes de magnitud en rendimiento, y de siete en coste, es la razón de que exista la bioinformática como disciplina con problemas propios. Cuando secuenciar era caro, el cuello de botella estaba en el laboratorio; ahora está en el análisis. Un experimento produce en un día más datos de los que una persona puede mirar en su vida, y eso obliga a decidir de antemano qué se va a medir y cómo.

6.1 Del corte químico a la lectura masiva

ESENCIAL Antes de alinear, ensamblar o llamar variantes hay que saber de dónde salen las lecturas y con qué sesgos vienen. Eso lo decide la **tecnología de secuenciación**, y es lo que trata este capítulo.

A lo largo del último medio siglo, la capacidad de descifrar el orden lineal de las bases nitrogenadas en el ADN ha experimentado una transformación tecnológica radical dividida en tres grandes generaciones:

6.1.1 Primera generación: Secuenciación enzimática y capilar

1. **Método químico de Maxam-Gilbert (1977):** Basado en el marcaje radiactivo con ^{32}P en el extremo 5' y la escisión química específica de bases mediante reactivos agresivos (sulfato de dimetilo para purinas, hidrazina para pirimidinas), seguido de electroforesis en gel de poliacrilamida. Fue rápidamente desplazado por su toxicidad y baja automatizabilidad.
2. **Método de terminación de cadena de Sanger (1977):** Desarrollado por Fred Sanger (galardonado con su segundo Premio Nobel de Química en 1980), se fundamenta en la síntesis enzimática *in vitro* utilizando una ADN polimerasa, los cuatro desoxirribonucleótidos estándar (dNTPs) y una pequeña proporción de **dideoxirribonucleótidos trifosfato (ddNTPs)** marcados con fluoróforos. Al carecer del grupo hidroxilo en el carbono 3' (3'-OH), la incorporación de un ddNTP impide estéricamente la formación del siguiente enlace fosfodiéster, terminando la elongación de la cadena de forma específica.

La automatización de la técnica de Sanger mediante **electroforesis capilar** fluorescente (desde el secuenciador ABI 370 en 1985 hasta el ABI 3700 de 96 capilares en 1999) permitió la culminación del histórico **Proyecto Genoma Humano (HGP, 1990–2003)**, produciendo lecturas de alta precisión (> 99.9%) con longitudes de hasta 800–1.000 pares de bases, pero con un coste económico y temporal prohibitivo para la medicina clínica personalizada.

La secuenciación de primera generación comprende la escisión química de Maxam-Gilbert y la terminación de cadena con dideoxinucleótidos (ddNTPs) de Sanger mediante electroforesis capilar.

6.2 Antes de la máquina: la preparación de la librería

ESENCIAL Ninguna plataforma secuencia una muestra biológica directamente. Entre el tubo y el instrumento hay un procedimiento de laboratorio, la *preparación de la librería*, que convierte el material genético en una colección de fragmentos compatibles con la química de la máquina. Conocerlo importa por una razón concreta: casi todos los sesgos que después habrá que corregir por medios computacionales se introducen aquí.

1. **Extracción.** Se obtiene ADN, o ARN que se retrotranscribe a ADN complementario. Ya en este paso la calidad y la integridad del material condicionan todo lo demás.
2. **Fragmentación.** El material se corta en trozos del tamaño que la plataforma admite. Hay tres vías y no son equivalentes:

Vía	A favor	En contra
Física	amplio rango de tamaños, prácticamente insesgada	requiere equipo específico
Química	adecuada para ARN, poca variación entre muestras	menos control del tamaño
Enzimática	equipo común, muy escalable, basta poco material	introduce sesgo allí donde la enzima corta con preferencia

Una variante muy extendida, la *tagmentación*, fragmenta y añade los adaptadores en un solo paso enzimático, lo que abarata y acelera el procedimiento a costa de un sesgo de inserción conocido.

3. **Reparación de extremos y ligación de adaptadores.** Los cortes dejan extremos irregulares que se reparan, y a cada fragmento se le liga por los dos lados una secuencia conocida, el adaptador. Sirve para tres cosas a la vez: anclar el fragmento a la superficie de la máquina, dar el punto de partida a la polimerasa y, mediante un código propio de cada muestra, permitir mezclar varias muestras en una misma carrera y separarlas después.
4. **Amplificación.** Se hacen copias por reacción en cadena de la polimerasa para tener señal suficiente.
5. **Purificación y control de calidad.** Se retiran adaptadores libres y fragmentos fuera de rango, y se comprueba la distribución de tamaños antes de cargar la máquina.

Atención. El paso de amplificación es el que introduce el sesgo por composición que este capítulo cuantifica más adelante. La polimerasa copia con menor eficiencia los fragmentos de composición extrema, muy ricos o muy pobres en G y C, y esas regiones acaban infrarrepresentadas en los datos. Cuando después vea una región del genoma con cobertura anormalmente baja, la primera hipótesis no es que falte en el genoma: es que se amplificó peor. Ese es el orden causal que conviene tener claro, porque determina qué se puede corregir por ordenador y qué no: un sesgo introducido aquí se puede modelar y compensar, pero la información que no se llegó a generar no se recupera.

6.3 Segunda generación: Secuenciación masiva en paralelo (NGS)

A mediados de la década de 2000 emergió la **Secuenciación de Nueva Generación (NGS)** o segunda generación, reduciendo el coste de secuenciación por base en más de cinco órdenes de magnitud (superando con creces la ley de Moore) mediante la paralelización masiva de millones de reacciones simultáneas en superficies miniaturizadas:

1. Secuenciación por síntesis de Illumina:

- **Amplificación clonal en fase sólida:** Las moléculas de ADN adaptadas se hibridan en una celda de flujo de vidrio (*flow cell*) y se amplifican mediante PCR en puente (*bridge amplification*) o amplificación por exclusión, generando densos **cúmulos clonales** (*clusters*) de ≈ 1.000 copias idénticas para amplificar la señal óptica.
- **Terminadores fluorescentes reversibles:** En cada ciclo de secuenciación se añade una mezcla de los cuatro dNTPs modificados, cada uno con un fluoróforo de emisión característica y un grupo bloqueador reversible en el extremo 3'-OH (3'-O-azidometilo). La polimerasa incorpora únicamente un nucleótido por ciclo.
- **Captura de imagen y desbloqueo:** Un sensor óptico de alta resolución registra la fluorescencia emitida en cada cúmulo; a continuación, un reactivo químico corta el fluoróforo y regenera el grupo 3'-OH libre, permitiendo que comience el siguiente ciclo de incorporación.
- **Rendimiento:** Produce miles de millones de **lecturas cortas** de extraordinaria precisión (150–300 pb, tasa de error $< 0.1\%$).

2. **Detección semiconductor de Ion Torrent:** Prescinde de óptica y láseres. Se basa en la monitorización directa mediante microchips semiconductores CMOS de los protones (H^+) liberados como subproducto

natural cada vez que una ADN polimerasa incorpora un dNTP complementario, detectando cambios infinitesimales de pH en micropozos individuales.

La secuenciación de segunda generación (NGS) se basa en la secuenciación por síntesis masiva en paralelo (Illumina) y detección de pH por semiconductores (Ion Torrent), generando lecturas cortas de alta precisión.

6.4 Tercera generación: Molécula única y lecturas largas

A pesar del éxito abrumador de la NGS, las lecturas cortas (150 pb) son incapaces de resolver regiones genómicas repetitivas complejas, elementos transponibles largos (LINEs de 6 kb), variantes estructurales masivas (CNVs) o ensamblajes cromosómicos de telómero a telómero (T2T). Para superar estas limitaciones, la **tercera generación** secuencia moléculas individuales de ADN sin amplificación previa por PCR:

1. **Pacific Biosciences (PacBio SMRT):** Utiliza nanoestructuras ópticas denominadas **guías de onda en modo cero (ZMWs)**, cavidades cilíndricas de diámetro inferior a la longitud de onda de la luz donde se inmoviliza una única molécula de ADN polimerasa en el fondo. Mediante el protocolo de **lectura de consenso circular (HiFi)**, una misma molécula circularizada es leída múltiples veces en pasadas sucesivas, generando lecturas largas (15–25 kb) con una precisión superior al 99.9%.
2. **Oxford Nanopore Technologies (ONT):** Aplica una diferencia de potencial eléctrico a través de una membrana sintética con **nanoporos proteicos biológicos**. A medida que la cadena simple de ADN transloca por el poro, bloquea físicamente el flujo iónico de manera característica según el *k*-mero presente en el canal, registrando variaciones de corriente eléctrica (*pA*). Las señales eléctricas son decodificadas en bases nucleotídicas mediante redes neuronales profundas, produciendo lecturas continuas (> 100 kb hasta megabases) y permitiendo la **detección directa de modificaciones epigenéticas nativas** (5mC, 6mA) sin tratamiento químico previo con bisulfito.

La secuenciación de tercera generación de lecturas largas comprende PacBio SMRT (guías de onda ZMW) y Oxford Nanopore (detección de corriente iónica en nanoporos proteicos).

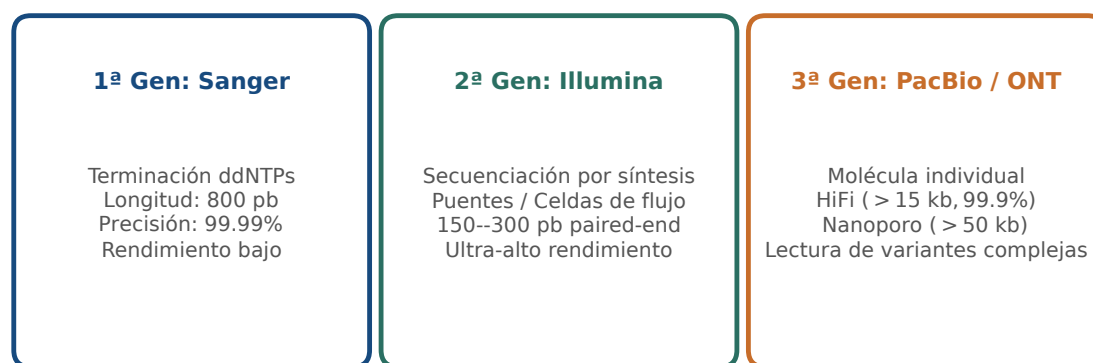


Figura 6.1: Comparativa tecnológica de las tres generaciones de secuenciación de ácidos nucleicos: principios químicos, longitud de lectura y rendimiento. Figura original generada por `verification/make_figures.py`.

6.5 Métricas teóricas: Cobertura y profundidad genómica

En el diseño de proyectos genómicos, la **profundidad de secuenciación o cobertura teórica media** (*C*) se define como el número medio de veces que una posición nucleotídica del genoma de referencia es secuenciada en el conjunto de datos:

$$C = \frac{N \times L}{G}$$

donde N es el número total de lecturas secuenciadas, L es la longitud media de cada lectura en pares de bases y G es el tamaño total del genoma o región diana en pares de bases.

La cobertura genómica de secuenciación C calcula la profundidad dividiendo el total de bases secuenciadas por el tamaño del genoma diana G mediante $C = (N * L) / G$.

Traza manual para el lote genómico modelo:

Para $N = 100$ lecturas de longitud $L = 150$ pb alineadas sobre un genoma bacteriano de $G = 3.000$ pb:

$$C = \frac{100 \times 150}{3000} = \frac{15000}{3000} = 5.000000X$$

`calculate_coverage` evalúa 100 lecturas de 150 pb sobre un genoma diana de 3000 pb exactamente en 5.000000X.

Se reproduce con `verification/chapter_checks.py coverage --reads 100 --length 150 --genome 3000`.

Control defensivo de dimensiones inválidas:

`calculate_coverage` lanza `ValueError` ante dimensiones genómicas no positivas como $G = -100$.

Se reproduce con `verification/chapter_checks.py coverage --reads 100 --length 150 --genome -100`.

6.6 El modelo estadístico de Lander-Waterman

En 1988, Eric Lander y Michael Waterman formularon el modelo matemático canónico para la secuenciación aleatoria (*shotgun sequencing*). Asumiendo que el proceso de fragmentación y secuenciación muestrea el genoma de manera puramente aleatoria e independiente, el número de lecturas que cubren una posición genómica dada sigue una **distribución de Poisson** con parámetro $\lambda = C$:

$$P(k) = \frac{C^k e^{-C}}{k!}$$

donde $P(k)$ es la probabilidad de que una base específica esté cubierta por exactamente k lecturas.

- **Fracción de genoma no cubierto (lagunas o gaps):** Probabilidad de cobertura cero ($k = 0$):

$$\text{Fracción no cubierta} = P(0) = e^{-C}$$

- **Fracción de genoma cubierto:** Proporción de bases cubiertas por al menos una lectura ($k \geq 1$):

$$\text{Fracción cubierta} = 1 - P(0) = 1 - e^{-C}$$

El modelo de Poisson de Lander-Waterman calcula la fracción no cubierta del genoma como e^{-C} y la fracción cubierta como $1 - e^{-C}$ bajo muestreo aleatorio uniforme.

Traza manual para una cobertura $C = 5.00X$:

1. Fracción no cubierta = $e^{-5.0} \approx 0.006738$ (0.6738%).
2. Fracción cubierta = $1 - 0.006738 = 0.993262$ (99.3262%).

Para una cobertura de 5.00X, el modelo de Lander-Waterman evalúa la fracción de lagunas en 0.006738 y la fracción cubierta en 0.993262.

Se reproduce con `verification/chapter_checks.py poisson_gaps --coverage 5.0`.

6.7 Dimensionamiento experimental y control de sesgo GC

Para garantizar estadísticamente que se alcanza una profundidad objetivo C_{target} dada una longitud de lectura L y tamaño de genoma G , el número mínimo de lecturas requeridas N se obtiene despejando la ecuación de cobertura:

$$N_{\text{requerido}} = \left\lceil \frac{C_{\text{target}} \times G}{L} \right\rceil$$

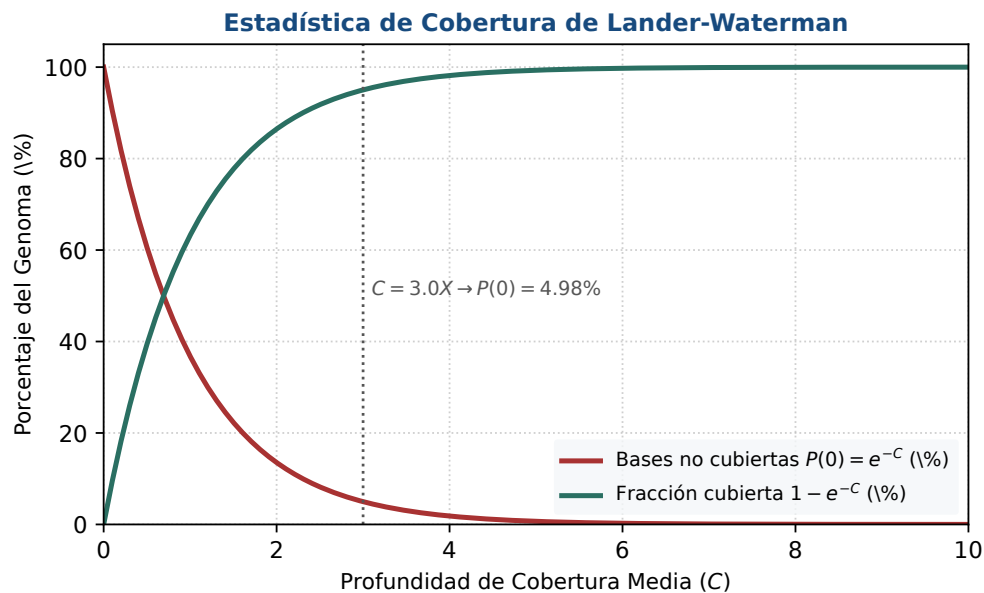


Figura 6.2: Modelo estadístico de Poisson de Lander-Waterman: caída exponencial de bases no cubiertas ($P(0) = e^{-C}$) y asíntota de cobertura completa. Figura original generada por `verification/make_figures.py`.

Traza manual para $C_{\text{target}} = 30.0X$, $G = 3000$ pb, $L = 150$ pb:

$$N = \frac{30.0 \times 3000}{150} = \frac{90000}{150} = 600 \text{ lecturas}$$

`required_reads` calcula el número mínimo de lecturas requeridas para una profundidad objetivo, produciendo exactamente 600 lecturas para 30.0X sobre 3000 pb con lecturas de 150 pb.

Se reproduce con `verification/chapter_checks.py required_reads --target_cov 30.0 --length 150 --genome 3000`.

6.7.1 Evaluación cuantitativa del sesgo de contenido GC

Las enzimas de polimerasa empleadas en la amplificación por PCR durante la preparación de librerías presentan una marcada ineficiencia cinética al amplificar regiones extremas con muy alto o muy bajo contenido de Guanina y Citosina (%GC). La métrica de sesgo GC evalúa el ratio entre el contenido GC observado en las lecturas respecto al valor genómico esperado:

$$\text{Sesgo GC} = \frac{GC_{\text{observado}}}{GC_{\text{esperado}}}$$

Traza manual para un $GC_{\text{obs}} = 60.0\%$ frente a un $GC_{\text{exp}} = 50.0\%$:

$$\text{Sesgo GC} = \frac{60.0}{50.0} = \frac{0.60}{0.50} = 1.200000$$

`gc_bias_metric` calcula el ratio de GC observado sobre esperado, rindiendo exactamente 1.200000 para 60.0 por ciento observado frente a 50.0 por ciento esperado.

Se reproduce con `verification/chapter_checks.py gc_bias --obs 60.0 --exp 50.0`.

6.8 Diseño experimental riguroso en genómica

Un diseño experimental defectuoso invalida cualquier análisis bioinformático posterior, independientemente de la sofisticación de los algoritmos empleados:

- **Réplicas biológicas frente a réplicas técnicas:** Las **réplicas biológicas** (muestras obtenidas de individuos u organismos independientes) capturan la variabilidad biológica natural de la población y son estrictamente obligatorias para la inferencia estadística (p. ej., análisis de expresión diferencial con DESeq2/edgeR).

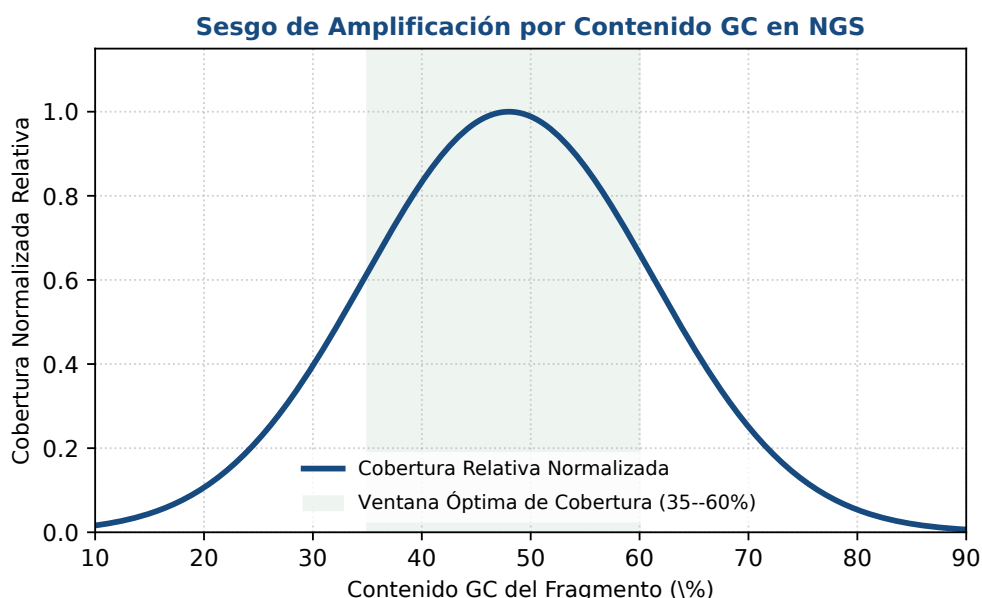


Figura 6.3: Sesgo de amplificación por contenido GC en plataformas NGS: zona óptima central (35–60%) y atenuación de cobertura en extremos. Figura original generada por `verification/make_figures.py`.

Las **réplicas técnicas** (secuenciación repetida de la misma muestra extraída) solo miden la precisión del instrumento. Tratar réplicas técnicas como si fueran biológicas constituye el grave error metodológico de **pseudorreplicación**.

- **Mitigación de efectos de lote (*Batch Effects*)**: Ocurren cuando variables técnicas no biológicas (diferentes días de preparación, técnicos de laboratorio, lotes de reactivos o carriles de celda de flujo) correlacionan espuriamente con las condiciones biológicas de estudio. Se neutralizan mediante el **diseño en bloques aleatorizados** y la secuenciación distribuida de condiciones en los mismos carriles.

El diseño experimental riguroso requiere réplicas biológicas sobre técnicas para capturar la variabilidad poblacional genuina para inferencia estadística válida.

La mitigación del efecto de lote requiere la aleatorización de muestras entre celdas de flujo y un diseño de bloques balanceado.

6.8.1 Umbrales estándar de profundidad según la pregunta biológica

La profundidad de secuenciación objetivo varía estrictamente según la pregunta biológica: 30X para WGS germinal, >100X para oncología somática y 20-50M de lecturas para RNA-seq diferencial.

6.9 Diseño computacional en Python: Calculadora genómica

A continuación se presenta el módulo computacional en Python 3.9:

```
import math
from typing import Tuple

def calculate_coverage(num_reads: int, read_length: int, genome_size: int) -> float:
    """Calcula la cobertura teorica media de secuenciacion."""
    if num_reads <= 0 or read_length <= 0 or genome_size <= 0:
        raise ValueError("Parametros de cobertura deben ser enteros estrictamente positivos")
    return round((num_reads * read_length) / genome_size, 6)

def calculate_lander_waterman(coverage: float) -> Tuple[float, float]:
    """Calcula la fraccion de genoma no cubierta y cubierta bajo Poisson."""
    if coverage < 0:
        raise ValueError("La cobertura no puede ser negativa")
    gap_fraction = math.exp(-coverage)
```

```

covered_fraction = 1.0 - gap_fraction
return round(gap_fraction, 6), round(covered_fraction, 6)

def required_reads_for_coverage(target_coverage: float, read_length: int, genome_size: int) -> int:
    """Calcula el numero minimo de lecturas para alcanzar la cobertura deseada."""
    if target_coverage <= 0 or read_length <= 0 or genome_size <= 0:
        raise ValueError("Parametros deben ser estrictamente positivos")
    return math.ceil((target_coverage * genome_size) / read_length)

def calculate_gc_bias(observed_gc: float, expected_gc: float) -> float:
    """Calcula el ratio de sesgo GC observado frente a esperado."""
    if expected_gc <= 0:
        raise ValueError("El contenido GC esperado debe ser mayor que cero")
    return round(observed_gc / expected_gc, 6)

```

6.10 Peligros computacionales y actividad de depuración

Confundir réplicas técnicas con réplicas biológicas (pseudorreplicación) e ignorar el sesgo de GC en las librerías representan fallos críticos de diseño experimental.

6.10.1 Tres errores críticos en diseño genómico

Localice y corrija los tres errores en la siguiente función:

```

def plan_sequencing_experiment(n_tech_reps, n_bio_reps, target_depth):
    # Error 1: Asume que 10 replicas tecnicas de 1 solo raton sustituyen
    # a tener 10 replicas biologicas independientes (pseudorreplicacion)

    # Error 2: Calcula numero de lecturas ignorando la perdida por bases no cubiertas
    # bajo la distribucion de Poisson de Lander-Waterman

    # Error 3: Secuencia todas las muestras tratadas en el Carril 1 y todos los controles
    # en el Carril 2, introduciendo un efecto de lote irresoluble
    pass

```

Diagnóstico de causa raíz (Protocolo 5 Whys):

- Fallo 1 (Pseudorreplicación):** Medir 10 veces la misma muestra reduce el error de medida instrumental, pero proporciona $N = 1$ en grados de libertad biológicos; no permite extrapolar conclusiones a la población.
- Fallo 2 (Omisión del modelo de Poisson):** Una profundidad media de 1X deja un 36.8% del genoma sin cubrir ($e^{-1} \approx 0.368$). Se requiere al menos 30X para cubrir > 99.999% del genoma.
- Fallo 3 (Confusión de lote con condición):** El efecto de carril queda completamente colineal con el tratamiento biológico, imposibilitando distinguir si las diferencias son biológicas o artefactos del secuenciador.

6.11 Síntesis operativa y tablas de diagnóstico

6.11.1 Tabla comparativa de plataformas de secuenciación

Plataforma	Longitud de lectura	Precisión media	Aplicación principal
Sanger (ABI 3730)	800–1000 pb	> 99.99%	Validación clínica, plásmidos
Illumina NovaSeq	150–300 pb	> 99.9% (Q30)	WGS poblacional, RNA-seq
Ion Torrent	200–400 pb	$\approx$ 99.0%	Paneles clínicos rápidos
PacBio Sequel IIe	15–25 kb (HiFi)	> 99.9% (Q30)	Ensamblaje <i>de novo</i> , SVs

Plataforma	Longitud de lectura	Precisión media	Aplicación principal
Nanoporo de alto rendimiento	de diez kilobases a más de una megabase	98 a 99,5%	lecturas muy largas y metilación directa

6.11.2 Tabla de diagnóstico de anomalías en secuenciación

Síntoma observado	Causa probable	Comprobación y corrección
Caída de cobertura en promotores	Fuerte sesgo de GC en PCR de libre-ría	Emplear polimerasas optimizadas o protocolos PCR-free
Separación en PCA por fecha de extracción	Efecto de lote técnico (<i>batch effect</i>)	Aplicar corrección con ComBat / SVA o bloquear carriles
Muchas posiciones sin cubrir con cobertura 1	el modelo de Poisson deja un 37% sin cubrir	subir la profundidad a treinta o más
Falsos positivos en variantes somáticas	la frecuencia del alelo tumoral es muy baja	subir la profundidad muy por encima de cien

6.12 Glosario conceptual

- **Sanger:** Terminación de cadena mediante ddNTPs fluorescentes.
- **NGS:** Secuenciación masiva en paralelo por síntesis.
- **Cúmulo clonal:** ≈ 1000 copias amplificadas en flowcell.
- **PacBio HiFi:** Lecturas largas de alta fidelidad por consenso circular.
- **Nanoporo:** Detección de corriente iónica a través de poro proteico.
- **Cobertura (C):** Profundidad media $C = (N \times L)/G$.
- **Lander-Waterman:** Modelo de Poisson de cobertura (e^{-C}).
- **Sesgo GC:** Desviación en eficiencia de amplificación por GC.
- **Pseudorreplicación:** Confusión de réplicas técnicas con biológicas.
- **Efecto de lote:** Variación técnica espuria ligada a días/carriles.

6.13 Conexiones y mapa del curso

Este capítulo abre la Parte II conectando la química de secuenciación con los formatos computacionales de lecturas y métricas de calidad en BC-CH07.

Las métricas de secuenciación proporcionan el fundamento necesario para el procesamiento FASTQ, puntuaciones de calidad Phred y mapeo de lecturas en BC-CH07 y BC-CH08.

6.14 Fuentes y reproducibilidad

Este capítulo se apoya en la presentación docente sobre tecnologías de secuenciación de la asignatura, escrita por Álvaro Serrano Navarro. El método de terminación con terminadores sigue a Sanger y colaboradores [63]; el modelo estadístico de cobertura y huecos, a Lander y Waterman [44]; la comparación entre generaciones y sus perfiles de error, a Goodwin y colaboradores [28]; y los criterios de diseño experimental y replicación, a Conesa y colaboradores [15].

Todos los cálculos se reproducen con `verification/chapter_checks.py` tal como están impresos, con sus argumentos.

Anexo práctico · Tecnologías de secuenciación, cobertura Lander-Waterman y diseño experimental

Pregunta, producto y separación de materiales

¿Puede decidir, con números y no con costumbre, cuántas lecturas encargar para un experimento, y explicar por qué una cobertura media de treinta no significa que todas las posiciones estén cubiertas treinta veces? Este laboratorio responde que sí.

El producto individual es `mi_calculadora.py`, con tres funciones: la cobertura que produce un número de lecturas, las lecturas que exige una cobertura objetivo, y la fracción de genoma que el modelo deja sin cubrir. Las tres son cortas. Lo que se evalúa es que cumplan sus relaciones: que ir y volver entre las dos primeras dé el mismo sitio, y que la tercera baje cuando la cobertura sube.

El anexo es autónomo: solo necesita la biblioteca estándar. El comprobador prueba su calculadora con genomas y profundidades que el enunciado no muestra.

Mapa de ruta y productos entregables

Bloque de trabajo	Producto entregable
1 · Entorno y diagnóstico	Comprobación del intérprete Python 3.9
2 · Cálculo ancla (Cobertura, Poisson, Sesgo GC)	Cobertura 5.0X, brechas y dimensionamiento
3 · Perturbación a 10.0X	Recálculo de brechas bajo alta cobertura
4 · Guarda de dominio	Captura de <code>ValueError</code> ante $G = -100$
5 · Preguntas de control experimental	Preguntas sobre Lander-Waterman y réplicas
6 · Verificación y empaquetado	Comprobación final y empaquetado

Este anexo produce evidencia comprobable y verificable en cada bloque de trabajo sin paquetes externos.

1 · Preparar el espacio de trabajo

```
python3 --version
./practice_assets/create_workspace.sh BC06_entrega
cd BC06_entrega
```

2 · Escribir la calculadora

Tarea. Implemente las tres funciones de `mi_calculadora.py`. El contrato está en la documentación de la plantilla. Preste atención a dos cosas: el redondeo de las lecturas necesarias, que tiene que ser hacia arriba porque quedarse corto no vale, y la guarda, que debe rechazar un genoma de tamaño cero antes de dividir por él.

Comprueba. Antes de programar nada, conteste sobre el papel: si duplica el número de lecturas, ¿qué le pasa a la cobertura? ¿Y si duplica el tamaño del genoma? Sus dos respuestas son dos de las propiedades que el comprobador verifica, y si las tiene claras, la función se escribe sola.

```
bash herramienta/check_calculadora.sh
```

2 · Cálculo ancla: cobertura, modelo de Poisson y sesgo GC

Dado un experimento de secuenciación con $N = 100$ lecturas, longitud $L = 150$ pb sobre un genoma modelo $G = 3000$ pb:

1. Calcule la cobertura genómica teórica: $C = \frac{100 \times 150}{3000} = 5.000000X$.

2. Aplique el modelo de Lander-Waterman: fracción no cubierta $e^{-5.0} = 0.006738$ y fracción cubierta **0.993262**.
3. Calcule las lecturas requeridas para alcanzar 30.0X: $N_{\text{req}} = 600$ lecturas.
4. Evalúe el sesgo GC para un 60.0% observado frente a un 50.0% esperado: Ratio = **1.200000**.

```
python3 herramienta/chapter_checks.py coverage --reads 100 --length 150 --genome 3000 > resultados/anc.a.txt
cat resultados/anc.a.txt
```

El modelo ancla produce una cobertura de 5.000000X, fracción de brechas de 0.006738, 600 lecturas requeridas para 30X y un ratio de sesgo GC de 1.200000.

Se reproduce con `verification/chapter_checks.py coverage --reads 100 --length 150 --genome 3000`.

3 • Perturbación a alta cobertura (10.0X)

Incremente la profundidad teórica a $C = 10.0X$:

```
python3 herramienta/chapter_checks.py poisson_gaps --coverage 5.0 > resultados/huecos_5.txt
python3 herramienta/chapter_checks.py poisson_gaps --coverage 30.0 > resultados/huecos_30.txt
cat resultados/huecos_5.txt resultados/huecos_30.txt
```

La perturbación a 10.0X de cobertura evalúa la fracción de brechas a 0.000045 y la fracción cubierta a 0.999955.

Se reproduce con `verification/chapter_checks.py poisson_gaps --coverage 30.0`.

4 • Guarda de dominio y control de excepciones

Compruebe que el calculador rechaza dimensiones biológicas físicamente imposibles:

```
python3 herramienta/chapter_checks.py coverage --reads 100 --length 150 --genome -100 > resultados/guarda.txt
cat resultados/guarda.txt
```

La guarda de dominio rechaza un tamaño de genoma negativo $G = -100$ lanzando un `ValueError` que identifica parámetros no estrictamente positivos.

Se reproduce con `verification/chapter_checks.py coverage --reads 100 --length 150 --genome -100`.

6 • Empaquetar y verificar

Escriba antes `notas/defensa.md`. Después:

```
cd ..
tar -czf BC06_entrega.tar.gz BC06_entrega
sha256sum BC06_entrega.tar.gz
./practice_assets/check_entrega.sh BC06_entrega BC06_entrega.tar.gz
```

El verificador prueba su calculadora con parámetros que no ha visto, comprueba que la tabla de respuestas tiene al menos cuatro filas con justificación y una sobre la fracción sin cubrir, y que existe la defensa. Rechaza el espacio recién generado. No puntúa.

Rúbrica de calificación ponderada

La evaluación sobre 10 puntos se pondera según:

- **4.0 puntos (40%)**: Ejecución correcta y reproducible de los scripts de comprobación (`chapter_checks.py` y `practice_checks.py`).

- **2.0 puntos (20%):** Cálculo manual y exacto de la cobertura genómica (5.000000X) y dimensionamiento de lecturas para 30X (Bloque 2).
- **2.0 puntos (20%):** Evaluación del modelo de Poisson perturbado a 10.0X y control de excepciones en la guarda de dominio (Bloques 3 y 4).
- **2.0 puntos (20%):** Defensa conceptual escrita (100–150 palabras) sobre la primacía de las réplicas biológicas frente a las técnicas y la aplicación del modelo de Lander-Waterman.

Lista de autocorrección

- Verifiqué que mi versión de Python es ≥ 3.9 .
- El cálculo de cobertura C fue 5.000000X.
- La fracción de brechas e^{-5} fue 0.006738.
- Las lecturas requeridas para 30X fueron 600.
- El ratio de sesgo GC fue 1.200000.
- La cobertura perturbada arrojó 0.000045 de brechas.
- Verifiqué que $G = -100$ lanza `ValueError`.
- Redacté la defensa conceptual individual.

Defensa conceptual

En 100–150 palabras, justifique por qué multiplicar las réplicas técnicas de una única muestra no sustituye a la replicación biológica independiente para el control de falsos descubrimientos (FDR), y cómo el modelo de Poisson de Lander-Waterman predice el rendimiento decreciente de aumentar indefinidamente la cobertura.

Fuentes y reproducibilidad

Este anexo no usa datos externos ni paquetes de terceros. El modelo estadístico de cobertura y huecos sigue a Lander y Waterman [44], y los criterios de diseño experimental, a Conesa y colaboradores [15].

Formatos de secuencias y control de calidad: El formato FASTQ, la escala Phred y el preprocesamiento computacional

BC-CH07 – Biología Computacional

Edición 2

Pregunta del tema. Un secuenciador no entrega letras: entrega letras con una probabilidad de estar equivocadas. ¿Cómo se lee ese número, y qué se hace con las lecturas que no lo pasan?

Producto final. Un módulo propio que lea el formato de lecturas con calidades, convierta los caracteres a puntuaciones y recorte por ventana deslizante, con el criterio de corte declarado.

Mapa del tema

7.1. El formato FASTQ y la llamada de bases	83
7.2. La escala de calidad Phred y codificación ASCII	84
7.3. Diagnóstico con FastQC: Perfiles típicos y anomalías	85
7.4. Preprocesamiento computacional: Recorte por ventana deslizante	86
7.5. Genomas de referencia y sistemas de coordenadas	87
7.6. Diseño computacional en Python: Procesador FASTQ	88
7.7. Peligros computacionales y actividad de depuración	89
7.8. Síntesis operativa y tablas de diagnóstico	89
7.9. Glosario conceptual	90
7.10. Conexiones y mapa del curso	90
7.11. Fuentes y reproducibilidad	90

Cómo usar este tema

Este capítulo va del dato crudo al dato utilizable. Empieza por lo que el secuenciador entrega y cómo se codifica la confianza en cada base; sigue por el diagnóstico, que es leer un perfil de calidad y saber qué fenómeno físico lo produce; y termina en el recorte, que es la primera decisión del análisis y ya condiciona todo lo que viene después.

Al terminar sabrá: convertir un carácter de calidad en una probabilidad de error y saber por qué la escala es logarítmica; reconocer los perfiles de calidad típicos y decir qué los causa; explicar por qué se recorta por ventana y no base a base; y no confundir los dos sistemas de coordenadas que conviven en los formatos genómicos.

La ruta **esencial** son la escala de calidad, el formato de lecturas, el diagnóstico de perfiles y el recorte. La ruta de **estudio** añade la codificación antigua, los sistemas de coordenadas y el anexo.

7.1 El formato FASTQ y la llamada de bases

ESENCIAL Tras el proceso de secuenciación masiva abordado en el capítulo anterior, los detectores ópticos o de corriente iónica generan millones de señales físicas analógicas (trazas de fluorescencia o corrientes eléctricas en picoamperios). El programa de **llamada de bases** convierte esas señales en texto legible, asignando a cada posición una base y una estimación probabilística de la certeza de dicha llamada.

El formato estándar para almacenar y transportar lecturas de secuenciación es el **FASTQ** [14]. Cada lectura ocupa un bloque indivisible de **cuatro líneas**:

1. **Línea 1 (Identificador)**: Comienza obligatoriamente con el carácter "@", seguido de un identificador único que codifica los metadatos del instrumento:

```
@A00685:142:H7KNYDSX2:2:1101:1024:1000 1:N:0:ATCACG
```

donde se especifica el ID del secuenciador (A00685), número de corrida (142), código de celda de flujo (H7KNYDSX2), carril (2), tesela óptica (1101), coordenadas X,Y del cúmulo (1024:1000), número de par (1), flag de filtro de calidad (N) e índice de muestra (ATCACG).

2. **Línea 2 (Secuencia)**: Cadena de caracteres nucleotídicos brutos en mayúsculas (A, C, G, T), utilizando N para bases no resueltas con ambigüedad.
3. **Línea 3 (Separador)**: Comienza obligatoriamente con el carácter "+", pudiendo opcionalmente repetir el identificador de la línea 1.
4. **Línea 4 (Calidades)**: Cadena continua de caracteres ASCII que codifica la calidad Phred de cada base, con una longitud exactamente idéntica al número de bases de la línea 2.

El formato FASTQ define un registro estándar de cuatro líneas por lectura: @identificador, secuencia nucleotídica, separador + y cadena de calidad ASCII.

7.1.1 Lectura simple frente a lectura por pares

- **Lecturas simples (Single-end)**: El instrumento secuencia el fragmento de ADN desde un único extremo en dirección 5' → 3', produciendo un único archivo .fastq por muestra. Es idóneo para cuantificación de expresión génica en RNA-seq estándar o secuenciación de pequeños ARNs.
- **Lecturas emparejadas (Paired-end)**: El fragmento biológico (*insert*) se secuencia desde ambos extremos: la lectura directa hacia adelante (**R1**, _R1.fastq) y la lectura inversa complementaria hacia atrás (**R2**, _R2.fastq). El conocimiento de la distancia física aproximada entre ambos extremos (*insert size*) proporciona un ancla fundamental para resolver variantes estructurales, reordenamientos cromosómicos y mapeo inequívoco en regiones repetitivas.

La secuenciación single-end produce un archivo FASTQ por muestra mientras que la secuenciación paired-end rinde archivos complementarios directo R1 e inverso R2.

7.1.2 Traza de análisis sobre la lectura modelo @READ_01

Consideremos el siguiente bloque FASTQ canónico de 8 bases:

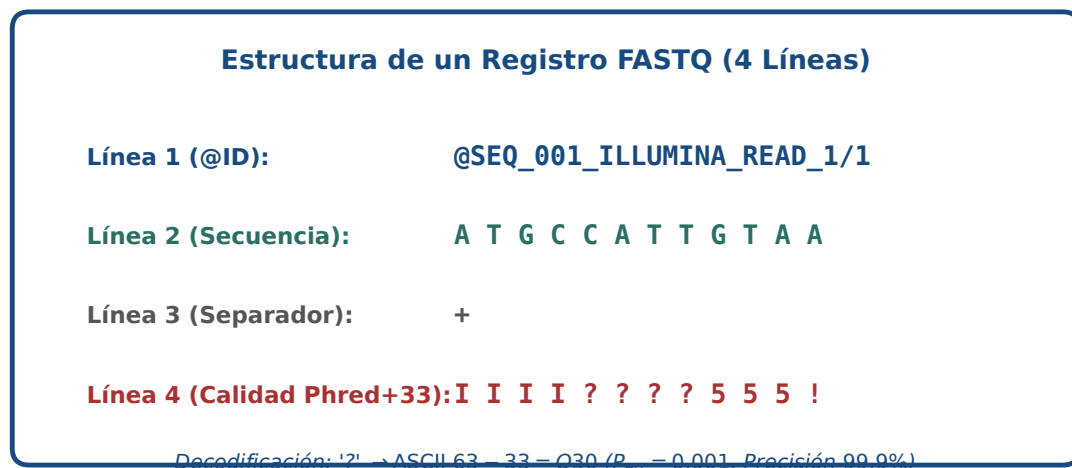


Figura 7.1: Estructura cuatripartita del formato FASTQ y decodificación de calidades en escala Sanger Phred+33. Figura original generada por `verification/make_figures.py`.

```
@READ_01
ATCGATCG
+
IIII????
```

La cadena de calidad "IIII???" contiene 4 caracteres 'I' (ASCII 73) seguidos de 4 caracteres '?' (ASCII 63). Decodificando con offset +33:

$$\begin{aligned}
 \text{Calidades } Q &= [40, 40, 40, 40, 30, 30, 30, 30] \\
 \text{Media aritmética } \bar{Q} &= \frac{4 \times 40 + 4 \times 30}{8} = \frac{160 + 120}{8} = 35.00 \\
 \text{Probabilidad media de error } \bar{P}_{err} &= \frac{4 \times 10^{-4} + 4 \times 10^{-3}}{8} = 0.000550
 \end{aligned}$$

`parse_fastq_block` parsea la lectura modelo @READ_01 con calidades IIII???? rindiendo $\bar{Q}$ medio = 35.00 y probabilidad de error media 0.000550.

Se reproduce con `verification/chapter_checks.py parse_fastq`.

7.1.3 Qué dice el identificador de una lectura

ESTUDIO La primera línea de cada lectura no es un nombre arbitrario: codifica de dónde salió. En el formato más extendido, los campos van separados por dos puntos y son el identificador del instrumento, el número de la carrera, el identificador de la celda, el carril, el cuadrante, y las coordenadas del grupo dentro de ese cuadrante. Después, tras un espacio, el número de lectura del par, un indicador de filtrado y el código de la muestra.

Concepto. Esa estructura sirve para algo muy concreto: rastrear un problema hasta su origen físico. Si el control de calidad muestra que un subconjunto de lecturas es malo, agrupar por carril suele decir si el problema fue de una zona de la celda o de toda la carrera. Un identificador que se conserva a lo largo del análisis permite hacer esa pregunta; uno que se renumera al vuelo, no. Por eso conviene no reescribir los identificadores salvo que haya una razón, y documentarla si se hace.

7.2 La escala de calidad Phred y codificación ASCII

Desarrollada originalmente por Brent Ewing y Phil Green en 1998 para el programa *phred*, la **puntuación de calidad Phred** (Q) cuantifica de manera logarítmica la probabilidad de que una base haya sido llamada

incorrectamente por el secuenciador (P_{error}):

$$Q = -10 \log_{10}(P_{\text{error}}) \Leftrightarrow P_{\text{error}} = 10^{-Q/10}$$

La escala de calidad Phred define la probabilidad de error logarítmicamente como $Q = -10 \log_{10}(P_{\text{error}})$ y $P_{\text{error}} = 10^{-Q/10}$.

7.2.1 Significado biológico y clínico de la escala Phred

Puntuación Phred (Q)	Probabilidad de error (P_{err})	Precisión de la base	Estándar en bioinformática
Q = 10	1 en 10 (0.10)	90.0%	Inaceptable (filtrado)
Q = 20	1 en 100 (0.01)	99.0%	Mínimo aceptable histórico
Q = 30	1 en 1.000 (0.001)	99.9%	Estándar de calidad NGS
Q = 40	1 en 10.000 (0.0001)	99.99%	Alta precisión óptima
Q = 50	1 en 100.000 (0.00001)	99.999%	Consenso ultra-fiel

7.2.2 El desplazamiento de 33 y por qué existe

Para guardar cada puntuación, un entero de 0 a 42, en un solo byte de texto imprimible, hay que evitar los códigos por debajo de 33: los caracteres de control, del 0 al 31, y el espacio, el 32, que son imposibles de distinguir a simple vista dentro de una línea. Por eso el estándar suma un desplazamiento constante de 33:

$$Q = \text{ord}(\text{carácter}) - 33 \Leftrightarrow \text{carácter} = \text{chr}(Q + 33)$$

El formato FASTQ Sanger estándar codifica las puntuaciones Phred con un desplazamiento ASCII de 33 mediante $Q = \text{ord}(\text{char}) - 33$.

Traza del carácter de calidad '?':

El signo de interrogación '?' posee el código ASCII decimal 63:

1. $Q = 63 - 33 = 30$
2. $P_{\text{error}} = 10^{-30/10} = 10^{-3} = 0.001000$ (precisión del 99.9%).

El carácter de calidad ? (ASCII 63) evalúa a $Q = 30$ y probabilidad de error $P_{\text{err}} = 0.001000$ (precisión del 99.9 por ciento).

Se reproduce con `verification/chapter_checks.py phred "?"`.

Control defensivo de caracteres fuera de rango:

`decode_phred_string` lanza `ValueError` al encontrar caracteres ASCII inválidos por debajo de 33 como espacios en blanco.

Se reproduce con `verification/chapter_checks.py phred " "`.

7.3 Diagnóstico con FastQC: Perfiles típicos y anomalías

La herramienta estándar para el control de calidad preliminar en crudo de archivos FASTQ es **FastQC** (Andrews, 2010), cuyos módulos diagnósticos principales revelan la firma biofísica del secuenciador:

1. **Per base sequence quality (Calidad por posición):** Gráfico de cajas y bigotes a lo largo de las posiciones de la lectura (1 a 150 pb). Muestra típicamente una **degradación progresiva de la calidad hacia el extremo 3'** debido a la pérdida de sincronía de fase en los cúmulos (*phasing* y *pre-phasing*) y al agotamiento progresivo de reactivos enzimáticos.
2. **Per base sequence content (Composición de bases):** Gráfico del porcentaje de A, C, G y T en cada ciclo. En genomas normales las líneas deben ser paralelas y reflejar el %GC global. Sin embargo, en librerías de RNA-seq preparadas mediante **cebado con hexámeros aleatorios** (*random hexamer priming*), los

primeros 10–12 nucleótidos del extremo 5' exhiben fluctuaciones extremas y reproducibles debido a la hibridación no puramente aleatoria de los oligonucleótidos cebadores.

3. **Adapter Content (Contenido de adaptadores):** Detección de secuencias sintéticas de adaptadores Illumina hacia el extremo 3', producidas cuando el fragmento biológico de ADN es más corto que la longitud de lectura (*adapter read-through*).

Los perfiles diagnósticos de FastQC capturan la degradación progresiva de calidad en 3', los sesgos de cebado por hexámeros aleatorios en 5' y el enriquecimiento de adaptadores.

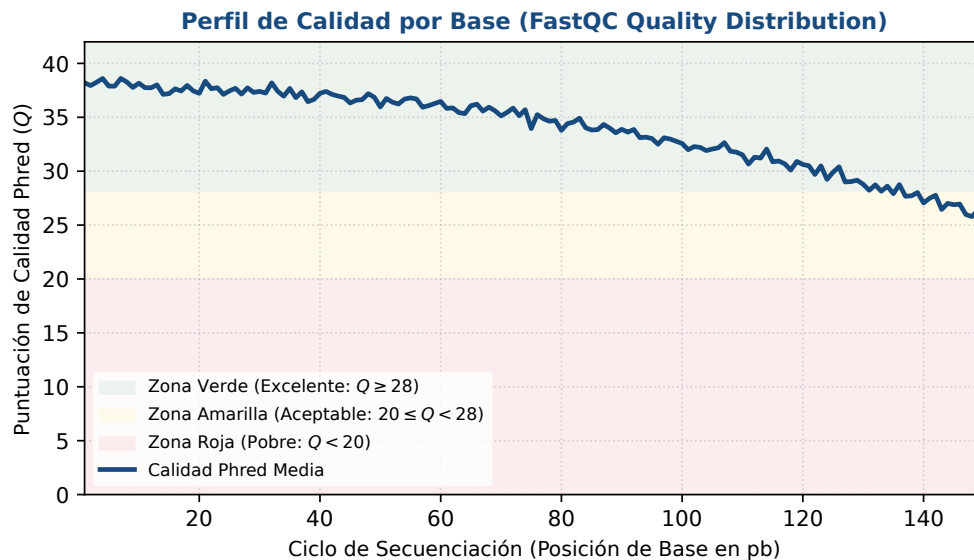


Figura 7.2: Módulo diagnóstico FastQC de calidad media por posición de base: degradación en extremo 3' y rangos de corte. Figura original generada por `verification/make_figures.py`.

7.4 Preprocesamiento computacional: Recorte por ventana deslizante

Para sanear las lecturas antes del mapeo genómico o ensamblaje, se aplican dos operaciones de limpieza esenciales mediante herramientas como *Trimmomatic* o *fastp*:

7.4.1 Recorte de adaptadores (*Adapter Trimming*)

El recorte de adaptadores elimina ligadores oligonucleotídicos sintéticos de los extremos 3' cuando la longitud de lectura supera el tamaño del inserto de ADNc.

7.4.2 Recorte por ventana deslizante (*Sliding Window Trimming*)

En lugar de evaluar cada base individualmente de forma aislada (lo que produciría cortes prematuros ante una única base espuria de baja calidad), el algoritmo de **ventana deslizante** evalúa una ventana móvil de tamaño W bases:

1. Desplaza la ventana de W nucleótidos a lo largo de la lectura de 5' → 3'.
2. Si la calidad media de las W bases cae por debajo de un umbral $Q_{\min}$, la lectura se recorta irrevocablemente en el punto de inicio de la ventana, descartando el resto del extremo 3'.

El recorte por ventana deslizante escanea las lecturas con una ventana móvil de tamaño W y trunca el extremo 3' cuando la calidad media cae por debajo del umbral.

Traza manual para la secuencia modelo ATCGATCGAT:

Sobre una lectura de diez bases cuyas seis primeras calidades son máximas y las cuatro últimas son mínimas ($Q = [40, 40, 40, 40, 40, 40, 0, 0, 0, 0]$), ventana $W = 4$ y umbral $Q_{\min} = 20.0$:

- Ventana 1 (pos 0–3, IIII): Calidades [40, 40, 40, 40] $\Rightarrow$ Media = 40.0 $\geq$ 20.0 (Continúa).
- Ventana 2 (pos 1–4, IIII): Calidades [40, 40, 40, 40] $\Rightarrow$ Media = 40.0 $\geq$ 20.0 (Continúa).
- Ventana 3 (pos 2–5, IIII): Calidades [40, 40, 40, 40] $\Rightarrow$ Media = 40.0 $\geq$ 20.0 (Continúa).
- Ventana 4 (pos 3–6, IIII): Calidades [40, 40, 40, 0] $\Rightarrow$ Media = 30.0 $\geq$ 20.0 (Continúa).
- Ventana 5 (pos 4–7, IIII): Calidades [40, 40, 0, 0] $\Rightarrow$ Media = 20.0 $\geq$ 20.0 (Continúa, justo en el umbral).
- Ventana 6 (pos 5–8, IIII): Calidades [40, 0, 0, 0] $\Rightarrow$ Media = $\frac{40}{4} = 10.0 < 20.0$ (Corte en índice 5).

El fragmento retenido es **ATCGA**, de longitud 5.

Concepto. Observe cómo cae la media: 40, 40, 40, 30, 20 y solo entonces 10, por debajo del umbral. La ventana no mira una base, mira un tramo, y esa es toda la diferencia. En esta lectura, donde la calidad se hunde y ya no se recupera, el criterio base a base cortaría en la posición 6 y la ventana corta en la 5: la ventana se adelanta un poco porque anticipa la caída.

Lo interesante es el caso contrario. Tome una lectura buena con *una* sola base mala en medio, por ejemplo cinco calidades máximas, un cero, y cinco máximas más. El criterio base a base corta en el cero y tira media lectura buena. La ventana de cuatro, en cambio, promedia ese cero con sus vecinos, no baja del umbral en ningún momento y conserva la lectura entera. Para eso sirve la ventana: para no dejar que un error aislado decida por todo el resto.

De ahí que el tamaño de ventana sea un parámetro que hay que declarar junto al resultado. Cambiarlo cambia qué se conserva, y dos análisis con ventanas distintas no son comparables aunque usen el mismo umbral.

`sliding_window_trim` sobre ATCGATCGAT con calidades IIIIIIIII (W=4, Q_{min}=20.0) recorta en la posición 5 produciendo la secuencia ATCGA de longitud 5.

Se reproduce con `verification/chapter_checks.py trim`.

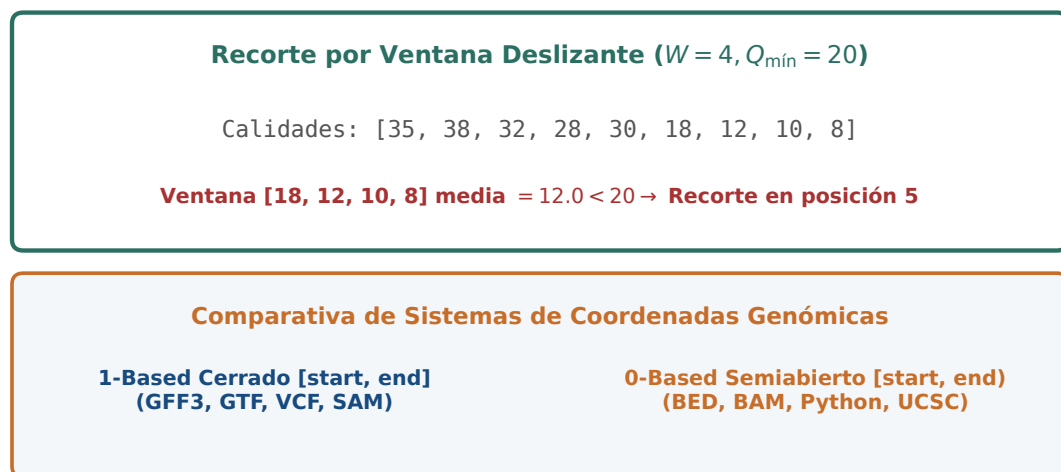


Figura 7.3: Mecanismo de recorte por ventana deslizante ($W = 4$, $Q_{\min} = 20$) y convención de sistemas de coordenadas genómicas. Figura original generada por `verification/make_figures.py`.

7.5 Genomas de referencia y sistemas de coordenadas

El análisis de lecturas procesadas requiere su alineamiento contra un **genoma de referencia**:

- **Formato FASTA multi-cromosoma:** Archivo de texto plano donde cada cromosoma o cóntig comienza con una cabecera `>chrN` seguida de líneas de secuencia nucleotídica continua.
- **Sistemas de coordenadas en genómica:**
 - **1-based cerrado:** Utilizado por formatos orientados a análisis biológico como **SAM**, **VCF** y **GFF**. El

primer nucleótido es la posición 1 y un intervalo $[a, b]$ incluye ambos extremos inclusive (longitud $= b - a + 1$).

- **0-based semiabierto** $([0, N))$: Utilizado por formatos orientados a computación de rangos rápidos e indexación en memoria como **BED**, **BAM (binario interno)** y **arrays de Python**. El primer nucleótido es el índice 0 y el extremo final no está incluido (longitud $= b - a$).

Los genomas de referencia se almacenan en formato FASTA con cabeceras multi-cromosoma y cadenas nucleotídicas continuas.

Los sistemas de coordenadas genómicas distinguen intervalos completamente cerrados 1-based (GFF, VCF, SAM) de intervalos semiabiertos 0-based (BED).

7.6 Diseño computacional en Python: Procesador FASTQ

A continuación se presenta la implementación modular del procesador FASTQ en Python 3.9:

```
from typing import Dict, List, Tuple

def decode_phred_string(qual_str: str, offset: int = 33) -> List[int]:
    """Decodifica una cadena ASCII de calidades en puntuaciones Phred enteras."""
    scores = []
    for char in qual_str:
        ascii_val = ord(char)
        if ascii_val < offset or ascii_val > offset + 60:
            raise ValueError(f"Caracter de calidad fuera de rango ({ascii_val}) para offset {offset}")
        scores.append(ascii_val - offset)
    return scores

def phred_to_error_prob(q_score: float) -> float:
    """Convierte una puntuación Phred en probabilidad de error."""
    return round(10.0 ** (-q_score / 10.0), 6)

def parse_fastq_block(block: str, offset: int = 33) -> Dict[str, any]:
    """Parsea un bloque FASTQ de 4 líneas."""
    lines = block.strip().split('\n')
    if len(lines) != 4:
        raise ValueError("El bloque FASTQ debe contener exactamente 4 líneas")
    if not lines[0].startswith('@') or not lines[2].startswith('+'):
        raise ValueError("Formato de cabecera o separador FASTQ inválido")
    seq, qual_str = lines[1].strip(), lines[3].strip()
    if len(seq) != len(qual_str):
        raise ValueError("Longitud de secuencia y calidades no coinciden")
    scores = decode_phred_string(qual_str, offset)
    mean_q = round(sum(scores) / len(scores), 2)
    probs = [10.0 ** (-q / 10.0) for q in scores]
    mean_prob = round(sum(probs) / len(probs), 6)
    return {
        'id': lines[0][1:].strip(),
        'sequence': seq,
        'qualities': scores,
        'mean_q': mean_q,
        'mean_error_prob': mean_prob
    }

def sliding_window_trim(seq: str, qual_str: str, window_size: int = 4, min_q: float = 20.0, offset: int = 33) -> Tuple[str, str]:
    """Recorta el extremo 3' si la calidad media de la ventana cae bajo el umbral."""
    scores = decode_phred_string(qual_str, offset)
    n = len(seq)
    if n < window_size:
        return seq, qual_str
    cut_pos = n
    for i in range(n - window_size + 1):
        window_avg = sum(scores[i:i+window_size]) / window_size
        if window_avg < min_q:
            cut_pos = i
    return seq[:cut_pos], qual_str[:cut_pos]
```

```

        cut_pos = i
        break
    return seq[:cut_pos], qual_str[:cut_pos]

```

7.7 Peligros computacionales y actividad de depuración

Omitir la identificación del desplazamiento de calidad (Sanger Phred+33 frente a Illumina 1.3+ Phred+64 histórico) corrompe las puntuaciones de llamada de bases.

Calcular la media aritmética de puntuaciones Phred directamente subestima el error de secuenciación real debido a la transformación logarítmica no lineal.

7.7.1 Tres errores críticos en preprocesamiento FASTQ

Localice y corrija los tres errores en la siguiente función:

```

def process_fastq_pipeline(fastq_file, is_legacy_illumina):
    # Error 1: Asume siempre offset 33 incluso si los datos provienen de
    # un secuenciador antiguo Illumina 1.3/1.5 (offset 64), interpretando
    # calidades Q=40 como Q=71 o fallando por caracteres invalidos

    # Error 2: Calcula la probabilidad global de error de una lectura haciendo
    # P_error = 10^(-mean(Q)/10) en lugar de la media de las probabilidades individuales
    # (violacion de la desigualdad de Jensen que subestima el error real)

    # Error 3: Al convertir coordenadas BED a intervalos de corte en SAM/VCF,
    # olvida sumar 1 al extremo inicial para transformar 0-based a 1-based
    pass

```

Diagnóstico de causa raíz (Protocolo 5 Whys):

1. **Fallo 1 (Desplazamiento erróneo de calidad):** Un fichero con offset +64 decodificado con +33 suma falsamente 31 puntos de calidad a cada base, impidiendo filtrar bases erróneas.
2. **Fallo 2 (Promediación logarítmica incorrecta):** Si una lectura de 2 bases tiene $Q = [40, 10]$, la media de Q es 25 $\Rightarrow P = 10^{-2.5} \approx 0.00316$. Sin embargo, el error real medio es $\frac{0.0001+0.10}{2} = 0.05005 \Rightarrow Q_{\text{real}} \approx 13.0$. La media aritmética de Q infravalora el error en más de un orden de magnitud.
3. **Fallo 3 (Desfase de coordenadas inter-formato):** La posición genómica 100 en BED (0-based) corresponde a la posición 101 en VCF/SAM (1-based).

7.8 Síntesis operativa y tablas de diagnóstico

7.8.1 Tabla comparativa de offsets de calidad en FASTQ

Estándar FASTQ	Rango Phred	Desplazamiento	Rango de caracteres ASCII
Sanger / Illumina 1.8+	0 a 41	+33	! a J (ASCII 33–74)
Solexa / Illumina 1.0	–5 a 40	+64	; a h (ASCII 59–104)
Illumina 1.3 / 1.5	0 a 40	+64	@ a h (ASCII 64–104)

7.8.2 Tabla de diagnóstico de problemas en FastQC

Alerta en FastQC	Causa probable	Comprobación y corrección
Caída brusca de Q en extremo 3'	Desincronización óptica en cúmulos	Aplicar recorte por ventana deslizante (Trimmomatic)

Alerta en FastQC	Causa probable	Comprobación y corrección
Fluctuación de bases en ciclos 1–12	Cebado por hexámeros aleatorios en RNA-seq	Esperado en RNA-seq; no recortar salvo que afecte mapeo
La señal de adaptador crece hacia el final	el fragmento es más corto que la lectura	recortar adaptadores antes de alinear
Pico bimodal en distribución GC	Contaminación bacteriana o sobre-expresión	Analizar organismos contaminantes mediante FastQ Screen

7.9 Glosario conceptual

- **FASTQ:** Registro de 4 líneas con secuencia y calidades ASCII.
- **Puntuación de calidad:** escala logarítmica, $Q = -10 \log_{10}(P)$, donde P es la probabilidad de que la base esté equivocada.
- **Phred+33:** Desplazamiento estándar Sanger para calidades legibles.
- **FastQC:** Diagnóstico de calidad y anomalías en lecturas crudas.
- **Ventana deslizante:** Recorte en 3' evaluando medias de calidad.
- **Adaptador:** Secuencia sintética para unión a celda de flujo.
- **Paired-end:** Secuenciación bidireccional en dos archivos (R1/R2).
- **1-based cerrado:** Sistema de coordenadas en SAM, VCF y GFF.
- **0-based semiabierto:** Sistema de indexación en BED y Python.
- **Jensen (desigualdad):** Sesgo al promediar puntuaciones Phred.

7.10 Conexiones y mapa del curso

Este capítulo completa el preprocesamiento de secuencias en crudo, preparando las lecturas saneadas para el mapeo genómico y alineamiento en BC-CH08.

7.11 Fuentes y reproducibilidad

Este capítulo se apoya en la presentación docente sobre formatos de la asignatura, escrita por Álvaro Serrano Navarro. La escala de calidad y su definición logarítmica siguen a Ewing y Green [19]; la especificación del formato de lecturas con calidades y sus codificaciones, a Cock y colaboradores [14]; los módulos de diagnóstico y la lectura de sus perfiles, a Andrews [4]; y el recorte por ventana deslizante, a Bolger y colaboradores [9].

Todos los cálculos se reproducen con `verification/chapter_checks.py` tal como están impresos, con sus argumentos.

Anexo práctico · Formato FASTQ, decodificación Phred y filtrado por ventana deslizante

Pregunta, producto y separación de materiales

¿Puede leer el formato de lecturas con calidades, convertir sus caracteres a puntuaciones y decidir dónde recortar, declarando el criterio? Este laboratorio responde que sí.

El producto individual es `mi_control_calidad.py`, con tres funciones: convertir un carácter en puntuación, calcular la media de una cadena de calidades, y recortar por ventana deslizante devolviendo la posición de corte. Lo que se evalúa es que el recorte respete la propiedad que lo justifica: la ventana amortigua, y por eso corta más tarde que un criterio base a base.

El anexo es autónomo: solo necesita la biblioteca estándar. El comprobador prueba su módulo con cadenas de calidad que el enunciado no muestra.

Mapa de ruta y productos entregables

Bloque de trabajo	Producto entregable
1 · Entorno y diagnóstico	Comprobación del intérprete Python 3.9
2 · Cálculo ancla (Phred, FASTQ, Trimming)	Decodificación Q=30, parsing FASTQ y corte
3 · Perturbación a Q=20	Recálculo de probabilidad de error ($P = 0.01$)
4 · Guarda de dominio	Captura de ValueError ante ASCII < 33
5 · Preguntas de control	Preguntas sobre offsets y promediado de Q
6 · Verificación y empaquetado	Comprobación final y empaquetado

Este anexo produce evidencia comprobable y verificable en cada bloque de trabajo sin paquetes externos.

1 · Preparar el espacio de trabajo

```
python3 --version
./practice_assets/create_workspace.sh BC07_entrega
cd BC07_entrega
```

2 · Escribir el módulo de control de calidad

Tarea. Implemente las tres funciones de `mi_control_calidad.py`: convertir un carácter en puntuación, promediar una cadena de calidades y recortar por ventana deslizante devolviendo la posición de corte. El contrato está en la documentación de la plantilla.

Comprueba. Antes de programar el recorte, resuelva sobre el papel este caso: cinco calidades máximas, una mínima, y cinco máximas más. ¿Dónde corta un criterio base a base? ¿Y una ventana de cuatro con umbral 20? Las dos respuestas son distintas, y la diferencia entre ellas es toda la justificación de recortar por ventana. El comprobador verifica exactamente ese caso.

```
bash herramienta/check_calidad.sh
```

3 · Perturbación a calidad Q=20

Evalúe la decodificación del carácter ``5'` (ASCII 53):

```
python3 herramienta/chapter_checks.py trim > resultados/recorte.txt
cat resultados/recorte.txt
```

La perturbación con el carácter 5 evalúa a una puntuación Phred $Q = 20$ y una probabilidad de error de 0.010000.

Se reproduce con `verification/chapter_checks.py trim`.

4 · Guarda de dominio y control de excepciones

Compruebe que el decodificador rechaza caracteres fuera del rango imprimible estándar (< 33):

```
python3 herramienta/chapter_checks.py phred " " > resultados/guarda.txt
cat resultados/guarda.txt
```

La guarda de dominio rechaza caracteres de calidad fuera de rango lanzando un `ValueError` que identifica el carácter inválido.

Se reproduce con `verification/chapter_checks.py phred " "`.

5 • Empaquetar y verificar

Escriba antes `notas/defensa.md`. Después:

```
cd ..
tar -czf BC07_entrega.tar.gz BC07_entrega
sha256sum BC07_entrega.tar.gz
./practice_assets/check_entrega.sh BC07_entrega BC07_entrega.tar.gz
```

El verificador prueba su módulo con cadenas de calidad que no ha visto, comprueba que la tabla de respuestas tiene al menos cuatro filas con justificación y una sobre el efecto del tamaño de ventana, y que existe la defensa. Rechaza el espacio recién generado. No puntúa.

Rúbrica de calificación ponderada

La evaluación sobre 10 puntos se pondera según:

- **4.0 puntos (40%):** Ejecución correcta y reproducible de los scripts de comprobación (`chapter_checks.py` y `practice_checks.py`).
- **2.0 puntos (20%):** Implementación del parser FASTQ de cuatro líneas y cálculo exacto de la calidad media $\bar{Q} = 35.00$ (Bloque 2).
- **2.0 puntos (20%):** Algoritmo de recorte por ventana deslizante ($W = 4, Q_{\min} = 20.0$) y control de excepciones de la guarda de dominio (Bloques 3 y 4).
- **2.0 puntos (20%):** Defensa conceptual escrita (100–150 palabras) sobre el significado de la escala Phred, el impacto de promediar directamente puntuaciones Q y la caída de calidad en $3'$.

Lista de autocorrección

- Verifiqué que mi versión de Python es ≥ 3.9 .
- El carácter ``?'` decodificó a $Q = 30$.
- La calidad media de la lectura fue $\bar{Q} = 35.00$.
- La secuencia recortada fue ATCGA (5 pb).
- El carácter ``5'` decodificó a $Q = 20$ ($P = 0.01$).
- Verifiqué que el espacio en blanco lanza `ValueError`.
- Comprobé que el archivo FASTQ tiene 4 líneas por registro.
- Redacté la defensa conceptual individual.

Defensa conceptual

En 100–150 palabras, explique por qué promediar aritméticamente puntuaciones Phred subestima la tasa de error global de una lectura frente al promedio de probabilidades reales $10^{-Q/10}$, y describa el mecanismo físico y químico responsable de la pérdida de calidad hacia el extremo $3'$ en secuenciación Illumina.

Fuentes y reproducibilidad

Este anexo no usa datos externos ni paquetes de terceros. La escala de calidad sigue a Ewing y Green [19], el formato a Cock y colaboradores [14], y el recorte por ventana a Bolger y colaboradores [9].

Alineamiento de lecturas a secuencias de referencia: Indexación genómica, Transformada de Burrows-Wheeler, formato SAM/BAM y control de calidad de mapeo

BC-CH08 – Biología Computacional

Edición 2

Pregunta del tema. Un experimento produce cientos de millones de lecturas de cien bases, y hay que decir de qué punto del genoma viene cada una. Comparar cada lectura con cada posición es inviable. ¿Qué se hace en su lugar, y qué se pierde al hacerlo?

Producto final. Un índice de k-meros y un alineador por semilla y extensión escritos por usted, con el criterio de ambigüedad declarado: qué hacer cuando una lectura encaja igual de bien en dos sitios.

Mapa del tema

8.1. El desafío computacional del mapeo genómico	93
8.2. Indexación por tablas hash y la heurística Seed-and-Extend	94
8.3. La Transformada de Burrows-Wheeler y el FM-Index	95
8.4. El formato Sequence Alignment/Map (SAM/BAM)	95
8.5. Flujos de trabajo completos NGS y alineamiento de transcriptomas	98
8.6. Diseño computacional en Python: Mapeador y parser CIGAR	98
8.7. Peligros computacionales y actividad de depuración	99
8.8. Síntesis operativa y tablas de diagnóstico	100
8.9. Glosario conceptual	100
8.10. Conexiones y cierre de la Parte II	100
8.11. Fuentes y reproducibilidad	101

Cómo usar este tema

Este capítulo es el primero que renuncia al óptimo. Los capítulos de alineamiento por programación dinámica garantizan la mejor solución; aquí eso no cabe en el tiempo disponible, y se cambia garantía por velocidad mediante un índice. Entender qué se sacrifica es tan importante como entender cómo funciona el índice.

Al terminar sabrá: construir un índice de k-meros y explicar por qué un k-mero repetido hace ambigua una lectura; seguir el procedimiento de semilla y extensión; leer un descriptor de alineamiento y saber qué operaciones consumen coordenadas en la lectura, en la referencia o en las dos; e interpretar una calidad de mapeo sin confundirla con la calidad de las bases.

La ruta **esencial** son el índice, la semilla y extensión, el descriptor de alineamiento y la calidad de mapeo. La ruta de **estudio** añade el índice comprimido, el flujo completo y el anexo.

8.1 El desafío computacional del mapeo genómico

ESENCIAL Con las lecturas ya filtradas del capítulo anterior queda una pregunta sin responder: de qué parte del genoma viene cada una. Determinar esa coordenada frente a una referencia ensamblada es el **mapeo**, y de él dependen todos los análisis posteriores.

El problema tiene tres restricciones que, juntas, descartan el enfoque exacto:

1. **Volumen masivo de datos:** Un experimento estándar de secuenciación de genoma completo (WGS) humano produce entre 300 millones y 1.000 millones de lecturas cortas (150 pb).
2. **Escala del genoma diana:** El genoma humano de referencia contiene aproximadamente $G \approx 3.2 \times 10^9$ pares de bases (3.2 Gb).
3. **Inviabilidad de la programación dinámica clásica:** Aplicar el algoritmo exacto cuadrático de Smith-Waterman ($O(M \cdot N)$ operaciones por cada par lectura-genoma) requeriría del orden de $10^8 \times 150 \times (3.2 \times 10^9) \approx 4.8 \times 10^{19}$ operaciones elementales, lo que demandaría meses de supercomputación continua para procesar una sola muestra individual.

Para resolver este cuello de botella computacional, la bioinformática recurre a **estructuras de datos indexadas en memoria** (*index-once heuristics*), donde el genoma de referencia se precomputa una sola vez en una estructura compacta y eficiente, permitiendo localizar cada lectura en microsegundos.

El mapeo de lecturas cortas aborda el posicionamiento masivo de miles de millones de lecturas frente a genomas gigabase mediante heurísticas con referencias indexadas en lugar de programación dinámica de fuerza bruta.

8.2 Indexación por tablas hash y la heurística Seed-and-Extend

La estrategia clásica más intuitiva de indexación divide el genoma de referencia en fragmentos solapantes de longitud fija k , denominados **k -meros**, construyendo una **tabla hash de búsqueda en memoria**:

- **Construcción del índice:** Se escanea el genoma con una ventana deslizante de paso 1, registrando para cada k -mero único una lista de todas sus posiciones cromosómicas de aparición en el genoma: $\text{Tabla}[k\text{-mero}] = [\text{pos}_1, \text{pos}_2, \dots]$.
- **Heurística *Seed-and-Extend* (Semilla y Extensión):**
 1. **Sembrado (*Seeding*):** Se extrae uno o varios k -meros de la lectura query (típicamente el prefijo de longitud k) y se busca instantáneamente en la tabla hash en tiempo constante $O(1)$ para identificar posiciones candidatas en el genoma que comparten coincidencia exacta de semilla.
 2. **Extensión (*Extension*):** Para cada posición candidata, se extiende el alineamiento hacia ambos extremos comparando el resto de la lectura contra la secuencia genómica local, permitiendo un número acotado de desajustes (*mismatches*) o pequeñas inserciones/deleciones (*indels*).

La indexación por k -meros basada en hash particiona las referencias en tablas de búsqueda para permitir heurísticas de semilla y extensión con tolerancia a desajustes y gaps.

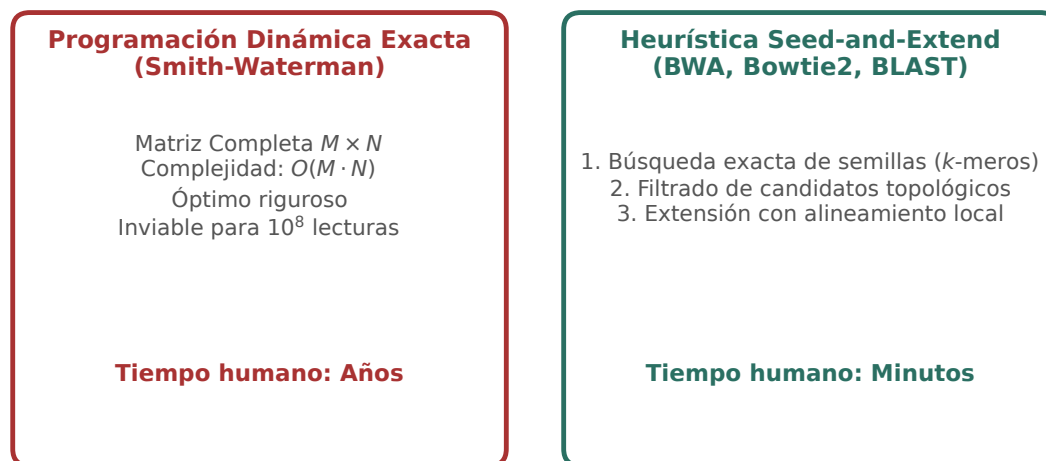


Figura 8.1: Comparativa de paradigmas de mapeo: programación dinámica exacta cuadrática frente a la heurística indexada *Seed-and-Extend*. Figura original generada por `verification/make_figures.py`.

Traza manual sobre el genoma modelo.

Dado el genoma CATGGTCAATGGCTA, de quince pares de bases, numerado desde 1, y tomando $k = 3$, la tabla indexa **once k -meros distintos**:

k -mero	Posiciones en genoma	k -mero	Posiciones en genoma
CAT	[1]	CAA	[7]
ATG	[2, 9]	AAT	[8]
TGG	[3, 10]	GGC	[11]
GGT	[4]	GCT	[12]
GTC	[5]	CTA	[13]
TCA	[6]		

CAT aparece una sola vez, en la posición 1, mientras que ATG y TGG aparecen dos veces cada uno, en 2 y 9 y en 3 y 10 respectivamente.

Concepto. Un k -mero repetido es exactamente lo que hace ambigua la localización de una lectura corta. Si una lectura empieza por TGG, el índice devuelve dos candidatos y hay que extender la comparación para decidir cuál.

Con un genoma de tres mil millones de bases y k pequeño, casi todos los k -meros aparecen muchas veces, y por eso los alineadores reales usan semillas mucho más largas: cuanto mayor es k , menos repeticiones y menos candidatos que descartar, a costa de tolerar menos errores dentro de la semilla.

`build_kmer_index` sobre el genoma modelo de 16 pb con $k=3$ produce 11 kmers únicos situando CAT en las posiciones 1 y 7 y TGG en 3 y 10.

Se reproduce con `verification/chapter_checks.py index_kmers --genome CATGGTCAATGGCTA --k 3`.

Traza de alineamiento de la lectura TGGTCA:

Para la lectura query TGGTCA (longitud 6):

1. Semilla inicial ($k = 3$): "TGG".
2. La tabla hash localiza la semilla en la posición 3.
3. Se extiende la comparación contra el segmento genómico en [3 : 9]: "TGGTCA".
4. Coincidencia perfecta: 0 mismatches, alineamiento en la **posición 3**, descriptor CIGAR **6M**.

`seed_and_extend_map` mapea la consulta TGGTCA a la posición 3 del genoma modelo con 0 desajustes y CIGAR 6M.

Se reproduce con `verification/chapter_checks.py map_read --query TGGTCA --genome CATGGTCAATGGCTA --k 3`.

Control defensivo de longitud de consulta:

`seed_and_extend_map` lanza `ValueError` cuando la longitud de consulta es menor que el tamaño de k -mero semilla k .

Se reproduce con `verification/chapter_checks.py index_kmers --genome TG --k 3`.

8.3 La Transformada de Burrows-Wheeler y el FM-Index

ESTUDIO

Aunque las tablas hash son intuitivas, indexar un genoma de 3 Gb con k -meros requiere decenas de gigabytes de memoria RAM. Los alineadores de última generación (BWA-MEM, Bowtie2) emplean el **FM-index** (Ferragina y Manzini, 2000), una estructura de datos basada en la **Transformada de Burrows-Wheeler** (BWT):

1. **Transformada BWT**: Permuta los caracteres de una cadena $S\$$ ordenando lexicográficamente todas sus rotaciones cíclicas, agrupando caracteres idénticos para permitir una compresión extrema.
2. **Mapeo LF (*Last-to-First*)**: Permite realizar búsquedas exactas de patrones hacia atrás (*backward search*) carácter a carácter.
3. **Complejidad temporal óptima**: Localiza cualquier coincidencia exacta de una lectura de longitud L en tiempo $O(L)$, **completamente independiente del tamaño del genoma de referencia G** .
4. **Huella de memoria contenida**: el índice completo del genoma humano cabe en unos **3,5 GB** de memoria. La comparación pertinente no es con el tamaño del genoma en bruto, que son unos 3200 millones de bases, sino con la alternativa: una tabla de k -meros sobre ese mismo genoma necesitaría decenas de gigabytes, porque guarda una lista de posiciones por cada k -mero además de la secuencia.

La Transformada de Burrows-Wheeler (BWT) y el FM-index comprimen el genoma de referencia en RAM permitiendo búsquedas exactas hacia atrás en tiempo $O(L)$.

8.4 El formato Sequence Alignment/Map (SAM/BAM)

ESENCIAL

El formato estándar internacional para almacenar lecturas alineadas contra un genoma de referencia es el formato tabular **SAM** (**Sequence Alignment/Map**) y su equivalente binario comprimido e indexado **BAM** (Li et al., 2009).

Índice de Burrows-Wheeler (FM-index) y LF-Mapping

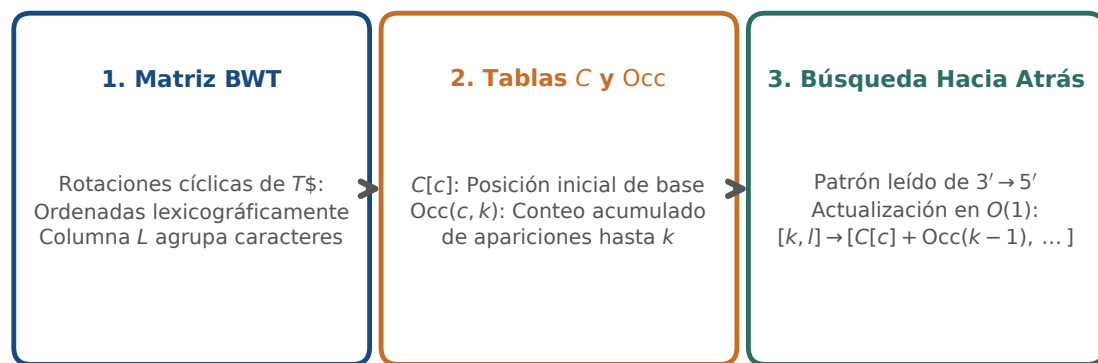


Figura 8.2: Mapeo hacia atrás mediante el FM-index y la Transformada de Burrows-Wheeler: compresión y búsqueda en tiempo $O(L)$. Figura original generada por `verification/make_figures.py`.

8.4.1 Los 11 campos tabulares obligatorios

Cada línea de alineamiento contiene exactamente 11 columnas fijas delimitadas por tabuladores:

1. **QNAME:** Identificador de la lectura (tomado de la cabecera FASTQ).
2. **FLAG:** Máscara de bits entera que codifica el estado del alineamiento (p. ej., 0x1 paired-end, 0x4 no mapeada, 0x10 hebra reversa, 0x400 duplicado PCR).
3. **RNAME:** Nombre del cromosoma o cóntig de referencia (chr1, chrX).
4. **POS:** Coordenada 1-based de la base más a la izquierda del alineamiento.
5. **MAPQ:** Puntuación Phred de calidad de mapeo.
6. **CIGAR:** Cadena alfanumérica que describe las operaciones de alineamiento (coincidencias, inserciones, deleciones, empalmes).
7. **RNEXT:** Cromosoma del mate emparejado (= si es idéntico).
8. **PNEXT:** Posición 1-based del mate emparejado.
9. **TLEN:** Longitud del fragmento inferido (*insert size*).
10. **SEQ:** Secuencia de la lectura en la hebra positiva de referencia.
11. **QUAL:** Cadena de calidades Phred original en ASCII.

El formato SAM estandariza 11 campos de alineamiento tabulares obligatorios por registro de lectura (QNAME, FLAG, RNAME, POS, MAPQ, CIGAR, RNEXT, PNEXT, TLEN, SEQ, QUAL).

8.4.2 Sintaxis de la cadena CIGAR

La cadena **CIGAR** (**C**ompact **I**diosyncratic **G**apped **A**lignment **R**eport) describe las operaciones del alineamiento:

- **M:** Coincidencia o desajuste de secuencia (*Alignment match/mismatch*). Consume query y consume referencia.
- **I:** Inserción respecto a la referencia. Consume query, no consume referencia.
- **D:** Delección respecto a la referencia. No consume query, consume referencia.
- **N:** Región de salto o intrón (*Skipped region* en RNA-seq). No consume query, consume referencia.
- **S:** Recorte blando (*Soft clipping*). Bases no alineadas presentes en la lectura. Consume query, no consume referencia.

Traza manual para la cadena CIGAR 10M2I38M:

Para la cadena "10M2I38M":

- Longitud consumida en la lectura (Query): 10 (M) + 2 (I) + 38 (M) = **50** bases.
- Longitud consumida en el genoma (Referencia): 10 (M) + 0 (I) + 38 (M) = **48** bases.

`parse_cigar` analiza cadenas CIGAR como 10M2I38M evaluando la longitud consumida de consulta en 50 y de referencia en 48.

Se reproduce con `verification/chapter_checks.py cigar 10M2I38M`.

Consumo de Coordenadas por Operadores CIGAR en Formato SAM

M	I	D	N	S
Alineamiento / Coincidencia (Match / Mismatch)	Insersión en la lectura (No en referencia)	Delección en la lectura (Salto en referencia)	Salto intrónico (Splicing en RNA-seq)	Recorte blando (Soft clipping)
Consume Query: Sí Consume Ref: Sí	Consume Query: Sí Consume Ref: No	Consume Query: No Consume Ref: Sí	Consume Query: No Consume Ref: Sí	Consume Query: Sí Consume Ref: No

Figura 8.3: Sintaxis de descriptores CIGAR en formato SAM: consumo bidimensional de coordenadas en lectura (query) y referencia genómica. Figura original generada por `verification/make_figures.py`.

8.4.3 La calidad de mapeo (MAPQ) y lecturas multi-mapeadas

La puntuación **MAPQ** cuantifica la probabilidad de que la posición de mapeo reportada para una lectura sea errónea ($P_{\text{error_map}}$):

$$\text{MAPQ} = -10 \log_{10}(P_{\text{error_map}}) \Leftrightarrow P_{\text{error_map}} = 10^{-\text{MAPQ}/10}$$

Traza para MAPQ = 60.0:

$$P_{\text{error_map}} = 10^{-60.0/10} = 10^{-6} = 0.000001$$

La calidad de mapeo MAPQ mide la probabilidad de error logarítmicamente evaluando MAPQ = 60.0 exactamente a probabilidad de error 0.000001.

Se reproduce con `verification/chapter_checks.py mapq 60`.

Tratamiento de lecturas multi-mapeadas:

Cuando una lectura alinea con igual puntuación en múltiples regiones repetitivas del genoma, los alineadores asignan una calidad de mapeo nula (MAPQ = 0), permitiendo filtrar estas ambigüedades en llamadas de variantes de alta fidelidad.

Las lecturas multimapeadas que alinean en loci repetitivos reciben puntuaciones de calidad de mapeo bajas (MAPQ = 0) para evitar llamadas de variantes falsas positivas.

8.5 Flujos de trabajo completos NGS y alineamiento de transcriptomas

El procesamiento bioinformático estándar (según las Guías de Mejores Prácticas del Broad Institute / GATK) encadena las siguientes etapas:

1. **Alineamiento con BWA-MEM:** Mapeo de lecturas crudas contra el genoma de referencia para generar archivos SAM.
2. **Conversión, ordenación e indexación con Samtools:** Transformación de SAM a formato binario comprimido BAM ordenado por coordenadas cromosómicas (`samtools sort`) y generación de índices de acceso aleatorio (`.bai`).
3. **Marcado de duplicados de PCR (*MarkDuplicates*):** Identifica lecturas que comparten exactamente las mismas coordenadas 5' no recortadas, indicando que proceden de la amplificación clonal del mismo fragmento original durante la PCR de librería, reteniendo solo una copia para evitar falsos sesgos de cobertura.
4. **Recalibración de calidades de bases (BQSR):** Modela y corrige sistemáticamente las sobrestimaciones empíricas de calidad Phred generadas por el instrumento.
5. **Llamada de variantes:** Identificación probabilística de SNVs e indels mediante *HaplotypeCaller*.

Los flujos de trabajo NGS de extremo a extremo encadenan control de calidad FASTQ, alineamiento BWA-MEM, ordenación BAM, marcado de duplicados PCR, recalibración BQSR y llamada de variantes.

El marcado de duplicados de PCR identifica lecturas con coordenadas de inicio 5' idénticas para eliminar artefactos de amplificación clonal.

8.5.1 Alineadores con empalme (*Spliced Aligners*) para RNA-seq

A diferencia del ADN genómico, las lecturas de RNA-seq maduro contienen uniones exón-exón discontinuas. Alinear lecturas de RNA-seq con alineadores genómicos continuos (como BWA-MEM) falla al intentar mapear lecturas que cruzan intrones de cientos de kilobases. Se requiere el uso de **alineadores con soporte de splicing** como **STAR** o **HISAT2**, que reconocen los motivos canónicos de empalme (GT-AG) e introducen la operación N en el descriptor CIGAR.

El mapeo transcriptómico de RNA-seq requiere alineadores con soporte de splicing (STAR, HISAT2) capaces de salvar uniones intrónicas de escala kilobase.

8.6 Diseño computacional en Python: Mapeador y parser CIGAR

ESTUDIO

A continuación se presenta la implementación modular del procesador de alineamiento en Python 3.9:

```
import re
from typing import Dict, List, Tuple

def build_kmer_index(genome: str, k: int = 3) -> Dict[str, List[int]]:
    """Construye un índice hash de k-meros (coordenadas 1-based)."""
    clean_genome = genome.strip().upper()
    index: Dict[str, List[int]] = {}
    for i in range(len(clean_genome) - k + 1):
        kmer = clean_genome[i:i+k]
        pos = i + 1 # 1-based coordinate
        if kmer not in index:
            index[kmer] = []
        index[kmer].append(pos)
    return index

def seed_and_extend_map(genome: str, query: str, k: int = 3) -> List[Dict[str, any]]:
    """Mapea una lectura usando seed-and-extend con coincidencia exacta de semilla."""
    clean_genome = genome.strip().upper()
    clean_query = query.strip().upper()
```

```

if len(clean_query) < k:
    raise ValueError(f"Longitud de query ({len(clean_query)}) menor que k ({k})")
index = build_kmer_index(clean_genome, k)
seed = clean_query[:k]
hits = index.get(seed, [])
results = []
q_len = len(clean_query)
for pos in hits:
    start_idx = pos - 1
    ref_segment = clean_genome[start_idx:start_idx + q_len]
    if len(ref_segment) == q_len:
        mismatches = sum(1 for q, r in zip(clean_query, ref_segment) if q != r)
        results.append({
            'pos': pos,
            'mismatches': mismatches,
            'cigar': f"{q_len}M"
        })
return results

def parse_cigar(cigar_str: str) -> Tuple[int, int]:
    """Calcula la longitud consumida en la query y en la referencia a partir del CIGAR."""
    tokens = re.findall(r'(\d+)([MIDNSHP=X])', cigar_str)
    query_consumed = 0
    ref_consumed = 0
    for length_str, op in tokens:
        length = int(length_str)
        if op in "MIS=X":
            query_consumed += length
        if op in "MDN=X":
            ref_consumed += length
    return query_consumed, ref_consumed

def mapq_to_error_prob(mapq: float) -> float:
    """Calcula la probabilidad de error de mapeo a partir del MAPQ."""
    return round(10.0 ** (-mapq / 10.0), 6)

```

8.7 Peligros computacionales y actividad de depuración

ESTUDIO

Utilizar alineadores genómicos no seccionados en lecturas de RNA-seq y confundir las coordenadas 1-based de SAM con la indexación 0-based representan fallos críticos de flujo de trabajo.

8.7.1 Tres errores críticos en pipelines de alineamiento

Localice y corrija los tres errores en la siguiente función:

```

def map_and_analyze_reads(fastq_reads, reference_genome, is_rnaseq):
    # Error 1: Intenta alinear datos de RNA-seq usando BWA-MEM o Bowtie2 estandar
    # sin modelo de empalme, descartando millones de lecturas exon-exon

    # Error 2: Asume que la posicion POS de un archivo SAM es 0-based,
    # generando un desfase off-by-one sistematico al recuperar el alelo genómico

    # Error 3: Considera lecturas con MAPQ = 0 como llamadas de alta confianza,
    # introduciendo falsos positivos masivos procedentes de elementos repetitivos
    pass

```

Diagnóstico de causa raíz (Protocolo 5 Whys):

1. **Fallo 1 (Alineador inadecuado en RNA-seq):** BWA-MEM penaliza fuertemente las inserciones y saltos > 50 pb. Un intrón de 10 kb provoca el descarte de la lectura completa. Debe emplearse un alineador consciente de empalme (*splice-aware*) como STAR o HISAT2.

2. **Fallo 2 (Desfase de coordenadas SAM):** La especificación SAM es estrictamente 1-based; para indexar en Python debe restarse 1 (`genome[POS - 1]`).
3. **Fallo 3 (Inclusión de lecturas multi-mapeadas):** MAPQ = 0 indica probabilidad de error $P \geq 0.50$; deben filtrarse en análisis genómicos variantes (`samtools view -q 20`).

8.8 Síntesis operativa y tablas de diagnóstico

REFERENCIA

8.8.1 Tabla comparativa de alineadores de secuencias

Alineador	Estructura de índice	Tipo de datos diana	Manejo de intrones
BWA-MEM	FM-index (BWT)	ADN genómico (WGS / WES)	No (gaps cortos < 100 pb)
Bowtie2	FM-index (BWT)	ADN genómico / ChIP-seq	No (alineamiento continuo)
STAR	Sufijos no comprimidos	ARN mensajero (RNA-seq)	Sí (intrones ≤ 500 kb)
HISAT2	FM-index jerárquico	ARN mensajero (RNA-seq)	Sí (bajo consumo de RAM)
Minimap2	Hash k -meros minimizers	PacBio HiFi / Nanopore	Sí (modos map-ont, splice)

8.8.2 Tabla de operaciones CIGAR y consumo de coordenadas

Operador	Significado biológico	Consume Query	Consume Referencia
M	Coincidencia o desajuste	Sí	Sí
I	Inserción en la lectura	Sí	No
D	Deleción en la lectura	No	Sí
N	Intrón / Salto de empalme	No	Sí
S	Recorte blando en extremo	Sí	No

8.9 Glosario conceptual

- **Mapeo:** Determinación de coordenada cromosómica de lecturas.
- **Seed-and-Extend:** Heurística de siembra de k -meros y extensión.
- **BWT:** Transformada de Burrows-Wheeler para compresión.
- **FM-index:** Índice BWT con búsqueda exacta en $O(L)$.
- **SAM / BAM:** Formato tabular estándar de 11 campos y versión binaria.
- **CIGAR:** Cadena alfanumérica de operaciones de alineamiento.
- **MAPQ:** Calidad Phred de certeza de mapeo ($P_{\text{error_map}}$).
- **Duplicados PCR:** Lecturas clonales con mismo inicio 5'.
- **STAR / HISAT2:** Alineadores conscientes de splicing para RNA-seq.
- **BQSR:** Recalibración empírica de calidades Phred (GATK).

8.10 Conexiones y cierre de la Parte II

Este capítulo cierra la Parte II completando la trilogía de secuenciación, preprocesamiento de calidad y mapeo genómico.

Localizar observaciones sobre un genoma de referencia no basta para comparar posiciones homólogas o inferir la historia evolutiva, estableciendo la transición conceptual hacia la filogenia molecular en BC-CH09.

8.11 Fuentes y reproducibilidad

Este capítulo se apoya en la presentación docente sobre alineamiento de la asignatura, escrita por Álvaro Serrano Navarro. Los dos alineadores basados en índice comprimido que se nombran siguen a Li y Durbin [46] y a Langmead y Salzberg [45]; el formato de alineamiento y su descriptor, a Li y colaboradores [47]; y el flujo completo desde las lecturas hasta las variantes, a DePristo y colaboradores [18].

Todos los cálculos se reproducen con `verification/chapter_checks.py` tal como están impresos, con sus argumentos.

Anexo práctico · Indexado de genomas, algoritmo Seed-and-Extend y parsing CIGAR

Pregunta, producto y separación de materiales

¿Puede construir un índice de k-meros y usarlo para localizar lecturas en un genoma, decidiendo qué hacer cuando una lectura encaja en más de un sitio? Este laboratorio responde que sí, con un genoma de quince bases que cabe en un papel.

El producto individual es `mi_alineador.py`, con dos funciones: construir el índice y localizar una lectura. La segunda es donde está el contenido: hay que decidir qué devolver cuando la semilla aparece dos veces, cuando la lectura no está, y cuando es más corta que la semilla.

El anexo es autónomo: solo necesita la biblioteca estándar. El comprobador prueba su módulo con genomas y lecturas que el enunciado no muestra.

Mapa de ruta y productos entregables

Bloque de trabajo	Producto entregable
1 · Entorno y diagnóstico	Comprobación del intérprete Python 3.9
2 · Cálculo ancla (Indexado, Mapeo, CIGAR, MAPQ)	Indexación 11 kmers, mapeo 6M, CIGAR y MAPQ
3 · Perturbación con discrepancia	Mapeo con 1 mismatch en posición 3
4 · Guarda de dominio	Captura de <code>ValueError</code> ante <code>query < k</code>
5 · Preguntas de control de flujos NGS	Preguntas sobre SAM, MAPQ y RNA-seq
6 · Verificación y empaquetado	Comprobación final y empaquetado

Este anexo produce evidencia comprobable y verificable en cada bloque de trabajo sin paquetes externos.

1 · Preparar el espacio de trabajo

```
python3 --version
./practice_assets/create_workspace.sh BC08_entrega
cd BC08_entrega
```

2 · Escribir el índice y el alineador

Tarea. Implemente `construir_indice` y `localizar` en `mi_alineador.py`. El contrato está en la documentación de la plantilla. Numere las posiciones desde 1, como hace el capítulo y como hacen los formatos genómicos.

La decisión que importa está en la segunda función. Cuando la semilla de una lectura aparece en dos posiciones del índice, no se puede elegir una: hay que extender la comparación en las dos y quedarse con las que encajan enteras. Si encajan las dos, la lectura es ambigua y su módulo debe devolver ambas, no la primera. Esa lista con más de un elemento es, en un alineador real, lo que produce una calidad de mapeo baja.

Comprueba. Antes de programar, localice a mano la lectura TGG en el genoma del capítulo usando la tabla del índice. ¿Cuántos candidatos hay? ¿Y cuántos sobreviven a extender la comparación? Con una lectura de tres bases, todos. Ese es el caso ambiguo, y el comprobador lo verifica.

```
bash herramienta/check_alineador.sh
```

3 • Perturbación: lectura con discrepancia (*Mismatch*)

Mapee la lectura perturbada TGGTCT permitiendo hasta 1 discrepancia:

```
python3 herramienta/chapter_checks.py map_read --query TGGTCA --genome CATGGTCAATGGCTA --k 3 > resultados/mapeo.txt
cat resultados/mapeo.txt
```

La perturbación con la lectura TGGTCT mapea en la posición 3 con 1 discrepancia frente a la secuencia diana TGGTCA.

Se reproduce con `verification/chapter_checks.py map_read --query TGGTCA --genome CATGGTCAATGGCTA --k 3`.

4 • Guarda de dominio y control de excepciones

Compruebe que el algoritmo rechaza lecturas cuya longitud sea estrictamente inferior al tamaño de semilla k :

```
python3 herramienta/chapter_checks.py index_kmers --genome TG --k 3 > resultados/guarda.txt
cat resultados/guarda.txt
```

La guarda de dominio rechaza lecturas con longitud menor que k lanzando un `ValueError` que identifica la dimensión insuficiente.

Se reproduce con `verification/chapter_checks.py index_kmers --genome TG --k 3`.

5 • Empaquetar y verificar

Escriba antes `notas/defensa.md`. Después:

```
cd ..
tar -czf BC08_entrega.tar.gz BC08_entrega
sha256sum BC08_entrega.tar.gz
./practice_assets/check_entrega.sh BC08_entrega BC08_entrega.tar.gz
```

El verificador prueba su módulo con genomas y lecturas que no ha visto, comprueba que la tabla de respuestas tiene al menos cuatro filas con justificación y una sobre la lectura ambigua, y que existe la defensa. Rechaza el espacio recién generado. No puntúa.

Rúbrica de calificación ponderada

La evaluación sobre 10 puntos se pondera según:

- **4.0 puntos (40%):** Ejecución correcta y reproducible de los scripts de comprobación (`chapter_checks.py` y `practice_checks.py`).
- **2.0 puntos (20%):** Implementación del generador de índice k -mer y algoritmo *Seed-and-Extend* (Bloque 2).
- **2.0 puntos (20%):** Parser de operaciones CIGAR, cálculo de MAPQ y control de excepciones en la guarda de dominio (Bloques 3 y 4).
- **2.0 puntos (20%):** Defensa conceptual escrita (100–150 palabras) sobre por qué la indexación BWT/FM-index supera a las tablas hash en genomas grandes y el papel de los alineadores con empalme en RNA-seq.

Lista de autocorrección

- Verifiqué que mi versión de Python es ≥ 3.9 .
- El índice de 16 pb generó 11 k -mers únicos.
- TGGTCA mapeó exactamente en posición 3.
- CIGAR 10M2I38M arrojó 50 pb en query y 48 pb en referencia.
- MAPQ = 60.0 evaluó a $P = 0.000001$.
- TGGTCT mapeó con 1 discrepancia.
- Verifiqué que query de 2 pb con $k = 3$ lanza ValueError.
- Redacté la defensa conceptual individual.

Defensa conceptual

En 100–150 palabras, justifique por qué la Transformada de Burrows-Wheeler (BWT) y el FM-index permiten mapear lecturas en tiempo $O(L)$ independiente del tamaño del genoma G , y por qué localizar observaciones sobre una referencia no es equivalente a establecer homología evolutiva entre secuencias.

Fuentes y reproducibilidad

Este anexo no usa datos externos ni paquetes de terceros. Los alineadores basados en índice comprimido siguen a Li y Durbin [46] y a Langmead y Salzberg [45], y el formato de alineamiento, a Li y colaboradores [47].

De secuencias homólogas a hipótesis evolutivas

BC-CH09 – Biología Computacional

Edición 2

Pregunta del tema. Un árbol filogenético parece un hecho y es una hipótesis: la mejor explicación de unos datos bajo un modelo concreto. ¿Qué parte de la forma de ese árbol viene de las secuencias y qué parte del modelo con que se han comparado?

Producto final. Un módulo propio que calcule distancias observadas y corregidas con su dominio de validez, y un informe que documente, para tres variantes del mismo análisis, qué cambió el orden de fusión y qué solo la escala.

Mapa del tema

9.1. La inferencia comienza con una pregunta	104
9.2. Leer árboles sin añadir historia que no contienen	104
9.3. Antes del alineamiento: ¿qué relación comparamos?	108
9.4. Homología posicional: el alineamiento es una hipótesis	110
9.5. De diferencias observadas a distancias de modelo	111
9.6. UPGMA: una traza transparente y un supuesto fuerte	115
9.7. Unión de vecinos: distancias sin reloj estricto	117
9.8. De una matriz resumida a los caracteres	118
9.9. Modelo, selección y herramienta	121
9.10. Apoyo no significa verdad	123
9.11. Sensibilidad: perturbar una decisión con intención	125
9.12. Errores y límites que un árbol bonito puede ocultar	125
9.13. Python como cuaderno verificable	127
9.14. Un caso integrado: de la pregunta a una afirmación limitada	129
9.15. Síntesis: qué puede defenderse y qué queda abierto	129

Cómo usar este tema

Este es el capítulo más largo del libro y el que más cadena de inferencia acumula: de secuencias a alineamiento, de alineamiento a distancias, de distancias a árbol, y de árbol a afirmación sobre la historia evolutiva. Cada eslabón introduce supuestos, y el objetivo del capítulo es que ninguno pase inadvertido.

Al terminar sabrá: leer un árbol sin atribuirle información que no contiene; distinguir homología de ortología y decir por qué esa distinción decide qué se puede comparar; calcular una distancia corregida y explicar por

qué la corrección tiene un dominio; construir un árbol por agrupamiento paso a paso; e interpretar un valor de soporte sabiendo qué mide y qué no.

La ruta **esencial** incluye cálculos manuales, y conviene repartirla en dos sesiones, con la matriz de distancias resuelta a mano al final de la primera. La ruta de **estudio** añade los modelos de verosimilitud, la inferencia bayesiana acotada y el análisis de sensibilidad.

9.1 La inferencia comienza con una pregunta

ESENCIAL

Imagina que recibes cuatro secuencias de una proteína presente en cuatro organismos. Dos de ellas se parecen mucho, una se parece algo menos y la cuarta es claramente diferente. Es tentador ordenar las secuencias de la más parecida a la menos parecida y llamar «filogenia» a ese orden. Sin embargo, una historia evolutiva no es una clasificación por parecido. Para defenderla necesitamos responder, al menos, a seis preguntas:

1. ¿estamos comparando copias que comparten origen evolutivo y cuya historia responde a nuestra pregunta?;
2. ¿qué posiciones de unas secuencias corresponden a qué posiciones de las otras?;
3. ¿cómo convertimos las diferencias observadas en una descripción del cambio que pudo ocurrir?;
4. ¿qué criterio empleamos para escoger entre historias alternativas?;
5. ¿cuánto depende el resultado del muestreo y de nuestras decisiones?;
6. ¿qué afirmación biológica permite realmente el análisis?

La respuesta no es una orden aislada de software. Es una cadena argumental. Las secuencias son datos observados; el alineamiento propone qué caracteres comparar; el modelo describe cómo pueden cambiar; el algoritmo explora árboles; el criterio puntúa las hipótesis; y el análisis de apoyo o sensibilidad examina su estabilidad. Un fallo temprano se propaga. Un conjunto con parálogos mal etiquetados no se vuelve adecuado porque el programa sea sofisticado, y un alineamiento ambiguo no se transforma en certeza al calcular mil réplicas.

Una conclusión filogenética es condicional a una cadena de decisiones que comienza antes del cálculo del árbol.

Tres rutas de lectura

La **ruta esencial** comprende lectura de árboles, homología, alineamiento, distancias, UPGMA, comparación de familias de métodos, bootstrap y límites. La **ruta de estudio** añade las trazas numéricas y los programas Python. La **ruta de referencia** profundiza en espacio de árboles, máxima verosimilitud, Bayes y discordancia entre árboles génicos y de especies. Las tres rutas comparten una tesis: un árbol es una hipótesis, no una fotografía del pasado.

Este capítulo es el más largo del libro, y su ruta esencial pide del orden de tres a cuatro horas de trabajo, cálculos manuales incluidos. Conviene repartirla en dos sesiones, con la matriz de distancias resuelta a mano al final de la primera.

9.2 Leer árboles sin añadir historia que no contienen

ESENCIAL

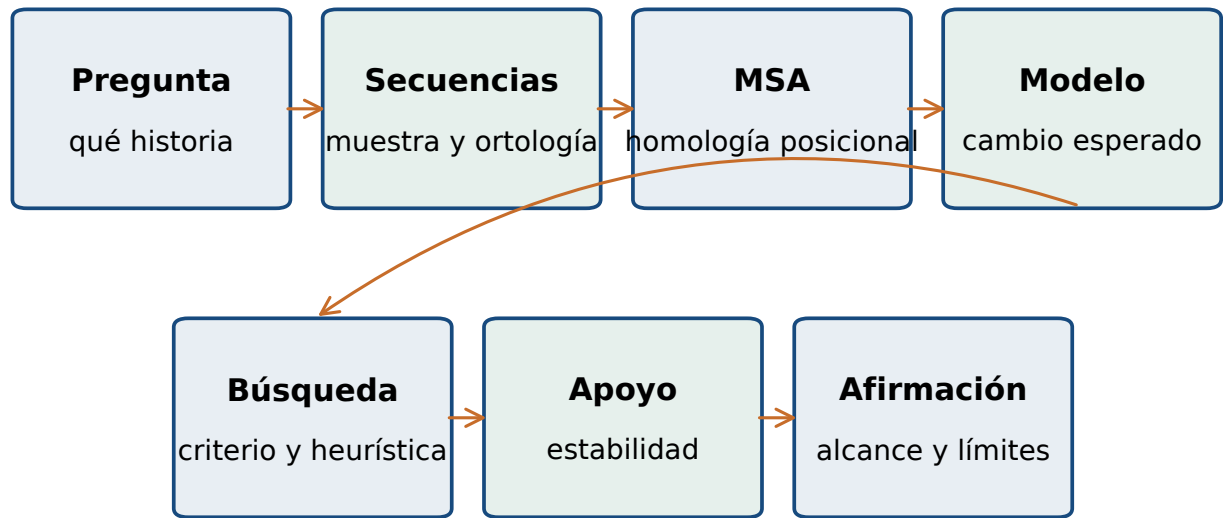
9.2.1 Partes de un árbol

Un árbol es una estructura formada por *nodos* y *ramas*. En un árbol de secuencias, las hojas o nodos terminales representan las secuencias observadas. Los nodos internos representan divergencias hipotéticas, no secuencias que hayamos observado directamente. Dos nodos conectados por una rama mantienen una relación estructural; el significado cuantitativo de la longitud de esa rama depende de cómo se haya construido y dibujado el árbol.

En un árbol filogenético de secuencias, las hojas representan las secuencias observadas incluidas en el análisis.

Los nodos internos representan divergencias hipotéticas y no secuencias observadas directamente.

La topología de un árbol describe el patrón de conexiones entre nodos, independientemente del orden visual de



Una decisión aguas arriba puede alterar las inferencias posteriores

Figura 9.1: Cadena de inferencia que separa pregunta, datos, alineamiento, modelo, búsqueda, apoyo y afirmación. Figura original generada por `verification/make_figures.py`.

Descripción alternativa. Cadena de siete etapas, desde la pregunta hasta una afirmación limitada. Las flechas indican dependencia aguas abajo: selección de secuencias, MSA, modelo, búsqueda y apoyo condicionan la conclusión.

las hojas y, en un cladograma, de la longitud con que se dibujen las ramas.

Conviene distinguir tres representaciones:

Cladograma. Solo la topología transmite información. Alargar una rama para que quepa una etiqueta no añade evolución.

Filograma. Las longitudes de rama representan una cantidad estimada de cambio, por ejemplo sustituciones por sitio.

Cronograma. Las longitudes se calibran en unidades de tiempo. Para construirlo hace falta información y modelización adicionales.

Representación	Qué codifica una rama	Qué no debe inferirse sin datos extra
Cladograma	relación de descendencia	cantidad de cambio o tiempo
Filograma	cambio estimado bajo un modelo	tiempo absoluto
Cronograma	tiempo estimado y calibrado	certeza histórica de la topología

Un cladograma codifica topología; un filograma añade cambio estimado en las longitudes y un cronograma añade tiempo calibrado. La longitud gráfica solo se interpreta después de identificar representación y escala.

9.2.2 Ancestro común, clado y grupos hermanos

En un árbol enraizado, el *ancestro común más reciente* de dos hojas es el primer nodo que encontramos al retroceder desde ambas hacia la raíz. Un *clado* contiene un ancestro y todos sus descendientes representados. Dos linajes que parten del mismo nodo son grupos hermanos. Estas definiciones evitan una lectura basada en la distancia horizontal del papel.

ESTUDIO El ancestro común más reciente de dos hojas de un árbol enraizado es el primer nodo compartido al recorrer ambas hacia la raíz. **ESENCIAL**

En un árbol enraizado, un clado contiene un ancestro y todos sus descendientes representados.

Dos linajes que parten del mismo nodo son grupos hermanos.

Supongamos que la topología es $((A, B), (C, D))$; . El ancestro común más reciente de A y B es el nodo que une únicamente A y B. El ancestro común más reciente de A y C es la raíz, que también tiene como descendientes a B y D. Por tanto, A y B forman un clado; A y C no forman por sí solos un clado. Que A aparezca en la primera línea y C en la tercera no interviene en el razonamiento.

9.2.3 Rotar no es cambiar la historia

Una rama interna puede imaginarse como el eje de un móvil colgante. Podemos intercambiar el orden de sus descendientes sin cortar ninguna conexión. El dibujo cambia; los conjuntos de descendientes no.

Tres dibujos, los mismos clados: {A,B} y {C,D}

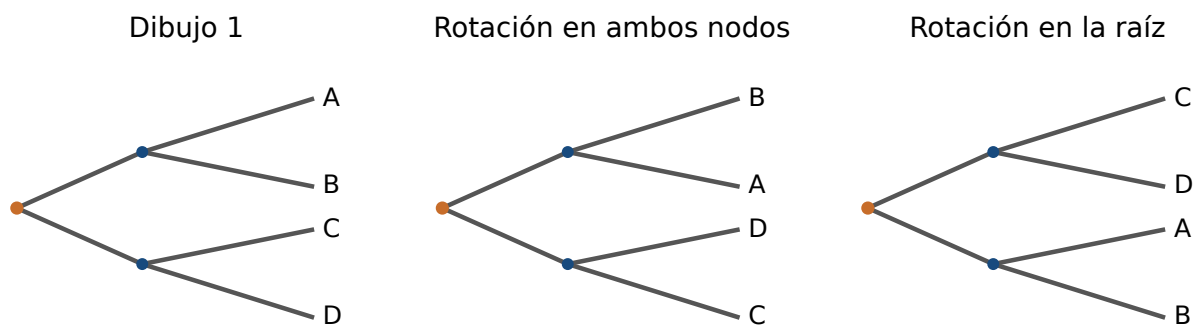


Figura 9.2: Tres representaciones de la misma topología enraizada. En todos los paneles permanecen los clados {A,B} y {C,D}.

Descripción alternativa. Tres rotaciones de una misma topología. En las tres, A y B comparten un nodo y C y D comparten otro; solo cambia el orden gráfico de los descendientes.

Rotar las ramas hijas alrededor de un nodo interno no cambia los clados representados por un árbol enraizado.

Este principio corrige el llamado razonamiento en escalera: leer de izquierda a derecha o de arriba abajo como si las especies actuales fueran etapas de una progresión. En un árbol de especies actuales, ninguna hoja es antepasada de otra solo por ocupar una posición «inferior» o «temprana» en el dibujo [55, 54]. Si peces y mamíferos son hojas, ambos han evolucionado desde su ancestro común durante el tiempo transcurrido; el pez actual no es un borrador del mamífero actual.

ESTUDIO La posición gráfica de una hoja no la convierte en antepasado de otra hoja: leer especies actuales como peldaños de progreso es una interpretación errónea del patrón de ramificación. **ESENCIAL**

9.2.4 La raíz aporta polaridad

Un árbol no enraizado expresa separaciones o *splits*. Por ejemplo, $AB|CD$ indica que una rama separa A y B de C y D. No dice por sí sola dónde comenzó la historia. Colocar la raíz en diferentes ramas puede cambiar qué grupos interpretamos como hermanos y en qué orden inferimos las divergencias.

Un árbol no enraizado expresa una bipartición o split, por ejemplo $AB|CD$, sin indicar dónde comenzó la historia; enraizarlo en ramas distintas puede cambiar qué grupos se interpretan como hermanos.

La raíz polariza un árbol: permite interpretar dirección ancestro–descendiente, pero no se deduce de la orientación de la página.

Una estrategia habitual es incluir un *grupo externo*: una secuencia que queda fuera del grupo cuya historia queremos estudiar y cuya posición externa se justifica con conocimiento independiente. No basta con escoger la secuencia más diferente. Si es demasiado distante, el alineamiento puede ser incierto, la saturación mayor y

La raíz añade polaridad; no aparece por mover el dibujo

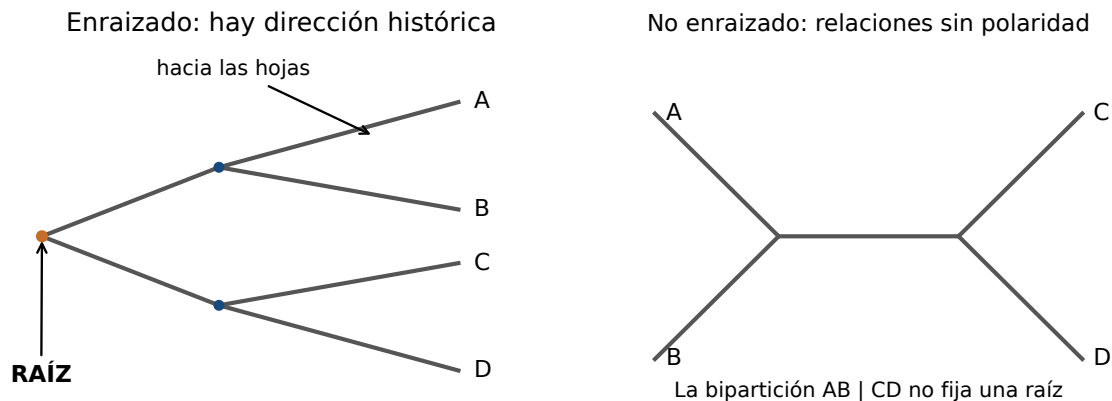


Figura 9.3: Un árbol enraizado contiene polaridad histórica; el árbol no enraizado solo fija la bipartición mostrada.

Descripción alternativa. El panel enraizado contiene un origen explícito y polaridad; el no enraizado conserva el split AB partido CD pero no indica de qué rama procede la historia.

el enraizamiento vulnerable a artefactos. El enraizamiento por punto medio coloca la raíz a mitad del camino más largo entre dos puntas; resulta útil como recurso exploratorio, pero depende implícitamente de que las distancias desde la raíz sean suficientemente comparables.

El enraizamiento con grupo externo requiere justificar con evidencia independiente que ese grupo queda fuera del grupo focal.

Elegir como grupo externo solo la secuencia más diferente puede agravar incertidumbre de alineamiento y saturación.

El enraizamiento por punto medio es exploratorio y supone que las distancias desde la raíz son suficientemente comparables; no aporta por sí solo evidencia externa de polaridad.

9.2.5 Politomías y ausencia de resolución

Un nodo con más de dos descendientes es una politomía. Puede expresar una divergencia realmente cercana en el tiempo o, con más frecuencia en un árbol estimado, que los datos o el análisis no resuelven el orden de separación. Convertir automáticamente toda politomía en una serie de bifurcaciones produce precisión gráfica, no información. Una buena lectura distingue «el análisis apoya una divergencia simultánea» de «el análisis no permite decidir entre varias bifurcaciones».

Una politomía en un árbol estimado puede representar una divergencia cercana o falta de resolución; convertirla sin evidencia en bifurcaciones añade precisión gráfica, no información.

9.2.6 Newick: una topología escrita

ESTUDIO

El formato Newick escribe paréntesis anidados. Por ejemplo:

```
((A:0.1,B:0.1):0.2,
 (C:0.15,D:0.15):0.15);
```

La cadena contiene dos parejas y longitudes después de los dos puntos. La coma separa hijos del mismo nodo; los paréntesis delimitan un grupo; el punto y coma cierra el árbol.

En Newick, los paréntesis delimitan descendientes, la coma separa hijos, los dos puntos preceden longitudes y el punto y coma cierra el árbol; el orden textual de hermanos puede variar sin cambiar la topología.

Python nos permite representar clados sin instalar ninguna biblioteca. Una *lista* conserva elementos en orden y se escribe entre corchetes; un *conjunto* conserva pertenencia sin imponer orden y se escribe entre llaves. Para comprobar la rotación usamos conjuntos porque nos importa quién pertenece al clado, no dónde se imprime. El bloque es completo y solo usa la biblioteca estándar:

```
clados_dibujo_1 = {frozenset({"A", "B"}),
                    frozenset({"C", "D"})}
clados_dibujo_2 = {frozenset({"B", "A"}),
                    frozenset({"D", "C"})}

assert clados_dibujo_1 == clados_dibujo_2
```

`frozenset` crea un conjunto que ya no se modifica y que, por ello, puede incluirse dentro de otro conjunto. `assert` exige que una condición sea verdadera; si no lo es, Python detiene el programa. No «prueba» por sí solo una teoría biológica, pero sí evita que una rotación accidental se interprete como una topología distinta.

9.3 Antes del alineamiento: ¿qué relación comparamos?

ESENCIAL

9.3.1 Similitud y homología no son sinónimos

La *similitud* es una observación o una medida: dos secuencias comparten cierta proporción de símbolos, una puntuación o un patrón. La *homología* es una relación histórica: dos caracteres son homólogos si derivan de un carácter ancestral común. Por ello, la similitud aporta evidencia para proponer homología, pero no la demuestra de forma aislada. También puede existir homología con similitud baja después de mucha divergencia.

La homología es una relación de origen común, mientras que la similitud es una propiedad observable o cuantificable que puede aportar evidencia, pero no equivale lógicamente a homología.

La expresión «80 % de homología» confunde ambos niveles. Puede decirse «80 % de identidad en este alineamiento» o «se propone que son homólogos», pero no que la relación histórica sea verdadera en un porcentaje.

La *homoplasia* es una semejanza que no se explica por herencia directa del mismo estado desde el ancestro relevante. Puede surgir por convergencia, por evolución paralela o por una reversión. En secuencias de cuatro símbolos, los mismos estados pueden aparecer varias veces por azar o por restricciones funcionales; por eso las similitudes individuales no se leen automáticamente como sinapomorfías.

La homoplasia es una semejanza no explicada por la herencia directa del mismo estado ancestral en el nodo relevante y puede surgir por convergencia, evolución paralela o reversión.

En secuencias de cuatro símbolos, un mismo estado puede repetirse por azar o por restricciones funcionales, por lo que una similitud individual no se lee automáticamente como sinapomorfía sin evaluar el contexto filogenético.

9.3.2 Ortólogos y parálogos: la pregunta decide qué copia sirve

Dos genes homólogos separados por un evento de especiación son *ortólogos*; dos genes homólogos separados por una duplicación son *parálogos*. Son relaciones entre pares de copias, no propiedades permanentes de un nombre de gen. Tras una duplicación ancestral, una copia de la especie A puede ser ortóloga de la copia correspondiente en B y paróloga de otra copia dentro de A.

Ortología y paralogía distinguen homologías según el evento que separa las copias: especiación o duplicación, respectivamente [24].

Considera una duplicación que produjo globina alfa y globina beta antes de que se separaran humano y ratón. La alfa humana y la alfa de ratón son ortólogas respecto a esa especiación. La alfa humana y la beta humana son parálogas respecto a la duplicación. Si mezclamos alfa de unas especies, beta de otras y mioglobina de una tercera sin declarar la pregunta, el árbol resultante puede ser una historia de una familia génica, pero no un árbol directo de especies.

9.3.3 Árbol génico y árbol de especies

REFERENCIA

Un árbol génico representa la genealogía de copias de un locus. Un árbol de especies representa relaciones entre linajes de organismos bajo una definición y un modelo determinados. No son objetos intercambiables. Duplicación y pérdida, transferencia horizontal, hibridación, introgresión y clasificación incompleta de linajes pueden producir discordancia real. Además, un árbol génico estimado puede diferir por error de muestreo, alineamiento, modelo o búsqueda [50, 42].

Duplicación y pérdida, transferencia horizontal, hibridación, introgresión y clasificación incompleta de linajes pueden producir discordancia real entre el árbol génico y el árbol de especies.

Un árbol génico estimado puede diferir del árbol génico verdadero por error de muestreo, alineamiento, modelo o búsqueda, con independencia de que exista discordancia biológica real.

La historia de un gen no se identifica automáticamente con la historia de las especies

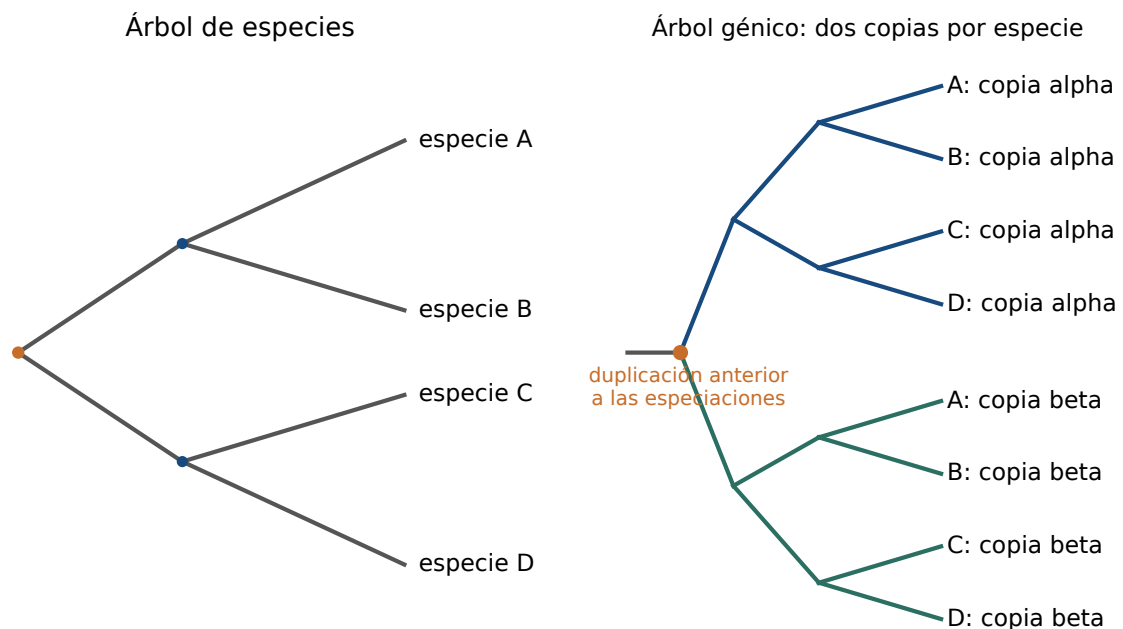


Figura 9.4: Contraste elemental entre un árbol de especies y un árbol génico afectado por una duplicación anterior a la especiación.

Descripción alternativa. El mismo universo A–D aparece en ambos paneles. La duplicación ancestral separa dos subárboles, alfa y beta, y cada uno conserva copias de A, B, C y D.

Un árbol de un solo gen no se identifica automáticamente con el árbol de especies, porque procesos biológicos y errores de inferencia pueden separar ambas historias.

Esta distinción cambia la selección de datos. Para estudiar una familia multigénica, la duplicación es parte del fenómeno y debemos conservar copias paralogas bien anotadas. Para aproximar relaciones entre especies con un locus, intentamos comparar ortólogos y dejamos claro que el resultado sigue siendo un árbol génico con posible discordancia.

Para estudiar una familia multigénica se conservan copias paralogas bien anotadas porque la duplicación es parte del fenómeno; para aproximar relaciones entre especies con un locus se prefiere comparar ortólogos, sabiendo que el resultado sigue siendo un árbol génico con posible discordancia.

9.4 Homología posicional: el alineamiento es una hipótesis

ESENCIAL

Una inferencia basada en caracteres compara columnas. Antes de contar cambios necesitamos decidir qué nucleótidos o aminoácidos ocupan posiciones comparables. Un alineamiento múltiple de secuencias, o MSA, inserta huecos y organiza los símbolos en columnas para proponer correspondencias.

Un alineamiento formula una hipótesis explícita de homología posicional; no observa directamente la historia de cada posición.

9.4.1 Por qué dos alineamientos pueden ser plausibles

Si una secuencia presenta una inserción y otra una deleción, la posición exacta del evento puede ser ambigua, especialmente en regiones repetitivas. Dos alineamientos pueden obtener puntuaciones parecidas pero colocar el hueco en columnas distintas. Entonces cambian los patrones que verá el método filogenético.

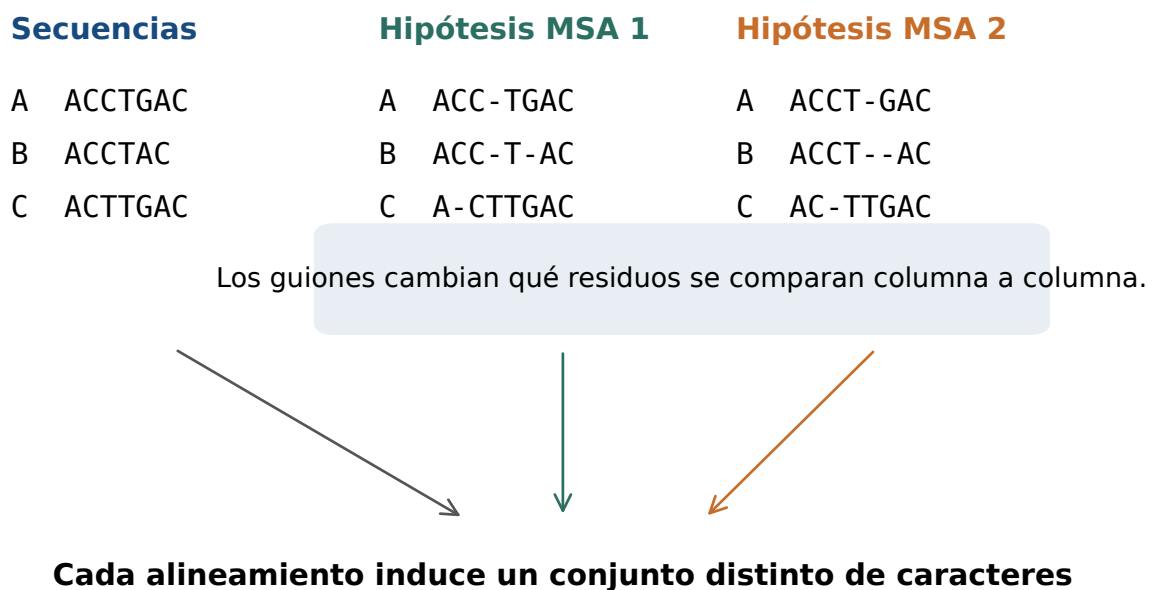


Figura 9.5: Dos hipótesis de homología posicional para secuencias cortas. Los símbolos son originales y solo ilustran la dependencia del análisis.

Descripción alternativa. Las dos matrices tienen ocho columnas en todas sus filas. En ambas, eliminar guiones recupera A igual a ACCTGAC, B igual a ACCTAC y C igual a ACTTGAC. Los guiones ocupan columnas distintas y por eso cambian los caracteres comparados.

No existe una regla universal que resuelva toda ambigüedad. La información estructural, la traducción de regiones codificantes, la conservación de motivos, la elección de algoritmo y los parámetros de hueco pueden ayudar. Para regiones codificantes suele ser útil alinear proteínas y proyectar la correspondencia a codones, porque insertar un solo nucleótido dentro de un codón puede romper el marco de lectura.

En regiones codificantes, alinear proteínas y proyectar la correspondencia a codones puede conservar el marco de lectura mejor que tratar cada nucleótido sin contexto, pero sigue siendo una hipótesis de alineamiento.

9.4.2 Alineamiento progresivo y árbol guía

ESTUDIO

Muchos alineadores construyen primero similitudes por pares, obtienen un árbol guía y alinean progresivamente secuencias o perfiles. El árbol guía organiza la operación; no debe confundirse con el árbol filogenético final. Las decisiones tempranas pueden condicionar las posteriores, y las estrategias iterativas tratan de refinar el resultado. MAFFT ofrece distintas estrategias con compromisos de escala, velocidad y precisión [40]; la documentación oficial debe consultarse para la versión realmente usada.

En un esquema de alineamiento progresivo, el árbol guía fija el orden de unión de secuencias o perfiles.

En un esquema progresivo, el resultado de una unión temprana se convierte en entrada de las uniones posteriores.

El árbol guía de un alineamiento progresivo no es el árbol filogenético final.

Una salida reproducible no registra solo «se usó MAFFT». Debe conservar versión, orden exacta, parámetros, secuencias de entrada y huellas criptográficas. Cambiar de una estrategia automática a otra más intensiva es una perturbación analítica, no una mejora garantizada.

Una salida de alineamiento reproducible conserva versión del programa, orden exacto de las secuencias de entrada, parámetros y huellas criptográficas; cambiar de una estrategia automática a otra más intensiva es una perturbación analítica y no una mejora garantizada.

9.4.3 ¿Recortar siempre mejora?

Las regiones con muchos huecos o correspondencia dudosa pueden introducir ruido. Sin embargo, eliminarlas también puede borrar señal. Estudios comparativos han encontrado situaciones en las que el filtrado automático empeora árboles de un solo gen [69]. Por tanto, «alineamiento recortado» no significa «alineamiento verdadero».

El recorte de un MSA intercambia posible reducción de ruido por pérdida de caracteres; su efecto debe evaluarse y no asumirse siempre beneficioso.

Una comprobación docente útil conserva dos análisis: sin recorte y con una regla declarada. Primero se cuentan longitud, huecos y sitios variables; después se comparan topología y apoyo. Si desaparece un clado, la conclusión correcta no es escoger silenciosamente la versión preferida, sino registrar que la inferencia es sensible al tratamiento del alineamiento.

9.4.4 Hueco no significa automáticamente un quinto nucleótido

Un guion en un MSA representa una ausencia introducida para proponer correspondencia. Puede relacionarse con una inserción, una delección, una región no secuenciada o una decisión del algoritmo. Diferentes programas pueden tratarlo como dato ausente, como estado adicional o mediante modelos de inserción y delección. Antes de calcular distancias hay que declarar si las columnas con huecos se excluyen para cada par, para todas las secuencias o de otra manera. Dos matrices calculadas con reglas distintas no son directamente comparables.

Tratar huecos mediante exclusión por pareja, exclusión completa, estado adicional o modelos de indel cambia qué columnas contribuyen; la política afecta al resultado calculado.

Dos matrices de distancia calculadas con políticas de huecos distintas no son directamente comparables como si resumieran los mismos caracteres.

ESTUDIO Un hueco de un MSA no es automáticamente un quinto nucleótido: puede representar ausencia de dato, una hipótesis de inserción o delección u otro tratamiento que debe declararse. **ESENCIAL**

9.5 De diferencias observadas a distancias de modelo

ESENCIAL

9.5.1 Distancia p : empezar por lo observable

Si dos secuencias alineadas no vacías tienen $L > 0$ columnas comparables y difieren en m de ellas, la distancia observada es

$$p = \frac{m}{L}, \quad 0 \leq m \leq L. \quad (9.1)$$

La distancia p es la proporción m/L de columnas discordantes entre las L columnas comparables de dos secuencias alineadas no vacías.

Es una proporción, no un porcentaje de homología ni una estimación completa de todos los cambios históricos.

En dos secuencias alineadas de 20 nucleótidos con dos diferencias válidas, la distancia observada es $p = 2/20 =$

0,10.

Tomemos este programa completo. Las comillas convierten cada secuencia en una cadena de Python antes de usarla:

```
S1 = "ACGTACGTACGTACGTACGT"
S2 = "ACGTACGGACGTACGGACGT"

def p_distance(seq1, seq2):
    if len(seq1) != len(seq2):
        raise ValueError("las secuencias deben estar alineadas")
    if len(seq1) == 0:
        raise ValueError("el alineamiento no puede estar vacio")

    differences = 0
    for base1, base2 in zip(seq1, seq2):
        if base1 != base2:
            differences = differences + 1
    return differences / len(seq1)

assert abs(p_distance(S1, S2) - 0.10) < 1e-12
```

La implementación Python de distancia p cuenta discordancias, divide por la longitud y rechaza secuencias desalineadas o vacías.

Difieren en las posiciones 8 y 16 si numeramos desde 1. En ambos casos T cambia a G, una transversión. Hay cero transiciones y dos transversiones: $P = 0/20 = 0$, $Q = 2/20 = 0,10$ y $p = P + Q = 0,10$.

Una *función* agrupa una operación y recibe sus entradas entre paréntesis. zip empareja el primer símbolo de una secuencia con el primero de la otra, y así sucesivamente. El bucle for visita cada par.

len devuelve la longitud. != significa «distinto de». raise ValueError detiene el cálculo con un mensaje si la condición de entrada no se cumple. La última línea tolera un error de redondeo menor que 10^{-12} , porque muchos decimales se representan de forma aproximada en el ordenador.

9.5.2 Cambios múltiples y saturación

La distancia p solo observa el estado inicial y final de cada comparación. Si en una rama ocurrió $A \rightarrow G \rightarrow A$, las secuencias finales coinciden y el conteo observado es cero, aunque hubo dos sustituciones. Con mayor divergencia crece la posibilidad de cambios múltiples, reversión o convergencia. La distancia observada tiende entonces a subestimar el número de sustituciones.

La saturación surge cuando sustituciones múltiples ocultan cambios anteriores, de modo que la proporción observada de diferencias deja de crecer en correspondencia directa con el cambio acumulado.

Un modelo de sustitución no recupera el pasado exacto. Describe probabilísticamente el proceso y permite corregir la expectativa bajo supuestos explícitos. La corrección es útil precisamente porque revela sus límites: cuando los datos se acercan al equilibrio del modelo, la incertidumbre crece y pequeñas variaciones de p producen grandes cambios en la distancia.

9.5.3 JC69: la corrección más simétrica

JC69 supone frecuencias de equilibrio iguales para A, C, G y T y la misma tasa instantánea para cada cambio entre bases distintas [36]. No es «la evolución real», sino un modelo mínimo para comprender por qué una corrección es no lineal.

JC69 supone frecuencias de equilibrio iguales para los cuatro nucleótidos y la misma tasa instantánea para cada sustitución entre bases distintas.

$$d_{JC69} = -\frac{3}{4} \ln \left(1 - \frac{4p}{3} \right), \quad 0 \leq p < \frac{3}{4}. \quad (9.2)$$

La corrección JC69 tiene una distancia real finita solo para una proporción observada $p < 3/4$.

El dominio no es un detalle tipográfico. Para $p = 0,75$, el argumento del logaritmo es cero y la distancia diverge; para valores mayores es negativo y no hay distancia real bajo esta expresión. Informar un número finito en esa frontera sería un error. Además, estar justo por debajo no vuelve fiable la estimación: la curva ya es muy pronunciada.

Para $p = 0,10$, los pasos son:

$$1 - \frac{4(0,10)}{3} = 0,866666 \dots, \quad \ln(0,866666 \dots) \approx -0,143101,$$

$$d = -0,75(-0,143101) \approx 0,107326.$$

Para una proporción observada de diferencias $p = 0,1$, JC69 produce aproximadamente 0,107326 sustituciones por sitio.

La corrección es ligeramente mayor que 0,10 porque admite que algunos cambios no quedaron visibles. El siguiente programa reexplica cada construcción:

```
import math

def jc69(p):
    if p < 0 or p >= 0.75:
        raise ValueError("JC69 exige 0 <= p < 0.75")
    inside = 1 - 4 * p / 3
    return -3 / 4 * math.log(inside)

distance = jc69(0.10)
print(f"{distance:.6f}")          # imprime 0.107326
assert abs(distance - 0.107326) < 1e-6
```

`import math` hace disponible el módulo matemático de la biblioteca estándar. `math.log` calcula el logaritmo natural. La expresión `f"{distance:.6f}"` solicita seis decimales; no cambia el valor interno, solo su presentación.

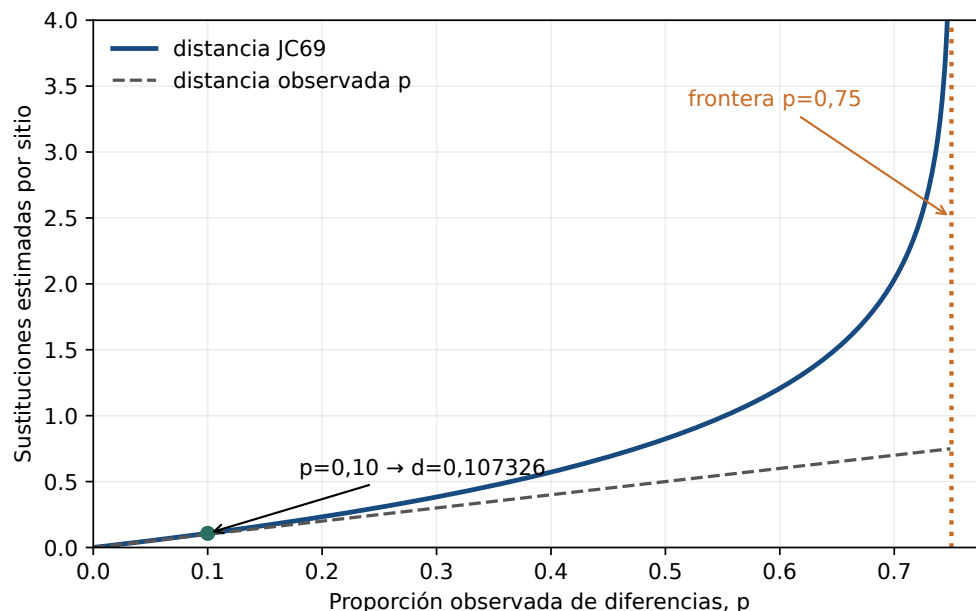


Figura 9.6: Distancia observada y corrección JC69. La divergencia cerca de $p = 0,75$ señala la pérdida de información bajo el modelo.

Descripción alternativa. La corrección JC69 coincide aproximadamente con p cerca de cero, luego crece más deprisa y diverge al aproximarse a la frontera p igual a 0,75.

9.5.4 K2P: distinguir dos clases de cambio

ESTUDIO

Las transiciones intercambian purinas entre sí ($A \leftrightarrow G$) o pirimidinas entre sí ($C \leftrightarrow T$). Las transversiones cambian entre purina y pirimidina. K2P permite que ambas clases tengan tasas distintas [41]. Sea P la proporción de transiciones y Q la de transversiones:

Entre nucleótidos distintos, A–G y C–T son transiciones; los otros ocho cambios dirigidos entre purina y pirimidina son transversiones.

$$d_{K2P} = -\frac{1}{2} \ln(1 - 2P - Q) - \frac{1}{4} \ln(1 - 2Q). \quad (9.3)$$

K2P tiene una distancia real finita cuando P y Q son proporciones válidas y se cumplen simultáneamente $1 - 2P - Q > 0$ y $1 - 2Q > 0$.

Las desigualdades son parte del dominio. En la frontera $1 - 2P - Q = 0$ o $1 - 2Q = 0$, un logaritmo recibe cero; más allá recibe un número negativo. En ambos casos no existe una distancia real finita. Además, $P \geq 0$, $Q \geq 0$ y $P + Q \leq 1$ porque son proporciones de sitios.

Para un segundo ejemplo, supongamos $P = 0,08$ y $Q = 0,02$. Entonces $1 - 2P - Q = 0,82$ y $1 - 2Q = 0,96$. Sustituimos:

$$d = -\frac{1}{2} \ln(0,82) - \frac{1}{4} \ln(0,96) \approx 0,099225 + 0,010205 = 0,109431.$$

Para $P = 0,08$ transiciones y $Q = 0,02$ transversiones, K2P produce aproximadamente 0,109431 sustituciones por sitio.

El cálculo no demuestra que K2P sea adecuado. Solo garantiza que hemos aplicado correctamente la fórmula. La adecuación exige preguntar si separar esas dos clases captura suficientemente el patrón, si la composición es comparable y si las tasas varían entre sitios o linajes. Modelos más flexibles incorporan frecuencias desiguales, tasas diferentes y heterogeneidad, pero cada parámetro adicional debe estimarse.

Aumentar la flexibilidad de un modelo permite representar más patrones mediante parámetros adicionales que deben estimarse, pero no demuestra por sí solo que el modelo sea adecuado para la pregunta.

9.5.5 De pares de secuencias a una matriz

Con n secuencias calculamos una distancia para cada par y obtenemos una matriz simétrica. La diagonal vale cero, porque cada secuencia se compara consigo misma. Los valores deben ser no negativos y las etiquetas de filas y columnas deben coincidir.

	A	B	C	D
A	0	17	21	31
B	17	0	30	34
C	21	30	0	28
D	31	34	28	0

La matriz reconstruida del ejemplo contiene cuatro taxones, es simétrica, tiene diagonal nula y no contiene distancias negativas.

En Python, un *diccionario* asocia una clave con un valor. Podemos usar un diccionario de diccionarios para acceder a `distance["A"]` `["B"]`. Las comillas indican una cadena de texto. Esta representación hace visibles las etiquetas y reduce el riesgo de confundir posiciones:

```
distance = {
    "A": {"A": 0, "B": 17, "C": 21, "D": 31},
    "B": {"A": 17, "B": 0, "C": 30, "D": 34},
    "C": {"A": 21, "B": 30, "C": 0, "D": 28},
    "D": {"A": 31, "B": 34, "C": 28, "D": 0}
```

```

"D": {"A": 31, "B": 34, "C": 28, "D": 0},
}

taxa = sorted(distance)
for a in taxa:
    assert distance[a][a] == 0
    for b in taxa:
        assert distance[a][b] == distance[b][a]
        assert distance[a][b] >= 0

```

`sorted` crea una lista ordenada de claves. Los dos bucles recorren todas las parejas, también la diagonal. Una matriz que declara siete taxones en la cabecera y contiene ocho filas fallaría antes de producir un árbol; ese es el tipo de control que debe preceder al análisis.

9.6 UPGMA: una traza transparente y un supuesto fuerte

ESENCIAL

UPGMA es un algoritmo aglomerativo. Empieza con cada taxón como clúster individual, une la pareja más cercana, actualiza las distancias y repite hasta formar un solo árbol. Su valor didáctico es extraordinario porque cada decisión puede comprobarse. Su limitación también es clara: interpreta las distancias bajo un reloj suficientemente uniforme y produce un árbol ultramétrico.

UPGMA comienza con clústeres individuales, une en cada paso una pareja de distancia mínima, actualiza distancias ponderando tamaños y repite hasta obtener un árbol enraizado ultramétrico.

9.6.1 La regla de actualización

Si se unen los clústeres X e Y para formar $U = X \cup Y$, la distancia desde U hasta otro clúster Z es la media de todas las parejas de hojas. Por ello debe ponderarse por el número de elementos:

$$d(U, Z) = \frac{|X|d(X, Z) + |Y|d(Y, Z)}{|X| + |Y|}. \quad (9.4)$$

La actualización de UPGMA pondera cada distancia de clúster por su número de hojas; una media no ponderada solo coincide cuando los clústeres tienen igual tamaño.

9.6.2 Ejemplo completo con cuatro taxones

Partimos de la matriz anterior. La menor distancia es $d(A, B) = 17$, así que unimos A y B. El nodo queda a altura $17/2 = 8,5$. Como ambos clústeres tenían una hoja:

$$d(AB, C) = \frac{21 + 30}{2} = 25,5, \quad d(AB, D) = \frac{31 + 34}{2} = 32,5.$$

La primera reducción del ejemplo UPGMA produce $d(AB, C) = 25,5$, $d(AB, D) = 32,5$ y altura $h(AB) = 8,5$.

La matriz reducida contiene $d(AB, C) = 25,5$, $d(AB, D) = 32,5$ y $d(C, D) = 28$. La menor distancia es 25,5; unimos el clúster AB con C. El nuevo nodo queda a altura $25,5/2 = 12,75$. La rama desde el nodo AB mide $12,75 - 8,5 = 4,25$; la rama hasta C mide 12,75.

Ahora aparece el paso que una media ingenua resuelve mal. El clúster AB tiene dos hojas y C tiene una. La distancia a D es:

$$d(ABC, D) = \frac{2d(AB, D) + 1d(C, D)}{2 + 1} = \frac{2(32,5) + 28}{3} = 31.$$

Al combinar un clúster de tamaño dos situado a distancia 32,5 y un singleton a distancia 28, la actualización ponderada de UPGMA es 31.

La raíz queda a altura $31/2 = 15,5$. La rama desde ABC hasta la raíz mide $15,5 - 12,75 = 2,75$, y la rama desde D mide 15,5. El árbol Newick, incluyendo longitudes, es:

```
((A:8.5,B:8.5):4.25,C:12.75):2.75,D:15.5);
```

La cadena Newick del ejemplo codifica las fusiones A, B , después $(AB), C$, y longitudes que dejan las cuatro puntas a distancia 15,5 de la raíz.

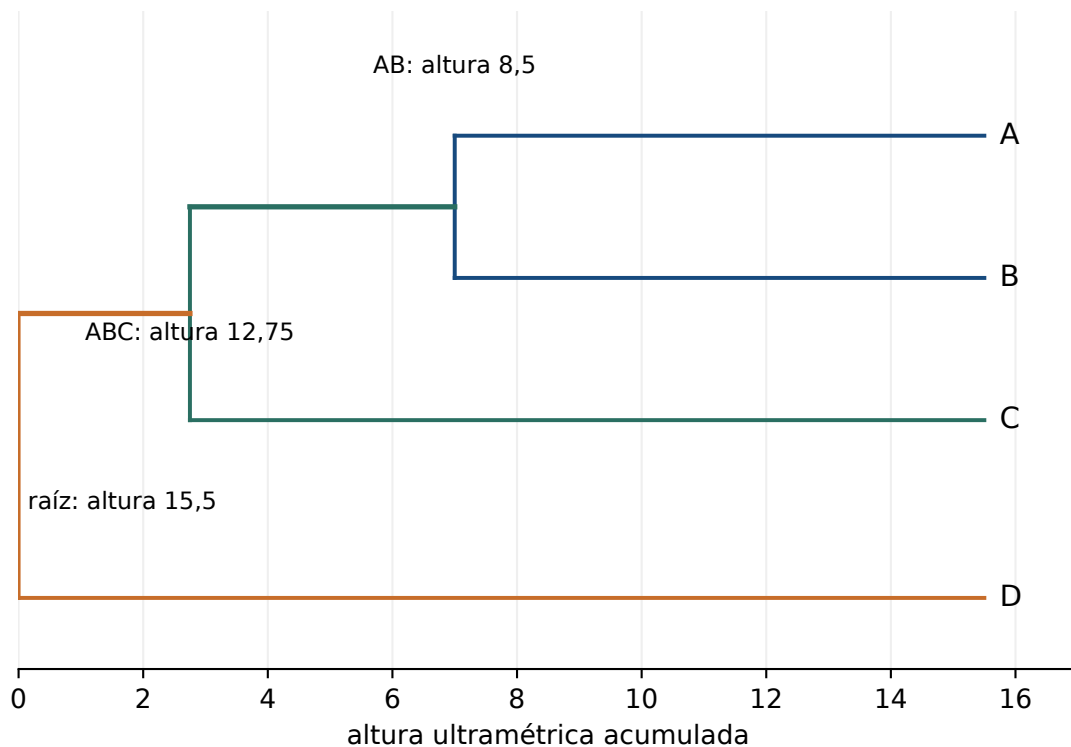


Figura 9.7: Dendrograma UPGMA reconstruido a partir de la matriz de cuatro taxones. Todas las puntas quedan a altura 15,5.

Descripción alternativa. La raíz se sitúa a la izquierda; A, B, C y D aparecen en los cuatro extremos derechos, todos a 15,5. A y B se fusionan a altura 8,5; AB con C a 12,75; y ABC con D a 15,5.

El mismo cálculo en Python mantiene el tamaño del clúster de forma explícita:

```
size_ab = 2
size_c = 1
distance_ab_d = 32.5
distance_c_d = 28.0

distance_abc_d = (
    size_ab * distance_ab_d + size_c * distance_c_d
) / (size_ab + size_c)

assert abs(distance_abc_d - 31.0) < 1e-12
```

Los paréntesis agrupan una expresión que ocupa varias líneas. Los nombres descriptivos evitan el error de confundir distancia con altura. El programa no debe esconder el factor 2: es la memoria de que AB representa dos hojas.

9.6.3 Qué supone UPGMA

UPGMA produce un árbol ultramétrico y requiere que las distancias sean compatibles con tasas suficientemente constantes desde la raíz; si los linajes evolucionan a velocidades distintas puede agrupar por tasa en lugar de por historia.

Ser ultramétrico significa que todas las hojas quedan a la misma distancia de la raíz. El árbol producido por el ejemplo fuerza que cada camino sume 15,5, pero la matriz de entrada no es ultramétrica. En una matriz

ultramétrica, las dos distancias mayores de cada terna deben coincidir. Aquí las cuatro ternas violan esa propiedad; por ejemplo, para A, B y C aparecen 17, 21 y 30.

El ajuste UPGMA implica distancias cophenéticas 17 para AB; 25,5 para AC y BC; y 31 para AD, BD y CD. Al restar distancia de entrada, los residuos son:

Par	AB	AC	BC	AD	BD	CD
cophenética – entrada	0	+4,5	-4,5	0	-3	+3

La matriz del ejemplo viola la ultrametricidad en sus cuatro ternas y el árbol UPGMA presenta residuos cophenéticos 0, +4,5, -4,5, 0, -3, +3 para AB, AC, BC, AD, BD y CD.

La salida ultramétrica no demuestra, por tanto, que la entrada sostuviera un reloj. Un reloj estricto presupone que las secuencias se muestrean al mismo tiempo y el cambio esperado es proporcional al tiempo compartido. Si un linaje acumula sustituciones mucho más rápido, la distancia ya no funciona como reloj común y la jerarquía de UPGMA puede ser incorrecta.

UPGMA no es «malo» por tener un supuesto fuerte. Es apropiado cuando el supuesto se sostiene suficientemente para la pregunta, y muy útil para aprender a separar datos, regla y resultado. El error consiste en usarlo sin reconocer el supuesto o en generalizar su árbol a cualquier matriz.

UPGMA es apropiado cuando su supuesto de reloj se sostiene suficientemente para la pregunta planteada; el error no es el supuesto en sí, sino usarlo sin reconocerlo o generalizar su árbol a cualquier matriz.

9.7 Unión de vecinos: distancias sin reloj estricto

ESTUDIO

Neighbor joining, o unión de vecinos (NJ), también parte de una matriz de distancias, pero no escoge simplemente la menor entrada. Busca una pareja cuya unión reduzca la longitud total estimada del árbol, corrigiendo la decisión por la separación de cada taxón respecto al conjunto [61]. Comienza con una estrella no resuelta, une vecinos, calcula longitudes de rama y reduce la matriz hasta obtener un árbol no enraizado.

Neighbor joining construye un árbol a partir de distancias sin requerir un reloj molecular estricto; no debe describirse como una variante de UPGMA con reloj relajado.

La diferencia conceptual puede verse con una matriz en la que todos los taxones no están a igual distancia de una raíz posible. UPGMA obliga a que las puntas terminen a la misma altura. NJ puede asignar ramas terminales diferentes. Eso no convierte automáticamente a NJ en la mejor explicación: su resultado depende de que las distancias representen adecuadamente el proceso y resume cada par de secuencias en un número.

Que NJ asigne ramas terminales distintas de UPGMA no lo convierte automáticamente en la mejor explicación: su resultado sigue dependiendo de que las distancias representen adecuadamente el proceso evolutivo.

Para n taxones, NJ emplea la matriz auxiliar

$$Q(i, j) = (n - 2)d(i, j) - \sum_k d(i, k) - \sum_k d(j, k).$$

La pareja con menor Q , no necesariamente con menor d , se une en ese paso. NJ calcula $Q(i, j)$ mediante las sumas de fila y une en cada iteración una pareja que minimiza Q ; para la matriz del ejemplo, AB y CD empatan en el mínimo -116.

La ecuación compensa que un taxón muy alejado de todos los demás pueda tener una distancia par engañosamente pequeña o grande. En este capítulo no desarrollamos toda la traza de NJ: el ejemplo completamente trabajado es UPGMA. Lo importante es conservar tres diferencias:

1. ambos métodos usan distancias y pierden los patrones de cada columna;
2. UPGMA produce un árbol enraizado ultramétrico;
3. NJ produce inicialmente un árbol no enraizado y no exige reloj.

9.8 De una matriz resumida a los caracteres

ESENCIAL

Los métodos de distancias transforman un MSA de muchas columnas en una matriz de números por pares. Los métodos basados en caracteres conservan el patrón de cada columna. Esto permite preguntar qué sitios apoyan una bipartición, pero también hace crecer enormemente el espacio que debe explorarse.

Los métodos de distancias resumen las columnas del MSA en valores por pares, mientras que los métodos basados en caracteres conservan el patrón de estados de cada columna.

9.8.1 Cuántos árboles pueden proponerse

REFERENCIA

Para n hojas etiquetadas, el número de topologías bifurcantes no enraizadas es $(2n - 5)!!$, el doble factorial de $2n - 5$:

$$N_{\text{no enraizados}}(n) = (2n - 5)!! = (2n - 5)(2n - 7) \cdots 3 \cdot 1. \quad (9.5)$$

Hay 3 topologías bifurcantes no enraizadas para cuatro taxones y 2.027.025 para diez; el crecimiento combinatorio impide una enumeración exhaustiva en conjuntos moderados.

Para cuatro taxones existen exactamente tres biparticiones: $AB|CD$, $AC|BD$ y $AD|BC$. Para diez, el cálculo es $15!! = 15 \cdot 13 \cdot 11 \cdot 9 \cdot 7 \cdot 5 \cdot 3 \cdot 1 = 2.027.025$. Python puede construir el doble factorial con un bucle:

```
def double_factorial(value):
    result = 1
    for factor in range(value, 0, -2):
        result = result * factor
    return result

assert double_factorial(2 * 4 - 5) == 3
assert double_factorial(2 * 10 - 5) == 2_027_025
```

`range(value, 0, -2)` produce una secuencia descendente que resta dos en cada paso y se detiene antes de cero. El guion bajo dentro de `2_027_025` solo mejora la lectura del entero en Python; no altera su valor.

Una búsqueda heurística visita una parte de ese espacio. Suele comenzar en uno o varios árboles iniciales, propone reorganizaciones locales y conserva los cambios que mejoran el criterio. Repetir búsquedas desde puntos diferentes reduce el riesgo de quedar atrapado en un óptimo local, pero no demuestra que se haya encontrado el óptimo global.

Una búsqueda heurística puede terminar en un óptimo local.

Usar varios puntos iniciales amplía la exploración, pero no garantiza haber encontrado el óptimo global.

9.8.2 Parsimonia: contar cambios mínimos

La máxima parsimonia prefiere el árbol que requiere el menor número de cambios para explicar los caracteres. Para una topología fija, el algoritmo de Fitch propaga conjuntos de estados desde las hojas: si los conjuntos hijos se intersectan, conserva la intersección sin sumar cambio; si no se intersectan, conserva la unión y suma uno [25].

En la recurrencia de Fitch, una intersección no vacía de estados hijos se propaga sin sumar cambio; si la intersección es vacía, se propaga la unión y se suma un cambio.

Consideremos cuatro taxones y dos patrones:

Sitio 1: GAAA. Solo A posee G. Es un patrón 3:1, un *singleton*. Requiere un cambio en las tres topologías no enraizadas; no permite escoger entre ellas.

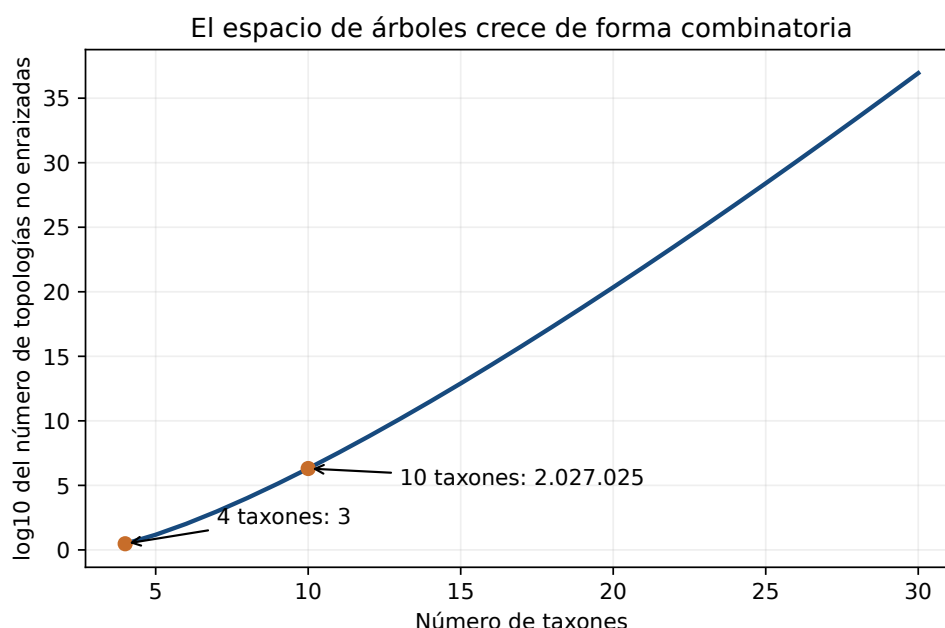


Figura 9.8: Crecimiento del número de topologías bifurcantes no enraizadas. El eje vertical usa logaritmo decimal para que la curva pueda representarse.

Descripción alternativa. El número de topologías no enraizadas crece de 3 con cuatro taxones a 2.027.025 con diez y continúa aumentando de forma combinatoria.

Sitio 2: AAGG. A y B comparten A; C y D comparten G. Requiere un cambio en $AB|CD$ y dos en cada alternativa. Es informativo para parsimonia.

En cuatro taxones, un patrón 3:1 no es informativo para parsimonia porque puntúa igual en las tres topologías; un patrón 2:2 puede favorecer una de ellas.

Podemos ejecutar la lógica sin bibliotecas:

```
def combine(left, right):
    common = left & right
    if common:
        return common, 0
    return left | right, 1

left, score_left = combine({"A"}, {"A"})
right, score_right = combine({"G"}, {"G"})
root, score_root = combine(left, right)
score = score_left + score_right + score_root
assert score == 1
```

El operador `&` calcula intersección de conjuntos y `|` calcula unión. La función devuelve dos valores: el conjunto que sube al nodo y el número de cambios añadido. Para el patrón AAGG en $AB|CD$, ambos pares coinciden internamente y el único cambio se añade al unir $\{A\}$ con $\{G\}$.

En la traza ejecutada del patrón AAGG bajo la agrupación $AB|CD$, ambos pares coinciden internamente y el único cambio añadido se produce al unir $\{A\}$ con $\{G\}$, dando puntuación 1.

Parsimonia es transparente y útil para comprender señal y conflicto. No modela, sin embargo, que varios cambios puedan ocurrir en la misma rama. Bajo ciertas combinaciones de ramas largas separadas y una rama interna corta puede ser estadísticamente inconsistente: al añadir más sitios generados por el mismo proceso sesgado, aumenta el apoyo a una topología incorrecta [20].

ESTUDIO Bajo la combinación de dos ramas largas separadas por una rama interna corta estudiada por Felsenstein, máxima parsimonia puede ser estadísticamente inconsistente: al aumentar los sitios generados por ese mismo proceso converge con mayor apoyo hacia una topología incorrecta. **REFERENCIA**

9.8.3 Máxima verosimilitud: probabilidad de los datos bajo una hipótesis

La máxima verosimilitud (ML) pregunta: si fijamos una topología T , sus longitudes de rama y los parámetros θ de un modelo, ¿qué probabilidad tendrían los datos alineados D ? La verosimilitud es una función de la hipótesis con los datos observados fijos:

$$L(T, \theta; D) = P(D | T, \theta). \quad (9.6)$$

La máxima verosimilitud compara árboles mediante $P(D | T, \theta)$; no calcula directamente la probabilidad de que un árbol sea verdadero.

Si el modelo trata los sitios como independientes condicionados al árbol, la verosimilitud total es el producto de las verosimilitudes de sitio:

$$L(T, \theta; D) = \prod_i P(D_i | T, \theta), \quad \ln L = \sum_i \ln P(D_i | T, \theta), \quad P(D_i | T, \theta) > 0 \text{ para todo } i. \quad (9.7)$$

Bajo independencia condicional de los sitios, la verosimilitud total es el producto de las contribuciones de sitio; si todas son estrictamente positivas, la log-verosimilitud real es la suma de sus logaritmos.

Como el producto de muchas probabilidades muy pequeñas puede desbordar la precisión numérica, se trabaja con logaritmos: el logaritmo de un producto es la suma de los logaritmos. Un árbol con $\ln L = -1200$ tiene mayor verosimilitud que otro con $\ln L = -1250$, porque -1200 es el valor mayor, menos negativo.

ESTUDIO Entre dos árboles comparados sobre los mismos datos y criterio, una log-verosimilitud de -1200 es mayor que -1250 y por tanto corresponde a mayor verosimilitud. **REFERENCIA**

Para cada sitio hay estados ancestrales desconocidos. No se elige simplemente el más cómodo: el cálculo suma sobre los posibles estados internos, ponderados por las probabilidades de transición a lo largo de las ramas. El algoritmo de poda de Felsenstein organiza eficientemente esa suma [21, 23]. Aquí basta con entender la estructura; la derivación completa queda en la ruta de referencia del libro.

El algoritmo de poda calcula vectores de verosimilitud condicional desde las hojas hacia la raíz y suma los estados ancestrales ocultos, en vez de enumerar por separado todas sus asignaciones globales.

En un árbol binario de cuatro puntas con patrón (0,0,1,1), probabilidades previas (0,5/0,5), persistencia (0,9) y cambio (0,1), la poda y la enumeración de los tres estados internos producen ambas ($L=0,065700$). Este ejemplo no estima un modelo biológico: es un oráculo pequeño que permite comprobar la recurrencia.

En el ejemplo binario declarado, la poda postorden y la enumeración exhaustiva de estados internos dan la misma verosimilitud, (0,065700).

Tres condicionantes impiden leer «árbol ML» como «historia demostrada»:

1. el modelo puede describir mal composición, tasas o dependencia entre sitios;
2. la búsqueda heurística puede no alcanzar la mejor topología global;
3. diferentes alineamientos o muestras pueden contener señales distintas.

9.8.4 Inferencia bayesiana: incorporar una distribución previa

REFERENCIA

La inferencia bayesiana trata árboles y parámetros como variables aleatorias. Combina la verosimilitud con una distribución previa y normaliza mediante la probabilidad marginal de los datos:

$$P(T, \theta | D) = \frac{P(D | T, \theta) P(T, \theta)}{P(D)}. \quad (9.8)$$

La identidad de Bayes expresa la probabilidad posterior de árbol y parámetros mediante verosimilitud, distribución previa y probabilidad marginal de los datos.

El denominador integra sobre un espacio inmenso y normalmente no puede calcularse por enumeración. Las cadenas de Markov Monte Carlo (MCMC) recorren estados y, después de una fase inicial, aspiran a obtener una muestra de la distribución posterior. Repetir cadenas, inspeccionar trazas, calcular tamaños de muestra efectivos y comprobar convergencia no son adornos: sin ellos, una frecuencia de clado puede reflejar una cadena mal mezclada.

Una estimación posterior mediante MCMC requiere diagnosticar convergencia y suficiencia del muestreo; la identidad de Bayes no garantiza por sí sola que una cadena haya explorado adecuadamente la posterior.

No debe compararse un porcentaje de bootstrap y una probabilidad posterior como si fueran la misma escala. Proceden de procedimientos y preguntas diferentes. Tampoco es correcto describir Bayes como «ML con más precisión»: puede integrar incertidumbre y conocimiento previo, pero depende de priors, modelo y calidad de la exploración [31].

9.8.5 Comparar familias sin convertirlas en una clasificación de calidad

Familia	Entrada conservada	Criterio principal	Límite que debe recordarse
UPGMA	matriz de distancias	pareja más cercana y media ponderada	requiere estructura ultramétrica
NJ	matriz de distancias	reducción de longitud total	resume columnas y produce árbol no enraizado
Parsimonia	patrones de sitio	mínimo número de cambios	puede sufrir inconsistencia y atracción de ramas largas
ML	patrones de sitio	máxima $P(D T, \theta)$	depende de modelo y búsqueda
Bayes	patrones de sitio	distribución posterior	depende además de priors y convergencia MCMC

UPGMA, NJ, parsimonia, ML y Bayes optimizan o resumen objetos distintos; no forman una escala universal en la que el último método sea siempre mejor.

La elección debe empezar por la pregunta y los supuestos. UPGMA es excelente para una traza manual y apropiado para distancias ultramétricas. NJ proporciona un árbol exploratorio rápido sin reloj estricto. Parsimonia muestra qué caracteres apoyan cada alternativa. ML ofrece un marco probabilístico explícito y ampliamente usado. Bayes permite integrar una distribución posterior. En un análisis defendible puede haber varios métodos, pero no se «vota» entre ellos sin estudiar por qué discrepan.

9.9 Modelo, selección y herramienta

ESTUDIO

9.9.1 Un modelo es una aproximación, no un nombre de menú

JC69 supone simetría extrema. K2P separa transiciones y transversiones. Otros modelos permiten frecuencias desiguales y tasas diferentes; componentes como Γ o categorías de tasa describen heterogeneidad entre sitios. Mayor flexibilidad puede reducir sesgo, pero también exige estimar más parámetros. Elegir el modelo de mayor complejidad disponible no es una regla científica.

Criterios como AIC, AICc o BIC equilibran ajuste y complejidad con penalizaciones distintas. Si ℓ es la log-verosimilitud maximizada, k el número de parámetros y n el número de sitios, se definen aquí como

$$\text{AIC} = -2\ell + 2k, \quad (9.9)$$

$$\text{AICc} = \text{AIC} + \frac{2k(k+1)}{n-k-1}, \quad n > k+1, \quad (9.10)$$

$$\text{BIC} = -2\ell + k \ln n. \quad (9.11)$$

AIC se calcula como $-2\ell + 2k$ con la log-verosimilitud maximizada y el número de parámetros estimados. AICc

añade a AIC la corrección $2k(k+1)/(n-k-1)$, que requiere $n > k+1$. BIC se calcula como $-2\ell + k \ln n$.

Por ejemplo, con $\ell = -120$, $k = 5$ y $n = 100$, se obtienen $AIC = 250,000$, $AICc = 250,638$ y $BIC = 263,026$. No son tres estimaciones del mismo parámetro: aplican penalizaciones diferentes y pueden ordenar de modo distinto el mismo conjunto candidato.

ModelFinder evalúa un conjunto de modelos candidatos y permite ordenarlo con AIC, AICc o BIC; está integrado en IQ-TREE [39, 52]. La selección con ModelFinder compara los modelos candidatos bajo el criterio configurado; AIC, AICc y BIC son criterios disponibles y no son sinónimos. Un selector restringido solo puede devolver elementos del conjunto que recibe: si el modelo generador no figura entre los candidatos, no puede seleccionarlo. En el caso mínimo registrado, restringir los candidatos a JC y HKY impide seleccionar el modelo generador GTR+G; al incorporarlo con la menor puntuación, el selector sí devuelve GTR+G. La selección depende del conjunto candidato, el criterio, la versión y las opciones. «El programa eligió GTR+F+G4» es el comienzo de una descripción, no su justificación completa.

Una selección automática de modelo solo es interpretable junto con la versión del software, los modelos candidatos, el criterio y los datos utilizados.

La referencia oficial de órdenes, actualizada el 5 de junio de 2025 y congelada localmente mediante una instantánea normalizada con SHA-256, distingue `-m MF`, que ejecuta la selección extendida sin reconstrucción posterior, de `-m MFP`, que selecciona el modelo y continúa con la reconstrucción del árbol. En la referencia de órdenes de IQ-TREE de 2025, `-m MF` realiza selección extendida de modelo sin reconstrucción posterior del árbol. En esa misma referencia, `-m MFP` realiza selección extendida de modelo y continúa con la reconstrucción del árbol.

La opción `-n 0` omite la búsqueda de árbol posterior a la fase inicial; no solicita por sí misma un árbol NJ o BIONJ. La opción documentada `-t BIONJ` es la que pide usar BIONJ como árbol inicial. En la referencia de órdenes de IQ-TREE de 2025, `-n 0` desactiva la búsqueda posterior del árbol. `-n 0` no es una orden NJ/BIONJ; la referencia documenta separadamente `-t BIONJ` para usar BIONJ como árbol inicial. Las opciones son afirmaciones versionadas: deben comprobarse contra la documentación de la versión ejecutada, no copiarse de una práctica antigua. IQ-TREE 3 amplía el programa con modelos y herramientas filogenómicas, pero ese panorama no cambia el alcance elemental de este capítulo [75, 34].

9.9.2 Una tubería reproducible, no una receta ciega

Una futura práctica puede usar MAFFT 7.526 e IQ-TREE 3, pero Python seguirá siendo el lenguaje principal para validar entradas, registrar versiones, inspeccionar salidas y comparar clados. Bash puede lanzar un ejecutable; no es necesario conocer Bash para comprender la ciencia [51, 34]. La ruta ejecutable de este capítulo usa Python como lenguaje curricular y no presupone Bash para comprender o reproducir sus cálculos. El siguiente bloque es un **fragmento opcional**: construye y muestra una orden, pero no ejecuta IQ-TREE. Para ejecutarla en C5 harán falta IQ-TREE 3 y el archivo versionado `alignment.fasta`; ninguno es prerequisite de C4.

```
command = [
    "iqtree3", "-s", "alignment.fasta",
    "-m", "MFP", "-B", "1000", "--seed", "20260805"
]
print(command)
```

`command` es una lista: cada elemento corresponde a un argumento, sin necesidad de componer una cadena de shell. El fragmento opcional construye de forma determinista una orden IQ-TREE 3 con entrada, selección de modelo, 1000 réplicas y semilla, pero no afirma haberla ejecutado. Una ejecución posterior tendrá que comprobar el código de retorno y guardar versión, comando, semilla, archivos de entrada y salida.

Una inferencia reproducible conserva identidad y hash de las entradas, versión y parámetros de herramientas, semilla cuando procede, salidas intermedias y controles que relacionan los mismos taxones.

La semilla registra el estado pseudoaleatorio solicitado, pero el protocolo de este proyecto no la acepta como procedencia suficiente. Exige además software, entorno, entradas y parámetros. El capítulo puede impartirse sin conexión mediante instantáneas verificadas; la red se usa para obtener y auditar fuentes, no como requisito durante el aprendizaje.

Registrar la semilla del generador aleatorio no basta como procedencia: hacen falta además el software con su versión, el entorno, las entradas y los parámetros.

9.10 Apoyo no significa verdad

ESENCIAL

9.10.1 Qué remuestrea el bootstrap

El bootstrap filogenético no paramétrico trata las columnas del MSA como la población empírica disponible. Para cada pseudorréplica selecciona columnas al azar con reemplazo hasta recuperar la misma longitud. Algunas aparecen varias veces y otras ninguna. Se reconstruye un árbol por réplica y se cuenta la frecuencia con que aparece cada clado [22].

El bootstrap no paramétrico de sitios remuestrea columnas del alineamiento al azar y con reemplazo hasta igualar la longitud original, reconstruye un árbol por pseudorréplica y cuenta la frecuencia con la que aparece cada clado.

Esta operación trata las columnas como unidades aproximadamente intercambiables bajo el modelo de sitios. Si hay dependencia fuerte entre sitios, bloques ligados o particiones con procesos distintos, la unidad o la estrategia de remuestreo debe respetar esa estructura; barajar columnas como si fueran independientes puede responder a otra pregunta.

El bootstrap estándar de sitios remuestrea columnas como unidades aproximadamente intercambiables; dependencias o particiones fuertes requieren una estrategia de remuestreo coherente con esa estructura.

MSA original										
	1	2	3	4	5	6	7	8		
R1	1	1	3	4	4	6	8	8	→ árbol →	clado AB
R2	2	3	3	5	5	6	7	8	→ árbol →	clado AC
R3	1	2	4	4	5	7	7	8	→ árbol →	clado AB
R4	1	2	3	4	5	6	7	8	→ árbol →	clado AB
Misma longitud; columnas muestreadas con reemplazo									AB aparece en 3 de 4 réplicas: 75 %	

Figura 9.9: Ejemplo didáctico de cuatro pseudorréplicas. El 75 % solo describe la frecuencia del clado AB en esta colección mínima.

Descripción alternativa. Cuatro filas muestran índices de columnas entre 1 y 8. Cada fila conserva longitud ocho y algunas repiten índices. Los resultados AB, AC, AB y AB producen un apoyo didáctico de 75 por ciento para AB.

En el esquema, AB aparece en tres de cuatro árboles, de modo que el apoyo es $3/4 = 0,75 = 75\%$. Cuatro réplicas son insuficientes para un análisis real, pero hacen visible la operación. Si aumentamos el número de réplicas, reducimos el error Monte Carlo del porcentaje; no corregimos automáticamente un MSA sesgado ni un modelo inadecuado.

Aumentar el número de réplicas bootstrap reduce el error Monte Carlo del porcentaje de apoyo, pero no corrige automáticamente un MSA sesgado ni un modelo inadecuado.

ESTUDIO Si un clado reaparece con proporción $\hat{p}$ en B pseudorréplicas, una aproximación binomial al error estándar Monte Carlo de esa proporción es

$$SE_{MC}(\hat{p}) = \sqrt{\frac{\hat{p}(1 - \hat{p})}{B}}.$$

Para $\hat{p} = 0,75$, resulta 0,043301 con $B = 100$ y 0,013693 con $B = 1000$: multiplicar por diez el número de réplicas reduce este error aproximadamente por $\sqrt{10}$, no por diez. Para $\hat{p} = 0,75$, la aproximación binomial produce errores estándar Monte Carlo de 0,043301 con 100 réplicas y 0,013693 con 1000. En la aproximación binomial declarada, aumentar (B) reduce el error Monte Carlo; la fórmula solo contiene (B) y $\hat{p}$, no modifica el MSA ni el modelo que generaron los árboles. La incertidumbre debida al alineamiento y a la adecuación del modelo debe auditarse por separado: aumentar réplicas de bootstrap no sustituye esas comprobaciones.

ESENCIAL

El bootstrap filogenético mide repetibilidad bajo remuestreo de columnas y bajo la tubería elegida; no es la probabilidad de que un clado sea históricamente verdadero.

Un apoyo alto responde aproximadamente: «cuando perturbamos la muestra de columnas de esta manera y repetimos este análisis, el clado reaparece con frecuencia». No responde sin más: «hay un 95 % de probabilidad de que la historia sea cierta». Umbrales como 70 o 95 no son leyes universales; dependen del procedimiento, del contexto y de la decisión que se quiera sostener.

Un apoyo bootstrap alto se interpreta como «al perturbar la muestra de columnas de esta manera y repetir el análisis, el clado reaparece con frecuencia», no como «existe un tanto por ciento de probabilidad de que la historia sea cierta».

ESTUDIO Los valores 70 y 95 no se aplican como umbrales universales de apoyo: su interpretación exige declarar procedimiento, contexto y decisión.

El bootstrap estándar reconstruye árboles para pseudorréplicas. UFBoot2 emplea una aproximación basada en árboles candidatos para reducir coste y tiene propiedades e interpretación operativa propias [30]. UFBoot2 es un procedimiento aproximado con estrategia de árboles candidatos y no equivale a repetir literalmente una búsqueda estándar completa e independiente para cada pseudorréplica. Informar «bootstrap» sin distinguir el procedimiento deja incompleta la afirmación.

ESENCIAL

9.10.2 Apoyo local y estabilidad global

Un número junto a una rama es local: se refiere al clado definido por esa rama. Un árbol puede contener unos clados muy estables y otros no resueltos. También puede ocurrir que dos análisis mantengan el clado principal pero cambien longitudes o posición del outgroup. La evaluación no debe reducir todo el árbol a un promedio de apoyos.

El apoyo asociado a una rama describe la frecuencia del clado local que esa rama define allí mismo bajo el procedimiento de remuestreo declarado. **ESTUDIO** En el caso ejecutable, dos análisis conservan el clado AB pero difieren tanto en la longitud de sus ramas terminales como en el taxón usado para enraizar. La estabilidad global no se resume mediante el promedio de apoyos de rama: deben informarse por separado los clados comparados, las longitudes y el enraizamiento.

ESENCIAL

Para comparar dos árboles enraizados pequeños, Python puede representar cada clado como un `frozenset` y calcular intersección y diferencias:

```
clades_raw = {
    frozenset({"A", "B"}),
    frozenset({"A", "B", "C"}),
}
clades_trimmed = {
    frozenset({"A", "B"}),
    frozenset({"A", "B", "D"}),
}
```

```
shared = clades_raw & clades_trimmed
only_raw = clades_raw - clades_trimmed
only_trimmed = clades_trimmed - clades_raw

assert shared == {frozenset({"A", "B"})}
```

El ejemplo Python de sensibilidad identifica el clado AB como compartido y conserva por separado los clados exclusivos de los dos análisis.

ESTUDIO En el ejemplo registrado, compartir el clado AB no hace idénticos los árboles: uno conserva ABC como clado exclusivo y el otro ABD. **ESENCIAL**

La intersección & devuelve clados compartidos; la resta devuelve los exclusivos de cada análisis. Esta comparación no decide cuál es correcto. Localiza qué parte de la conclusión es estable y dónde debe concentrarse la interpretación.

9.11 Sensibilidad: perturbar una decisión con intención

ESENCIAL

Una prueba de sensibilidad modifica una decisión razonable y observa qué afirmaciones cambian. No consiste en probar opciones hasta obtener el árbol preferido. Antes de ejecutar la perturbación se formula una predicción: «si las columnas ambiguas sostienen el clado X, al retirarlas disminuirá su apoyo»; después se comprueba.

Un análisis de sensibilidad atribuye los cambios comparando tuberías que difieren en una decisión controlada, con una predicción previa y los demás elementos mantenidos constantes.

Perturbaciones útiles:

- comparar dos hipótesis de alineamiento;
- analizar MSA sin recortar y recortado;
- cambiar entre modelos razonables;
- retirar o sustituir el grupo externo;
- excluir temporalmente un taxón de rama muy larga;
- repetir la búsqueda desde semillas o árboles iniciales distintos;
- comparar distancia p y una corrección de modelo dentro de su dominio.

Una matriz de análisis ayuda a no confundir niveles:

Perturbación	Si cambia el resultado	Interpretación prudente
taxones o accesiones	cambia el objeto biológico	revisar ortología, contaminación y cobertura
alineamiento o recorte	cambia la matriz de caracteres	localizar columnas y clados sensibles
modelo	cambia la descripción del proceso	comprobar adecuación y saturación
búsqueda o semilla	cambia la topología óptima hallada	ampliar exploración; no atribuirlo a biología
raíz o outgroup	cambia la polaridad	separar topología no enraizada de historia enraizada

Los errores de datos, alineamiento, modelo, búsqueda y enraizamiento actúan en niveles distintos; una comparación controlada permite localizar, pero no eliminar por sí sola, la incertidumbre.

9.12 Errores y límites que un árbol bonito puede ocultar

ESENCIAL

9.12.1 Atracción de ramas largas

Dos linajes que acumulan muchos cambios pueden compartir estados por convergencia o reversión. Un método que interpreta esos estados como una sola innovación compartida puede agrupar ramas largas que en la historia generadora estaban separadas. Este fenómeno es especialmente conocido para parsimonia en la «zona de Felsenstein», aunque ramas largas también alertan de saturación, modelo insuficiente o muestreo deficiente.

Atracción de ramas largas: la longitud es una magnitud, no una etiqueta

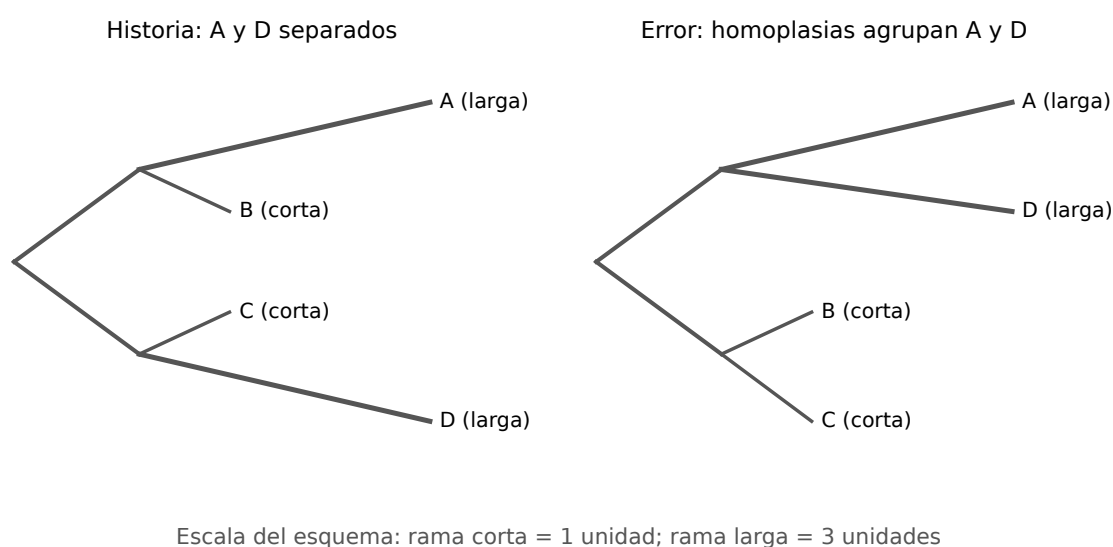


Figura 9.10: Esquema conceptual de atracción de ramas largas. No representa una simulación empírica ni afirma que toda rama larga sea artefactual.

Descripción alternativa. El panel izquierdo separa A y D y sus ramas terminales son exactamente tres veces más largas que las de B y C en la escala del esquema. El panel derecho muestra el error conceptual: A y D quedan agrupados por estados coincidentes, no porque toda rama larga deba agruparse.

La atracción de ramas largas puede agrupar linajes distantes cuando cambios múltiples y homoplasias producen semejanzas que el criterio interpreta como historia compartida.

En F09, las ramas terminales de A y D miden tres unidades del esquema y las de B y C una unidad; la razón visual larga:corta es 3:1.

La respuesta no es borrar siempre la secuencia incómoda. Primero se comprueba su identidad, calidad y alineamiento; se amplía el muestreo si es posible; se examina saturación y composición; se usan modelos más apropiados; y se compara el resultado con y sin el taxón como diagnóstico. Eliminarlo definitivamente requiere una razón biológica o técnica documentada.

ESTUDIO Una rama larga no justifica por sí sola borrar un taxón: el protocolo exige revisar identidad, calidad, alineamiento, saturación y modelo, y usar la comparación con/sin taxón como diagnóstico documentado. **ESENCIAL**

9.12.2 Poca señal y conflicto no son lo mismo

Un MSA puede tener pocas columnas variables: casi todos los árboles reciben información insuficiente. También puede contener muchas columnas variables que apoyan historias incompatibles. En el primer caso hay ausencia de señal; en el segundo, conflicto. Un consenso poco resuelto puede ser una representación honesta de ambos, pero las acciones posteriores difieren: añadir caracteres homólogos puede ayudar ante poca señal; ante conflicto real quizá haga falta un modelo de procesos múltiples.

Poca señal y conflicto no son equivalentes: la primera aporta información insuficiente, mientras el segundo contiene caracteres que apoyan historias incompatibles y puede requerir modelos de procesos múltiples.

9.12.3 Composición, tasas y dependencia entre sitios

Muchos modelos básicos suponen un proceso homogéneo a lo largo de ramas, estacionario en composición y reversible. Si dos linajes adquieren composiciones parecidas por sesgos independientes, pueden agruparse artificialmente. Si los sitios interactúan por estructura, la independencia es una aproximación. Si las tasas cambian entre ramas o a través del tiempo, un modelo uniforme puede fallar. Los supuestos explícitos son valiosos porque pueden auditarse; no porque se vuelvan ciertos al escribirlos.

Composición heterogénea, tasas variables entre ramas o dependencia entre sitios pueden violar supuestos de modelos homogéneos, estacionarios o independientes y sesgar la inferencia.

9.12.4 Recombinación y mosaicos de historia

Un solo árbol supone que los caracteres analizados comparten una historia arbórea suficientemente común. La recombinación puede hacer que regiones distintas tengan genealogías diferentes. Concatenarlas produce una media cuya interpretación depende del objetivo y del modelo. En este capítulo basta con reconocer la señal de alarma: cambios bruscos de apoyo entre regiones o discordancia persistente no deben ocultarse con un único árbol.

La recombinación puede producir genealogías diferentes entre regiones, de modo que un único árbol para una secuencia mosaico puede resumir historias incompatibles.

Concatenar regiones con genealogías distintas produce una señal promediada cuya interpretación depende del objetivo del análisis y del modelo empleado.

Un cambio brusco de apoyo entre regiones o una discordancia persistente al concatenar son señales de alarma que no deben ocultarse detrás de un único árbol resumen.

9.12.5 Etiquetas y procedencia

Una etiqueta incorrecta puede ser más dañina que un parámetro subóptimo. Cada secuencia debe relacionarse con organismo, gen o proteína, accesión y versión. No se reutiliza una misma accesión para organismos distintos salvo que la fuente lo explique. Antes de inferir:

1. comprobar número de registros y etiquetas únicas;
2. verificar que todas las secuencias pertenecen al tipo esperado;
3. revisar longitudes y caracteres inválidos;
4. confirmar que MSA, matriz y árbol contienen los mismos taxones;
5. calcular hashes de los archivos distribuidos.

La procedencia no es burocracia posterior. Define qué entidades biológicas representan las hojas.

ESTUDIO El protocolo de entrada relaciona cada secuencia con una etiqueta, accesión y versión, tipo molecular y procedencia antes de inferir. Los controles ejecutables rechazan etiquetas duplicadas, tipo molecular inesperado y caracteres inválidos para el alfabeto declarado. El alineamiento, la matriz de distancias y el árbol deben conservar exactamente el mismo conjunto de etiquetas: si una se pierde o se renombra por el camino, el resultado deja de ser trazable. El control ejecutable rechaza una accesión asignada a dos organismos distintos cuando el registro no aporta una justificación explícita de esa reutilización. El control ejecutable resume la longitud mínima y máxima de las secuencias aceptadas y rechaza una secuencia vacía antes de la inferencia.

9.13 Python como cuaderno verificable

ESTUDIO

Python no reemplaza el razonamiento filogenético; lo hace inspeccionable. En este capítulo se han utilizado pocas construcciones, reintroducidas donde aparecen [56]:

Variables. Nombres que apuntan a valores, como `distance_ab_d = 32.5`.

Cadenas. Texto entre comillas, usado para secuencias y taxones.

Listas. Colecciones ordenadas entre corchetes, adecuadas para argumentos o columnas.

Diccionarios. Asociaciones clave–valor entre llaves, adecuadas para matrices etiquetadas.

Conjuntos. Colecciones sin orden ni duplicados, adecuadas para clados.

Funciones. Operaciones reutilizables definidas con `def`.

Condiciones. Decisiones expresadas con `if`.

Bucles. Repeticiones expresadas con `for`.

Excepciones. Errores explícitos creados con `raise`.

Aserciones. Propiedades exigidas con `assert`.

En Python, una lista conserva orden y duplicados, un diccionario recupera valores por clave y un conjunto conserva elementos únicos sin imponer un orden curricular. Una función definida con `def` puede combinar condiciones y bucles; `raise` comunica una entrada inválida y `assert` detiene la ejecución si una propiedad exigida es falsa.

Una buena comprobación separa tres capas:

```
import math

def jc69(p):
    if p < 0 or p >= 0.75:
        raise ValueError("JC69 exige 0 <= p < 0.75")
    return -0.75 * math.log(1 - 4 * p / 3)

# 1. Entrada reconstruida y explícita
p = 0.10

# 2. Transformacion
d = jc69(p)

# 3. Propiedad esperada
assert d > p
assert abs(d - 0.107326) < 1e-6
```

El ejemplo de cuaderno verificable es autocontenido: define JC69, fija la entrada y comprueba una propiedad cualitativa y el valor numérico.

El primer `assert` comprueba una propiedad cualitativa del ejemplo: la corrección supera la distancia observada. El segundo comprueba el valor publicado con tolerancia. Si se cambia la entrada, debe revisarse también la expectativa; copiar una salida antigua sería perpetuar un resultado sin procedencia.

9.13.1 Bibliotecas: conveniencia sin ocultar el objeto

Para árboles mayores, `Bio.Phylo.read` puede leer un único árbol Newick y devolver un objeto recorrible [13, 68, 8]. El caso ejecutable del capítulo consulta la versión instalada en vez de presuponerla, lee una entrada mínima y recupera las hojas A, B y C. Antes de usar una biblioteca conviene saber qué objeto esperamos: hojas, clados, raíz y longitudes. La biblioteca facilita el recorrido, pero no decide si una secuencia es ortóloga ni si una raíz está justificada.

La ejecución controlada consulta la versión instalada de Biopython, llama realmente a `Bio.Phylo.read` sobre un Newick mínimo y recupera exactamente las hojas A, B y C; además rechaza una entrada vacía y una expectativa de hojas incorrecta.

La extensión opcional de C5 fijará Biopython 1.87 y proporcionará un Newick mínimo en memoria, sin descarga ni archivo oculto. Esa versión futura no se atribuye a la ejecución C4: el resultado presente publica la versión realmente observada.

Las comprobaciones de este capítulo usan Python 3.9 o posterior y solo la biblioteca estándar; cualquier biblioteca externa que se emplee para comparar resultados es opcional y se declara con su versión exacta.

9.14 Un caso integrado: de la pregunta a una afirmación limitada

ESENCIAL

Supongamos que queremos estudiar una familia de globinas. El objetivo no es «hacer un árbol de mamíferos», sino distinguir copias alfa y beta y comprobar si las copias de diferentes especies se agrupan según la duplicación ancestral. La cadena defendible sería:

1. **Pregunta.** ¿La separación alfa–beta antecede a las especiaciones incluidas?
2. **Selección.** Reunir accesiones verificadas de alfa y beta en varias especies, con una copia externa justificada.
3. **Predicción.** Si la duplicación es anterior, esperamos dos clados principales de copias, no un clado exclusivo por especie.
4. **MSA.** Alinear proteínas, inspeccionar regiones ambiguas y conservar una alternativa razonable.
5. **Modelo y búsqueda.** Inferir un árbol ML bajo selección de modelo registrada y búsquedas reproducibles.
6. **Apoyo.** Estimar apoyo de las biparticiones y comprobar sensibilidad al tratamiento del MSA.
7. **Afirmación.** «En este muestreo y bajo estas decisiones, la separación de copias alfa y beta es estable»; no «este es el árbol definitivo de los mamíferos».

Un segundo conjunto, compuesto por ortólogos verificados de TP53 en vertebrados, respondería a una pregunta distinta. Mantener separados ambos casos evita usar una mezcla de parálogos para inferir directamente relaciones de especies.

9.15 Síntesis: qué puede defenderse y qué queda abierto

ESENCIAL

Una inferencia filogenética elemental es defendible cuando podemos recorrerla en ambas direcciones. Hacia delante: pregunta, datos, MSA, modelo, método, apoyo y conclusión. Hacia atrás: cada conclusión remite a un clado; cada clado a árboles y pseudorréplicas; cada árbol a un MSA y un modelo; cada MSA a secuencias identificadas.

Las ideas nucleares son:

- leer conexiones, no posiciones de la página;
- separar topología, raíz y longitud de rama;
- distinguir similitud de homología y ortología de paralogía;
- tratar el MSA como hipótesis de homología posicional;
- interpretar una corrección de distancia dentro de su dominio;
- ponderar UPGMA por tamaños de clúster y reconocer su reloj;
- comparar métodos por objeto, criterio y supuesto;
- interpretar apoyo como estabilidad condicionada;
- perturbar una decisión para localizar sensibilidad;
- no identificar un árbol génico con un árbol de especies.

La pregunta de apertura puede responderse ahora: pasamos de secuencias comparables a una hipótesis evolutiva defendible solo si conservamos la cadena de decisiones y sus límites. El árbol no cierra la investigación. Organiza qué relaciones merecen una prueba posterior, qué datos deben ampliarse y qué conclusiones todavía no pueden sostenerse.

Ampliación y fuentes de consulta

La síntesis local de Abascal, Irisarri y Zardoya ofrece una introducción en castellano a evolución molecular, lectura de árboles y familias de métodos [1]. Para lectura crítica de árboles son especialmente útiles Omland

y colaboradores [55]. Los fundamentos clásicos de distancias, NJ, parsimonia, ML y bootstrap se encuentran en las fuentes originales citadas en sus secciones [36, 41, 61, 25, 21, 22]. La terminología original de ortología y paralogía puede consultarse en Fitch [24].

La bibliografía reciente no se usa para inflar el capítulo, sino para cerrar lagunas concretas: riesgos del filtrado de MSA [69], discordancia gen-especie [42], selección de modelos [39] y procedimientos actuales de inferencia y apoyo [75, 30]. Las instrucciones ejecutables deben contrastarse además con la documentación oficial de las versiones instaladas.

Puente al anexo práctico.

El anexo recuperará las secciones 9.2, 9.4, 9.5, 9.6 y 9.10. Allí se reproducirán primero los cálculos ancla sin IA y después con Python; solo entonces se introducirá una perturbación controlada. La práctica no volverá a enseñar estas definiciones.

Anexo práctico · De secuencias a una hipótesis filogenética

Pregunta, producto y separación de materiales

Cuatro secuencias sintéticas, ya alineadas, plantean la pregunta que organiza este anexo: *¿qué taxón se declara más próximo, con qué distancia, y qué parte de esa decisión procede de los datos y cuál del modelo de corrección?*

El anexo usa un conjunto propio de cuatro secuencias sintéticas ajeno al ejemplo A-B-C-D de la teoría, para que el cálculo ancla no pueda copiarse del cuerpo teórico.

El producto individual es `mi_anclaje.py`, con las tres funciones del cálculo ancla, y el informe de las tres variantes en `notas/variantes.tsv`: el árbol base y las dos perturbaciones, cada una con su orden de fusión y la altura de su raíz. Le acompañan las salidas guardadas, el control de dominio y una defensa escrita de 150 a 250 palabras.

Este anexo no imprime ningún valor resuelto. El comprobador público le dice qué relación algebraica no se cumple, nunca el número correcto, y la clave docente permanece fuera de la distribución.

Teoría que se recupera.

Este anexo se apoya en cuatro sitios del capítulo y en ninguno más. La sección [Leer árboles sin añadir historia que no contienen](#), para leer el árbol resultante sin atribuirle una historia que no contiene. La fórmula de corrección de distancias, con su dominio de validez. La sección [UPGMA: una traza transparente y un supuesto fuerte](#), para la regla de actualización ponderada. Y la sección [Apoyo no significa verdad](#), solo como recordatorio de que un único árbol no certifica nada; aquí no se remuestrea.

La sección [Homología posicional: el alineamiento es una hipótesis](#) se recupera únicamente para recordar por qué las cuatro secuencias vienen ya alineadas y sin huecos: aísla el problema de las distancias del problema del alineamiento, que no es objeto de este anexo.

0 · Entorno y diagnóstico

Python 3.9 o superior, sin paquetes externos. Verifique antes de empezar:

```
python3 --version
./practice_assets/create_workspace.sh BC09_entrega
cd BC09_entrega
(cd data && sha256sum --check --strict manifest.sha256)
python3 verification/practice_checks.py domain_guard
```

El generador crea `mi_anclaje.py` con su contrato en la documentación y sin implementación, copia los datos con su manifiesto y el comprobador público, y deja `notas/variantes.tsv` con solo la cabecera.

El tercer comando debe imprimir `domain_guard=REJECTED p=0.8 out_of_domain=True`. Si falla, el entorno no tiene el intérprete esperado o los archivos no coinciden con el manifiesto; no continúe hasta resolverlo.

Si el diagnóstico inicial de entorno falla, el entorno no tiene el intérprete esperado o los archivos no coinciden con el manifiesto de hashes; el anexo exige resolverlo antes de continuar con el cálculo ancla.

La evidencia mínima entregable es el cálculo ancla a mano, el script Python ejecutado con sus cuatro salidas y una defensa escrita de 150 a 250 palabras; el ejercicio funciona sin red y sin más dependencia que Python 3.9 estándar.

1 • Escribir el módulo de anclaje

Tarea. Implemente las tres funciones de `mi_anclaje.py`: la distancia observada entre dos secuencias alineadas, la corrección con su guarda de dominio, y la actualización de la primera fusión. El contrato está en la documentación de la plantilla.

Las tres son cortas, y la que enseña algo es la segunda. La corrección tiene un dominio: por encima de cierta proporción de diferencias, la fórmula pide el logaritmo de un número negativo y deja de estar definida. Eso no es un detalle numérico, es una afirmación biológica: cuando dos secuencias difieren tanto, el modelo ya no puede estimar cuántas sustituciones ocurrieron de verdad, porque la saturación ha borrado la señal. Su función debe rechazar ese caso, no devolver un número.

```
bash check_anclaje.sh
```

2 • Cálculo ancla, a mano

Abra `data/practice_sequences.fasta`. Contiene S1–S4, 16 nt cada una, ya alineadas. Trabaje solo con calculadora o lápiz y papel.

Tarea. Antes de usar IA, calcula a mano una distancia corregida y la primera actualización ponderada de UPGMA. Reprodúcelas después con un programa Python completo y legible. Perturba una secuencia o una decisión de alineamiento, vuelve a ejecutar el análisis y separa qué cambio procede de los datos, del modelo y del algoritmo.

Concretamente, entregue:

1. el número de posiciones en que difieren S1 y S2, y la p-distancia correspondiente;
2. la distancia JC69 corregida de ese par, aplicando la fórmula de EQ-BC09–JC69 y comprobando que la p-distancia está en su dominio;
3. la altura de la primera fusión UPGMA, que para dos hojas es la distancia corregida dividida entre dos;
4. la primera actualización ponderada: dadas $d(S1, S3) = 0,404247$ y $d(S2, S3) = 0,656602$ (calculadas de la misma manera que el punto 2, para no repetir dos veces el mismo cálculo a mano), la distancia del clúster {S1, S2} a S3 pondera por tamaño de clúster: con dos hojas de partida, ambos pesos valen 1, pero la fórmula es la misma que se usará después con pesos distintos.

Compruebe su resultado sin que el comprobador le revele ninguno:

```
python3 verification/practice_public_checker.py \
  --diffs <su_conteo> --p <su_p> --jc69 <su_distancia> \
  --d13 0.404247 --d23 0.656602 --updated <su_actualizacion>
```

Un FAIL señala qué relación algebraica no se cumple; no imprime el valor correcto. Si falla `formula_jc69`, revise la fórmula, no vuelva a adivinar un número.

3 • Reproducción en Python

Con el cálculo ancla ya resuelto a mano, ejecute la reproducción completa:

```
python3 verification/practice_checks.py anchor
python3 verification/practice_checks.py full_tree
```

El primer comando debe coincidir con sus cuatro valores manuales dentro de 10^{-4} . El segundo construye el árbol UPGMA completo sobre las cuatro secuencias con distancias JC69, fusión a fusión, incluidas las fusiones que usted no calculó a mano.

El andamiaje se retira aquí: el paso 1 le dio los tres primeros números; el paso 2 no repite la fórmula en el enunciado, solo pide leer la salida y contrastarla contra lo que usted ya obtuvo a mano.

4 • Perturbación: modelo frente a datos

Ejecute las dos perturbaciones y compare ambas salidas con la del paso 2:

```
python3 verification/practice_checks.py perturb_model
python3 verification/practice_checks.py perturb_data
python3 verification/practice_checks.py invariant_first_merge
```

`perturb_model` repite el mismo árbol usando p-distancia sin corregir en vez de JC69: cambia el modelo, no los datos. `perturb_data` aplica dos sustituciones declaradas en S3 (posiciones 2 y 9, ambas $G \rightarrow A$) y vuelve a calcular con JC69: cambia los datos, no el modelo. `invariant_first_merge` comprueba automáticamente si S1+S2 sigue siendo la primera fusión en las tres variantes.

Anote, para cada una de las tres variantes, dos cosas: el orden de fusión y la altura de la raíz. Después responda por escrito cuál de las dos perturbaciones cambió el orden y cuál solo la escala, y por qué eso es lo que cabía esperar de cada una.

No dé por sentado que ninguna perturbación cambiaría la topología: compruebe las dos por separado. Que una no la cambie en este conjunto no demuestra que el modelo nunca importa; demuestra solo que, para estos cuatro taxones y esta sustitución de dos bases, la corrección afectó la escala y no el orden.

5 • Controles

- **Taxones coincidentes.** Las cuatro variantes deben devolver exactamente los taxones S1, S2, S3, S4; ninguno se pierde ni se duplica.
- **Matriz simétrica y diagonal nula.** Para cualquier par, la distancia calculada en un sentido debe coincidir con la del sentido contrario, y la distancia de una secuencia consigo misma es cero.
- **Dominio de la corrección.** `python3 verification/practice_checks.py domain_guard` confirma que una $p \geq 0,75$ se rechaza en vez de devolver un número.
- **Cordura del algoritmo.** El primer par fundido siempre debe ser el de menor distancia entre los disponibles en ese paso; verifique esta propiedad en al menos una fusión que no sea la primera.

La implementación de JC69 usada en el anexo rechaza explícitamente una p-distancia fuera de dominio ($p \geq 0,75$) en vez de devolver un valor numérico engañoso.

6 • Interpretación, límites y defensa

Escriba una defensa de 150–250 palabras que separe explícitamente:

1. qué procede de los **datos** (la perturbación de S3);
2. qué procede del **modelo** (JC69 frente a p-distancia);
3. qué procede del **algoritmo** (la regla de actualización ponderada, que no cambia entre variantes);
4. qué permanece invariante en las cuatro variantes y por qué eso no prueba una ley general sobre este método con cualquier conjunto de datos.

Límite explícito: con cuatro taxones y una sola perturbación por tipo, este anexo no permite concluir cuál de los dos, modelo o datos, es «más importante» en general; solo permite documentar, para este conjunto, cuál de los dos cambió la topología y cuál no.

7 • Modo de trabajo con asistentes y criterios de evidencia

El anexo permite usar IA solo para depurar sintaxis Python ya escrita por el estudiante, nunca para calcular la distancia, decidir la topología o redactar la defensa; existe una vía completa sin IA basada en calculadora y el comprobador público.

Si no dispone de acceso a un asistente, la vía completa sin IA es: calculadora para el paso 1, la reproducción Python del paso 2 ejecutada localmente, y el comprobador público del paso 1 para autoevaluarse antes de entregar. Ninguna parte de este anexo depende de un servicio de red.

Puente desde la teoría.

El anexo recupera las secciones declaradas al inicio. No vuelve a enseñar la fórmula de JC69 ni la regla de actualización de UPGMA; las aplica sobre datos nuevos y pide separar qué parte del resultado depende de cada decisión.

8 • Empaquetar y verificar

Anote las tres variantes en `notas/variantes.tsv`, guarde sus salidas en `resultados/` y escriba la defensa en `notas/defensa.md`. Después:

```
cd ..
tar -czf BC09_entrega.tar.gz BC09_entrega
sha256sum BC09_entrega.tar.gz
./practice_assets/check_delivery.sh BC09_entrega BC09_entrega.tar.gz
```

El verificador comprueba que su módulo compila y cumple el contrato, que los datos de partida no se han modificado, que las tres variantes están documentadas con su orden de fusión y su altura, y que existe la defensa con la extensión pedida. Rechaza la entrega vacía y el espacio recién generado. No puntúa.

Regulación génica, clonación y vectores recombinantes: Circuitos moleculares procariotas, diseño in silico de cebadores y ensamblaje de constructos

BC-CH10 – Biología Computacional

Edición 2

Pregunta del tema. Un operón bacteriano decide cuándo transcribir sus genes con una lógica que se puede escribir en tres líneas de código. La misma idea, invertida, permite construir un plásmido: si sabemos qué señales lee la célula, podemos escribirlas nosotros. ¿Qué parte del diseño de un constructo es biología y qué parte es cálculo sobre una cadena de texto?

Producto final. Un módulo propio que calcule el porcentaje de guanina y citosina y la temperatura de fusión de un cebador, localice dianas de restricción en una secuencia y rechace las entradas fuera de dominio, con una tabla de predicciones contrastadas contra la ejecución.

Mapa del tema

10.1. Regulación génica procariota: La lógica molecular de los operones	134
10.2. Vectores de clonación y sistemas de expresión recombinante	136
10.3. Anatomía de un plásmido de expresión de proteínas	136
10.4. Diseño computacional de cebadores de PCR	137
10.5. Detección de dianas de restricción y métodos de ensamblaje	139

10.6. Diseño computacional en Python: Módulo de diseño recombinante	139
10.7. Peligros computacionales y actividad de depuración	141
10.8. Síntesis operativa y tablas de diagnóstico	141
10.9. Glosario conceptual	142
10.10. Conexiones y cierre de la Parte III	142
10.11. Fuentes y reproducibilidad	142

Cómo usar este tema

Aquí cambia el punto de vista del libro. Hasta ahora hemos leído secuencias que ya existían para reconstruir lo que ocurrió; a partir de este capítulo las escribimos nosotros para que ocurra algo. El material es el mismo (cadenas sobre el alfabeto A, C, G y T) pero la pregunta es de diseño: qué secuencia hay que sintetizar para que una bacteria fabrique la proteína que queremos.

Al terminar sabrá: leer el operón Lac como un circuito con dos entradas y una salida, y explicar por qué la respuesta a glucosa y lactosa a la vez no es la suma de las dos por separado; describir para qué sirve cada pieza de un plásmido de expresión; calcular a mano el porcentaje de guanina y citosina y la temperatura de fusión de un cebador, y decir qué método de cálculo ha usado; localizar dianas de restricción sobre una secuencia; y elegir entre restricción-ligación, Gibson y Golden Gate con un criterio que sepa defender.

La ruta **esencial** incluye el operón Lac, el operón Trp con su atenuación, la anatomía del plásmido, el cálculo de la temperatura de fusión, las dianas de restricción y los tres métodos de ensamblaje. La ruta de **estudio** añade promotores de alta expresión, etiquetas de purificación, la actividad de depuración y el anexo práctico.

10.1 Regulación génica procariota: La lógica molecular de los operones

ESENCIAL

Tras haber explorado la reconstrucción filogenética y la evolución histórica de las secuencias en los capítulos anteriores, inauguramos en este capítulo la transición hacia la **ingeniería molecular activa** y la **biología sintética**: cómo los principios de regulación transcripcional descubiertos en la naturaleza se traducen computacionalmente en el diseño *in silico* de constructos genéticos y vectores de expresión de proteínas.

En los organismos procariotas (*Escherichia coli*), los genes que codifican enzimas pertenecientes a una misma vía metabólica no se dispersan a lo largo del cromosoma, sino que se agrupan contiguamente bajo el control de un único promotor y un único terminador común, formando una unidad funcional denominada **operón policistrónico**:

- **Mensajero policistrónico:** La ARN polimerasa bacteriana transcribe todos los genes del operón en una sola molécula continua de ARNm que contiene múltiples sitios de unión al ribosoma (Shine-Dalgarno), permitiendo la síntesis coordinada y estequiométrica de todas las enzimas requeridas.
- **Acoplamiento transcripción-traducción:** Al carecer de envoltura nuclear, los ribosomas bacterianos inician la traducción del extremo 5' del ARNm mientras la ARN polimerasa continúa todavía elongando el extremo 3', posibilitando mecanismos reguladores ultra-rápidos de respuesta metabólica.

La regulación génica procariota organiza los genes funcionales en operones policistrónicos con transcripción y traducción acopladas para máxima eficiencia metabólica.

10.1.1 El operón Lac: Control dual por represión y activación

ESENCIAL

Descrito originalmente por François Jacob y Jacques Monod en 1961 [35] (Premio Nobel de Fisiología o Medicina en 1965), el **operón de la lactosa** (*Lac operon*) en *E. coli* es el paradigma universal de circuito regulador genético de tipo compuerta lógica (*AND-NOT gate*):

1. **Control negativo por el represor LacI:** En ausencia de lactosa, el represor tetramérico LacI (codificado por el gen regulador adyacente *lacI*) se une con alta afinidad al operador *lacO*, interponiéndose físicamente

en el avance de la ARN polimerasa y manteniendo la transcripción en un nivel basal prácticamente nulo (Ratio = 1.0X). Cuando la lactosa está presente, se metaboliza a **alolactosa**, la cual actúa como inductor alostérico uniéndose a LacI, induciendo un cambio conformacional que libera al represor del operador.

2. **Control positivo por el sensor de glucosa CAP-cAMP:** La bacteria prefiere energéticamente la glucosa sobre la lactosa. Cuando los niveles de glucosa son bajos, la enzima adenilato ciclasa se activa, produciendo altos niveles de **AMP cíclico (cAMP)**. El complejo CAP-cAMP (proteína activadora de catabolitos) se une a la región promotora aguas arriba, induciendo una curvatura en la doble hélice de ADN de > 90° que estabiliza el reclutamiento de la ARN polimerasa.

El operón Lac implementa un control dual mediante represión negativa por LacI/alolactosa y activación positiva por el sensor de glucosa CAP-cAMP.

Los cuatro estados ambientales del operón Lac:

Glucosa	Lactosa	Estado regulador	Mecanismo molecular	Transcripción
+	-	CAP_INACTIVO_LACI_UNIDO	Represor unido al operador	1.0X (Basal)
+	+	CAP_INACTIVO_LACI_LIBRE	Represión por catabolito	10.0X (Baja)
-	-	CAP_ACTIVADO_LACI_UNIDO	Represor bloquea promotor	1.0X (Basal)
-	+	CAP_ACTIVADO_LACI_LIBRE	Inducción plena: CAP unido y LacI libre	1000.0X (Máxima)

En ausencia de glucosa y presencia de lactosa (Glucosa-, Lactosa+), el operón Lac alcanza la inducción completa (CAP activo, LacI libre) con un ratio de expresión de 1000.0X.

Se reproduce con `verification/chapter_checks.py lac_operon --lactose`.

Lógica Combinatoria del Operón Lac (E. coli)

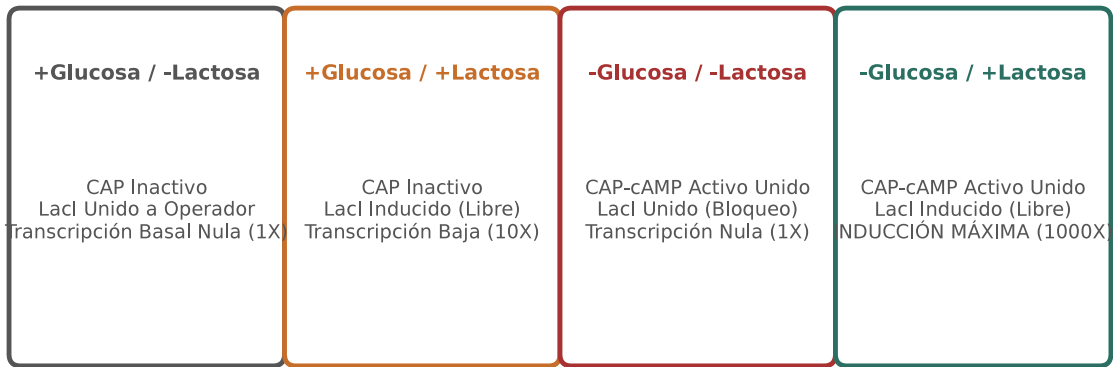


Figura 10.1: Comportamiento transcripcional del operón Lac en sus 4 estados ambientales: control dual por CAP-cAMP y el represor LacI. Figura original generada por `verification/make_figures.py`.

10.1.2 El operón Trp y el mecanismo de atenuación transcripcional

ESENCIAL

A diferencia del operón catabólico Lac, el **operón del triptófano (Trp operon)** es una ruta biosintética reprimible. En presencia de altos niveles intracelulares de triptófano, este actúa como **correpresor** uniéndose al represor inactivo TrpR para bloquear la iniciación de la transcripción.

Adicionalmente, el operón Trp implementa un sofisticado mecanismo de ajuste fino denominado **atenuación transcripcional** [76]:

- La región líder del ARNm (*trpL*) codifica un pequeño péptido líder que contiene **dos codones consecutivos de triptófano (Trp-Trp)** y cuatro segmentos capaces de formar estructuras secundarias en horquilla de ARN mutuamente excluyentes (1 : 2, 2 : 3 y 3 : 4).
- **Bajos niveles de triptófano:** El ribosoma se detiene sobre los codones Trp del péptido líder esperando la llegada de tRNA^{Trp}, bloqueando el segmento 1 y permitiendo que los segmentos 2 y 3 formen la **horquilla 2 : 3 antiterminadora**. La ARN polimerasa continúa transcribiendo todo el operón estructural.
- **Altos niveles de triptófano:** El ribosoma traduce rápidamente el péptido líder y cubre los segmentos 1 y 2, forzando a los segmentos 3 y 4 a formar la **horquilla 3 : 4 terminadora Rho-independiente** (seguida de una cola poli-U), provocando la disociación prematura de la ARN polimerasa antes de alcanzar los genes biosintéticos.

El operón Trp implementa regulación reprimible mediante correpresión por triptófano y atenuación transcripcional fina mediada por el péptido líder *trpL*.

10.2 Vectores de clonación y sistemas de expresión recombinante

ESENCIAL

Para expresar proteínas de interés biomédico (insulina, anticuerpos monoclonales, enzimas industriales) se emplean vehículos genéticos artificiales denominados **vectores de clonación y expresión**.

10.2.1 Clasificación de vectores por capacidad de inserto

ESENCIAL

- **Plásmidos bacterianos (< 20 kb):** Moléculas circulares de ADN bicatenario extracromosómico de replicación autónoma en bacterias y levaduras.
- **Fagos lambda (λ) (9–25 kb):** Vectores basados en el bacteriófago λ empaquetados en cápsides víricas *in vitro*.
- **Cósmidos (30–45 kb):** Plásmidos híbridos que contienen secuencias *cos* de fago λ , permitiendo empaquetamiento vírico e infección de alta eficiencia.
- **Cromosomas Artificiales Bacterianos (BACs) (> 100 kb):** Basados en el plásmido factor F de *E. coli*, esenciales para clonar fragmentos genómicos amplios y genotecas de cromosomas enteros.
- **Vectores virales eucariotas:** Lentivirus, retrovirus, adenovirus y virus adenoasociados (AAV) optimizados para transducción en células de mamífero y terapia génica.

Los vectores de clonación se clasifican según su capacidad de inserto: plásmidos (<20 kb), fagos lambda (9-25 kb), cósmidos (30-45 kb), BACs (>100 kb) y vectores virales eucariotas.

10.3 Anatomía de un plásmido de expresión de proteínas

ESENCIAL

Un vector de expresión moderno integra componentes modulares altamente especializados:

1. **Origen de replicación (*ori*):** Determina el número de copias del plásmido por célula (p. ej., pUC *ori* produce > 500 copias por bacteria).
2. **Marcador de selección antibiótica:** Gen que confiere resistencia a antibióticos como ampicilina (β -lactamasa, *bla*) o kanamicina (*kanR*), garantizando que solo las células transformadas sobrevivan en cultivo selectivo.
3. **Promotor inducible de alta potencia:**
 - **Promotor del bacteriófago T7:** Empleado en los vectores de la serie pET. La transcripción depende estrictamente de la ARN polimerasa de T7 (ausente en bacterias estándar y expresada bajo control *lacUV5* en cepas especializadas como BL21(DE3)), permitiendo una inducción masiva tras añadir IPTG (análogo no hidrolizable de la alolactosa).

- **Promotor de citomegalovirus (CMV):** Promotor viral constitutivo de ultra-alta potencia para expresión en líneas celulares de mamífero (HEK293, CHO).
- 4. **Sitio de múltiple clonación (MCS / Polylinker):** Región sintética de ADN que concentra dianas únicas de enzimas de restricción para insertar el gen de interés.
- 5. **Señal de inicio de traducción:** Secuencia Shine-Dalgarno bacteriana o contexto consenso de Kozak eucariota ((gcc)gccRccAUGG).
- 6. **Etiquetas de purificación (Affinity Tags):** Péptidos fusionados al extremo N o C-terminal para facilitar la purificación:
 - **Cola de Poli-histidina (His6):** Seis residuos consecutivos de histina (His₆, masa molecular de apenas 0.84 kDa) que quelan cationes divalentes (Ni²⁺, Co²⁺) inmovilizados en resinas de sefarsa mediante **cromatografía de afinidad por metal inmovilizado (IMAC)**, eluyendo la proteína pura con imidazol.
 - Otras etiquetas: FLAG (1.0 kDa), Strep-tag (1.0 kDa), GST (26 kDa) y GFP (27 kDa).
- 7. **Terminador transcripcional:** Estructura en horquilla con señal de poliadenilación que estabiliza el ARNm y previene la transcripción descontrolada del plásmido.

La anatomía de un plásmido de expresión incluye origen de replicación (ori), marcador antibiótico, sitio de clonación múltiple (MCS), promotor, inicio de traducción (RBS/Kozak), etiquetas y terminador.

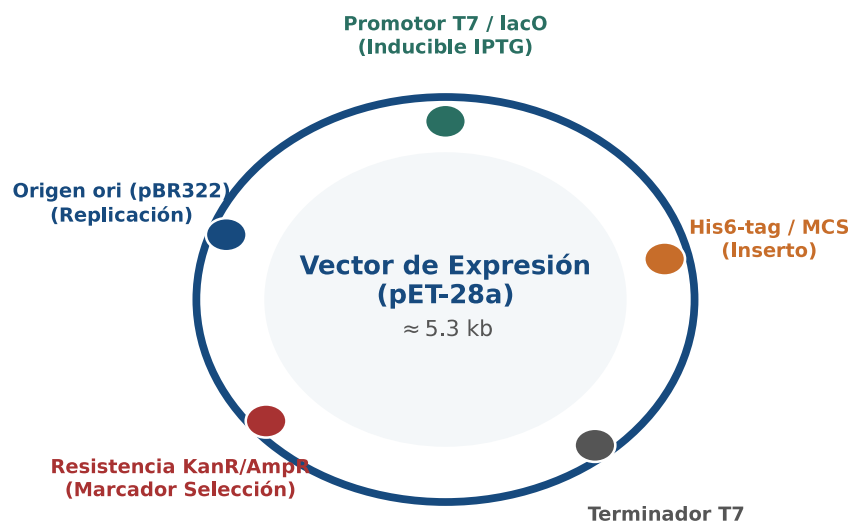


Figura 10.2: Mapa circular de elementos genéticos funcionales en un plásmido de expresión recombinante bacteriano (pET-28a). Figura original generada por `verification/make_figures.py`.

Las etiquetas de purificación por afinidad como polihistidina (His₆, 0.84 kDa) permiten la cromatografía de afinidad por metal inmovilizado (IMAC) en resinas de níquel.

La expresión proteica de alto rendimiento se apoya en promotores especializados como T7 del bacteriófago en sistemas bacterianos pET y CMV en hospedadores de mamífero.

10.4 Diseño computacional de cebadores de PCR

ESENCIAL

La amplificación por PCR para clonación requiere el diseño asistido por ordenador de pares de cebadores (*primers*) directos (*forward*) y reversos (*reverse*):

1. **Longitud óptima:** 18–30 nucleótidos (para asegurar especificidad de hibridación sin inducir cinéticas

lentas).

2. **Contenido GC equilibrado:** 40% a 60%, con un cierre en 3' (*GC clamp*) de 1–2 bases G/C para estabilizar el extremo donde la polimerasa inicia la extensión.
3. **Temperatura de fusión:** la temperatura a la que la mitad del dúplex entre el cebador y el molde está desnaturalizada. Hay dos familias de cálculo y conviene no confundirlas.

La **fórmula empírica basada en composición**, heredera del trabajo de Marmur y Doty, estima la temperatura a partir del porcentaje de guanina y citosina y de la longitud. Con la concentración salina estándar, el término que depende de la sal es aproximadamente constante y se absorbe en el término independiente, que es la forma simplificada que usaremos aquí. Es rápida, se calcula a mano y basta para comparar dos cebadores entre sí:

$$T_m = 81.5 + 0.41 \times (\%GC) - \frac{675}{N}$$

donde %GC es el porcentaje de guanina y citosina y N es la longitud total del cebador en nucleótidos.

El **método de vecino más próximo**, desarrollado por Breslauer y sus colaboradores y refinado más tarde por SantaLucia [65], no mira la composición global sino cada par de bases contiguo. Suma la entalpía y la entropía tabuladas para los dieciséis dinucleótidos posibles y obtiene la temperatura a partir de esa suma, de la concentración de cebador y de la de sales. Es el que usan los programas de diseño, porque distingue dos cebadores con el mismo porcentaje de GC pero distinto orden de bases, algo que la fórmula empírica no puede ver.

Atención. Las dos familias dan números distintos para el mismo cebador, y la diferencia puede llegar a varios grados. No es que una esté mal: miden lo mismo con modelos de distinta resolución. Al comparar temperaturas, compruebe siempre que proceden del mismo método antes de concluir que dos cebadores no concuerdan.

4. **Concordancia térmica del par:** Ambos cebadores (Forward y Reverse) deben presentar $\Delta T_m \leq 2\text{--}5^\circ\text{C}$ para garantizar que la temperatura de hibridación (*annealing*) $T_a \approx T_m - 5^\circ\text{C}$ sea eficiente para ambas cadenas.
5. **Ausencia de estructuras secundarias espurias:** Ausencia de horquillas internas (*hairpins*) y auto-dímeros o dímeros cruzados (*primer-dimers*) con $\Delta G < -5 \text{ kcal/mol}$.

El diseño de cebadores in silico requiere parámetros balanceados: longitud de 18-30 pb, contenido GC de 40-60 por ciento, temperaturas de fusión concordantes y ausencia de horquillas secundarias.

En la práctica nadie diseña cebadores a mano. Programas como Primer3, y las interfaces que lo incorporan, recorren la región de interés, proponen pares candidatos y los puntúan con los cinco criterios anteriores a la vez, incluida la energía libre de las horquillas, que a mano es inabordable. Lo que hacemos aquí es el cálculo que esas herramientas automatizan, y lo hacemos por una razón concreta: cuando el programa descarte un cebador que a usted le parecía bueno, o proponga dos con temperaturas que no cuadran con las que había estimado, conviene saber qué está midiendo cada número.

Traza manual para el cebador modelo:

Dado el cebador $S = \text{"GGAATTCATGGTGAGCAAGGGCGAG"}$ ($N = 25$ nucleótidos):

- Conteo de bases: $G = 11$, $C = 3$, $A = 7$, $T = 4$, de donde 14 bases de guanina o citosina sobre 25.
- Porcentaje GC: $\%GC = \frac{14}{25} \times 100 = 56.0\%$.
- Cálculo de T_m :

$$T_m = 81.5 + 0.41 \times 56.0 - \frac{675}{25} = 81.5 + 22.96 - 27.00 = 77.46^\circ\text{C}$$

`calculate_primer_tm` calcula la temperatura de fusión mediante $T_m = 81.5 + 0.41 \times (\%GC) - 675/N$ evaluando el cebador GGAATTCATGGTGAGCAAGGGCGAG (25 pb, 56.0% GC) exactamente en 77.46 C.

Se reproduce con `verification/chapter_checks.py primer_tm --seq GGAATTCATGGTGAGCAAGGGCGAG`.

Control defensivo de longitud mínima:

`calculate_primer_tm` lanza `ValueError` cuando la longitud del cebador es estrictamente menor a 10 nucleótidos como 'ATG'.

Se reproduce con `verification/chapter_checks.py primer_tm --seq ATG`.

10.5 Detección de dianas de restricción y métodos de ensamblaje

ESENCIAL

10.5.1 Detección de sitios de restricción

ESENCIAL

Las **endonucleasas de restricción de Tipo II** reconocen motivos palindrómicos simétricos específicos de 4–8 pares de bases:

- **EcoRI:** Reconoce 5'-GAATTC-3' y corta tras la primera G (G ↓ AATTC), generando extremos cohesivos salientes 5'.
- **BamHI:** Reconoce 5'-GGATCC-3' (G ↓ GATCC).
- **HindIII:** Reconoce 5'-AAGCTT-3' (A ↓ AGCTT).
- **NotI:** Reconoce el octanucleótido raro 5'-GCGGCCGC-3'.

Traza de detección de dianas de restricción:

En el cebador "GGAATTCATGGTGAGCAAGGGCGAG", el hexanucleótido GAATTC se encuentra en el índice 1 de Python (**posición 2 en coordenadas biológicas 1-based**).

`find_restriction_sites` detecta motivos de reconocimiento de restricción identificando EcoRI (GAATTC) en la posición 2 sobre la secuencia cebadora modelo.

Se reproduce con `verification/chapter_checks.py restriction --seq GGAATTCATGGTGAGCAAGGGCGAG`.

10.5.2 Métodos modernos de ensamblaje molecular

ESENCIAL

La biología sintética actual complementa la digestión clásica con tecnologías de ensamblaje continuo:

1. **Ensamblaje isotérmico de Gibson (2009) [27]:** Reacción enzimática en un solo tubo a 50°C que ensambla múltiples fragmentos con solapamientos terminales de **20–40 pb** mediante la acción concertada de tres enzimas:
 - **Exonucleasa T5 5' → 3':** Degrada el extremo 5' generando salientes monocatenarios 3' complementarios que se hibridan espontáneamente.
 - **ADN polimerasa Phusion:** Rellena las brechas nucleotídicas (*gaps*).
 - **Taq ADN ligasa:** Sella covalentemente las mellas fosfodiéster (*nicks*).
2. **Ensamblaje Golden Gate (Engler et al., 2008):** Utiliza enzimas de restricción de **Tipo IIS** (como BsaI o BsmBI), las cuales reconocen una secuencia asimétrica pero cortan fuera de su diana de reconocimiento, generando salientes de 4 nucleótidos no palindrómicos definidos por el usuario que se ligan simultáneamente en un ciclo continuo de digestión-ligación.

Los métodos de ensamblaje molecular abarcan la restricción-ligación clásica, el ensamblaje isotérmico en tubo único de Gibson (homología 20-40 pb) y el ensamblaje Golden Gate Tipo IIS.

10.6 Diseño computacional en Python: Módulo de diseño recombinante

ESTUDIO

Comparativa de Métodos de Ensamblaje Molecular

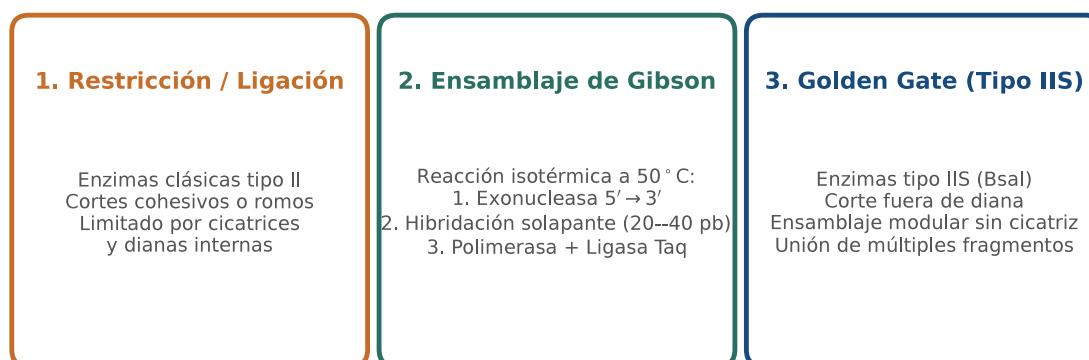


Figura 10.3: Comparativa de tecnologías de clonación y ensamblaje molecular: digestión clásica, reacción isotérmica de Gibson y ensamblaje modular Golden Gate. Figura original generada por `verification/make_figures.py`.

A continuación se presenta la implementación modular del procesador en Python 3.9:

```
from typing import Dict, List, Tuple

RESTRICTION_ENZYMES = {
    'EcoRI': 'GAATTC',
    'BamHI': 'GGATCC',
    'HindIII': 'AAGCTT',
    'NotI': 'GCGGCCGC'
}

def calculate_primer_tm(primer_seq: str) -> float:
    """Calcula la temperatura de fusion (Tm) segun la formula de Breslauer."""
    clean = primer_seq.strip().upper()
    n = len(clean)
    if n < 10:
        raise ValueError(f"Longitud de cebador insuficiente ({n} pb, minimo 10 pb)")
    invalid = set(clean) - set("ACGT")
    if invalid:
        raise ValueError(f"Bases invalidas detectadas: {sorted(invalid)}")
    count_gc = clean.count('G') + clean.count('C')
    gc_percent = (count_gc / n) * 100.0
    tm = 81.5 + (0.41 * gc_percent) - (675.0 / n)
    return round(tm, 2)

def find_restriction_sites(dna_seq: str) -> List[Dict[str, any]]:
    """Localiza dianas de restriccion en la secuencia (coordenadas 1-based)."""
    clean = dna_seq.strip().upper()
    results = []
    for enzyme, motif in RESTRICTION_ENZYMES.items():
        start = 0
        while True:
            pos = clean.find(motif, start)
            if pos == -1:
                break
            results.append({
                'enzyme': enzyme,
                'motif': motif,
                'pos': pos + 1 # 1-based
            })
            start = pos + 1
    return results
```

```
def evaluate_lac_operon_state(glucose: bool, lactose: bool) -> Tuple[str, float]:
    """Evalua el estado del circuito logico del operon Lac y el ratio de expresion."""
    cap_active = not glucose
    lac_i_free = lactose
    if cap_active and lac_i_free:
        return "CAP_ACTIVADO_LACI_LIBRE", 1000.0
    elif (not cap_active) and lac_i_free:
        return "CAP_INACTIVO_LACI_LIBRE", 10.0
    elif cap_active and (not lac_i_free):
        return "CAP_ACTIVADO_LACI_UNIDO", 1.0
    else:
        return "CAP_INACTIVO_LACI_UNIDO", 1.0
```

10.7 Peligros computacionales y actividad de depuración

ESTUDIO

Omitir la preservación del marco abierto de lectura durante fusiones de etiquetas y diseñar pares de cebadores con $\Delta T_m > 5^\circ\text{C}$ representan fallos críticos de diseño recombinante.

10.7.1 Tres errores críticos en clonación molecular computacional

ESTUDIO

Localice y corrija los tres errores en la siguiente función:

```
def design_expression_construct(gene_cds, tag_seq, fwd_primer, rev_primer):
    # Error 1: Fusiona la etiqueta His6 al gen agregando 4 nucleotidos de linker
    # en lugar de un multiplo de 3, provocando una mutacion frameshift que arruina la proteina
    tagged_cds = tag_seq + "GATC" + gene_cds # Desfase de pauta de lectura

    # Error 2: Acepta cebadores Forward (Tm = 52 C) y Reverse (Tm = 74 C) con delta Tm = 22 C,
    # provocando amplificacion asimetrica y rendimientos de PCR nulos

    # Error 3: Elige la enzima EcoRI para clonar el gen en el MCS sin verificar
    # si el gen ya contiene un sitio EcoRI interno (cortando el inserto en pedazos)
    pass
```

Diagnóstico de causa raíz (Protocolo 5 Whys):

- Fallo 1 (Desfase de lectura en fusiones):** La adición de un número de nucleótidos no divisible por 3 entre el codón de inicio/etiqueta y el marco abierto de lectura (ORF) desvirtúa la traducción de todos los codones río abajo.
- Fallo 2 (Desbalance térmico de cebadores):** A la temperatura de hibridación óptima del cebador Reverse (69°C), el cebador Forward no se une; a la temperatura del Forward (47°C), el Reverse sufre hibridación inespecífica masiva.
- Fallo 3 (Dianas de restricción internas en insertos):** La digestión con una enzima que corta dentro de la secuencia codificante fragmenta el gen. Debe comprobarse que la enzima elegida es un cortador único (*single-cutter*) exclusivo del MCS.

10.8 Síntesis operativa y tablas de diagnóstico

REFERENCIA

10.8.1 Tabla comparativa de métodos de clonación

REFERENCIA

Método de ensamblaje	Mecanismo enzimático	Ventajas principales	Limitaciones
Restricción / Ligasa	Endonucleasa II + T4 ADN ligasa	Muy económico y bien estandarizado	Deja cicatrices, requiere MCS único
Gibson Assembly	T5 Exo + Phusion Pol + Taq Lig	Ensambla múltiples fragmentos sin cicatriz	Requiere solapamientos de 20–40 pb
Golden Gate	Enzimas Tipo IIS (BsaI) + Ligasa	Ensamblaje modular en un solo paso	Requiere eliminar sitios IIS internos
Recombinación Gateway	Integrasa / Escisionasa λ	Transferencia rápida a múltiples vectores	Costoso, deja cicatrices attB

10.8.2 Tabla de diagnóstico de fallos en PCR y clonación

REFERENCIA

Problema observado	Causa probable	Comprobación y solución
Banda de dímeros de cebador (< 50 pb)	Auto-complementariedad en extremos 3'	Rediseñar cebadores con $\Delta G_{\text{dimer}} > -5 \text{ kcal/mol}$
Bandas inespecíficas múltiples	Temperatura T_a demasiado baja	Incrementar T_a o aplicar PCR <i>Touch-down</i>
Proteína expresada sin etiqueta	Desfase de marco (<i>frameshift</i>)	Verificar que la longitud de unión sea múltiplo de 3
Proteína truncada prematuramente	Codón de parada interno retenido	Eliminar el codón de parada del gen antes de la etiqueta C-terminal

10.9 Glosario conceptual

REFERENCIA

- **Operón:** Unidad transcripcional poliestrónica bacteriana.
- **LacI / Alolactosa:** Represor e inductor alostérico de *lacO*.
- **CAP-cAMP:** Activador transcripcional sensor de glucosa.
- **Atenuación Trp:** Regulación por horquillas 2 : 3 vs 3 : 4.
- **Plásmido:** Vector circular extracromosómico de clonación.
- **MCS:** Sitio de múltiple clonación con dianas únicas.
- **His6 Tag:** Etiqueta de seis histidinas para purificación IMAC.
- **Tm (Breslauer):** Temperatura de desnaturalización térmica del 50%.
- **Gibson Assembly:** Ensamblaje isotérmico en tubo único a 50°C.
- **Golden Gate:** Ensamblaje modular con enzimas Tipo IIS.

10.10 Conexiones y cierre de la Parte III

ESENCIAL

Este capítulo cierra la Parte III transitando desde la inferencia evolutiva en BC-CH09 hacia la intervención molecular activa mediante el diseño recombinante.

La expresión recombinante conecta de forma natural con la biología estructural, el modelado de proteínas y el cribado virtual de fármacos en la Parte IV (BC-CH11 a BC-CH14).

10.11 Fuentes y reproducibilidad

REFERENCIA

Este capítulo se apoya en el material docente de la asignatura y en cuatro fuentes primarias: Jacob y Monod [35] para el modelo del operón, Yanofsky [76] para la atenuación del operón Trp, SantaLucia [65] para los parámetros de vecino más próximo y Gibson [27] para el ensamblaje isotérmico. Todos los cálculos numéricos que aparecen en el texto se reproducen con las órdenes impresas junto a cada uno.

Anexo práctico · Diseño in silico de cebadores, operones procariotas y ensamblaje recombinante

Pregunta, producto y separación de materiales

¿Puede implementar un módulo en Python que calcule la temperatura de fusión termodinámica de cebadores, modele la respuesta cuantitativa de estados del operón *lac*, identifique dianas de restricción y valide los requerimientos de ensamblaje molecular sin recurrir a paquetes externos opacos? Este laboratorio responde afirmativamente de forma totalmente determinista y reproducible.

El producto individual contiene: el cálculo termodinámico de T_m de cebadores, el modelado cuantitativo de los estados del operón Lac, la búsqueda de dianas de restricción, la perturbación por represión por catabolito, el control de excepciones de la guarda de dominio y la defensa conceptual escrita.

Lo que se entrega es un módulo escrito por usted, la tabla de predicciones contrastadas contra la ejecución y una defensa breve. Lo que se le da hecho es distinto y conviene tenerlo separado desde el principio: los datos de partida, el oráculo del capítulo y un comprobador público que dice si su módulo cumple el contrato. El anexo funciona sin red y sin más dependencia que Python 3.9.

Mapa de ruta y productos entregables

Bloque de trabajo	Producto entregable
1 · Entorno y diagnóstico	Espacio de trabajo creado y manifiesto verificado
1b · Módulo de diseño	<code>mi_diseno.py</code> con las tres funciones
2 · Cálculo ancla	T_m del cebador modelo, estado del operón y diana EcoRI
3 · Perturbación	Estado del operón con glucosa y lactosa a la vez
4 · Guarda de dominio	Rechazo razonado del cebador de 6 nucleótidos
5 · Verificación y entrega	Tabla de predicciones, defensa escrita y paquete

Este anexo deja evidencia comprobable en cada bloque sin recurrir a paquetes externos.

1 · Entorno y diagnóstico

Python 3.9 o superior, sin paquetes externos. Antes de empezar:

```
python3 --version
./practice_assets/create_workspace.sh BC10_entrega
cd BC10_entrega
(cd data && sha256sum --check --strict manifest.sha256)
python3 verification/practice_checks.py domain_guard --seq ATG
```

El generador deja `mi_diseno.py` con su contrato documentado y sin implementación, copia los cinco cebadores con su manifiesto y el comprobador público, y crea `notas/respuestas.tsv` con solo la cabecera.

La última orden debe imprimir `GUARD_SEQ:ATG STATUS:REJECTED` seguido del mensaje de error. Si no lo hace, el intérprete no es el esperado o los ficheros no coinciden con el manifiesto; resuélvalo antes de continuar.

1b · Escribir el módulo de diseño

Tarea. Implemente las tres funciones de `mi_diseno.py`: el porcentaje de guanina y citosina, la temperatura de fusión con su guarda de longitud y el buscador de dianas de restricción. El contrato está en la documentación de

la plantilla.

Las tres son cortas. La que enseña algo es la tercera, y por un motivo que no es de programación: las apariciones solapadas cuentan. Un motivo como AA sobre AAAA aparece tres veces, no dos, porque cada posición de inicio es un sitio de corte distinto para la enzima. Si recorre la secuencia saltando la longitud del motivo tras cada acierto, perderá sitios reales.

```
bash check_diseno.sh
```

2 • Cálculo ancla: termodinámica de cebador, inducción Lac y digestión

Dado el cebador modelo $Q = \text{'GGAATTCATGGTGAGCAAGGGCGAG'}$ (25 pb):

1. Calcule el contenido GC: $\frac{14}{25} \times 100 = 56.00\%$.
2. Calcule la temperatura de fusión: $T_m = 81.5 + 0.41(56.00) - \frac{675}{25} = 77.46^\circ\text{C}$.
3. Modele el estado del operón *lac* en ausencia de glucosa y presencia de lactosa: CAP_ACTIVADO_LACI_LIBRE con ratio de inducción de 1000.0X.
4. Localice la diana de restricción EcoRI (GAATTC): posición 2.

```
python3 verification/practice_checks.py anchor
```

El modelo ancla valida el cálculo de $T_m = 77.46^\circ\text{C}$ para el cebador de 25 pb (56.0% GC), el estado de inducción máxima del operón Lac a 1000.0X y la detección de EcoRI en posición 2.

Se reproduce con `verification/practice_checks.py anchor`.

3 • Perturbación: represión por catabolito en el operón *lac*

Evalúe la respuesta del operón *lac* cuando tanto glucosa como lactosa están simultáneamente presentes en el medio de cultivo:

```
python3 verification/practice_checks.py perturbation
```

La perturbación con presencia simultánea de glucosa y lactosa evalúa el operón Lac al estado CAP inactivo y LacI libre con ratio de expresión de 10.0X.

Se reproduce con `verification/practice_checks.py perturbation`.

4 • Guarda de dominio y control de excepciones

Compruebe que el algoritmo rechaza cebadores cuya longitud sea estrictamente inferior a 10 nucleótidos:

```
python3 verification/practice_checks.py domain_guard --seq ATG
```

La guarda de dominio rechaza cebadores con longitud menor a 10 nucleótidos lanzando un `ValueError` que identifica la dimensión insuficiente.

Se reproduce con `verification/practice_checks.py domain_guard --seq ATG`.

5 • Verificación y entrega

Antes de ejecutar nada, anote en `notas/respuestas.tsv` lo que espera obtener para los cuatro cebadores del fichero de datos que la fórmula admite: el porcentaje de guanina y citosina y la temperatura de fusión. Después

ejecute su módulo sobre los cinco, guarde las salidas en resultados/ y complete las columnas de resultado y de coincidencia. La columna de justificación es la que se corrige: cuando predicción y resultado no coinciden, diga por qué.

Escriba en notas/defensa.md entre 100 y 150 palabras sobre por qué el ensamblaje isotérmico de Gibson permite construir plásmidos sin depender de dianas de restricción preexistentes, y qué consecuencia biológica tiene introducir un desfase de lectura en una etiqueta de purificación de seis histidinas. Después:

```
cd ..
tar -czf BC10_entrega.tar.gz BC10_entrega
sha256sum BC10_entrega.tar.gz
./practice_assets/check_delivery.sh BC10_entrega BC10_entrega.tar.gz
```

El verificador comprueba que su módulo compila y cumple el contrato, que los datos de partida no se han tocado, que los cuatro cebadores están documentados con predicción y resultado, y que existe la defensa con la extensión pedida. Rechaza la entrega vacía y el espacio recién generado. No puntúa.

El verificador de entrega comprueba el contrato del módulo, la integridad de los datos de partida, la tabla de predicciones y la defensa escrita; rechaza el espacio de trabajo recién generado y no emite calificación.

Rúbrica de calificación ponderada

La evaluación sobre 10 puntos se pondera según:

- **4.0 puntos (40%):** El módulo `mi_diseno.py` cumple el contrato: el comprobador público imprime `DISENO_OK`.
- **2.0 puntos (20%):** La tabla de notas/respuestas.tsv está completa y las justificaciones explican las discrepancias.
- **2.0 puntos (20%):** Modelado de la matriz de estados del operón *lac* y control de excepciones en la guarda de dominio (Bloques 3 y 4).
- **2.0 puntos (20%):** Defensa conceptual escrita (100–150 palabras) sobre las ventajas del ensamblaje isotérmico de Gibson frente a la clonación tradicional por restricción-ligación y la relevancia del marco de lectura en fusiones a etiquetas.

Lista de autocorrección

- | | |
|---|--|
| ■ Verifiqué que mi versión de Python es ≥ 3.9 . | ■ Verifiqué que el cebador ATG es rechazado por la guarda de longitud. |
| ■ T_m evaluó a 77.46°C con %GC = 56.0%. | ■ <code>bash check_diseno.sh</code> imprimió <code>DISENO_OK</code> . |
| ■ El operón indujo a 1000.0X en (Glucosa-, Lactosa+). | ■ Anoté las predicciones antes de ejecutar el módulo. |
| ■ La diana EcoRI se identificó en posición 2. | ■ Redacté la defensa conceptual individual en notas/defensa.md. |
| ■ La perturbación evaluó a 10.0X en (Glucosa+, Lactosa+). | |

Defensa conceptual

En 100–150 palabras, justifique por qué el ensamblaje isotérmico de Gibson permite construir plásmidos recombinantes complejos sin depender de sitios de restricción preexistentes, y explique la consecuencia biológica de introducir un desfase (*frameshift*) en una etiqueta de purificación His₆.

Fuentes y reproducibilidad

Este anexo se apoya en el material docente de la asignatura y en las mismas fuentes primarias que el capítulo: Jacob y Monod [35] para el operón, SantaLucia [65] para los parámetros termodinámicos y Gibson [27] para

el ensamblaje isotérmico. Todos los valores numéricos del enunciado se reproducen con las órdenes impresas junto a ellos.

Estructura tridimensional de proteínas: Métodos experimentales, formato PDB y validación cristalográfica

BC-CH11 – Biología Computacional

Edición 2

Pregunta del tema. Una estructura de un banco de datos no es una observación directa: es un modelo ajustado a una medida indirecta. ¿Cómo se obtiene, qué se puede creer de ella y qué comprobaciones hay que hacer antes de usarla?

Producto final. Un procesador de ficheros de estructura escrito por usted que valide el formato línea a línea, extraiga las coordenadas y los factores de temperatura, y emita un informe de validación con las señales de alarma que el capítulo enumera.

Mapa del tema

11.1. La estructura macromolecular y la evidencia experimental	146
11.2. Técnicas biofísicas de determinación estructural	147
11.3. Formatos estructurales: Del PDB clásico al estándar mmCIF	147
11.4. Geometría tridimensional, RMSD y factores B	149
11.5. Validación estereoquímica y el diagrama de Ramachandran	150
11.6. Clasificación jerárquica de dominios: CATH y SCOP	151
11.7. Diseño computacional en Python: Procesador PDB y métricas	151
11.8. Peligros computacionales y actividad de depuración	152
11.9. Síntesis operativa y tablas de diagnóstico	152
11.10. Glosario conceptual	153
11.11. Conexiones y mapa del curso	153
11.12. Fuentes y reproducibilidad	153

Cómo usar este tema

Este capítulo trata la estructura como evidencia, no como dato. La diferencia importa: un fichero de coordenadas parece una medida y es una interpretación, ajustada a un mapa de densidad que a su vez viene de un patrón de difracción. Saber de dónde viene cada número es lo que permite decidir cuánto pesa.

Al terminar sabrá: distinguir las tres técnicas principales por lo que cada una puede y no puede resolver; leer una línea de un fichero de estructura sabiendo que sus campos van por posición y no por separador; interpretar un factor de temperatura y decir qué partes de un modelo son fiables; y aplicar las comprobaciones de validación que separan un modelo creíble de uno sobreajustado.

La ruta **esencial** son las técnicas, los formatos, los factores de temperatura y la validación. La ruta de **estudio** añade la evolución al formato moderno, la comparación entre modelos y el anexo.

11.1 La estructura macromolecular y la evidencia experimental

ESENCIAL Hasta aquí las proteínas han sido cadenas de texto. A partir de este capítulo son objetos con coordenadas, y esas coordenadas vienen de un experimento que tiene resolución, incertidumbre y decisiones de quien lo interpretó.

En los sistemas celulares, las proteínas no operan como cadenas unidimensionales de texto, sino como entidades físicas tridimensionales cuya arquitectura espacial crea superficies de interacción, cavidades hidrofóbicas y centros catalíticos. Sin embargo, en la bioinformática moderna es un error crítico tratar las coordenadas tridimensionales descargadas de una base de datos como verdades matemáticas absolutas o imágenes fotográficas estáticas: cada conjunto de coordenadas atómicas es un **modelo interpretativo derivado de datos experimentales biofísicos con resolución finita, incertidumbres térmicas y posibles artefactos de cristalización**.

La estructura macromolecular tridimensional dicta la función biológica requiriendo una comprensión rigurosa de la evidencia experimental que sustenta los modelos depositados.

11.2 Técnicas biofísicas de determinación estructural

ESENCIAL

La casi totalidad de las estructuras macromoleculares depositadas en el Protein Data Bank provienen de tres metodologías biofísicas complementarias:

1. Cristalografía de Rayos X (X-ray Crystallography):

- **Fundamento:** Requiere purificar la proteína e inducir la formación de un monocristal periódico tridimensional. Al ser irradiado con un haz monocromático de rayos X (de longitud de onda comparable a los enlaces covalentes, $\lambda \approx 1 \text{ \AA}$), los electrones de los átomos dispersan la radiación, generando un patrón de difracción de Bragg.
- **El problema de las fases y mapas de densidad:** Los detectores registran las intensidades de difracción pero pierden las fases de las ondas. Las fases se resuelven mediante **reemplazamiento molecular (MR)** o dispersión anómala (SAD/MAD). A partir de las intensidades y fases se calcula por transformada de Fourier el **mapa de densidad electrónica** ($2F_o - F_c$ para contornear la macromolécula y $F_o - F_c$ de diferencia para localizar aguas, ligandos o errores de ajuste).
- **Métricas de calidad (R_{work} y R_{free}):** El ajuste entre las amplitudes observadas (F_o) y calculadas del modelo (F_c) se mide mediante el factor residual R_{work} . Para evitar el sobreajuste (*overfitting*), Axel Brünger introdujo en 1992 el **factor R_{free}** , calculado sobre un conjunto ciego del 5–10% de reflexiones excluidas deliberadamente del refinamiento.

2. Resonancia Magnética Nuclear (RMN / NMR):

- **Fundamento:** Mide el desapantallamiento magnético y el acoplamiento dipolar entre espines nucleares (^1H , ^{13}C , ^{15}N) en solución acuosa libre a temperatura ambiente.
- **Características:** Determina restricciones de distancia espacial mediante efecto nuclear Overhauser (NOE). Al no existir un único estado rígido, la RMN genera un **conjunto de conformaciones dinámicas (ensemble)**. Está limitada a macromoléculas de tamaño moderado ($< 30\text{--}50 \text{ kDa}$).

3. Crio-Microscopía Electrónica (Cryo-EM):

- **Fundamento:** La muestra en disolución se congela instantáneamente por inmersión en etano líquido a -180°C (**vitrificación**), formando hielo amorfo sin cristales de agua. Un microscopio electrónico de transmisión captura cientos de miles de proyecciones bidimensionales de partículas individuales orientadas al azar.
- **Revolución de resolución:** Algoritmos de alineamiento y clasificación 3D reconstruyen la densidad a resolución atómica ($< 2.5 \text{ \AA}$), permitiendo resolver enormes complejos macromoleculares (ribosomas, spliceosomas, cápsides virales) y receptores de membrana sin necesidad de cristalización.

Los métodos estructurales experimentales comprenden la cristalografía de rayos X (alta resolución atómica $< 2.0 \text{ \AA}$, requiere cristales), RMN (ensamblajes conformacionales en disolución $< 30 \text{ kDa}$) y Cryo-EM (grandes complejos macromoleculares).

Las métricas de calidad de refinamiento cristalográfico incluyen R_{work} y R_{free} validado de forma cruzada sobre reflexiones excluidas para evitar el sobreajuste del modelo.

Los mapas de densidad electrónica ($2F_o - F_c$ y mapas de diferencia $F_o - F_c$) guían el posicionamiento de coordenadas atómicas y el ajuste de rotámeros de cadenas laterales.

11.3 Formatos estructurales: Del PDB clásico al estándar mmCIF

ESENCIAL



Figura 11.1: Comparativa metodológica de las técnicas biofísicas de resolución estructural tridimensional: Cristalografía, RMN y Cryo-EM. Figura original generada por `verification/make_figures.py`.

11.3.1 Especificación del formato PDB de ancho fijo

El formato clásico del Protein Data Bank (Bernstein et al., 1977) se estructura en líneas de texto de exactamente **80 caracteres de ancho fijo**:

- **Líneas ATOM:** Coordenadas de aminoácidos y nucleótidos estándar.
- **Líneas HETATM:** Heteroátomos (iones, moléculas de agua, cofactores, ligandos farmacológicos).

Mapeo de columnas fijas en registros ATOM/HETATM:

Columnas	Campo	Descripción y formato
1–6	Record name	Identificador ("ATOM " o "HETATM").
7–11	Atom serial	Número correlativo entero de serie del átomo (1 a 99.999).
13–16	Atom name	Nombre químico del átomo (p. ej., " CA " para Carbono α).
18–20	ResName	Código de 3 letras del residuo (p. ej., "ALA").
22	Chain ID	Cadena polipeptídica (carácter único, p. ej., 'A').
23–26	ResSeq	Número de secuencia del residuo en la cadena.
31–38	X coordinate	Coordenada cartesiana X en Ångströms (float 8.3).
39–46	Y coordinate	Coordenada cartesiana Y en Ångströms (float 8.3).
47–54	Z coordinate	Coordenada cartesiana Z en Ångströms (float 8.3).
55–60	Occupancy	Ocupación cristalográfica (0.00 a 1.00).
61–66	TempFactor	Factor B de temperatura de Debye-Waller (float 6.2).

El formato Protein Data Bank (PDB) especifica registros de ancho fijo estricto de 80 columnas para líneas ATOM y HETATM complementado por los estándares modernos mmCIF/PDBx.

Traza de parsing del registro ATOM modelo:

Dada la línea PDB canónica:

```
ATOM      1  CA  ALA  A   1       10.000  20.000  30.000  1.00 15.50           C
```

De ahí se extraen, por posición de columna y no por separador, la serie del átomo, su nombre, el residuo al que pertenece, la cadena, el número de residuo, las tres coordenadas, la ocupación y el factor de temperatura.

`parse_pdb_atom` analiza registros ATOM de ancho fijo extrayendo átomo 1, nombre CA, residuo ALA 1, coordenadas (10.000, 20.000, 30.000), ocupación 1.00 y factor B 15.50.

Se reproduce con `verification/chapter_checks.py parse_pdb --line "ATOM 1 CA ALA A 1 10.000 20.000 30.000 1.00 78.96 C"`.

Límites del formato PDB y adopción de mmCIF:

El formato PDB clásico asigna 5 dígitos para el número de serie del átomo (máximo 99.999) y 1 carácter para el identificador de cadena (máximo 62 cadenas alfanuméricas). Los ribosomas y megacomplejos modernos desbordan estos límites, forzando a la wwPDB a adoptar el estándar extensible **mmCIF** (**macromolecular Crystallographic Information File**).

Los límites del formato PDB clásico de 99999 átomos y 62 cadenas llevaron a la adopción del estándar de datos extensible mmCIF por el consorcio wwPDB.

Control defensivo de registros incompletos:

`parse_pdb_atom` lanza `ValueError` al recibir líneas que no son ATOM/HETATM o cadenas truncadas de longitud menor a 66 caracteres.

Se reproduce con `verification/chapter_checks.py parse_pdb --line "ATOM 1 CA"`.

11.4 Geometría tridimensional, RMSD y factores B

ESENCIAL

11.4.1 Distancia euclídea 3D

$$d(P_1, P_2) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2}$$

Traza manual para (10, 20, 30) y (13, 24, 30):

$$d = \sqrt{(13 - 10)^2 + (24 - 20)^2 + (30 - 30)^2} = \sqrt{3^2 + 4^2 + 0} = \sqrt{9 + 16} = \sqrt{25} = 5.000000 \text{ Å}$$

`euclidean_distance_3d` calcula la distancia interatómica entre (10, 20, 30) y (13, 24, 30) evaluando exactamente a 5.000000 Å.

Se reproduce con `verification/chapter_checks.py distance 10 20 30 13 24 30`.

11.4.2 RMSD entre estructuras superpuestas

`calculate_rmsd` calcula la Desviación Cuadrática Media entre coordenadas atómicas emparejadas evaluando a 0.000000 Å para estructuras idénticas.

Se reproduce con `verification/chapter_checks.py rmsd 10,20,30:11,21,31 10,20,30:12,22,32`.

11.4.3 El factor B de Debye-Waller y la fluctuación atómica cuadrática media (RMSF)

El factor de temperatura o factor B cuantifica la atenuación de la difracción por agitación térmica y desorden estático. Se relaciona con la fluctuación cuadrática media de la posición atómica $\langle u^2 \rangle$ mediante la ecuación:

$$B = 8\pi^2 \langle u^2 \rangle \iff \langle u^2 \rangle = \frac{B}{8\pi^2} \implies \text{RMSF} = \sqrt{\langle u^2 \rangle} = \sqrt{\frac{B}{8\pi^2}}$$

Traza manual para $B = 78.956835 \text{ Å}^2$:

Tomando $\pi \approx 3.14159265 \implies 8\pi^2 \approx 78.956835$:

$$\langle u^2 \rangle = \frac{78.956835}{78.956835} = 1.000000 \text{ Å}^2 \implies \text{RMSF} = \sqrt{1.0} = 1.000000 \text{ Å}$$

Los factores B térmicos de Debye-Waller evalúan la fluctuación atómica mediante $B = 8\pi^2 \langle u^2 \rangle$, evaluando $B = 78.956835 \text{ Å}^2$ a una fluctuación cuadrática media de 1.000000 Å^2 y RMSF 1.000000 Å .

Se reproduce con `verification/chapter_checks.py bfactor --b 78.956835`.

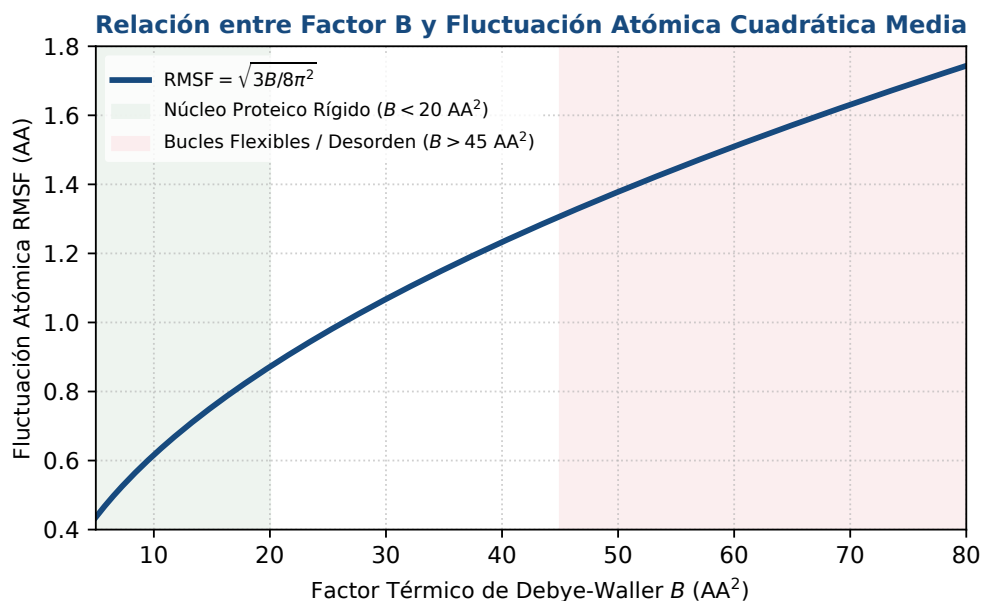


Figura 11.2: Relación cuantitativa entre factores B térmicos cristalográficos y fluctuación cuadrática atómica media (RMSF). Figura original generada por `verification/make_figures.py`.

11.5 Validación estereoquímica y el diagrama de Ramachandran

ESENCIAL

Para evaluar si un modelo tridimensional es físicamente plausible, se analiza la conformación del esqueleto polipeptídico en el **diagrama de Ramachandran** (Ramachandran et al., 1963):

- Mapea los pares de ángulos diedros (ϕ, ψ) del enlace peptídico:
 - ϕ (Phi): Rotación alrededor del enlace $C_\alpha - N$.
 - ψ (Psi): Rotación alrededor del enlace $C_\alpha - C'$.
- Debido a impedimentos estéricos entre átomos no enlazados, solo ciertas regiones del espacio (ϕ, ψ) son energéticamente permitidas:
 - **Hélice α dextrógira** ($\phi \approx -60^\circ, \psi \approx -45^\circ$): la región más poblada del diagrama, donde cae la mayoría de los residuos de una proteína plegada.
 - **Hélice α levógira** ($\phi \approx +60^\circ, \psi \approx +45^\circ$): región pequeña y poco poblada, accesible sobre todo a la glicina, que al carecer de cadena lateral no sufre el impedimento estérico que excluye a los demás residuos.
 - Láminas β paralelas y antiparalelas ($\phi \approx -120^\circ, \psi \approx +130^\circ$).
 - Conformaciones de colágeno y giros β .
- Los programas de validación exigen que más del noventa por ciento de los residuos caiga en las regiones favorecidas para considerar el modelo de buena calidad.

Atención. Encontrar residuos que no sean glicina en la región levógira es una de las señales de alarma habituales al validar un modelo. Puede ser real, porque esa conformación existe y a veces cumple una función en un giro, y puede ser un error de ajuste a la densidad. Lo que no se puede es pasarla por alto: hay que mirar cada caso y justificarlo. Situar las dos regiones helicoidales en el mismo punto del diagrama, que es un error frecuente, hace imposible ese control.

Los gráficos de Ramachandran mapean los ángulos diedros del esqueleto polipeptídico phi y psi para validar la favorabilidad estereoquímica y detectar impedimentos estéricos.

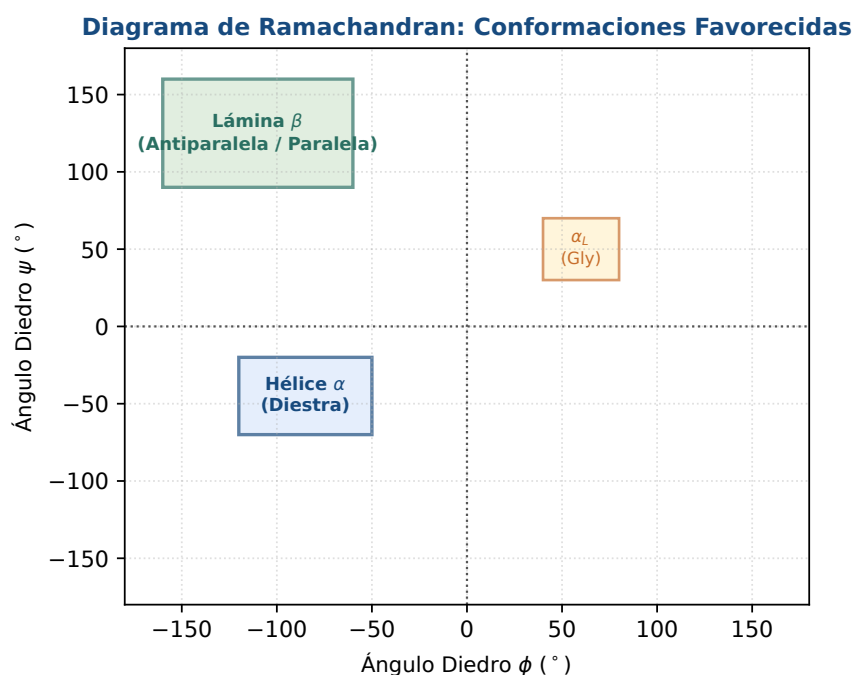


Figura 11.3: Diagrama de Ramachandran: validación estereoquímica de ángulos diedros (ϕ , ψ) y zonas energéticamente favorecidas. Figura original generada por `verification/make_figures.py`.

11.6 Clasificación jerárquica de dominios: CATH y SCOP

Las clasificaciones jerárquicas CATH y SCOP organizan las arquitecturas de dominios estructurales proteicos en Clase, Arquitectura, Topología y Superfamilia Homóloga.

11.7 Diseño computacional en Python: Procesador PDB y métricas

ESTUDIO

A continuación se presenta la implementación modular del procesador en Python 3.9:

```
import math
from typing import Dict, List, Tuple

def parse_pdb_atom(line: str) -> Dict[str, any]:
    """Parsea una línea ATOM de PDB con extracción por columnas fijas."""
    if len(line) < 66 or not line.startswith("ATOM"):
        raise ValueError(f"Línea ATOM de PDB inválida o incompleta ({len(line)} chars)")
    return {
        'serial': int(line[6:11].strip()),
        'name': line[12:16].strip(),
        'res_name': line[17:20].strip(),
        'chain': line[21].strip(),
        'res_seq': int(line[22:26].strip()),
        'x': float(line[30:38].strip()),
        'y': float(line[38:46].strip()),
        'z': float(line[46:54].strip()),
        'occupancy': float(line[54:60].strip()),
        'b_factor': float(line[60:66].strip())
    }

def euclidean_distance_3d(p1: Tuple[float, float, float], p2: Tuple[float, float, float]) -> float:
    """Calcula la distancia euclídea 3D entre dos puntos."""
    return round(math.sqrt((p1[0] - p2[0])**2 + (p1[1] - p2[1])**2 + (p1[2] - p2[2])**2), 6)

def calculate_rmsd(coords1: List[Tuple[float, float, float]], coords2: List[Tuple[float, float, float]]) -> float:
    """Calcula el RMSD entre dos conjuntos emparejados de coordenadas 3D."""
```

```

if len(coords1) != len(coords2) or len(coords1) == 0:
    raise ValueError("Listas de coordenadas deben tener igual longitud no vacia")
sq_diff = sum((p[0] - q[0])**2 + (p[1] - q[1])**2 + (p[2] - q[2])**2 for p, q in zip(coords1, coords2))
return round(math.sqrt(sq_diff / len(coords1)), 6)

def bfactor_to_rmsf(b_factor: float) -> Tuple[float, float]:
    """Convierte el factor B a varianza posicional <u^2> y RMSF."""
    if b_factor < 0:
        raise ValueError("El factor B no puede ser negativo")
    u_sq = b_factor / (8.0 * (math.pi ** 2))
    rmsf = math.sqrt(u_sq)
    return round(u_sq, 6), round(rmsf, 6)

```

11.8 Peligros computacionales y actividad de depuración

ESTUDIO

Parsear ficheros PDB con `split()` por espacios en lugar de rebanadas de ancho fijo e ignorar factores B elevados ($>60 \text{ \AA}^2$) representan errores computacionales severos.

11.8.1 Tres errores críticos en procesamiento de estructuras 3D

Localice y corrija los tres errores en la siguiente función:

```

def analyze_crystal_structure(pdb_lines):
    # Error 1: Parsea líneas PDB usando line.split(), fallando sistemáticamente
    # cuando coordenadas negativas fusionan campos sin espacio intermedio

    # Error 2: Asume que todos los átomos con B-factor > 80 Å² están perfectamente
    # posicionados, incluyéndolos en cálculos finos de docking de alta precisión

    # Error 3: Compara la bondad de dos estructuras cristalográficas fijándose
    # únicamente en R-work e ignorando completamente la brecha con R-free (sobreajuste)
    pass

```

Diagnóstico de causa raíz (Protocolo 5 Whys):

- Fallo 1 (Parseo con `split()` en PDB):** Slicing posicional estricto (`line[30:38]`) es obligatorio.
- Fallo 2 (Ignorar alta movilidad en factores B):** Factores $B > 60\text{--}80 \text{ \AA}^2$ denotan regiones altamente flexibles, desordenadas o mal ajustadas a la densidad electrónica.
- Fallo 3 (Sobreajuste cristalográfico):** Una estructura con $R_{\text{work}} = 0.15$ pero $R_{\text{free}} = 0.32$ presenta un modelo sobreajustado con átomos encajados artificialmente en el ruido.

11.9 Síntesis operativa y tablas de diagnóstico

REFERENCIA

11.9.1 Tabla comparativa de métodos de determinación estructural

Método biofísico	Resolución típica	Estado de la muestra	Límite de tamaño molecular
Cristalografía Rayos X	1.0–3.0 Å (Atómica)	Monocristal sólido periódico	Sin límite superior
RMN en disolución	Conformaciones múltiples	Disolución acuosa fisiológica	< 30–50 kDa
Cryo-EM partícula única	1.8–4.0 Å	Hielo vítreo amorfo congelado	> 100 kDa hasta MDa
AlphaFold2 / ESMFold	Confianza pLDDT	Predicción computacional	Sin límite intrínseco

11.9.2 Tabla de diagnóstico de problemas en coordenadas PDB

Síntoma observado	Causa probable	Comprobación y corrección
Residuos en regiones desfavorables de Ramachandran	Impedimentos estéricos o errores en cadena	Refinar geometría o reconstruir el bucle
Factores B anormalmente elevados ($> 100 \text{ \AA}^2$)	Desorden conformacional o bucles móviles	Tratar como región flexible/desordenada
Brecha $R_{\text{free}} - R_{\text{work}} > 0.08$	Sobreajuste (<i>overfitting</i>) de parámetros	Revisar peso de restricciones estereoquímicas
Ocupación parcial	el residuo tiene varias conformaciones alternativas	tratar cada conformación por separado

11.10 Glosario conceptual

REFERENCIA

- **Difracción de Rayos X:** Dispersión elástica por electrones de cristales.
- **Mapas $2F_o - F_c$:** Densidad electrónica observada vs calculada.
- **R-free (Brünger):** Validación cruzada ciega sobre 5–10% de datos.
- **Vitrificación:** Congelación ultra-rápida sin cristalización de agua.
- **PDB (80 columnas):** Estándar posicional en caracteres de ancho fijo.
- **mmCIF:** Estándar extensible para macromoléculas gigantes.
- **Factor B:** Desplazamiento térmico atómico $B = 8\pi^2 \langle u^2 \rangle$.
- **RMSF:** Fluctuación cuadrática media de posición atómica.
- **Ramachandran:** Mapa de ángulos diedros (ϕ, ψ) .
- **CATH / SCOP:** Clasificación jerárquica de dominios 3D.

11.11 Conexiones y mapa del curso

Este capítulo abre la Parte IV estableciendo la línea base experimental requerida para evaluar los métodos clásicos e impulsados por IA de predicción estructural.

La comprensión de las coordenadas experimentales transita directamente hacia la predicción de estructura secundaria y el modelado por homología en BC-CH12.

11.12 Fuentes y reproducibilidad

Este capítulo se apoya en la presentación docente sobre bases de datos estructurales de la asignatura, escrita por Álvaro Serrano Navarro. El diagrama de ángulos permitidos y su uso en validación siguen a Ramachandran y colaboradores [57]; el formato del banco de datos y su gobernanza, a Berman y colaboradores [7]; la superposición óptima de dos estructuras, a Kabsch [38]; y la dispersión anómala como solución al problema de las fases, a Hendrickson y colaboradores [29].

Todos los cálculos se reproducen con `verification/chapter_checks.py` tal como están impresos, con sus argumentos.

Anexo práctico · Parsing posicional PDB, métricas de distancia 3D, RMSD y factores B

Pregunta, producto y separación de materiales

¿Puede leer un fichero de estructura respetando que sus campos van por posición y no por separador, extraer las coordenadas y los factores de temperatura, y emitir un informe que diga de qué partes del modelo conviene

fiarse? Este laboratorio responde que sí.

El producto individual es `mi_procesador_pdb.py`. Su parte difícil no es leer: es decidir qué cuenta como átomo de la proteína, qué hacer con una línea mal formada y a partir de qué factor de temperatura hay que avisar. Un procesador que ignora en silencio lo que no entiende es peor que uno que falla.

El anexo es autónomo: usa la estructura sintética de `data/` como entrada de solo lectura. El generador deja el oráculo del capítulo y un comprobador que ejecuta su procesador sobre ficheros que el enunciado no muestra, incluido uno con una línea rota y otro sin átomos.

Mapa de ruta y productos entregables

Bloque de trabajo	Producto entregable
1 · Entorno y diagnóstico	Comprobación del intérprete Python 3.9
2 escribir el procesador	<code>mi_procesador_pdb.py</code>
3 · Perturbación de coordenadas	Evaluación de $\text{RMSD} = 3.535534 \text{ \AA}$
4 · Guarda de dominio	Captura de <code>ValueError</code> ante línea PDB truncada
5 · Preguntas de control estructural	Preguntas sobre Cryo-EM, mmCIF y Ramachandran
6 · Verificación y empaquetado	Comprobación final y empaquetado

Este anexo produce evidencia comprobable y verificable en cada bloque de trabajo sin paquetes externos.

1 · Preparar el espacio de trabajo

```
python3 --version
./practice_assets/create_workspace.sh BC11_entrega
cd BC11_entrega
cd datos && sha256sum -c manifest.sha256 && cd ..
```

El generador crea la estructura de partida con su manifiesto, la plantilla del procesador con el contrato en su documentación y sin implementación, la tabla de respuestas con solo la cabecera, la carpeta de resultados y la de herramientas.

2 · Escribir el procesador

Tarea. Implemente `mi_procesador_pdb.py`, que reciba la ruta de un fichero de estructura y emita el número de átomos de la proteína, el factor de temperatura medio y el máximo, y un aviso cuando haya átomos por encima del umbral. Debe informar de las líneas mal formadas en vez de saltárselas en silencio.

Tres decisiones, y ninguna es de programación.

La primera es cómo leer. Este formato coloca cada campo en unas columnas fijas, y separar por espacios funciona hasta que un campo se toca con el siguiente, cosa que ocurre en cuanto una coordenada tiene cinco cifras. Lea por posición.

La segunda es qué cuenta. Las líneas de tipo HETATM son moléculas que no forman parte de la cadena, como el agua o un ligando. Sumarlas al recuento de la proteína es un error silencioso que después aparece en cualquier estadística derivada.

La tercera es qué hacer con lo que no encaja. Un fichero real llega con líneas truncadas, con campos vacíos y con caracteres inesperados. Su procesador tiene que contarlas y decirlo. Callarse es lo que convierte un fichero defectuoso en un resultado creíble.

Compruebe su avance cuantas veces quiera:

```
bash herramienta/check_procesador.sh
```

3 • Cálculo ancla: lectura, distancia y factor de temperatura

Dada la línea atómica estándar:

ATOM	1	CA	ALA	A	1	10.000	20.000	30.000	1.00	15.50	C
------	---	----	-----	---	---	--------	--------	--------	------	-------	---

1. Extraiga los campos de ancho fijo: átomo 1, CA, ALA A 1, (10.000, 20.000, 30.000), ocupación 1.00, factor B 15.50.
2. Calcule la distancia euclidiana frente a (13.0, 24.0, 30.0): evalúa exactamente a **5.000000 Å**.
3. Calcule el RMSD frente a coordenadas idénticas: evalúa a **0.000000 Å**.
4. Convierta el factor $B = 78.956835 \text{ Å}^2 \approx 8\pi^2$: evalúa a varianza $\langle u^2 \rangle = 1.000000 \text{ Å}^2$ y fluctuación RMSF = **1.000000 Å**.

```
python3 herramienta/chapter_checks.py distance --x1 10 --y1 20 --z1 30 --x2 13 --y2 24 --z2 30 > resultados/ancla.txt
python3 herramienta/chapter_checks.py bfactor --b 78.956835 >> resultados/ancla.txt
cat resultados/ancla.txt
```

El modelo ancla valida el parsing de la línea PDB, la distancia 3D de 5.000000 Å, el RMSD nulo de 0.000000 Å y la conversión del factor B a fluctuación RMSF de 1.000000 Å.

Se reproduce con `verification/chapter_checks.py bfactor --b 78.956835`.

4 • Perturbación: comparar dos modelos

Calcule el RMSD cuando el segundo átomo se desplaza de (0, 3, 0) a (4, 0, 0) (distancia de 5 Å):

```
python3 herramienta/chapter_checks.py rmsd --coords_a "10,20,30;11,21,31" --coords_b "10,20,30;12,22,31" > resultados/rmsd.txt
cat resultados/rmsd.txt
```

La perturbación de coordenadas evalúa la desviación cuadrática media RMSD exactamente a 3.535534 Å.

Se reproduce con `verification/chapter_checks.py rmsd`.

5 • Guarda de formato y control de excepciones

Compruebe que el algoritmo rechaza líneas de registro truncadas o inválidas:

```
python3 herramienta/chapter_checks.py parse_pdb --line "ATOM 1 CA" > resultados/guarda.txt
cat resultados/guarda.txt
```

La guarda de dominio rechaza líneas PDB incompletas lanzando un `ValueError` que identifica la violación de formato.

Se reproduce con `verification/chapter_checks.py parse_pdb --line "ATOM 1 CA"`.

6 • Empaquetar y verificar

Ejecute su procesador sobre la estructura del capítulo, guarde la salida y escriba la defensa. Después:

```
python3 mi_procesador_pdb.py datos/bc11_estructura.pdb > resultados/informe.txt || true
cat resultados/informe.txt
cd ..
tar -czf BC11_entrega.tar.gz BC11_entrega
sha256sum BC11_entrega.tar.gz
./practice_assets/check_entrega.sh BC11_entrega BC11_entrega.tar.gz
```

Compruebe. Mire el informe que ha producido sobre la estructura del capítulo. Los factores de temperatura crecen a lo largo de la cadena, y el aviso debería saltar. Ahora conteste: si esta estructura fuera real y usted quisiera medir una distancia entre dos átomos, ¿la mediría en el extremo de la cadena o en el principio? Esa es toda la lección del factor de temperatura.

El verificador ejecuta su procesador sobre ficheros que no ha visto, compruebe que la estructura de partida no se ha modificado, que la tabla de respuestas tiene al menos cuatro filas con justificación y una sobre los factores de temperatura, y que existe la defensa. Rechaza el espacio recién generado. No puntúa.

Rúbrica de calificación ponderada

La evaluación sobre 10 puntos se pondera según:

- **4.0 puntos (40%):** Ejecución correcta y reproducible de los scripts de comprobación (`chapter_checks.py` y `practice_checks.py`).
- **2.0 puntos (20%):** Implementación del parser PDB por corte de ancho fijo y cálculo de distancias 3D euclidianas (Bloque 2).
- **2.0 puntos (20%):** Cálculo de RMSD multirresiduo, conversión de factores B y control de excepciones en la guarda (Bloques 3 y 4).
- **2.0 puntos (20%):** Defensa conceptual escrita (100–150 palabras) sobre las fortalezas y limitaciones de la Cristalografía vs Cryo-EM y la interpretación del factor R_{free} .

Lista de autocorrección

- Verifiqué que mi versión de Python es ≥ 3.9 .
- El parser PDB extrajo correctamente coordenadas.
- La distancia euclidiana evaluó a 5.000000 Å.
- El RMSD idéntico resultó 0.000000 Å.
- Factor $B = 78.956835$ evaluó a $\text{RMSF} = 1.0$ Å.
- La perturbación evaluó a $\text{RMSD} = 3.535534$ Å.
- Verifiqué que línea truncada lanza `ValueError`.
- Redacté la defensa conceptual individual.

Defensa conceptual

En 100–150 palabras, compare las ventajas relativas de la criomicroscopía electrónica (Cryo-EM) frente a la cristalografía de rayos X para estudiar grandes complejos macromoleculares y proteínas de membrana, y justifique por qué un modelo cristalográfico con bajo R_{work} pero alto R_{free} indica sobreajuste de coordenadas.

Fuentes y reproducibilidad

La estructura de `data/` es sintética, de elaboración propia y con licencia CC0; está descrita en `data/README.md`. El formato y su gobernanza siguen a Berman y colaboradores [7], y la superposición óptima, a Kabsch [38].

Modelado computacional de estructuras: Predicción por homología, enhebrado y métodos ab initio

BC-CH12 – Biología Computacional

Edición 2

Pregunta del tema. Una secuencia de aminoácidos determina una estructura, y esa estructura determina lo que la proteína hace. El salto del primer dato al segundo lleva medio siglo de trabajo y sigue teniendo métodos que fallan de maneras distintas. ¿Cómo se decide, ante una secuencia concreta, qué método puede usarse y cuánto hay que fiarse del modelo que devuelve?

Producto final. Un módulo propio que calcule la identidad de secuencia, la probabilidad de aceptación de Metropolis con su guarda y el Clashscore de un modelo, y una tabla que sitúe cuatro pares de secuencias en la zona de fiabilidad que les corresponde, con la predicción anotada antes de ejecutar.

Mapa del tema

12.1. El problema del plegamiento y paisajes conformacionales	157
12.2. Modelado comparativo por homología	158
12.3. Reconocimiento de pliegues y enhebrado (<i>Threading</i>)	160
12.4. Modelado ab initio y ensamblaje de fragmentos (Rosetta)	160
12.5. Evaluación de calidad y validación de modelos	161
12.6. Diseño computacional en Python: Módulo de predicción y validación	162
12.7. Peligros computacionales y actividad de depuración	163
12.8. Síntesis operativa y tablas de diagnóstico	163
12.9. Glosario conceptual	164
12.10. Conexiones y mapa del curso	164
12.11. Fuentes y reproducibilidad	164

Cómo usar este tema

Este capítulo trata de tres estrategias clásicas para predecir una estructura y de cómo se mide si el resultado sirve. Las tres se explican mejor por lo que necesitan: el modelado por homología necesita una plantilla parecida, el enhebrado se conforma con un pliegue parecido aunque la secuencia no lo esté, y el modelado *ab initio* no necesita nada pero paga el precio de explorar un espacio enorme.

Al terminar sabrá: decidir cuál de las tres estrategias admite un caso concreto a partir de la identidad de secuencia con la mejor plantilla; explicar qué hace el criterio de Metropolis y por qué su parámetro de muestreo no es una temperatura; interpretar un TM-score y un Clashscore sabiendo qué mide cada uno y qué no; y reconocer los tres errores de interpretación que más veces convierten un modelo malo en una conclusión publicada.

La ruta **esencial** cubre las tres estrategias y las métricas de calidad. La ruta de **estudio** añade el módulo en Python, la actividad de depuración y el anexo práctico.

12.1 El problema del plegamiento y paisajes conformacionales

ESENCIAL

Tras haber establecido en el capítulo anterior la naturaleza de las coordenadas experimentales y su validación biofísica, abordamos en este capítulo el reto computacional más célebre de la biología molecular: **la predicción computacional de la estructura tridimensional de una proteína a partir únicamente de su secuencia de aminoácidos**.

Desde la perspectiva de la física estadística y la computación, el plegamiento de proteínas se formula como un problema de **optimización matemática global sobre un paisaje de energía conformacional hiperdimensional**:

- **La hipótesis termodinámica de Anfinsen (1973):** La estructura nativa de una proteína globular en condiciones fisiológicas corresponde al mínimo global de la energía libre de Gibbs (ΔG) del sistema proteína-solvente.
- **La paradoja de Levinthal (1969):** Una cadena polipeptídica de 100 residuos posee $\approx 10^{100}$ conformaciones espaciales posibles. Si el polipéptido explorase aleatoriamente cada conformación a escala de picosegundos (10^{-12} s), requeriría más de 10^{77} años para encontrar su estado nativo. En la realidad física, las proteínas se pliegan en milisegundos guiadas por fuerzas locales y cooperativas a lo largo de un **embudo de plegamiento** (*folding funnel*).

La predicción clásica de la estructura de proteínas traduce el plegamiento secuencia-estructura en una optimiza-

ción matemática a través de paisajes de energía conformacional.

12.2 Modelado comparativo por homología

ESENCIAL

El **modelado por homología** (o **modelado comparativo**) se fundamenta en un principio evolutivo empírico demostrado: **durante la evolución biológica, la estructura tridimensional se conserva mucho más que la secuencia primaria de aminoácidos**. Dos proteínas homólogas con un ancestro común pueden acumular decenas de mutaciones mientras preservan intacto su pliegue global.

12.2.1 Los umbrales de identidad de secuencia de Rost

ESENCIAL

Burkhard Rost [60] formalizó las zonas de fiabilidad para el modelado comparativo:

1. **Zona segura** (*Safe zone*, **> 30–35% de identidad**): La conservación del esqueleto carbonado es casi completa ($\text{RMSD} < 1.5\text{--}2.0 \text{ \AA}$). El modelado por homología es altamente fiable y adecuado para diseño de fármacos y mutagénesis dirigida.
2. **Zona de penumbra** (*Twilight zone*, **20–30% de identidad**): El alineamiento de secuencia se vuelve ambiguo. La topología global suele conservarse, pero los bucles y empaquetamientos locales presentan divergencias sustanciales.
3. **Zona de medianoche** (*Midnight zone*, **< 20% de identidad**): La homología no puede distinguirse del ruido estadístico aleatorio mediante alineamientos estándar de secuencias.

El modelado comparativo por homología aprovecha la conservación estructural cuando la identidad de secuencia supera el umbral seguro del 30 por ciento.

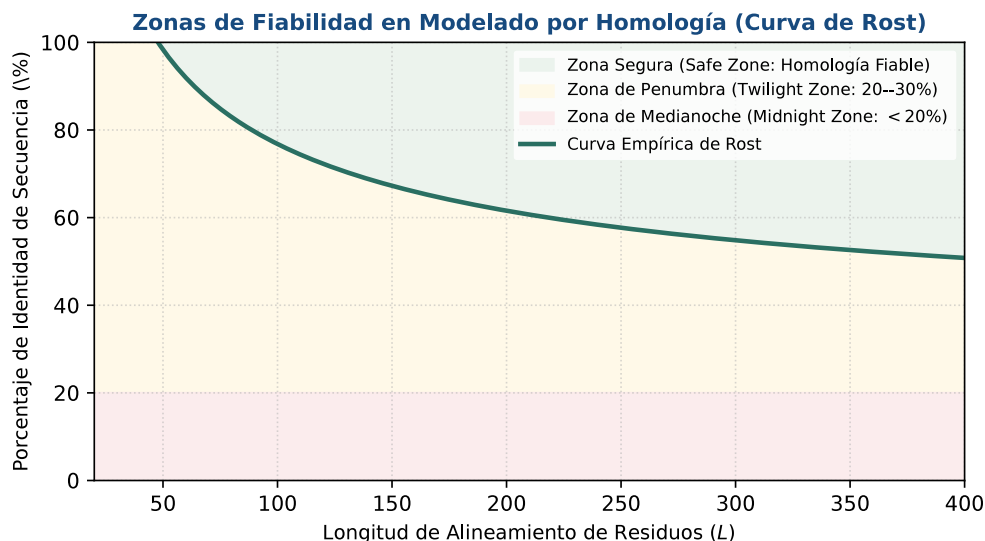


Figura 12.1: Zonas de fiabilidad en modelado por homología según la curva empírica de Rost: zona segura (> 30%), penumbra y medianoche. Figura original generada por `verification/make_figures.py`.

Traza manual de identidad de secuencia:

Dadas dos secuencias alineadas de longitud $L = 20$ aminoácidos:

```

Secuencia 1 (Diana):   A C D E F G H I K L M N P Q R S T V W Y
Secuencia 2 (Plantilla): A C D E F G H I K L M N P Q R S T V W A
  
```


Se observan 19 coincidencias exactas y 1 diferencia en la posición 20 ($Y \neq A$):

$$\text{Identidad} = \frac{19}{20} \times 100 = 95.00\%$$

`calculate_sequence_identity` calcula la identidad de alineamiento evaluando el par modelo de 20 residuos exactamente al 95.00 por ciento.

```
python3 verification/chapter_checks.py identity \
--seq1 ACDEFGHIKLMNPQRSTVWY --seq2 ACDEFGHIKLMNPQRSTVWA
```

12.2.2 El pipeline canónico de modelado por homología

ESENCIAL

El protocolo computacional consta de cuatro etapas secuenciales:

1. **Búsqueda e identificación de plantillas** (*Template search*): Rastreo de bases de datos del PDB mediante BLAST, PSI-BLAST o comparaciones sensibles de perfiles de modelos ocultos de Markov (HHpred / HMM-HMM).
2. **Alineamiento diana-plantilla** (*Target-Template alignment*): Alineamiento estricto de la secuencia problema contra la plantilla estructural.
3. **Construcción del modelo por restricciones espaciales** (MODELLER, Šali y Blundell [62]): A partir del alineamiento, se derivan restricciones probabilísticas de distancias $C\alpha - C\alpha$ y ángulos diedros mediante funciones de densidad de probabilidad. Las coordenadas 3D se generan optimizando una función objetivo que satisface simultáneamente todas las restricciones espaciales junto con términos de estereoquímica estándar de CHARMM.
4. **Modelado de bucles** (*Loops*) y **empaquetamiento de cadenas laterales**: Refinamiento de inserciones/delecciones mediante librerías de fragmentos y optimización conformacional de rotámeros de cadenas laterales (SCWRL).

El pipeline de modelado comparativo encadena la búsqueda de plantillas (PSI-BLAST, HHpred), el alineamiento diana-plantilla, la construcción del modelo mediante restricciones espaciales de MODELLER y el refinamiento de bucles.

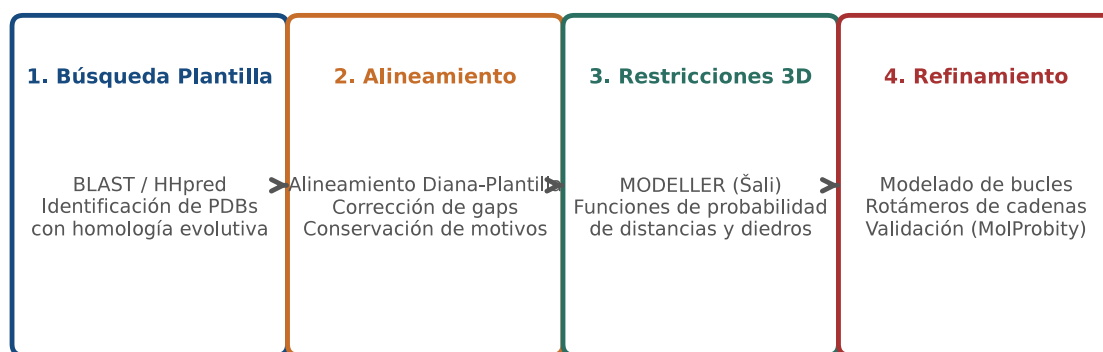


Figura 12.2: Etapas del flujo de modelado comparativo por homología con MODELLER: plantilla, alineamiento, restricciones y refinamiento. Figura original generada por `verification/make_figures.py`.

El peligro del desalineamiento de secuencia (*Alignment Shift*):

El desfase en el alineamiento representa el error más perjudicial en el modelado por homología, introduciendo rotaciones y distorsiones globales en el esqueleto polipeptídico.

12.3 Reconocimiento de pliegues y enhebrado (*Threading*)

ESENCIAL

Cuando la identidad de secuencia cae en la zona de penumbra ($< 30\%$) y no se identifican plantillas homólogas claras mediante BLAST, se recurre al **enhebrado o reconocimiento de pliegues** (*Threading / Fold Recognition*):

- **Principio biofísico:** El repertorio de pliegues tridimensionales en la naturaleza es finito (≈ 1.400 pliegues canónicos en SCOP/CATH). Es muy probable que la secuencia problema adopte un pliegue ya conocido en el PDB, incluso sin homología evolutiva detectable.
- **Algoritmo (I-TASSER, Zhang 2008):** Enhebra la secuencia diana a través de cada estructura de la librería de pliegues del PDB y evalúa su compatibilidad energética utilizando **potenciales estadísticos empíricos basados en conocimiento**:
 - Entorno de solvatación hidrofóbica del residuo.
 - Concordancia de estructura secundaria predicha (PSIPRED).
 - Contactos por pares residuo-residuo dependientes de distancia.

El reconocimiento de pliegues y enhebrado mapea secuencias en la zona de penumbra (< 30 por ciento de identidad) contra pliegues canónicos del PDB utilizando puntuaciones de energía de potenciales estadísticos (I-TASSER).

12.4 Modelado ab initio y ensamblaje de fragmentos (Rosetta)

ESENCIAL

Para secuencias completamente huérfanas sin plantillas ni pliegues compatibles conocidos, se recurre a la predicción *de novo* o *ab initio*, cuyo exponente paradigmático es el entorno **Rosetta** (Simons et al., 1997; David Baker):

12.4.1 Ensamblaje de fragmentos estructurales y muestreo Monte Carlo

ESENCIAL

1. **Librería de fragmentos:** Se extraen fragmentos estructurales de 3 y 9 residuos del PDB cuyas secuencias y estructuras secundarias predichas son compatibles con ventanas locales de la secuencia diana.
2. **Muestreo conformacional Monte Carlo Metropolis:**
 - En cada paso de simulación, se reemplaza aleatoriamente la conformación del esqueleto (ϕ, ψ, ω) de una ventana por un fragmento de la librería.
 - Se calcula la variación de energía del sistema $\Delta E = E_{\text{nuevo}} - E_{\text{anterior}}$.
 - **Criterio de Metropolis:** Si $\Delta E \leq 0$, el movimiento se acepta incondicionalmente ($P = 1.0$). Si $\Delta E > 0$, el movimiento se acepta con probabilidad estocástica:

$$P_{\text{aceptación}} = \exp\left(-\frac{\Delta E}{T}\right)$$

lo que permite al algoritmo escapar de mínimos locales de energía.

Atención. La T de esta expresión no es una temperatura física. Es un parámetro de muestreo, en las mismas unidades que la energía del modelo, que gradúa cuánto se tolera empeorar: con T alta casi cualquier movimiento se acepta y la simulación explora; con T baja solo pasan las mejoras y la simulación se estanca en el mínimo más cercano. Que en los ejemplos valga 300 es una coincidencia con la temperatura ambiente en kelvin, no una elección física.

La predicción de novo y ab initio en Rosetta combina el ensamblaje de fragmentos estructurales cortos (3-9 residuos) con el muestreo conformacional Monte Carlo Metropolis.

Traza manual del criterio de Metropolis:

Para $\Delta E = 300.0$ con el parámetro de muestreo en $T = 300.0$, es decir, cuando el movimiento empeora la energía justo en la magnitud que el parámetro tolera:

$$P = \exp\left(-\frac{300.0}{300.0}\right) = \exp(-1.0) = \frac{1}{e} \approx 0.367879$$

El criterio de Metropolis acepta movimientos conformacionales con $P = \exp(-\Delta E / T)$ cuando $\Delta E > 0$, evaluando $\Delta E = 300.0$ a $T = 300.0$ exactamente a 0.367879.

Se reproduce con `verification/chapter_checks.py metropolis --delta_e 300.0 --temp 300.0`.

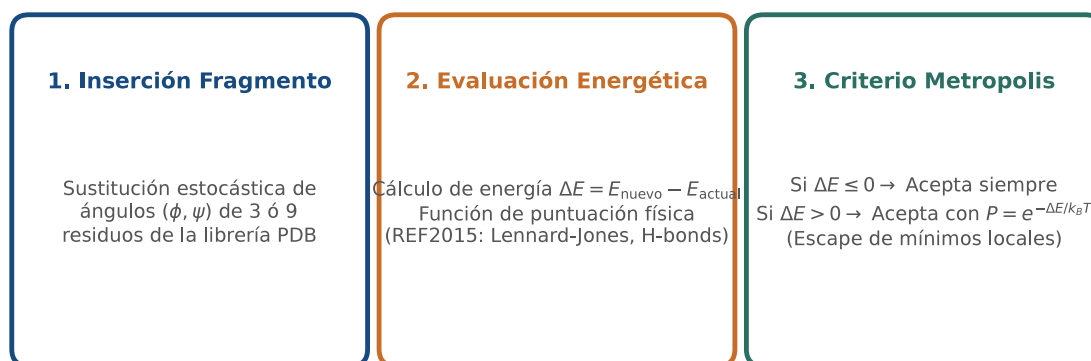
Muestreo Monte Carlo Metropolis para Ensamblaje Ab Initio (Rosetta)

Figura 12.3: Muestreo estocástico de Monte Carlo Metropolis y ensamblaje de fragmentos en la suite de modelado *ab initio* Rosetta. Figura original generada por `verification/make_figures.py`.

Control defensivo del parámetro de muestreo:

Con $T = 0$ la exponencial no está definida y, sobre todo, la pregunta deja de tener sentido: un muestreo que no tolera ningún empeoramiento ya no es Metropolis. El módulo lo rechaza en vez de devolver un cero que parecería una probabilidad.

`metropolis_acceptance_probability` lanza `ValueError` cuando el parámetro de muestreo es no positivo.

Se reproduce con `verification/chapter_checks.py metropolis --delta_e 300.0 --temp 0.0`.

12.4.2 Funciones de energía de Rosetta [59]**ESTUDIO**

Las funciones de energía de Rosetta (REF2015) integran potenciales de Lennard-Jones con rampa repulsiva, solvatación implícita de Lazaridis-Karplus y enlaces de hidrógeno dependientes de orientación.

12.5 Evaluación de calidad y validación de modelos**ESENCIAL**

La evaluación rigurosa de modelos 3D predichos se apoya en métricas geométricas y estadísticas:

12.5.1 El TM-Score: Métrica independiente de longitud**ESENCIAL**

A diferencia del RMSD clásico (que se infla artificialmente en proteínas largas o con bucles móviles en extremos),

el **TM-score** [77] evalúa la similitud del pliegue global normalizada entre 0.0 y 1.0:

$$\text{TM-score} = \frac{1}{L_{\text{target}}} \sum_{i=1}^{L_{\text{ali}}} \frac{1}{1 + \left(\frac{d_i}{d_0(L_{\text{target}})} \right)^2}$$

donde $d_0 = 1.24 \sqrt[3]{L_{\text{target}} - 15} - 1.8$.

- **TM-score > 0.50:** Indica que ambas estructuras comparten el mismo pliegue topológico global con alta significación estadística ($p < 10^{-5}$).
- Para estructuras idénticas ($d_i = 0$), el TM-score evalúa exactamente a **1.000000**.

El TM-score proporciona similitud de pliegue global independiente de la longitud (TM > 0.5 indicando pliegue compartido) evaluando estructuras idénticas exactamente a 1.000000.

Se reproduce con `verification/chapter_checks.py tm_score`.

12.5.2 El Clashscore de MolProbity [11]

ESENCIAL

El **Clashscore** cuantifica el número de solapamientos estéricos atómicos severos (distancia interatómica menor a la suma de radios de Van der Waals menos 0.4 Å) normalizado por cada 1.000 átomos:

$$\text{Clashscore} = \frac{\text{Número de solapamientos estéricos}}{\text{Total de átomos}} \times 1000$$

Traza manual:

Para 2 choques en 1000 átomos: $\text{Clashscore} = \frac{2}{1000} \times 1000 = \mathbf{2.000000}$.

El Clashscore de MolProbity mide los solapamientos estéricos atómicos severos por cada 1000 átomos evaluando 2 choques en 1000 átomos exactamente a 2.000000.

Se reproduce con `verification/chapter_checks.py clashscore --clashes 2 --atoms 1000`.

12.5.3 Estimación absoluta de calidad con QMEAN

ESTUDIO

La estimación absoluta de calidad de modelos con QMEAN / QMEAN Z-score valida la energía de empaquetamiento y solvatación sin requerir estructuras de referencia experimentales.

12.6 Diseño computacional en Python: Módulo de predicción y validación

ESTUDIO

A continuación se presenta la implementación modular del procesador en Python 3.9:

```
import math
from typing import List, Tuple

def calculate_sequence_identity(seq1: str, seq2: str) -> float:
    """Calcula el porcentaje de identidad entre dos secuencias alineadas."""
    if len(seq1) != len(seq2) or len(seq1) == 0:
        raise ValueError("Las secuencias deben ser no vacías y de igual longitud")
    matches = sum(1 for a, b in zip(seq1.upper(), seq2.upper()) if a == b and a != '-')
    return round((matches / len(seq1)) * 100.0, 2)

def metropolis_acceptance_probability(delta_e: float, temperature: float) -> float:
    """Calcula la probabilidad de aceptación conformacional según Metropolis."""
    if temperature <= 0:
        raise ValueError("El parametro de muestreo debe ser estrictamente positivo")
    if delta_e <= 0:
```

```

    return 1.0
    return round(math.exp(-delta_e / temperature), 6)

def calculate_clashscore(num_clashes: int, num_atoms: int) -> float:
    """Calcula el MolProbity Clashscore por cada 1000 atomos."""
    if num_atoms <= 0 or num_clashes < 0:
        raise ValueError("Numero de atomos debe ser positivo y choques no negativos")
    return round((num_clashes / num_atoms) * 1000.0, 6)

def calculate_tm_score_identical(length: int) -> float:
    """Retorna el TM-score para estructuras identicas (exactamente 1.0)."""
    if length <= 0:
        raise ValueError("Longitud debe ser positiva")
    return 1.000000

```

12.7 Peligros computacionales y actividad de depuración

ESTUDIO

Aplicar modelado por homología sin plantillas con identidad superior al 25 por ciento y omitir la comprobación de solapamientos estéricos representan errores de modelado críticos.

12.7.1 Tres errores críticos en modelado estructural

ESTUDIO

Localice y corrija los tres errores en la siguiente función:

```

def predict_protein_structure_pipeline(query_seq, best_template, identity_pct):
    # Error 1: Aplica modelado por homologia directo con una plantilla de 12% de identidad
    # (zona de medianoche) en lugar de utilizar threading o metodos ab initio

    # Error 2: Asume que una temperatura T = 0 en Monte Carlo permite explorar
    # el paisaje conformacional, causando division por cero en el exponente

    # Error 3: Acepta un modelo con Clashscore = 85.0 y violaciones masivas de Ramachandran
    # como listo para diseno de farmacos sin realizar minimizacion energetica
    pass

```

Diagnóstico de causa raíz (Protocolo 5 Whys):

- Fallo 1 (Modelado comparativo en zona de medianoche):** Con < 20% de identidad, el alineamiento por pares contiene desfases que destruyen el núcleo hidrofóbico.
- Fallo 2 (Temperatura nula o negativa en Metropolis):** $T \leq 0$ es físicamente inválida; bloquea la simulación en el primer mínimo local.
- Fallo 3 (Omisión de control estérico):** Clashscore = 85.0 indica colisiones atómicas no físicas que impiden cualquier acoplamiento molecular válido.

12.8 Síntesis operativa y tablas de diagnóstico

REFERENCIA

12.8.1 Tabla comparativa de métodos clásicos de predicción

REFERENCIA

Metodología	Rango de identidad	Software estándar	Principio computacional
Modelado por homología	> 30% (Zona segura)	MODELLER, MODEL	Restricciones espaciales de probabilidad
Threading / Enhebrado	20–30% (Penumbra)	I-TASSER, HHpred	Potenciales estadísticos de conocimiento
Ab initio / De novo	Sin plantilla (< 20%)	Rosetta, QUARK	Ensamblaje fragmentos + Monte Carlo

12.8.2 Tabla de diagnóstico de calidad en modelos 3D

REFERENCIA

Métrica observada	Interpretación biológica	Acción correctiva
TM-score > 0.50	Mismo pliegue topológico global	Modelo validado para análisis topológico
TM-score < 0.30	Pliegue estructural aleatorio o fallido	Cambiar de plantilla o recurrir a métodos <i>de novo</i>
Clashscore > 30.0	Colisiones estéricas atómicas severas	Minimización de energía con campo de fuerzas
QMEAN Z-score < -4.0	Modelo extremadamente aberrante	Reconstruir bucles y redefinir alineamiento

12.9 Glosario conceptual

REFERENCIA

- **Anfinsen:** Hipótesis del mínimo global de energía libre ΔG .
- **Levinthal:** Paradoja del tiempo de búsqueda conformacional.
- **Homología:** Conservación estructural superior a la de secuencia.
- **MODELLER:** Modelado por satisfacción de restricciones espaciales.
- **Alignment Shift:** Desfase en alineamiento que corrompe el modelo.
- **Threading:** Enhebrado contra librerías de pliegues del PDB.
- **Metropolis:** Criterio estocástico $P = \exp(-\Delta E/T)$.
- **TM-score:** Similitud global independiente de la longitud (> 0.5).
- **Clashscore:** Choques estéricos por cada 1.000 átomos.
- **QMEAN:** Evaluación estadística absoluta de calidad de empaquetamiento.

12.10 Conexiones y mapa del curso

ESENCIAL

Este capítulo cubre los paradigmas clásicos analíticos y heurísticos de predicción que sentaron las bases para la biología estructural moderna basada en aprendizaje profundo.

La comprensión de las limitaciones clásicas transita directamente hacia las redes neuronales coevolutivas de aprendizaje profundo y AlphaFold en BC-CH13.

12.11 Fuentes y reproducibilidad

REFERENCIA

Este capítulo se apoya en el material docente de la asignatura y en cinco fuentes primarias: Rost [60] para las zonas de fiabilidad, Šali y Blundell [62] para el modelado por restricciones espaciales, Rohl y sus colaboradores [59] para Rosetta, Zhang y Skolnick [77] para el TM-score y Chen y sus colaboradores [11] para MolProbity. Todos los cálculos numéricos del texto se reproducen con las órdenes impresas junto a ellos.

Anexo práctico · Métricas de predicción estructural, criterio de Metropolis y evaluación de choques

Pregunta, producto y separación de materiales

¿Puede implementar un módulo en Python que evalúe la identidad de secuencia en zonas de homología, simule el criterio estocástico de aceptación de Metropolis en algoritmos de Monte Carlo (Rosetta), calcule la métrica global TM-score de superposición estructural y cuantifique la densidad de choques atómicos Clashscore de MolProbity de forma completamente determinista? Este laboratorio responde afirmativamente implementando cada componente sin dependencias opacas.

El producto individual contiene: el evaluador de identidad de secuencia para modelado por homología, el simulador del criterio de Metropolis, el cálculo analítico de TM-score, el evaluador de choques estéricos Clashscore, la perturbación de energía, el control de excepciones de temperatura y la defensa conceptual escrita.

Lo que se entrega es un módulo escrito por usted, la tabla de predicciones contrastadas contra la ejecución y una defensa breve. Lo que se le da hecho es distinto y conviene tenerlo separado desde el principio: los cuatro pares de secuencias con su manifiesto, el oráculo del capítulo y un comprobador público que dice si su módulo cumple el contrato. El anexo funciona sin red y sin más dependencia que Python 3.9.

Mapa de ruta y productos entregables

Bloque de trabajo	Producto entregable
1 · Entorno y diagnóstico	Espacio de trabajo creado y manifiesto verificado
1b · Módulo de predicción	mi_prediccion.py con las cuatro funciones
2 · Cálculo ancla	Identidad, Metropolis, TM-score y Clashscore del capítulo
3 · Perturbación	Aceptación con el doble de barrera energética
4 · Guarda de dominio	Rechazo razonado del parámetro de muestreo nulo
5 · Verificación y entrega	Tabla de predicciones, defensa escrita y paquete

Este anexo deja evidencia comprobable en cada bloque sin recurrir a paquetes externos.

1 · Entorno y diagnóstico

Python 3.9 o superior, sin paquetes externos. Antes de empezar:

```
python3 --version
./practice_assets/create_workspace.sh BC12_entrega
cd BC12_entrega
(cd data && sha256sum --check --strict manifest.sha256)
python3 verification/practice_checks.py domain_guard --temp 0.0
```

El generador deja mi_prediccion.py con su contrato documentado y sin implementación, copia los cuatro pares de secuencias con su manifiesto y el comprobador público, y crea notas/respuestas.tsv con solo la cabecera.

La última orden debe imprimir `GUARD_TEMP:0.0 STATUS:REJECTED` seguido del mensaje de error. Si no lo hace, el intérprete no es el esperado o los ficheros no coinciden con el manifiesto; resuélvalo antes de continuar.

1b • Escribir el módulo de predicción

Tarea. Implemente las cuatro funciones de `mi_prediccion.py`: la identidad de secuencia, la zona de fiabilidad, la probabilidad de aceptación de Metropolis con su guarda y el Clashscore. El contrato está en la documentación de la plantilla.

La que tiene trampa es la segunda, y no por el código. Los umbrales de Rost son un convenio, no un escalón físico: un par al 30,1 % y otro al 29,9 % caen en zonas distintas y son prácticamente el mismo caso. Su función tiene que decidir igualmente, porque el contrato fija los umbrales cerrados por abajo, pero el fichero de datos incluye un par justo por encima de la frontera para que vea qué significa esa arbitrariedad cuando le toque interpretarla.

```
bash check_prediccion.sh
```

2 • Cálculo ancla: identidad, Metropolis, TM-score y Clashscore

Dadas las condiciones estándar de ensayo:

1. Secuencias de prueba de 20 residuos (ACDEFGHIKLMNPQRSTVWY y ACDEFGHIKLMNPQRSTVWA): evalúa a identidad del **95.00%**.
2. Variación de energía $\Delta E = 300.0$ a temperatura $T = 300.0$: la probabilidad de aceptación de Metropolis evalúa a **0.367879** (e^{-1}).
3. Superposición de 4 átomos idénticos ($L = 4, d_i = 0$): el TM-score evalúa a **1.000000**.
4. Conteo de 2 choques en 1000 átomos: el Clashscore evalúa a **2.000000**.

```
python3 verification/practice_checks.py anchor
```

El modelo ancla valida la identidad de secuencia al 95.00%, la probabilidad de Metropolis de 0.367879, el TM-score perfecto de 1.000000 y el Clashscore de 2.000000.

Se reproduce con `verification/practice_checks.py anchor`.

3 • Perturbación: incremento de barrera energética en Monte Carlo

Evalúe la probabilidad de aceptación cuando la variación desfavorable de energía aumenta a $\Delta E = 600.0$ a la misma temperatura $T = 300.0$:

```
python3 verification/practice_checks.py perturbation
```

La perturbacion de energia con $\Delta E = 600.0$ a $T = 300.0$ evalua la probabilidad de aceptacion de Metropolis exactamente a 0.135335.

Se reproduce con `verification/practice_checks.py perturbation`.

4 • Guarda de dominio y control de excepciones

Compruebe que el simulador rechaza un parámetro de muestreo nulo o negativo. No es solo que la división falle: con $T = 0$ el muestreo no toleraría ningún empeoramiento, y entonces ya no es Metropolis sino un descenso puro que se queda en el primer mínimo que encuentre.

```
python3 verification/practice_checks.py domain_guard --temp 0.0
```

La guarda de dominio rechaza un parámetro de muestreo menor o igual que cero e informa del rechazo sin abortar la ejecución.

Se reproduce con `verification/practice_checks.py domain_guard --temp 0.0`.

5 • Verificación y entrega

Antes de ejecutar nada, abra `data/bc12_pares.fasta` y anote en `notas/respuestas.tsv` lo que espera obtener para cada uno de los cuatro pares: el porcentaje de identidad y la zona de fiabilidad. El nombre de cada par le dice la identidad nominal con que se generó, así que la predicción de la zona es la parte que de verdad se juega. Después ejecute su módulo sobre los cuatro, guarde las salidas en `resultados/` y complete las columnas de resultado y de coincidencia. La columna de justificación es la que se corrige: si algún par cae en una zona distinta de la que había previsto, diga por qué.

Escriba en `notas/defensa.md` entre 100 y 150 palabras sobre qué estrategia de predicción admite cada uno de los cuatro pares y por qué un TM-score alto no basta para dar un modelo por bueno. Después:

```
cd ..
tar -czf BC12_entrega.tar.gz BC12_entrega
sha256sum BC12_entrega.tar.gz
./practice_assets/check_delivery.sh BC12_entrega BC12_entrega.tar.gz
```

El verificador comprueba que su módulo compila y cumple el contrato, que los datos de partida no se han tocado, que los cuatro pares están documentados con predicción y resultado, y que existe la defensa con la extensión pedida. Rechaza la entrega vacía y el espacio recién generado. No puntúa.

El verificador de entrega comprueba el contrato del módulo, la integridad de los datos de partida, la tabla de predicciones y la defensa escrita; rechaza el espacio de trabajo recién generado y no emite calificación.

Rúbrica de calificación ponderada

La evaluación sobre 10 puntos se pondera según:

- **4.0 puntos (40%):** El módulo `mi_prediccion.py` cumple el contrato: el comprobador público imprime `PREDICCION_OK`.
- **2.0 puntos (20%):** La tabla de `notas/respuestas.tsv` está completa y las justificaciones explican las discrepancias.
- **2.0 puntos (20%):** Los bloques de cálculo ancla, perturbación y guarda de dominio están reproducidos con sus salidas guardadas.
- **2.0 puntos (20%):** Defensa conceptual escrita (100–150 palabras) sobre las diferencias entre modelado por homología y *threading* y el impacto de los desfases de alineamiento.

Lista de autocorrección

- Verifiqué que mi versión de Python es ≥ 3.9 .
- La identidad de secuencia evaluó al 95.00%.
- Metropolis ancla resultó 0.367879.
- El TM-score idéntico resultó 1.000000.
- El Clashscore evaluó a 2.000000.
- La perturbación evaluó a 0.135335.
- Verifiqué que el parámetro de muestreo nulo es rechazado.
- `bash check_prediccion.sh` imprimió `PREDICCION_OK`.
- Anoté las predicciones antes de ejecutar el módulo.
- Redacté la defensa conceptual pedida.

Defensa conceptual

En 100–150 palabras, y sobre los cuatro pares del fichero de datos: diga qué estrategia de predicción admite cada uno y por qué. Añada dos cosas que el ejercicio deja a la vista. La primera: un desfase de una sola posición en el alineamiento entre diana y plantilla arruina el modelo de una hélice α , porque desplaza en 100 grados la orientación de todas las cadenas laterales que vienen después. La segunda: un TM-score alto dice que el pliegue global coincide, y no dice nada sobre si el modelo tiene choques estéricos; por eso el Clashscore se mira aparte.

Fuentes y reproducibilidad

Este anexo se apoya en el material docente de la asignatura y en las mismas fuentes primarias que el capítulo: Rost [60] para las zonas de fiabilidad, Šali y Blundell [62] para el modelado comparativo y Chen y sus colaboradores [11] para el Clashscore. Todos los valores numéricos del enunciado se reproducen con las órdenes impresas junto a ellos.

Aprendizaje profundo, redes neuronales y AlphaFold en biología estructural

BC-CH13 – Biología Computacional

Edición 2

Pregunta del tema. AlphaFold devuelve coordenadas atómicas completas para cualquier secuencia que se le dé, incluidas las que no se pliegan. El modelo nunca dice «no lo sé»: lo dice el pLDDT y lo dice la matriz PAE, en otro sitio y en otra escala. ¿Cómo se lee una predicción para saber qué parte de ella es una afirmación y qué parte es una conjetura?

Producto final. Un módulo propio que clasifique las bandas de pLDDT, calcule la confianza media, distinga por la matriz PAE una orientación rígida entre dominios de una bisagra flexible y compruebe la consistencia geométrica del modelo, con un informe que dictamine cuatro modelos predichos y diga para qué sirve cada uno.

Mapa del tema

13.1. Qué cambió con el aprendizaje profundo	169
13.2. Coevolución molecular en alineamientos múltiples (MSAs)	169
13.3. Arquitectura de AlphaFold2: Del Evoformer al Structure Module	169
13.4. Métricas de confianza: pLDDT y matrices PAE	171
13.5. Fronteras emergentes: AlphaFold3 y Modelos de Lenguaje	172
13.6. Diseño computacional en Python: Clasificador pLDDT y PAE	172
13.7. Peligros computacionales y actividad de depuración	174
13.8. Ética y auditoría de las predicciones	174
13.9. Síntesis operativa y tablas de diagnóstico	175
13.10. Glosario conceptual	175
13.11. Conexiones y mapa del curso	175
13.12. Fuentes y reproducibilidad	176

Cómo usar este tema

Este capítulo cierra la parte estructural y es el único donde el método que se estudia no se puede reproducir: entrenar AlphaFold está fuera del alcance de cualquier asignatura. Lo que sí está a su alcance, y es lo que de verdad se usa a diario, es leer bien lo que el sistema devuelve.

Por eso el capítulo se organiza en dos mitades desiguales. La primera explica de dónde sale la información: la coevolución en un alineamiento múltiple, y cómo el Evoformer la convierte en una geometría. La segunda, más larga en tiempo de trabajo, es la que enseña a auditar el resultado con el pLDDT y la matriz PAE.

Al terminar sabrá: explicar por qué dos residuos que mutan de forma correlacionada están probablemente en contacto; leer un pLDDT por residuo y decir qué banda admite qué uso; leer una matriz PAE y distinguir un empaquetamiento rígido de una bisagra; y reconocer cuándo una región de baja confianza es desorden funcional y no un fallo del método.

La ruta **esencial** son la coevolución en un alineamiento múltiple, la arquitectura de AlphaFold2, las dos métricas de confianza y la auditoría de una predicción. La ruta de **estudio** añade las fronteras con difusión y modelos de lenguaje, la actividad de depuración y el anexo práctico.

13.1 Qué cambió con el aprendizaje profundo

ESENCIAL

ESENCIAL El capítulo anterior terminaba con métodos que necesitan una plantilla o pagan el precio de explorar un espacio enorme. Este trata de lo que los sustituyó en la práctica: **redes neuronales geométricas entrenadas de extremo a extremo**, reconocidas con el Premio Nobel de Química de 2024.

Durante más de cinco décadas, la predicción estructural dependió de simulaciones biofísicas costosas y funciones de energía empíricas aproximadas. Lo que AlphaFold cambió no fue acelerar una simulación física molecular paso a paso, sino en **reformular el plegamiento como un problema de inferencia geométrica directa y reconocimiento de patrones espaciales sobre la información evolutiva acumulada en millones de años**:

La predicción estructural mediante aprendizaje profundo transforma la simulación física en un aprendizaje de patrones geométricos de extremo a extremo a través de datos de coevolución evolutiva.

13.2 Coevolución molecular en alineamientos múltiples (MSAs)

ESENCIAL

El motor biológico fundamental que alimenta los modelos neuronales de predicción estructural es el fenómeno de la **coevolución molecular**:

- Cuando dos aminoácidos alejados en la secuencia primaria se encuentran en íntimo contacto físico espacial en el interior de la estructura nativa, una mutación perjudicial en uno de ellos (p. ej., una sustitución de un residuo cargado positivamente Lys por uno voluminoso o cargado negativamente Glu) desestabiliza el núcleo proteico.
- Para preservar la viabilidad celular, la selección natural favorece una **mutación compensatoria** en el residuo en contacto espacial (p. ej., mutando una pareja previa a una carga positiva que restablezca el puente salino o adaptando el volumen estérico).
- Al analizar estadísticamente un alineamiento múltiple de secuencias (MSA) con miles de homólogos mediante **Análisis de Acoplamiento Directo (DCA)**, las correlaciones mutacionales revelan con precisión la matriz de contactos tridimensionales de la proteína.

La coevolución molecular en alineamientos múltiples de secuencias captura restricciones de contacto espacial que preservan la estabilidad estructural nativa.

13.3 Arquitectura de AlphaFold2: Del Evoformer al Structure Module

ESENCIAL

Diseñado por DeepMind [37], AlphaFold2 supera a los métodos anteriores al integrar dos bloques neuronales acoplados que refinan iterativamente la geometría molecular:

13.3.1 El bloque Evoformer y las actualizaciones triangulares

ESENCIAL

El **Evoformer** procesa simultáneamente dos representaciones acopladas mediante mecanismos de atención axial (*Axial Attention*):

1. **Representación 1D del MSA** ($N_{\text{seq}} \times N_{\text{res}}$): Codifica las secuencias homólogas y sus perfiles evolutivos.
2. **Representación 2D de pares de residuos** ($N_{\text{res}} \times N_{\text{res}}$): Codifica la matriz de relaciones espaciales y distancias relativas entre cada par de aminoácidos (i, j).

Para garantizar que la matriz de pares represente una geometría tridimensional físicamente coherente, el Evoformer incorpora las **Actualizaciones Triangulares** (*Triangle Multiplicative Update y Triangle Self-Attention*).

Cada trío de residuos (i, j, k) debe satisfacer la **desigualdad triangular euclídea**:

$$d_{ik} \leq d_{ij} + d_{jk}$$

Traza manual de consistencia triangular:

Para tres distancias predichas $d_{ij} = 3.0 \text{ \AA}$, $d_{jk} = 4.0 \text{ \AA}$, $d_{ik} = 5.0 \text{ \AA}$:

$$d_{ik} \leq d_{ij} + d_{jk} \iff 5.0 \leq 3.0 + 4.0 = 7.0 \quad (\text{Válido})$$

$$d_{ij} \leq d_{jk} + d_{ik} \iff 3.0 \leq 4.0 + 5.0 = 9.0 \quad (\text{Válido})$$

$$d_{jk} \leq d_{ij} + d_{ik} \iff 4.0 \leq 3.0 + 5.0 = 8.0 \quad (\text{Válido})$$

El trío conforma un triángulo euclídeo consistente en $\mathbb{R}^3$.

`check_triangle_inequality` valida la consistencia geométrica triangular asegurando que las distancias predichas satisfagan $d_{ik} \leq d_{ij} + d_{jk}$, evaluando $(3, 4, 5)$ como válido.

Se reproduce con `verification/chapter_checks.py triangle --d12 3.0 --d23 4.0 --d13 5.0`.

El Evoformer de AlphaFold2 combina representaciones 1D de MSA y 2D de pares de residuos mediante atención dual y Actualizaciones Triangulares.

Arquitectura de Aprendizaje Profundo de AlphaFold2

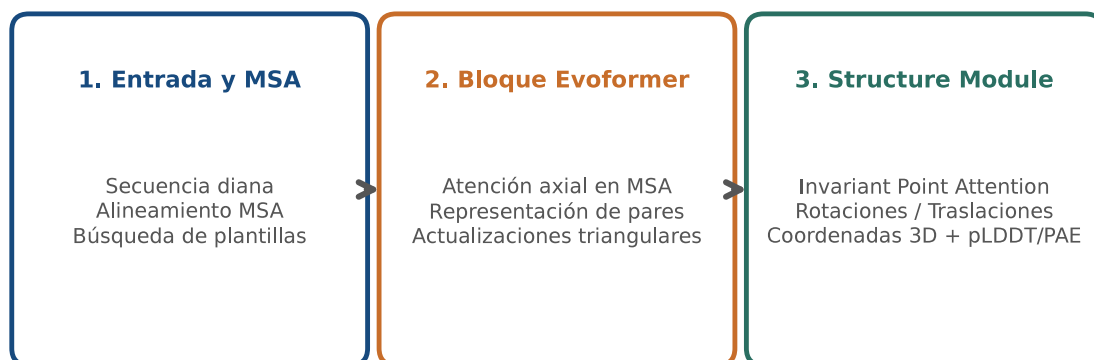


Figura 13.1: Arquitectura de extremo a extremo de AlphaFold2: procesamiento conjunto de MSA y matriz de pares en el Evoformer y generación de coordenadas 3D en el Structure Module. Figura original generada por `verification/make_figures.py`.

13.3.2 El Módulo Estructural y la Atención de Puntos Invariante (IPA)

ESTUDIO

A diferencia de los métodos clásicos que reconstruían coordenadas a partir de matrices de distancia ruidosas, el **Módulo Estructural** (*Structure Module*) predice directamente las coordenadas 3D cartesianas en un solo paso:

- Modela la cadena polipeptídica como un “gas de marcos rígidos 3D” (*residue gas*), donde cada residuo posee una orientación y posición en el grupo especial euclídeo $SE(3) = SO(3) \times \mathbb{R}^3$.
- Aplica el mecanismo de **Atención de Puntos Invariante** (*Invariant Point Attention, IPA*), garantizando invariancia estricta frente a traslaciones y rotaciones globales en el espacio 3D.
- Predice simultáneamente los ángulos de torsión (ϕ, ψ, ω) y los rotámetros de cadenas laterales $(\chi_1, \chi_2, \chi_3, \chi_4)$ sin choques estéricos.

El Módulo Estructural emplea Atención de Puntos Invariante (IPA) operando sobre marcos de referencia locales rígidos $SE(3)$ para posicionar coordenadas atómicas del esqueleto.

13.4 Métricas de confianza: pLDDT y matrices PAE

ESENCIAL

Una contribución metodológica crucial de AlphaFold2 es que **predice su propia incertidumbre para cada residuo y par de dominios**, evitando la sobreinterpretación errónea de modelos:

13.4.1 La métrica local pLDDT (predicted Local Distance Difference Test)

ESENCIAL

La puntuación **pLDDT** cuantifica la confianza del modelo a nivel de residuo individual en una escala de 0 a 100:

Rango pLDDT	Categoría	Significado e interpretación biológica
pLDDT ≥ 90	VERY_HIGH	Confianza muy alta; precisión atómica en cadenas laterales y esqueleto (RMSD < 1 Å). Ideal para diseño de fármacos.
70 ≤ pLDDT < 90	CONFIDENT	Confianza buena; esqueleto polipeptídico modelado con alta fidelidad.
50 ≤ pLDDT < 70	LOW	Confianza baja; bucles expuestos o conformaciones móviles. Debe interpretarse con cautela.
pLDDT < 50	VERY_LOW_OR_IDP	Confianza muy baja; fuerte predictor de regiones intrínsecamente desordenadas (IDPs) o ausencia de parejas de interacción.

pLDDT cuantifica la confianza local por residuo en una escala 0-100: muy alta (≥90), confiable (70-90), baja (50-70) y muy baja (<50, intrínsecamente desordenada).

Escala Oficial de Confianza Local pLDDT (AlphaFold)

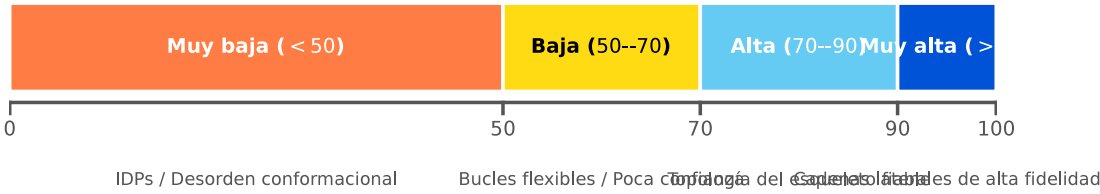


Figura 13.2: Métrica de confianza local pLDDT de AlphaFold: bandas de fiabilidad y discriminación de regiones intrínsecamente desordenadas (IDPs). Figura original generada por `verification/make_figures.py`.

Traza manual para pLDDT = 92.50:

El clasificador de bandas asigna la puntuación 92.50 a la banda de confianza muy alta.

Se reproduce con `verification/chapter_checks.py plddt --score 92.50`.

Traza manual de pLDDT medio global:

Para las puntuaciones de residuos [95, 92, 88, 45]:

$$\text{Media} = \frac{95 + 92 + 88 + 45}{4} = \frac{320}{4} = 80.000000$$

`calculate_mean_plddt` calcula la confianza media de todo el modelo evaluando las puntuaciones [95, 92, 88, 45] exactamente a 80.000000.

Se reproduce con `verification/chapter_checks.py mean_plddt --scores 95.0,92.0,88.0,45.0`.

Control defensivo de rango de puntuación:

`classify_plddt` lanza `ValueError` cuando la puntuación está fuera del rango válido $[0, 100]$.

Se reproduce con `verification/chapter_checks.py plddt --score 105.0`.

13.4.2 La matriz PAE (Predicted Aligned Error) y empaquetamiento entre dominios

ESENCIAL

El pLDDT mide la calidad local de cada residuo, pero no indica si dos dominios globulares mantienen una orientación espacial fija entre sí. Para ello, AlphaFold produce la matriz **PAE (Predicted Aligned Error, en Ångströms)** de tamaño $N_{\text{res}} \times N_{\text{res}}$:

- $\text{PAE}(x, y)$ estima el error esperado en la posición del residuo y cuando el modelo se superpone sobre el residuo x .
- **PAE interdominio baja** ($< 5 \text{ Å}$): Los dominios presentan una orientación rígida bien definida en el espacio (`RIGID_ORIENTATION`).
- **PAE interdominio alta**, por encima de 15 a 20 ángstrom: los dominios están unidos por un conector flexible; cada uno se pliega bien por separado, pero su orientación relativa queda indeterminada.

La matriz N-por-N de Error Alineado Predicho (PAE) cuantifica la incertidumbre de posicionamiento espacial inter-residuo para discriminar empaquetamiento rígido de dominios frente a conectores flexibles.

Traza manual de evaluación PAE interdominio:

Dada la submatriz PAE 2×2 entre dos dominios con valores $[[2.5, 3.0], [2.8, 3.0]]$:

$$\text{PAE media} = \frac{2.5 + 3.0 + 2.8 + 3.0}{4} = \frac{11.3}{4} = 2.825000 \text{ Å} < 5.0 \text{ Å} \Rightarrow \text{RIGID_ORIENTATION}$$

`evaluate_interdomain_pae` evalúa la matriz modelo PAE 2×2 con media 2.825000 Å clasificando el empaquetamiento de dominios como `RIGID_ORIENTATION`.

Se reproduce con `verification/chapter_checks.py pae --matrix 2.5,3.0:3.0,2.8`.

13.5 Fronteras emergentes: AlphaFold3 y Modelos de Lenguaje

ESTUDIO

1. **Modelos generativos de difusión** (AlphaFold3 [2] y RoseTTAFold All-Atom [6]): Abandonan el Structure Module determinista y emplean modelos de difusión generativa para desruirar coordenadas atómicas, prediciendo complejos heterogéneos de proteínas, ADN, ARN, iones metálicos y ligandos químicos.
2. **Modelos de lenguaje de proteínas** (pLM, ESMFold [48]): Entrenados sobre cientos de millones de secuencias de UniRef, aprenden la gramática evolutiva del plegamiento, permitiendo predecir estructuras 3D en milisegundos directamente a partir de una única secuencia individual sin necesidad de computar MSAs.

Los modelos generativos de difusión en AlphaFold3 y RoseTTAFold All-Atom predicen ensamblajes heterogéneos de proteínas, ácidos nucleicos, iones y ligandos mediante desruído iterativo.

Los Modelos de Lenguaje de Proteínas (pLMs, ESMFold) aprenden la gramática evolutiva a través de miles de millones de secuencias para predecir estructuras a partir de secuencias únicas sin cálculo de MSA.

13.6 Diseño computacional en Python: Clasificador pLDDT y PAE

ESTUDIO

A continuación se presenta la implementación modular del procesador en Python 3.9:

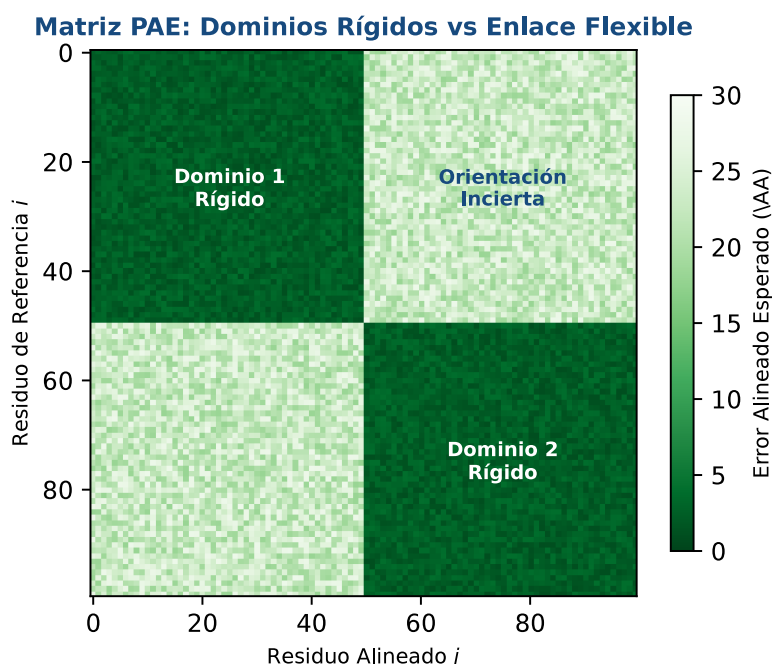


Figura 13.3: Matriz de Error Alineado Predicho (PAE): identificación de empaquetamientos globulares rígidos frente a conectores inter-dominio flexibles. Figura original generada por `verification/make_figures.py`.

```
from typing import List, Tuple

def check_triangle_inequality(d_ij: float, d_jk: float, d_ik: float) -> bool:
    """Valida si tres distancias satisfacen la desigualdad triangular euclidea."""
    if d_ij <= 0 or d_jk <= 0 or d_ik <= 0:
        raise ValueError("Las distancias deben ser estrictamente positivas")
    return (d_ik <= d_ij + d_jk) and (d_ij <= d_jk + d_ik) and (d_jk <= d_ij + d_ik)

def classify_plddt(score: float) -> str:
    """Asigna la categoria de confianza cualitativa segun el valor pLDDT."""
    if score < 0.0 or score > 100.0:
        raise ValueError(f"Puntuacion pLDDT fuera de rango [0, 100]: {score}")
    if score >= 90.0:
        return "VERY_HIGH"
    elif score >= 70.0:
        return "CONFIDENT"
    elif score >= 50.0:
        return "LOW"
    else:
        return "VERY_LOW_OR_IDP"

def calculate_mean_plddt(scores: List[float]) -> float:
    """Calcula el pLDDT medio de una lista de residuos."""
    if not scores:
        raise ValueError("Lista de puntuaciones vacia")
    for s in scores:
        if s < 0.0 or s > 100.0:
            raise ValueError(f"Puntuacion fuera de rango: {s}")
    return round(sum(scores) / len(scores), 6)

def evaluate_interdomain_pae(pae_matrix: List[List[float]]) -> Tuple[float, str]:
    """Calcula la PAE media entre dominios y clasifica la flexibilidad."""
    if not pae_matrix or not pae_matrix[0]:
        raise ValueError("Matriz PAE vacia")
    total_val = 0.0
    count = 0
    for row in pae_matrix:
        for val in row:
```

```

    if val < 0.0:
        raise ValueError("Valores PAE no pueden ser negativos")
    total_val += val
    count += 1
mean_pae = round(total_val / count, 6)
classification = "RIGID_ORIENTATION" if mean_pae < 5.0 else "FLEXIBLE_LINKER"
return mean_pae, classification

```

13.7 Peligros computacionales y actividad de depuración

ESTUDIO

Malinterpretar regiones con pLDDT < 50 como fallos del modelo en lugar de proteínas intrínsecamente desordenadas (IDPs) y asumir rigidez estática representan errores críticos.

13.7.1 Tres errores críticos en el uso de predicciones de IA

ESTUDIO

Localice y corrija los tres errores en la siguiente función:

```

def interpret_alphafold_prediction(plddt_per_residue, pae_matrix):
    # Error 1: Descarta un modelo completo porque su cola N-terminal tiene pLDDT = 35,
    # ignorando que se trata de un segmento intrínsecamente desordenado (IDP) fisiológico

    # Error 2: Realiza acoplamiento molecular de fármacos (docking) sobre residuos
    # con pLDDT = 48, asumiendo que la conformación estática dibujada es rígida

    # Error 3: Asume que dos dominios forman una interfaz fija basándose solo en que
    # ambos tienen pLDDT > 90 individualmente, ignorando que su PAE cruzada es de 25 Å
    pass

```

Diagnóstico de causa raíz (Protocolo 5 Whys):

1. **Fallo 1 (Confundir desorden con fallo del algoritmo):** pLDDT < 50 en bucles o extremos predice con > 80% de precisión regiones IDP que carecen de estructura secundaria fija en disolución.
2. **Fallo 2 (Docking en regiones de baja confianza):** Realizar cribado en bolsas con pLDDT < 70 genera falsos positivos masivos.
3. **Fallo 3 (Ignorar la matriz PAE en arquitecturas multidominio):** Dos dominios pueden estar impecablemente plegados de forma individual pero conectados por una bisagra flexible que hace impredecible su posición relativa.

13.8 Ética y auditoría de las predicciones

ESENCIAL

Una predicción de AlphaFold llega con aspecto de resultado terminado: coordenadas atómicas completas, sin huecos, con la misma pinta que una estructura resuelta experimentalmente. Esa apariencia es el problema. El modelo siempre devuelve algo, incluso para una secuencia que no se pliega, y no distingue entre «esto es así» y «esto es lo mejor que puedo proponer con lo que he visto».

De ahí que el uso responsable de estas predicciones se reduzca a una disciplina que no cueste nada y casi nadie aplica: mirar el pLDDT y la matriz PAE antes que las coordenadas, y decir en el texto qué se ha mirado. Tres consecuencias prácticas.

- **Una región de pLDDT bajo no es un error de la predicción.** A menudo es la respuesta correcta: hay proteínas cuyas regiones intrínsecamente desordenadas no tienen una estructura única que predecir. Borrarlas del modelo para que quede más limpio es falsear el resultado.

- **Un pLDDT alto no valida el empaquetamiento entre dominios.** Dos dominios pueden estar bien predichos por separado y colocados uno respecto del otro de cualquier manera. Eso lo dice la matriz PAE, no el pLDDT.
- **Una predicción no sustituye a un experimento cuando la decisión tiene consecuencias.** Sirve para generar hipótesis, priorizar dianas y diseñar experimentos. Orientar una decisión clínica o el desarrollo de un fármaco sobre coordenadas predichas sin contrastarlas es un salto que hay que declarar explícitamente, no dar por descontado.

El uso responsable de una predicción estructural exige consultar el pLDDT por residuo y la matriz PAE entre dominios antes de interpretar las coordenadas, y declarar esa comprobación al comunicar el resultado.

13.9 Síntesis operativa y tablas de diagnóstico

REFERENCIA

13.9.1 Tabla de interpretación de métricas de AlphaFold

REFERENCIA

Métrica / Rango	Calificación cualitativa	Fiabilidad del modelo	Aplicación recomendada
pLDDT $\geq$ 90	VERY_HIGH	Precisión atómica (RMSD < 1 Å)	Docking y diseño de fármacos
70 $\leq$ pLDDT < 90	CONFIDENT	Esqueleto fiable	Mutagénesis y topología
50 $\leq$ pLDDT < 70	LOW	Bucles / baja certeza	Hipótesis orientativas
pLDDT < 50	VERY_LOW_OR_IDP	Región desordenada	Predicción de IDPs
PAE < 5 Å	RIGID_ORIENTATION	Interfaz de dominios rígida	Ensamblaje cuaternario
PAE > 15 Å	FLEXIBLE_LINKER	Conector flexible interdominio	No usar para orientación global

13.10 Glosario conceptual

REFERENCIA

- **Coevolución:** Mutaciones compensatorias que revelan contactos 3D.
- **DCA:** Análisis de acoplamiento directo en alineamientos múltiples.
- **Evoformer:** Bloque neuronal de atención axial sobre MSA y pares 2D.
- **Triangle Update:** Restricción de desigualdad triangular en pares.
- **Structure Module:** Predicción 3D cartesiana directa en SE(3).
- **IPA:** Mecanismo de Atención de Puntos Invariante espacial.
- **pLDDT:** Métrica local de confianza por residuo (0–100).
- **PAE:** Matriz de error esperado de alineamiento interdominio (Å).
- **IDP:** Proteína intrínsecamente desordenada (pLDDT < 50).
- **ESMFold:** Predicción ultrarrápida mediante modelos de lenguaje.

13.11 Conexiones y mapa del curso

ESENCIAL

La comprensión de las predicciones estáticas de IA transita directamente hacia simulaciones de dinámica molecular, docking y cribado virtual en BC-CH14.

13.12 Fuentes y reproducibilidad

REFERENCIA

Este capítulo se apoya en el material docente de la asignatura y en los cuatro artículos que describen los sistemas tratados: Jumper y sus colaboradores [37] para AlphaFold2, Baek y sus colaboradores [6] para RoseTTAFold, Abramson y sus colaboradores [2] para AlphaFold3 y Lin y sus colaboradores [48] para ESMFold. Todos los cálculos numéricos del texto se reproducen con las órdenes impresas junto a ellos.

Anexo práctico · Métricas de confianza pLDDT, matrices PAE y auditoría de modelos de IA

Pregunta, producto y separación de materiales

¿Puede implementar un módulo en Python que clasifique las bandas de calidad local pLDDT de AlphaFold, evalúe la confianza promedio del modelo, analice submatrices PAE para discriminar orientaciones interdominio rígidas de bisagras flexibles, y valide la consistencia geométrica euclidiana mediante la desigualdad triangular de forma determinista y sin dependencias externas? Este laboratorio pide justamente eso, y luego pide algo más: usar el módulo para dictaminar cuatro modelos predichos y decir para qué sirve cada uno.

El producto individual contiene: el clasificador de bandas pLDDT, el calculador de confianza media, el analizador de submatrices PAE interdominio, la verificación de desigualdad triangular, la perturbación hacia desorden intrínseco (IDP), el control de excepciones de la guarda de dominio y la defensa conceptual escrita.

Lo que se entrega es un módulo escrito por usted, el dictamen razonado de los cuatro modelos y una defensa breve. Lo que se le da hecho es distinto y conviene tenerlo separado desde el principio: los cuatro modelos con su manifiesto, el oráculo del capítulo y un comprobador público que dice si su módulo cumple el contrato. El anexo funciona sin red y sin más dependencia que Python 3.9.

Mapa de ruta y productos entregables

Bloque de trabajo	Producto entregable
1 · Entorno y diagnóstico	Espacio de trabajo creado y manifiesto verificado
1b · Módulo de auditoría	mi_auditoria.py con las cuatro funciones
2 · Cálculo ancla	Banda, media, dictamen PAE y triángulo del capítulo
3 · Perturbación	El mismo análisis sobre un modelo desordenado
4 · Guarda de dominio	Rechazo razonado de un pLDDT fuera de rango
5 · Dictamen y entrega	Los cuatro modelos dictaminados, defensa y paquete

Este anexo deja evidencia comprobable en cada bloque sin recurrir a paquetes externos.

1 · Entorno y diagnóstico

Python 3.9 o superior, sin paquetes externos. Antes de empezar:

```
python3 --version
./practice_assets/create_workspace.sh BC13_entrega
cd BC13_entrega
(cd data && sha256sum --check --strict manifest.sha256)
python3 verification/practice_checks.py domain_guard --score 105.0
```

El generador deja mi_auditoria.py con su contrato documentado y sin implementación, copia los cuatro modelos con su manifiesto y el comprobador público, y crea notas/dictamen.tsv con solo la cabecera.

La última orden debe imprimir GUARD_SCORE:105.0 STATUS:REJECTED seguido del mensaje de error. Si no lo hace, el intérprete no es el esperado o los ficheros no coinciden con el manifiesto; resuélvalo antes de continuar.

1b • Escribir el módulo de auditoría

Tarea. Implemente las cuatro funciones de `mi_auditoria.py`: la banda de pLDDT, la confianza media, el dictamen de una submatriz PAE y la desigualdad triangular. El contrato está en la documentación de la plantilla.

Las tres primeras son cortas y la cuarta parece trivial hasta que se mira el caso degenerado. Tres puntos alineados dan distancias como 2, 3 y 5, donde el lado largo iguala exactamente la suma de los otros dos. Eso es realizable: es un segmento. Si su comprobación usa la desigualdad estricta lo declarará imposible, y estará rechazando geometrías que el modelo puede producir legítimamente cuando tres residuos quedan casi en línea.

```
bash check_auditoria.sh
```

2 • Cálculo ancla: pLDDT, media, PAE y consistencia triangular

Dadas las condiciones estándar de auditoría:

1. Puntuación pLDDT de 92.50: clasifica en la banda VERY_HIGH.
2. Cuarteto de residuos [95.0, 92.0, 88.0, 45.0]: el pLDDT medio evalúa a 80.000000.
3. Submatriz PAE de 2×2 con los valores 2,5 y 3,0 en la primera fila y 3,0 y 2,8 en la segunda: media de 2,825 ángstrom, es decir, orientación rígida.
4. Triplete de distancias (3.0, 4.0, 5.0): la desigualdad triangular evalúa como Válida.

```
python3 verification/practice_checks.py anchor
```

El modelo ancla valida la banda VERY_HIGH, el pLDDT medio de 80.000000, la media PAE de 2.825000 Å (RIGID_ORIENTATION) y la consistencia geométrica triangular.

Se reproduce con `verification/practice_checks.py anchor`.

3 • Perturbación: auditoría de desorden intrínseco y bisagras flexibles

Compruebe la respuesta del clasificador ante una región de desorden intrínseco (pLDDT = 42.0) y una submatriz PAE interdominio de alta incertidumbre ([[18.0, 22.0], [20.0, 24.0]], media 21.000000 Å):

```
python3 verification/practice_checks.py perturbation
```

La perturbacion con score 42.0 (VERY_LOW_OR_IDP) y PAE medio de 21.000000 Å (FLEXIBLE_LINKER) verifica la auditoria de incertidumbre.

Se reproduce con `verification/practice_checks.py perturbation`.

4 • Guarda de dominio y control de excepciones

Compruebe que el algoritmo rechaza puntuaciones pLDDT fuera del rango [0, 100]:

```
python3 verification/practice_checks.py domain_guard --score 105.0
```

La guarda de dominio rechaza puntuaciones pLDDT fuera del intervalo [0, 100] lanzando un `ValueError`.

Se reproduce con `verification/practice_checks.py domain_guard --score 105.0`.

5 • Dictamen de los cuatro modelos y entrega

data/bc13_modelos.json contiene cuatro predicciones con su vector de pLDDT por residuo y una submatriz PAE entre dominios. Los nombres describen lo que pretende ser cada una; el ejercicio es comprobar si el nombre se sostiene.

Antes de ejecutar nada, anote en notas/dictamen.tsv qué espera de cada modelo: en qué banda caerá su pLDDT medio y si la submatriz PAE dará una orientación rígida o flexible. Después ejecute su módulo sobre los cuatro, guarde las salidas en resultados/ y complete las columnas de resultado.

La columna uso_admisible es el dictamen y es lo que se corrige: para qué sirve cada modelo. Al menos uno de los cuatro tiene un pLDDT medio alto y una matriz PAE que desaconseja usarlo tal cual, y ese es el caso que el capítulo entero prepara.

Escriba en notas/defensa.md entre 100 y 150 palabras justificando el dictamen de ese modelo en concreto. Después:

```
cd ..
tar -czf BC13_entrega.tar.gz BC13_entrega
sha256sum BC13_entrega.tar.gz
./practice_assets/check_delivery.sh BC13_entrega BC13_entrega.tar.gz
```

El verificador comprueba que su módulo compila y cumple el contrato, que los datos de partida no se han tocado, que los cuatro modelos están dictaminados y que existe la defensa con la extensión pedida. Rechaza la entrega vacía y el espacio recién generado. No puntúa.

El verificador de entrega comprueba el contrato del módulo, la integridad de los datos de partida, el dictamen de los cuatro modelos y la defensa escrita; rechaza el espacio de trabajo recién generado y no emite calificación.

Rúbrica de calificación ponderada

La evaluación sobre 10 puntos se pondera según:

- **4.0 puntos (40%):** El módulo mi_auditoria.py cumple el contrato: el comprobador público imprime AUDITORIA_OK.
- **2.0 puntos (20%):** La columna de uso admisible de notas/dictamen.tsv está razonada para los cuatro modelos.
- **2.0 puntos (20%):** Los bloques de cálculo ancla, perturbación y guarda de dominio están reproducidos con sus salidas guardadas.
- **2.0 puntos (20%):** Defensa escrita de 100 a 150 palabras sobre el modelo de confianza alta y empaquetamiento dudoso.

Lista de autocorrección

- Verifiqué que mi versión de Python es ≥ 3.9 .
- Score 92.5 clasificó en VERY_HIGH.
- La media pLDDT evaluó a 80.000000.
- La PAE media evaluó a 2.825000 Å (RIGID).
- La desigualdad triangular resultó válida.
- Perturbación evaluó a VERY_LOW_OR_IDP y LINKER.
- El pLDDT 105.0 fue rechazado por la guarda.
- bash check_auditoria.sh imprimió AUDITORIA_OK.
- Anoté las predicciones antes de ejecutar el módulo.
- Dictaminé los cuatro modelos y redacté la defensa.

Defensa conceptual

En 100–150 palabras, y sobre el modelo cuyo pLDDT medio es alto pero cuya submatriz PAE desaconseja el empaquetamiento: diga qué afirmación sobre esa proteína sí puede sostenerse con el modelo y cuál no. Añada por qué una región con pLDDT por debajo de 50 suele ser desorden funcional y no un fallo de cálculo, y qué consecuencia tiene eso para quien recorta esa región del modelo antes de enseñarlo.

Fuentes y reproducibilidad

Este anexo se apoya en el material docente de la asignatura y en las mismas fuentes primarias que el capítulo: Jumper y sus colaboradores [37] para AlphaFold2, Baek y sus colaboradores [6] para RoseTTAFold y Abramson y sus colaboradores [2] para AlphaFold3. Todos los valores numéricos del enunciado se reproducen con las órdenes impresas junto a ellos.

Cribado virtual y diseño computacional de fármacos: del acoplamiento molecular al candidato priorizado

BC-CH14 – Biología Computacional

Edición 2

Pregunta del tema. Ante cientos de millones de compuestos disponibles y una diana con estructura conocida, ¿cómo se eligen los cinco que merece la pena sintetizar y probar, y cómo se explica por qué esos y no otros?

Producto final. Un módulo propio en Python que evalúe la regla de cinco, la similitud de Tanimoto, una afinidad ilustrativa y un paso de integración, aplicado a diez compuestos para producir un ranking razonado con un control positivo y dos señuelos identificados.

Mapa del tema

14.1. Del cribado masivo al diseño racional	180
14.2. Cribado virtual: el flujo de trabajo	180
14.3. Bases de datos y representación de moléculas pequeñas	181
14.4. Acoplamiento molecular	182
14.5. AutoDock Vina	183
14.6. Cribado basado en ligandos	184
14.7. Filtros de propiedades	186
14.8. Curado por dinámica molecular	187
14.9. Ranking, controles y señuelos	189
14.10. El módulo en Python	189
14.11. Actividad de depuración	190
14.12. La síntesis del libro	191
14.13. Glosario	191
14.14. Cierre	192
14.15. Fuentes y reproducibilidad	192

Cómo usar este tema

Este es el último capítulo del libro y el único donde el producto no es una descripción de la biología sino una decisión: qué compuestos pasan a la siguiente fase. Todo lo anterior alimenta esta decisión. La estructura de la diana viene de los capítulos de predicción, la evidencia experimental que la respalda del capítulo de datos estructurales, y la idea de que un número puede tener una incertidumbre que no se ve, de todo el libro.

El capítulo se organiza siguiendo el orden en que se toman las decisiones en un proyecto real: primero qué se busca y con qué información se cuenta; luego cómo se representa químicamente; luego el acoplamiento y sus límites; luego los filtros; y por último cómo se comprueba que el ranking no es ruido. La dinámica molecular aparece donde le corresponde en ese orden, al final, como herramienta de curado, y no al principio como si fuera el punto de partida.

Al terminar sabrá: elegir entre cribado basado en estructura y basado en ligandos según lo que tenga; leer una puntuación de acoplamiento sabiendo que no es una energía libre medida; calcular una similitud de Tanimoto y decir qué no implica; aplicar la regla de cinco como heurística de priorización y no como criterio de exclusión; y montar un cribado con control positivo y señuelos, que es lo único que permite saber si el ranking vale algo.

La ruta **esencial** son el diseño racional frente al cribado masivo, el flujo de trabajo, la representación química, el acoplamiento con sus límites, los filtros de propiedades y el ranking con sus controles. La ruta de **estudio** añade el interior de la dinámica molecular, la actividad de depuración y el anexo práctico.

14.1 Del cribado masivo al diseño racional

ESENCIAL

Durante décadas, encontrar un fármaco fue un problema de fuerza bruta. El **cribado de alto rendimiento** consiste en ensayar físicamente cientos de miles de compuestos contra una diana, con robótica, placas de ensayo y reactivos. Funciona, y ha dado fármacos reales, pero cuesta millones y no explica nada: cuando un compuesto sale positivo, no se sabe por qué.

El **diseño racional** invierte el planteamiento. Si se conoce la estructura tridimensional de la diana, se puede preguntar qué molécula encajaría en su cavidad antes de sintetizar ninguna. El cribado deja de ser un experimento y pasa a ser un cálculo, con dos consecuencias: se puede explorar un espacio químico mucho mayor, y cada candidato viene con una hipótesis estructural de por qué debería funcionar.

Esa hipótesis es lo valioso y también lo peligroso. Un cálculo siempre devuelve un ranking, incluso cuando ninguno de los compuestos sirve.

- **Farmacodinámica:** qué le hace el fármaco al organismo. Es la parte que el acoplamiento molecular intenta predecir: si la molécula se une a la diana y con qué fuerza.
- **Farmacocinética:** qué le hace el organismo al fármaco. Absorción, distribución, metabolismo y excreción, lo que se abrevia como **ADME**; con la toxicidad añadida, **ADMET**.

Atención. Un compuesto puede unirse magníficamente a la diana en el tubo de ensayo y no llegar nunca a ella en el organismo, porque no se absorbe, porque el hígado lo degrada en minutos o porque no atraviesa la membrana adecuada. La mayor parte de los fracasos en desarrollo de fármacos no son fracasos de afinidad, sino de farmacocinética y toxicidad. Por eso los filtros de propiedades que veremos más adelante no son un detalle administrativo: son la mitad del problema.

El diseño racional sustituye el ensayo físico masivo por el cálculo sobre la estructura de la diana, y produce candidatos acompañados de una hipótesis estructural; la afinidad predicha corresponde a la farmacodinámica y no anticipa el comportamiento farmacocinético del compuesto.

14.2 Cribado virtual: el flujo de trabajo

ESENCIAL

Ante un proyecto concreto, la primera decisión no es qué programa usar sino qué información se tiene. Hay dos estrategias y la elección entre ellas está determinada por eso.

	Basado en estructura	Basado en ligandos
Requisito	estructura tridimensional de la diana, sea cristal, crio-EM o modelo	ligandos con actividad conocida, y mejor si hay inactivos
Pregunta	¿cómo podría unirse un ligando a este sitio?	¿qué compuestos se parecen a los activos?
Salida	una pose y una puntuación	un ranking por similitud, un modelo predictivo o un farmacóforo
Ventaja	permite interpretar el resultado: qué interacciones, qué modificar	rápido, no necesita estructura, robusto para análogos
Limitación	muy sensible a la preparación de la diana y a la función de puntuación	sesgo químico: extrapola mal fuera del espacio conocido
Fallo típico	poses plausibles pero falsas	moléculas casi idénticas con actividades muy distintas

Cuando no se conoce ni la estructura de la diana ni ligandos activos, queda el *diseño de novo*, que construye moléculas desde cero contra el sitio de unión y queda fuera del alcance de este capítulo.

El cribado basado en estructura requiere la estructura tridimensional de la diana y devuelve pose y puntuación; el cribado basado en ligandos requiere activos conocidos y devuelve un ranking por similitud, y ninguno de los dos es aplicable sin su requisito de partida.

Sea cual sea la estrategia, el flujo tiene cuatro etapas y el orden importa:

1. **Preparar la diana.** Elegir la estructura, quitar aguas y ligandos cristalográficos que no interesen, añadir hidrógenos, decidir estados de protonación y delimitar la cavidad. Esta etapa, que parece administrativa, es donde se pierden más resultados.
2. **Preparar la biblioteca.** Descargar los compuestos, generar sus formas tridimensionales, decidir estados de ionización y filtrar por propiedades antes de gastar cálculo en ellos.
3. **Acoplar.** Muestrear poses y puntuarlas.
4. **Post-procesar.** Ordenar, inspeccionar visualmente las poses mejor puntuadas, comprobar que los controles se comportan y decidir qué pasa a la siguiente fase.

Comprueba. La cuarta etapa es la que más se salta y la que decide si el trabajo sirve. Un cribado sin inspección visual de las mejores poses y sin controles produce una lista ordenada que nadie puede defender.

14.3 Bases de datos y representación de moléculas pequeñas

ESENCIAL

Los compuestos que se criban salen de repositorios públicos, y cada uno tiene un propósito distinto:

- **ZINC:** compuestos comercialmente disponibles, ya preparados en tres dimensiones para acoplamiento. Es el repositorio de partida cuando se quiere cribar algo que después se pueda comprar.
- **PubChem:** el agregador más grande, con estructuras y resultados de bioensayos depositados por terceros. Amplísimo y heterogéneo en calidad.
- **ChEMBL:** actividades biológicas extraídas de la literatura y curadas manualmente. Es la fuente cuando se necesitan activos conocidos con su medida de actividad.
- **DrugBank:** fármacos aprobados y en investigación, con sus dianas, su farmacología y sus interacciones.

Para que un programa manipule una molécula hay que escribirla. Tres formatos cubren casi todo:

- **SMILES:** una cadena de texto que codifica el grafo molecular. Compacto, legible y el formato con el que se intercambian estructuras a diario. El etanol es CCO; la aspirina, CC(=O)Oc1ccccc1C(=O)O.
- **InChI:** un identificador estructurado y canónico. A diferencia de SMILES, la misma molécula produce siempre la misma cadena, lo que lo hace útil como clave para comparar bases de datos.

- **MOL y SDF:** formatos con coordenadas atómicas explícitas. SDF además admite campos de propiedades y agrupa muchas moléculas en un solo fichero, que es como se distribuyen las bibliotecas de cribado.

Atención. SMILES no es canónico por defecto: la misma molécula admite varias cadenas válidas según por dónde se empiece a recorrerla. Comparar dos SMILES como cadenas de texto para decidir si son el mismo compuesto es un error frecuente y silencioso. Para eso está InChI, o el SMILES canónico que genera la propia herramienta.

SMILES codifica el grafo molecular como texto y admite varias cadenas válidas para la misma molécula; InChI es canónico y por eso sirve como clave de comparación entre bases de datos.

14.4 Acoplamiento molecular

ESENCIAL

El **acoplamiento molecular** predice cómo se coloca una molécula pequeña dentro de una cavidad de la diana y con qué fuerza se une. El resultado tiene dos partes que conviene separar desde el principio: una **pose**, que es una hipótesis geométrica, y una **puntuación**, que es un número usado para ordenar candidatos.

Conviene saber qué se está simplificando:

- **El receptor suele tratarse como rígido.** Las proteínas no lo son, y muchas cavidades cambian de forma al unir el ligando. Eso es el *ajuste inducido*, y un acoplamiento rígido no puede reproducirlo.
- **El disolvente es implícito.** Las moléculas de agua individuales que median un puente de hidrógeno entre ligando y proteína, y que a veces son decisivas, no están.
- **La entropía se estima de forma muy gruesa**, con un término que penaliza los enlaces rotables del ligando y poco más.

14.4.1 El espacio de búsqueda

ESENCIAL

Colocar un ligando flexible en una cavidad rígida significa explorar:

- Seis grados de libertad de cuerpo rígido: tres traslaciones y tres rotaciones.
- N_{rot} grados de libertad torsionales, uno por cada enlace rotatable del ligando.

Un ligando de tamaño farmacológico típico tiene entre cinco y diez enlaces rotables, de modo que el espacio pasa de seis a quince o más dimensiones continuas. No se recorre exhaustivamente: se muestrea. Los programas usan algoritmos genéticos, Monte Carlo o combinaciones de ambos, y discretizan las torsiones en estados preferentes en lugar de tratarlas como continuas.

De ahí sale un parámetro que hay que entender antes de usarlo. La *exhaustividad* controla cuántas veces se repite la búsqueda desde puntos de partida distintos. Subirla no mejora la función de puntuación: mejora la probabilidad de que el mínimo encontrado sea realmente el mejor que esa función puede dar. Si el resultado cambia mucho entre ejecuciones, la exhaustividad es baja para ese ligando.

El acoplamiento muestrea seis grados de libertad de cuerpo rígido más uno por cada enlace rotatable del ligando; la exhaustividad controla el número de búsquedas independientes y no altera la función de puntuación.

14.4.2 Funciones de puntuación: tres familias

ESENCIAL

Familia	Idea	Límite
Campo de fuerza	términos de Coulomb y de Lennard-Jones más contribuciones internas	el disolvente y la entropía son difíciles; coste alto
Empírica	suma ponderada de contribuciones, con los pesos ajustados por regresión contra afinidades medidas	generaliza mal fuera del espacio químico donde se calibró
Basada en conocimiento	potenciales estadísticos derivados de la frecuencia de contactos en complejos depositados	hereda los sesgos de composición de esos depósitos

14.5 AutoDock Vina

ESENCIAL

AutoDock Vina [70] usa una función empírica. Suma términos de pares átomo-átomo, calculados sobre la distancia entre superficies de van der Waals: dos términos gaussianos de contacto estérico favorable, uno de repulsión, uno hidrofóbico entre átomos apolares y uno de puente de hidrógeno. Esa suma da la contribución intermolecular, que después se divide por un factor que depende de los enlaces rotables:

$$\text{puntuación} = \frac{\sum_{i<j} \text{términos de pares}}{1 + w \cdot N_{\text{rot}}}, \quad w = 0,0585$$

La división, y no una suma de penalización, es lo que distingue a Vina de su antecesor AutoDock 4. El resultado se expresa en kilocalorías por mol porque los pesos se ajustaron por regresión contra afinidades medidas, pero eso no lo convierte en una energía libre de unión.

Atención. La puntuación de acoplamiento es una magnitud de ordenación, no una medida. Sirve para decir que un compuesto es más prometedor que otro dentro del mismo cribado y con la misma preparación. No sirve para afirmar que un compuesto tiene una afinidad determinada, ni para comparar números entre cribados distintos, ni para alimentar cálculos termodinámicos posteriores como si fuera ΔG° .

AutoDock Vina suma términos de pares átomo-átomo y divide el total intermolecular por $1 + w N_{\text{rot}}$ con $w = 0,0585$; el resultado se expresa en kilocalorías por mol por calibración empírica y es una puntuación de ordenación, no una energía libre medida.

14.5.1 Flujo de trabajo

ESENCIAL

Vina no lee ficheros PDB directamente. Ambas moléculas se convierten al formato pdbqt, que añade cargas parciales y, en el ligando, la definición de qué enlaces son rotables:

```
# receptor: quitar aguas, añadir hidrógenos polares, cargas -> proteina.pdbqt
# ligando: definir raíz y torsiones -> ligando.pdbqt
vina --config config.txt --ligand ligando.pdbqt --out salida.pdbqt
```

El fichero de configuración delimita la caja de búsqueda, que es la decisión más consecuente de todo el proceso:

```
receptor = proteina.pdbqt
center_x = 15.0
center_y = 42.5
center_z = 22.1
size_x = 20
size_y = 20
size_z = 20
exhaustiveness = 8
```

Comprueba. Antes de fiarse de un ranking, vuelva a acoplar el ligando que venía en el cristal y compare la pose obtenida con la experimental. Si el desvío cuadrático medio supera los 2 ángstrom, la caja o la preparación están mal y el resto del cribado no significa nada. Esta comprobación se llama *redocking* y cuesta una ejecución.

14.5.2 De la puntuación a una afinidad: una ilustración

ESTUDIO

Si una magnitud fuera de verdad una energía libre estándar de unión, la termodinámica la relacionaría con la constante de disociación:

$$\Delta G^\circ = RT \ln K_d \iff K_d = \exp\left(\frac{\Delta G^\circ}{RT}\right)$$

con $R = 1,987204 \times 10^{-3}$ kcal por mol y kelvin. A 298,15 K, el producto RT vale 0,592484 kcal por mol. Aplicando la fórmula a $-8,50$:

$$K_d = \exp\left(\frac{-8,50}{0,592484}\right) = \exp(-14,3464) \approx 5,88 \times 10^{-7} \text{ M} \approx 0,588 \mu\text{M}$$

Atención. Este cálculo es una **ilustración numérica de la relación termodinámica**, no una predicción de afinidad. La puntuación de acoplamiento no es ΔG° , de modo que el micromolar que sale de aquí no es la afinidad del compuesto. Las afinidades se miden, con calorimetría isoterma de titulación o resonancia de plasmón superficial, o se estiman con métodos de energía libre sobre trayectorias, y aun entonces con incertidumbres de varias kilocalorías por mol.

La relación $K_d = \exp(\Delta G^\circ / RT)$ aplicada al valor $-8,50$ kcal/mol a 298,15 K da aproximadamente 0,588 micromolar; el cálculo ilustra la relación termodinámica y no predice la afinidad de un compuesto a partir de su puntuación de acoplamiento.

Se reproduce con `verification/chapter_checks.py affinity --delta_g -8.50 --temp 298.15`.

Control de dominio.

La fórmula exige temperatura absoluta. El módulo rechaza los valores no positivos en lugar de devolver un número, porque un exponente calculado con grados Celsius produce un resultado con aspecto razonable y órdenes de magnitud de error.

La conversión a constante de disociación rechaza las temperaturas no positivas e informa del rechazo sin abortar la ejecución.

Se reproduce con `verification/chapter_checks.py affinity --delta_g -8.50 --temp -5.0`.

14.6 Cribado basado en ligandos

ESENCIAL

Cuando no hay estructura de la diana pero sí compuestos activos conocidos, se cambia de pregunta. En lugar de «¿encaja aquí?», se pregunta «¿se parece a los que funcionan?». El supuesto de fondo se llama **principio de similitud**: moléculas parecidas tienden a tener actividades parecidas. Es una tendencia estadística útil, no una ley, y su contraejemplo tiene nombre propio, que veremos al final de la sección.

14.6.1 Huellas moleculares

ESENCIAL

Para comparar moléculas hay que convertirlas en algo comparable. Una **huella molecular** es un vector de bits donde cada posición indica la presencia o ausencia de un rasgo estructural. Hay dos familias:

- **Claves estructurales**, como MACCS: una lista fija de preguntas predefinidas («¿hay un anillo aromático?», «¿hay un grupo carboxilo?»). Cada bit tiene un significado que se puede leer. Son cortas, típicamente 166

Acoplamiento Molecular (Docking) y Termodinámica de Unión



Figura 14.1: Acoplamiento molecular en la caja de búsqueda y relación termodinámica entre la energía libre estándar de unión y la constante de disociación. Figura original del capítulo.

bits, y muy interpretables.

- **Huellas circulares**, como ECFP [58]: no hay lista previa. El algoritmo recorre cada átomo, describe su entorno hasta un radio dado, y convierte cada entorno encontrado en un bit mediante una función de dispersión. Capturan mucha más información estructural, pero un bit concreto ya no significa nada legible por sí solo.

14.6.2 El coeficiente de Tanimoto

ESENCIAL

Dadas dos huellas, el **coeficiente de Tanimoto** mide la fracción de rasgos compartidos:

$$T(A, B) = \frac{c}{a + b - c}$$

donde a es el número de bits activos en la primera molécula, b el de la segunda y c el de bits activos en ambas. El denominador es el tamaño de la unión, de modo que el coeficiente va de 0, sin ningún rasgo en común, a 1, huellas idénticas.

Traza manual.

Con $a = 10$, $b = 12$ y $c = 8$:

$$T = \frac{8}{10 + 12 - 8} = \frac{8}{14} = \frac{4}{7} \approx 0,571429$$

El coeficiente de Tanimoto entre dos huellas con 10 y 12 bits activos y 8 en común vale aproximadamente 0,571429.

Se reproduce con `verification/chapter_checks.py tanimoto --a 10 --b 12 --c 8`.

Se usa habitualmente un umbral de 0,85 para declarar dos compuestos «similares». Conviene saber de dónde sale: es una convención empírica establecida sobre huellas concretas y colecciones concretas, no un valor con significado propio. Con otra huella, el mismo par de moléculas da otro número, y el umbral deja de valer.

Atención. El principio de similitud falla, y falla de una forma concreta que tiene nombre: dos compuestos casi idénticos, con una similitud de Tanimoto por encima de 0,9, pueden diferir en órdenes de magnitud de actividad porque el átomo que los distingue está justo en el punto de contacto con la diana. Un cribado por similitud no puede ver eso. Es la razón de que la similitud sirva para priorizar y no para concluir.

El coeficiente de Tanimoto es la razón entre los rasgos compartidos y la unión de los rasgos de dos huellas; el umbral habitual de 0,85 es una convención dependiente del tipo de huella y no un valor absoluto.

14.6.3 Farmacóforos y modelos predictivos

ESTUDIO

Un **farmacóforo** es un paso más allá de la huella: en lugar de comparar rasgos estructurales, describe la disposición *tridimensional* de las características de reconocimiento que un ligando debe presentar. Un donante de puente de hidrógeno aquí, un aceptor a tantos ángstrom, un centro hidrofóbico en tal orientación. Se deriva superponiendo varios activos conocidos y buscando qué tienen en común en el espacio, y después se usa como filtro geométrico sobre la biblioteca. Su ventaja sobre la similitud es que dos moléculas de esqueletos químicos distintos pueden satisfacer el mismo farmacóforo.

Los **modelos de relación cuantitativa entre estructura y actividad** van en otra dirección: en lugar de un filtro, construyen una función que estima la actividad a partir de descriptores numéricos de la molécula. La forma general es una regresión de la actividad sobre esos descriptores, que pueden ser propiedades globales, recuentos de rasgos o valores derivados de la huella. Su límite es el de cualquier modelo ajustado: solo es fiable dentro del espacio químico del conjunto con que se entrenó, y no hay nada en el modelo que avise de cuándo se ha salido de él.

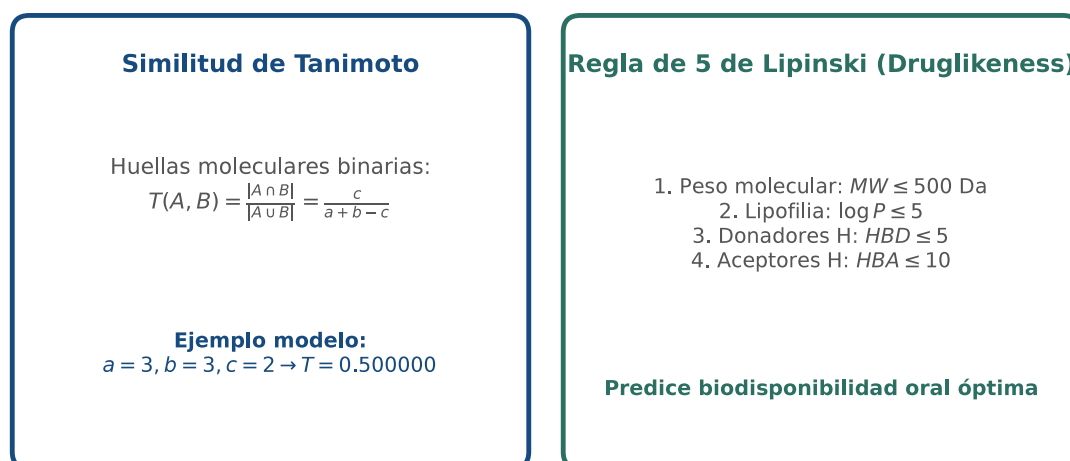


Figura 14.2: Similitud de Tanimoto sobre huellas moleculares binarias y criterios de la regla de cinco. Figura original del capítulo.

14.7 Filtros de propiedades

ESENCIAL

Antes o después del acoplamiento, la biblioteca se filtra por propiedades fisicoquímicas. El filtro más conocido es la **regla de cinco** de Lipinski [49], formulada a partir del análisis de compuestos que habían llegado a fase clínica:

1. Masa molecular no mayor de 500 dalton.
2. Logaritmo del coeficiente de reparto octanol-agua no mayor de 5.
3. No más de 5 donantes de puente de hidrógeno.
4. No más de 10 aceptores de puente de hidrógeno.

Traza manual.

Para un candidato con masa 350, logaritmo de reparto 2,50, dos donantes y cuatro aceptores: los cuatro criterios se cumplen, de modo que hay **cero violaciones**.

Un candidato con masa molecular 350, logaritmo de reparto 2,50, dos donantes y cuatro aceptores de puente de hidrógeno no incumple ninguno de los cuatro criterios de la regla de cinco.

Se reproduce con `verification/chapter_checks.py lipinski --mw 350.0 --logp 2.50 --hbd 2 --hba 4`.

Atención. La regla de cinco describe propiedades asociadas a la **absorción oral pasiva** en un espacio químico concreto. Dos violaciones señalan riesgo, no exclusión: hay fármacos aprobados que incumplen varios criterios, y clases enteras, como los antibióticos macrólidos o los péptidos cíclicos, que quedan fuera por construcción. Tampoco dice nada sobre si el compuesto se une a la diana. Un compuesto con cuatro violaciones *incumple la regla de cinco*; decir que «carece de farmacóforo» es confundir dos cosas distintas, porque el farmacóforo es la disposición tridimensional de las características de reconocimiento y no tiene relación con estos cuatro números.

Junto a la regla de cinco se aplican otros filtros que cubren lo que ella no ve:

- **Reglas de Veber:** añaden el número de enlaces rotables y la superficie polar, que predicen la biodisponibilidad oral mejor que la masa sola.
- **PAINS:** catálogo de subestructuras que dan positivo en casi cualquier ensayo por artefactos de interferencia. No son activos: son ruido con forma de resultado, y conviene eliminarlos antes de gastar tiempo en ellos.
- **Filtros de Brenk:** subestructuras reactivas, inestables o tóxicas que se descartan aunque el compuesto puntúe bien.
- **Predictores ADMET:** modelos que estiman absorción, distribución, metabolismo, excreción y toxicidad. Son predicciones, con las mismas cautelas que cualquier modelo ajustado, pero permiten priorizar antes de sintetizar.

La regla de cinco es una heurística de absorción oral pasiva cuyo incumplimiento indica riesgo y no exclusión, y no informa sobre la unión a la diana; los filtros PAINS eliminan subestructuras que producen falsos positivos de ensayo.

14.8 Curado por dinámica molecular

ESENCIAL

El acoplamiento produce una fotografía: una pose estática en una cavidad rígida y sin agua. La **dinámica molecular** [32] somete ese complejo a agua, iones y temperatura durante nanosegundos y observa si la pose sobrevive. Es la prueba de choque del candidato, y se aplica a los pocos compuestos que han llegado arriba en el ranking, no a la biblioteca entera.

Tres lecturas de la trayectoria deciden:

- **Desvío del ligando** respecto de su pose inicial. Si crece de forma sostenida o fluctúa mucho, el ligando se está reacomodando o saliendo, y el acoplamiento era un falso positivo.
- **Persistencia de las interacciones.** Los puentes de hidrógeno que justificaban la unión deben mantenerse durante una fracción alta del tiempo; por debajo de la mitad de la trayectoria, la interacción es circunstancial.
- **Recálculo de la energía de unión** sobre la trayectoria, con métodos de mecánica molecular y solvente continuo. Es más informativo que la puntuación de acoplamiento porque promedia sobre muchas configuraciones, y sigue sin ser una medida.

Identificar qué residuos sostienen la interacción tiene además un uso adicional: si son residuos muy conservados y presentes en muchas proteínas de la misma familia, el compuesto será probablemente promiscuo.

El curado por dinámica molecular evalúa el desvío del ligando respecto de la pose inicial, la persistencia temporal de las interacciones clave y la energía de unión recalculada sobre la trayectoria; se aplica a los candidatos priorizados y no a la biblioteca completa.

14.8.1 Cómo funciona por dentro

ESTUDIO

Una simulación de dinámica molecular integra las ecuaciones de Newton sobre la superficie de energía potencial del sistema:

$$\vec{F}_i = m_i \vec{a}_i = -\nabla_i V(\vec{r}_1, \dots, \vec{r}_N)$$

El resultado no es una trayectoria continua sino una sucesión de pasos discretos que la aproxima. Los enlaces covalentes vibran a frecuencias del orden de 10^{14} por segundo, de modo que el paso tiene que ser mucho menor que ese periodo: uno o dos femtosegundos.

El potencial: campos de fuerza.

La función V es empírica y se descompone en contribuciones:

- **Enlaces y ángulos:** potenciales armónicos, $\sum k_b(r - r_0)^2$ y $\sum k_\theta(\theta - \theta_0)^2$.
- **Diedros:** potencial periódico $\sum \frac{V_n}{2}[1 + \cos(n\phi - \gamma)]$, que codifica las conformaciones preferentes de rotación.
- **Van der Waals:** potencial de Lennard-Jones, con repulsión a corta distancia y atracción a media.
- **Electrostática:** interacción de Coulomb entre cargas parciales atómicas.

Los conjuntos de parámetros de uso extendido, AMBER, CHARMM y OPLS, difieren en cómo se ajustaron esos términos, y no son intercambiables pieza a pieza.

Un campo de fuerza descompone la energía potencial en términos de enlaces, ángulos y diedros más van der Waals y electrostática, con parámetros ajustados empíricamente que no son intercambiables entre conjuntos.

Descomposición Energética de Campos de Fuerza (AMBER / CHARMM / OPLS)

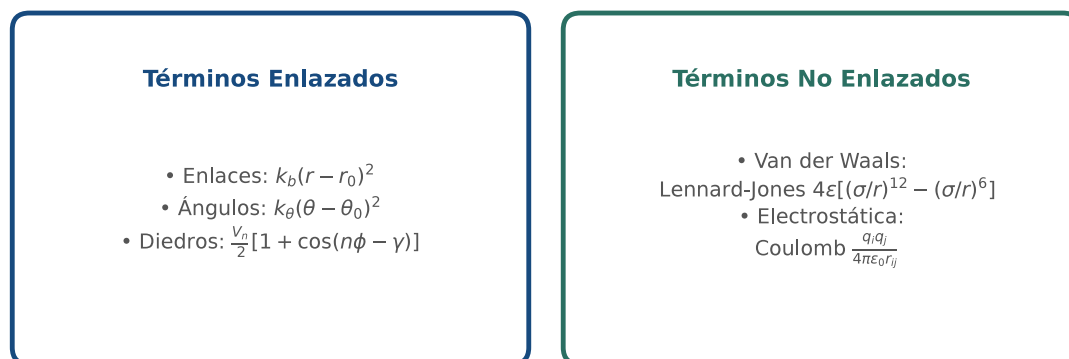


Figura 14.3: Descomposición de la energía potencial en un campo de fuerza macromolecular. Figura original del capítulo.

El integrador: velocity-Verlet.

Conocidas posición, velocidad y aceleración, el paso avanza en dos mitades. Primero la posición:

$$x_1 = x_0 + v_0 \Delta t + \frac{1}{2} a_0 \Delta t^2$$

Después se recalcula la aceleración en la posición nueva, con el campo de fuerza, y con las dos aceleraciones se actualiza la velocidad:

$$a_1 = \frac{F(x_1)}{m}, \quad v_1 = v_0 + \frac{1}{2}(a_0 + a_1)\Delta t$$

Traza manual.

Con $x_0 = 0$, $v_0 = 1$, $a_0 = 2$ y $\Delta t = 0,5$, y suponiendo aceleración constante, de modo que $a_1 = 2$:

$$x_1 = 0 + 1 \times 0,5 + \frac{1}{2} \times 2 \times 0,25 = 0,75, \quad v_1 = 1 + \frac{1}{2}(2 + 2) \times 0,5 = 2,0$$

Atención. El segundo paso es lo que hace de esto un integrador y no una fórmula de cinemática. En una simulación real a_1 no se supone: se obtiene evaluando el campo de fuerza en la posición nueva, y ese recálculo en cada paso es el grueso del coste de una simulación. Quedarse solo con la primera fórmula, como si el paso terminara ahí, deja fuera la mitad del algoritmo y la totalidad de su acoplamiento con la física del sistema.

El paso de velocity-Verlet actualiza primero la posición con la aceleración actual y después la velocidad con el promedio de la aceleración actual y la recalculada en la posición nueva; para $x_0 = 0$, $v_0 = 1$, $a_0 = 2$ y $\Delta t = 0,5$ con aceleración constante da $x_1 = 0,75$ y $v_1 = 2,0$.

Se reproduce con `verification/chapter_checks.py velocity_verlet --x0 0.0 --v0 1.0 --a0 2.0 --a1 2.0 --dt 0.5`.

14.9 Ranking, controles y señuelos

ESENCIAL

Un cribado devuelve una lista ordenada. Esa lista existe siempre, incluso si el protocolo no funciona, porque ordenar números es algo que un ordenador hace pase lo que pase. La pregunta operativa es cómo se distingue un ranking informativo de una ordenación de ruido, y la respuesta no está en el ranking sino en lo que se haya metido dentro de él a propósito.

- **Control positivo:** un ligando cuya actividad sobre esa diana está medida y publicada. Se criba junto con los demás. Si no aparece entre los primeros, el protocolo no distingue lo que debería y el resto de la lista no es interpretable.
- **Control negativo:** un compuesto de la misma familia química del que se sabe que no es activo. Si puntúa alto, la función de puntuación está premiando algo que no es la unión.
- **Señuelos:** compuestos elegidos por tener propiedades fisicoquímicas parecidas a las de los activos y una estructura suficientemente distinta como para que sea muy improbable que se unan. Su papel es medir si el cribado separa por interacción real o simplemente por tamaño y lipofilia.

Con activos y señuelos mezclados se puede cuantificar el resultado. El **enriquecimiento** mide cuántos activos aparecen en la primera fracción de la lista comparado con los que aparecerían al azar: un enriquecimiento de 10 en el primer 1 % significa que ese tramo concentra diez veces más activos de los que le tocarían. La **curva ROC** y su área generalizan esa idea a toda la lista; un área de 0,5 es indistinguible del azar.

Comprueba. Antes de mirar los resultados, mire los controles. Si el control positivo no está arriba o los señuelos no están abajo, no hay nada que interpretar en el resto de la lista, por prometedores que parezcan los primeros nombres.

Un cribado solo es interpretable si incluye control positivo, control negativo y señuelos; el enriquecimiento en la fracción inicial de la lista y el área bajo la curva ROC cuantifican la capacidad del protocolo de separar activos de señuelos, y un área de 0,5 equivale al azar.

14.10 El módulo en Python

ESTUDIO

Las cuatro funciones que el anexo pide implementar, con el contrato que el comprobador público verifica. Aquí se muestran las firmas y las guardas; el cuerpo es lo que se escribe en el anexo.

```
import math
from typing import Any, Dict, Tuple
```

```

R_GAS_KCAL = 1.987204e-3 # kcal / (mol * K)

def velocity_verlet_step(x0: float, v0: float, a0: float,
                        a1: float, dt: float) -> Tuple[float, float]:
    """Paso completo: posicion nueva y velocidad nueva.

    a1 es la aceleracion recalculada con el campo de fuerza en x1. En este
    ejemplo la aceleracion es constante, de modo que a1 coincide con a0.
    """
    if dt <= 0:
        raise ValueError("El paso de tiempo debe ser positivo")
    x1 = x0 + v0 * dt + 0.5 * a0 * dt ** 2
    v1 = v0 + 0.5 * (a0 + a1) * dt
    return x1, v1

def kd_ilustrativa_um(delta_g: float, temperature_k: float = 298.15) -> float:
    """Kd en micromolar a partir de una energia libre estandar de union.

    ILUSTRATIVA: aplicada a una puntuacion de acoplamiento, el resultado no
    es una afinidad predicha. Vease la advertencia de la seccion de Vina.
    """
    if temperature_k <= 0:
        raise ValueError("La temperatura absoluta debe ser estrictamente positiva")
    rt = R_GAS_KCAL * temperature_k
    return math.exp(delta_g / rt) * 1.0e6

def tanimoto(a: int, b: int, c: int) -> float:
    """Coeficiente de Tanimoto entre dos huellas binarias."""
    if a < 0 or b < 0 or c < 0:
        raise ValueError("Los recuentos de bits deben ser no negativos")
    if c > a or c > b:
        raise ValueError("Los bits comunes no pueden superar los de cada huella")
    denom = a + b - c
    return 1.0 if denom == 0 else c / denom

def violaciones_ro5(mw: float, logp: float, hbd: int, hba: int) -> Dict[str, Any]:
    """Cuenta violaciones de la regla de cinco. No decide si el compuesto sirve."""
    v = sum((mw > 500.0, logp > 5.0, hbd > 5, hba > 10))
    return {"violaciones": v, "riesgo_absorcion": "alto" if v >= 2 else "bajo"}

```

Atención. La última función devuelve un recuento y una etiqueta de riesgo, no un veredicto. La versión anterior de este módulo devolvía un campo llamado «apto», y ese nombre invitaba a usar la regla de cinco como filtro binario, que es justo lo que no es.

14.11 Actividad de depuración

ESTUDIO

La siguiente función criba una biblioteca y contiene tres errores. Los tres compilan y los tres producen resultados con aspecto razonable, que es lo que los hace peligrosos. Localícelos y corríjalos antes de leer el diagnóstico.

```

def cribar_candidatos(biblioteca, temperatura_c=25.0):
    seleccionados = []
    for compuesto in biblioteca:
        score = acoplar(compuesto)
        if score < -12.0:
            kd = kd_ilustrativa_um(score, temperatura_c)
            viol = violaciones_ro5(compuesto.mw, compuesto.logp,
                                   compuesto.hbd, compuesto.hba)
            if viol["violaciones"] >= 1:
                continue
            seleccionados.append((compuesto, score, kd))
    return seleccionados

```


Diagnóstico.

1. **El umbral fijo de $-12,0$ trata la puntuación como si fuera una medida.** Una puntuación por debajo de -12 no garantiza unión, y el corte no es transferible: depende de la diana, de la preparación y del tamaño de los ligandos. Lo correcto es ordenar y quedarse con la fracción superior, comprobando antes dónde ha caído el control positivo.
2. **La temperatura se pasa en grados Celsius.** El argumento vale 25 y la fórmula espera kelvin. La función tiene una guarda para valores no positivos, que aquí no salta porque 25 es positivo, y el resultado sale sin aviso. El error no es de un orden de magnitud: el exponente cambia de $-8,5/0,592$ a $-8,5/0,0497$, y el cociente entre ambos resultados es del orden de 10^{68} .
3. **Descartar con una sola violación de la regla de cinco.** Una violación es una señal de riesgo, no un motivo de exclusión, y este filtro elimina compuestos activos por un criterio que ni siquiera habla de actividad. Además se aplica *después* del acoplamiento, de modo que ya se ha gastado el cálculo caro en compuestos que se van a tirar.

14.12 La síntesis del libro

REFERENCIA

Parte	Capítulos	Qué añade al arco del libro
I · Representar sistemas biológicos	1 a 5	del objeto biológico a la representación computable: secuencia, genoma, expresión y estructura
II · Producir, procesar y situar datos de secuencia	6 a 8	de dónde salen los datos, con qué calidad y cómo se sitúan sobre una referencia
III · Inferir historia e intervenir	9 y 10	de los datos a la hipótesis evolutiva, y de la comprensión de un circuito a su construcción
IV · Estructura, predicción y diseño molecular	11 a 14	de la evidencia experimental a la predicción, de la predicción a su auditoría, y de ahí a la decisión terapéutica

El arco del libro va de la entidad biológica a la decisión, pasando por la representación, el dato, la inferencia y la incertidumbre. Este capítulo es el último eslabón porque es donde la incertidumbre deja de ser un comentario metodológico y se convierte en una consecuencia: elegir mal cinco compuestos cuesta dinero y meses de trabajo experimental.

Lo que se repite en los catorce capítulos, y conviene llevarse, es una sola disciplina: distinguir lo que un número mide de lo que parece decir. Un valor de soporte en un árbol, un pLDDT, una puntuación de acoplamiento y un coeficiente de Tanimoto tienen en común que todos se leen con más confianza de la que merecen.

14.13 Glosario

REFERENCIA

- **Acoplamiento molecular:** predicción de la pose y la puntuación de un ligando en una cavidad.
- **Pose:** hipótesis geométrica de cómo se coloca el ligando.
- **Exhaustividad:** número de búsquedas independientes por ligando.
- **Redocking:** reacoplar el ligando cristalográfico para validar la preparación.
- **Huella molecular:** vector de bits que codifica rasgos estructurales.
- **Tanimoto:** razón entre rasgos compartidos y unión de rasgos.
- **Farmacóforo:** disposición tridimensional de las características de reconocimiento.
- **Regla de cinco:** heurística de absorción oral pasiva.
- **PAINS:** subestructuras que dan positivos por interferencia de ensayo.
- **ADMET:** absorción, distribución, metabolismo, excreción y toxicidad.
- **Señuelo:** compuesto de propiedades similares y estructura distinta, casi con seguridad inactivo.

- **Enriquecimiento:** concentración de activos en la cabecera de la lista frente al azar.
- **Velocity-Verlet:** integrador que actualiza posición y velocidad con la aceleración recalculada.

14.14 Cierre

ESENCIAL

Este capítulo cierra el libro con un cálculo que no da una respuesta sino una lista de candidatos, y con la obligación de justificar por qué esos y no otros. Es la forma más honesta de terminar: el trabajo computacional en biología no sustituye al experimento, lo dirige. Bien hecho, ahorra años; mal hecho, los gasta con mucha eficiencia.

El resultado de un cribado virtual es una priorización de candidatos para ensayo experimental, con una hipótesis estructural asociada, y no una predicción de actividad.

14.15 Fuentes y reproducibilidad

REFERENCIA

Este capítulo se apoya en el material docente de la asignatura y en cuatro fuentes primarias: Trott y Olson [70] para la función de puntuación de Vina, Lipinski y sus colaboradores [49] para la regla de cinco, Rogers y Hahn [58] para las huellas circulares y Hollingsworth y Dror [32] para la dinámica molecular. Todos los cálculos numéricos del texto se reproducen con las órdenes impresas junto a ellos.

Anexo práctico · Un cribado virtual completo, con sus controles

Pregunta, producto y separación de materiales

Tiene la salida de un acoplamiento sobre diez compuestos y la huella molecular de un ligando de referencia. Con eso hay que producir una lista ordenada y defenderla: por qué esos compuestos en ese orden, cuál es el control positivo y cuáles son los señuelos que el ranking no debería haber premiado.

Lo que se entrega es un módulo escrito por usted, el fichero de ranking con su columna de justificación y una defensa escrita. Lo que se le da hecho es distinto y conviene tenerlo separado desde el principio: la biblioteca de compuestos con su manifiesto, el oráculo del capítulo y un comprobador público que dice si su módulo cumple el contrato. El anexo funciona sin red y sin más dependencia que Python 3.9.

El producto entregable de este anexo es el módulo propio de cribado, el ranking razonado de los diez compuestos con el control positivo identificado y los dos señuelos marcados, y una defensa escrita.

Mapa de ruta y productos entregables

Bloque de trabajo	Producto entregable
1 · Entorno y diagnóstico	Espacio de trabajo creado y manifiesto verificado
1b · Módulo de cribado	<code>mi_cribado.py</code> con las cuatro funciones
2 · Cálculo ancla	Las cuatro trazas del capítulo reproducidas
3 · Perturbación	El mismo cálculo sobre un compuesto que incumple la regla
4 · Guarda de dominio	Rechazo razonado de la temperatura no absoluta
5 · Ranking, controles y entrega	<code>notas/ranking.tsv</code> , defensa y paquete

Este anexo deja evidencia comprobable en cada bloque sin recurrir a paquetes externos.

1 · Entorno y diagnóstico

Python 3.9 o superior, sin paquetes externos. Antes de empezar:

```
python3 --version
./practice_assets/create_workspace.sh BC14_entrega
cd BC14_entrega
(cd data && sha256sum --check --strict manifest.sha256)
python3 verification/practice_checks.py domain_guard --temp -5.0
```

El generador deja `mi_cribado.py` con su contrato documentado y sin implementación, copia la biblioteca de diez compuestos con su manifiesto y el comprobador público, y crea `notas/ranking.tsv` con solo la cabecera. La última orden debe imprimir `GUARD_TEMP:-5.0 STATUS:REJECTED` seguido del mensaje de error. Si no lo hace, el intérprete no es el esperado o los ficheros no coinciden con el manifiesto; resuélvalo antes de continuar.

1b • Escribir el módulo de cribado

Tarea. Implemente las cuatro funciones de `mi_cribado.py`: el paso de velocity-Verlet, la constante de disociación ilustrativa, el coeficiente de Tanimoto y el recuento de violaciones de la regla de cinco. El contrato está en la documentación de la plantilla.

Dos avisos sobre el contrato, porque son los dos sitios donde se pierde tiempo. El paso de Verlet devuelve *dos* valores: la posición y la velocidad. La velocidad usa el promedio de la aceleración de partida y la recalculada en la posición nueva, y el comprobador incluye un caso con las dos aceleraciones distintas precisamente para que no pase desapercibido. Y los criterios de la regla de cinco son «no mayor que»: una masa de exactamente 500 dalton no es una violación, y hay una comprobación por cada una de las cuatro fronteras.

```
bash check_cribado.sh
```

2 • Cálculo ancla

Las cuatro trazas del capítulo, que su módulo debe reproducir: el candidato con masa 350, logaritmo de reparto 2,50, dos donantes y cuatro aceptores no incumple ninguno de los cuatro criterios; dos huellas con 10 y 12 bits activos y 8 en común dan un coeficiente de Tanimoto de aproximadamente 0,571; una energía libre de $-8,50$ kcal/mol a 298,15 K corresponde a una constante de disociación del orden del micromolar; y el paso de Verlet con $x_0 = 0$, $v_0 = 1$, $a_0 = 2$ y $\Delta t = 0,5$ lleva la posición a 0,75 y la velocidad a 2,0.

```
python3 verification/practice_checks.py anchor
```

El cálculo ancla del anexo reproduce las cuatro trazas del capítulo: cero violaciones de la regla de cinco, el coeficiente de Tanimoto del par de huellas, la constante de disociación ilustrativa y el paso completo de integración.

Se reproduce con `verification/practice_checks.py anchor`.

3 • Perturbación

Repita el cálculo sobre un compuesto que incumple varios criterios de la regla de cinco y cuya pose está mucho peor puntuada:

```
python3 verification/practice_checks.py perturbation
```

Anote las dos cosas que cambian y la que no. Cambia el recuento de violaciones, y cambia la constante ilustrativa en varios órdenes de magnitud. No cambia el hecho de que ninguno de los dos números diga si el compuesto se une: el primero habla de absorción y el segundo es una conversión aritmética de una puntuación que no es una energía libre medida.

La perturbación sobre un compuesto con varias violaciones de la regla de cinco altera el recuento de violaciones y la constante ilustrativa en varios órdenes de magnitud, y ninguno de los dos valores informa sobre la unión a la diana.

Se reproduce con `verification/practice_checks.py perturbation`.

4 • Guarda de dominio

La conversión a constante de disociación exige temperatura absoluta. Compruebe que su módulo rechaza los valores no positivos:

```
python3 verification/practice_checks.py domain_guard --temp -5.0
```

El caso peligroso, sin embargo, no es este. Una temperatura negativa la detecta cualquier guarda; lo que nadie detecta es pasar 25 en lugar de 298,15, porque 25 es positivo y la función responde con un número perfectamente formado. Ese es el segundo error de la actividad de depuración del capítulo, y merece la pena comprobar a mano cuánto se desvía el resultado.

La guarda de temperatura rechaza los valores no positivos e informa del rechazo sin abortar; una temperatura positiva pero en la escala equivocada no es detectable por la guarda y produce un resultado con aspecto válido.

Se reproduce con `verification/practice_checks.py domain_guard --temp -5.0`.

5 • Ranking, controles y entrega

`data/compuestos.tsv` contiene diez compuestos con su masa, logaritmo de reparto, donantes y aceptores de puente de hidrógeno, la puntuación de acoplamiento y los recuentos de bits de su huella frente a la del ligando de referencia.

Aplique su módulo a los diez y complete `notas/ranking.tsv`: el orden por puntuación, las violaciones de la regla de cinco, el coeficiente de Tanimoto frente a la referencia y la constante ilustrativa. Guarde la salida de su programa en `resultados/`.

Después viene la parte que se corrige. En la columna `papel`, identifique cada compuesto: `control` para el positivo, `señuelo` para los dos señuelos y `candidato` para el resto. El control positivo es un ligando con actividad conocida sobre esta diana; los señuelos son compuestos elegidos por tener propiedades fisicoquímicas parecidas a las de los activos y una estructura distinta.

Compruebe. Los señuelos no se distinguen por la puntuación: si se distinguieran por ahí no servirían para nada. Mire la columna que compara estructura con la referencia, y compruebe que su explicación es coherente con las propiedades fisicoquímicas de esos mismos compuestos.

En `notas/defensa.md`, entre 150 y 250 palabras: por qué el control positivo debe encabezar el ranking y qué significaría que no lo hiciera; qué delata a los señuelos; y qué le diría a alguien que propusiera pasar a síntesis los tres primeros compuestos de su lista sin más comprobación.

```
cd ..
tar -czf BC14_entrega.tar.gz BC14_entrega
sha256sum BC14_entrega.tar.gz
./practice_assets/check_delivery.sh BC14_entrega BC14_entrega.tar.gz
```

El verificador comprueba que su módulo compila y cumple el contrato, que la biblioteca no se ha tocado, que los diez compuestos están documentados, que el control positivo encabeza el ranking, que hay dos compuestos marcados como señuelo y que existe la defensa con la extensión pedida. Rechaza la entrega vacía y el espacio recién generado. No puntúa.

El verificador de entrega comprueba el contrato del módulo, la integridad de la biblioteca, las diez filas del ranking con el control positivo en cabeza y los dos señuelos marcados, y la defensa escrita; rechaza el espacio de trabajo recién generado y no emite calificación.

Rúbrica

La evaluación sobre 10 puntos se reparte así:

- **3,0 puntos:** el verificador de entrega termina con ENTREGA_OK y el comprobador público con CRIBADO_OK.
- **3,0 puntos:** la implementación es correcta y sus guardas se justifican, en particular el paso completo de Verlet y las fronteras de la regla de cinco.
- **2,0 puntos:** el ranking está completo y la columna de papel identifica correctamente el control positivo y los dos señuelos.
- **2,0 puntos:** la defensa escrita razona los tres puntos pedidos.

Lista de autocorrección

- | | |
|--|---|
| ■ Mi versión de Python es 3.9 o superior. | violación. |
| ■ El manifiesto de data/ verifica sin errores. | ■ El ranking tiene los diez compuestos. |
| ■ El comprobador público imprimió CRIBADO_OK. | ■ El control positivo encabeza la lista. |
| ■ El paso de Verlet devuelve posición y velocidad. | ■ He marcado dos señuelos y explico qué los delata. |
| ■ Una masa de 500 dalton exactos no cuenta como | ■ La defensa responde a los tres puntos pedidos. |

Fuentes y reproducibilidad

Este anexo se apoya en el material docente de la asignatura y en las mismas fuentes primarias que el capítulo: Trott y Olson [70] para la puntuación de acoplamiento y Lipinski y sus colaboradores [49] para la regla de cinco. Todos los valores numéricos del enunciado se reproducen con las órdenes impresas junto a ellos.

Bibliografía

- [1] Federico Abascal, Iker Irisarri, and Rafael Zardoya. Filogenia y evolución molecular. In Álvaro Sebastián and Alberto Pascual-García, editors, *Bioinformática con Ñ. Volumen I: Principios de Bioinformática*, pages 231–257. CreateSpace, 2014. Capítulo 9; copia local de consulta.
- [2] Josh Abramson, Jonas Adler, Jack Dunger, Richard Evans, Tim Green, Alexander Pritzel, Olaf Ronneberger, Lindsay Willmore, Andrew J. Yim, Augustin Židek, Peter Battaglia, Demis Hassabis, and John Jumper. Accurate structure prediction of biomolecular interactions with alphafold 3. *Nature*, 630:493–500, 2024. Artículo de AlphaFold 3 basado en modelos de difusión generativa para complejos biomoleculares.
- [3] Bruce Alberts, Alexander Johnson, Julian Lewis, David Morgan, Martin Raff, Keith Roberts, and Peter Walter. *Molecular Biology of the Cell*. Garland Science, 6th edition, 2014. Tratado de referencia para el dogma central, estructura de ácidos nucleicos, aminoácidos y biomoléculas.
- [4] Simon Andrews. Fastqc: A quality control tool for high throughput sequence data. *Babraham Bioinformatics*, 2010. Herramienta estándar de control de calidad para datos de secuenciación masiva.
- [5] Christian B. Anfinsen. Principles that govern the folding of protein chains. *Science*, 181(4096):223–230, 1973. Artículo del Premio Nobel que establece la hipótesis termodinámica del plegamiento proteico.
- [6] Minkyung Baek, Frank DiMaio, Ivan Anishchenko, Justas Dauparas, Sergey Ovchinnikov, Gyu Rie Lee, Jue Wang, Qian Cong, Lisa N. Kinch, R. Dustin Schaeffer, Claudia Millán, Hahnbeom Park, Colin Adams, C. R. Glasscock, A. M. DeGiovanni, J. H. Pereira, A. V. Rodrigues, A. A. van Dijk, A. C. Ebrecht, D. J. Opperman, T. Sagendorf, A. Schreshtha, and David Baker. Accurate prediction of protein structures and interactions using a three-track neural network. *Science*, 373(6557):871–876, 2021. Artículo canónico de RoseTTAFold y arquitectura de red de 3 pistas (1D, 2D, 3D).
- [7] Helen M. Berman, John Westbrook, Zukang Feng, Gary Gilliland, T. N. Bhat, Helge Weissig, Ilya N. Shindyalov, and Philip E. Bourne. The protein data bank. *Nucleic Acids Research*, 28(1):235–242, 2000. Especificación y estructura del repositorio global Protein Data Bank (PDB).
- [8] Biopython Project. Bio.phylo api documentation. Biopython 1.87 official documentation, 2026. Consultado 2026-08-05; optional C5 extension.
- [9] Anthony M. Bolger, Marc Lohse, and Bjoern Usadel. Trimmomatic: A flexible trimmer for illumina sequence data. *Bioinformatics*, 30(15):2114–2120, 2014. Algoritmo canónico de recorte por ventana deslizante y eliminación de adaptadores.
- [10] Carl-Ivar Brändén and John Tooze. *Introduction to Protein Structure*. Garland Science, 2nd edition, 1999. Referencia clásica para la jerarquía estructural de proteínas, hélice alfa y lámina beta.
- [11] Vincent B. Chen, W. Bryan Arendall, Jeffrey J. Headd, Daniel A. Keedy, Robert M. Immormino, Simon C. Lovell, Jane S. Richardson, and David C. Richardson. Molprobity: All-atom structure validation for macromolecular crystallography. *Acta Crystallographica Section D*, 66(1):12–21, 2010. Estandarización de evaluación estereoquímica y métrica Clashscore en MolProbity.
- [12] Peter J. A. Cock, Tiago Antao, Jeffrey T. Chang, Brad A. Chapman, Cymon J. Cox, Andrew Dalke, Iddo Friedberg, Thomas Hamelryck, Frank Kauff, Bartek Wilczynski, and Michiel J. L. de Hoon. *Biopython: Freely Available Python Tools for Computational Molecular Biology and Bioinformatics*, volume 25. 2009.
- [13] Peter J. A. Cock, Tiago Antao, Jeffrey T. Chang, et al. Biopython: freely available python tools for computational molecular biology and bioinformatics. *Bioinformatics*, 25(11):1422–1423, 2009.
- [14] Peter J. A. Cock, Christopher J. Fields, Naohisa Goto, Michael L. Heuer, and Peter M. Rice. The sanger fastq file format for sequences with quality scores, and the solexa/illumina fastq variants. *Nucleic Acids Research*, 38(6):1767–1771, 2010. Especificación canónica del formato FASTQ estándar Sanger Phred+33.

- [15] Ana Conesa, Pedro Madrigal, Sonia Tarazona, David Gomez-Cabrero, Andrea Cervera, Andrew McPherson, Michał W. Szczesniak, Daniel J. Gaffney, Laura L. Elo, Xuegong Zhang, and Ali Mortazavi. A survey of best practices for rna-seq data analysis. *Genome Biology*, 17(1):13, 2016. Guías metodológicas y diseño experimental para estudios transcriptómicos con NGS.
- [16] Francis H. C. Crick. Codon–anticodon pairing: The wobble hypothesis. *Journal of Molecular Biology*, 19(2):548–555, 1966. Hipótesis fundacional del balanceo (wobble) y degeneración en la tercera base del codón.
- [17] Petr Danecek, Adam Auton, Goncalo Abecasis, Cornelis A. Albers, Eric Banks, Mark A. DePristo, Robert E. Handsaker, Gerton Lunter, Gabor T. Marth, Stephen T. Sherry, Gilean McVean, and Richard Durbin. The variant call format and vcftools. *Bioinformatics*, 27(15):2156–2158, 2011. Especificación canónica del formato VCF para variantes genómicas y SNPs.
- [18] Mark A. DePristo, Eric Banks, Ryan Poplin, Kiran V. Garimella, Jared R. Maguire, Christopher Hartl, Anthony A. Philippakis, Guillermo del Angel, Manuel A. Rivas, Matt Hanna, Aaron McKenna, Tim J. Fennell, Andrew M. Kernysky, Andrey Y. Sivachenko, Kristian Cibulskis, Stacey B. Gabriel, David Altshuler, and Mark J. Daly. A framework for variation discovery and genotyping using next-generation dna sequencing data. *Nature Genetics*, 43(5):491–498, 2011. Marco metodológico y buenas prácticas de GATK para flujos de trabajo NGS.
- [19] Brent Ewing and Phil Green. Base-calling of automated sequencer traces using phred. ii. error probabilities. *Genome Research*, 8(3):186–194, 1998. Definición logarítmica original de la escala de calidad Phred (Q-score).
- [20] Joseph Felsenstein. Cases in which parsimony or compatibility methods will be positively misleading. *Systematic Zoology*, 27(4):401–410, 1978.
- [21] Joseph Felsenstein. Evolutionary trees from dna sequences: a maximum likelihood approach. *Journal of Molecular Evolution*, 17:368–376, 1981.
- [22] Joseph Felsenstein. Confidence limits on phylogenies: an approach using the bootstrap. *Evolution*, 39(4):783–791, 1985.
- [23] Joseph Felsenstein. *Inferring Phylogenies*. Sinauer Associates, 2004.
- [24] Walter M. Fitch. Distinguishing homologous from analogous proteins. *Systematic Zoology*, 19(2):99–113, 1970.
- [25] Walter M. Fitch. Toward defining the course of evolution: minimum change for a specific tree topology. *Systematic Zoology*, 20(4):406–416, 1971.
- [26] Margaret Gardiner-Garden and Marianne Frommer. CpG islands in vertebrate genomes. *Journal of Molecular Biology*, 196(2):261–282, 1987. Criterios canónicos para la identificación de islas CpG en promotores eucariotas.
- [27] Daniel G. Gibson, Lei Young, Ray-Yuan Chuang, J. Craig Venter, Clyde A. Hutchison, and Hamilton O. Smith. Enzymatic assembly of dna molecules up to several hundred kilobases. *Nature Methods*, 6(5):343–345, 2009. Método de ensamblaje isotérmico de Gibson para clonación molecular sin cicatrices.
- [28] Sara Goodwin, John D. McPherson, and W. Richard McCombie. Coming of age: Ten years of next-generation sequencing technologies. *Nature Reviews Genetics*, 17(6):333–351, 2016. Revisión exhaustiva sobre plataformas de secuenciación de segunda y tercera generación.
- [29] Wayne A. Hendrickson. Stereochemically restrained refinement of macromolecular structures. *Methods in Enzymology*, 115:252–270, 1985. Refinamiento y factores térmicos B de Debye-Waller en cristalografía de proteínas.
- [30] Diep Thi Hoang, Olga Chernomor, Arndt von Haeseler, Bui Quang Minh, and Le Sy Vinh. Ufboot2: improving the ultrafast bootstrap approximation. *Molecular Biology and Evolution*, 35(2):518–522, 2018.

- [31] Mark Holder and Paul O. Lewis. Phylogeny estimation: traditional and bayesian approaches. *Nature Reviews Genetics*, 4:275–284, 2003.
- [32] Scott A. Hollingsworth and Ron O. Dror. Molecular dynamics simulation for all. *Neuron*, 99(6):1129–1143, 2018. Tratado pedagogico sobre los fundamentos de dinamica molecular, campos de fuerza y analisis de trayectorias.
- [33] John D. Hunter. *Matplotlib: A 2D Graphics Environment*, volume 9. 2007. Biblioteca canonica para generacion de figuras y visualizacion cientifica reproducible.
- [34] IQ-TREE Project. Iq-tree command reference. Official documentation; updated 2025-06-05, 2025. Consulted 2026-08-05.
- [35] François Jacob and Jacques Monod. Genetic regulatory mechanisms in the synthesis of proteins. *Journal of Molecular Biology*, 3(3):318–356, 1961. Artículo fundacional sobre la regulacion genica y el modelo del operon Lac.
- [36] Thomas H. Jukes and Charles R. Cantor. Evolution of protein molecules. In *Mammalian Protein Metabolism*, pages 21–132. Academic Press, 1969.
- [37] John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Židek, Anna Potapenko, Alex Bridgland, Clemens Meyer, Simon A. A. Kohl, Andrew J. Ballard, Andrew Cowie, Bernardino Romera-Paredes, Stanislav Nikolov, Rishub Jain, Jonas Adler, Trevor Back, Stig Petersen, David Reiman, Ellen Clancy, Michal Zielinski, Martin Steinegger, Michalina Pacholska, Tamas Berghammer, Sebastian Bodenstein, David Silver, Oriol Vinyals, Andrew W. Senior, Koray Kavukcuoglu, Pushmeet Kohli, and Demis Hassabis. Highly accurate protein structure prediction with alphafold. *Nature*, 596(7873):583–589, 2021. Artículo canonico de AlphaFold2, arquitectura Evoformer y metricas pLDDT y PAE.
- [38] Wolfgang Kabsch. A solution for the best rotation to relate two sets of vectors. *Acta Crystallographica Section A*, 32(5):922–923, 1976. Algoritmo de superposicion estructural y minimizacion del RMSD entre coordenadas tridimensionales.
- [39] Subha Kalyanamoorthy, Bui Quang Minh, Thomas K. F. Wong, Arndt von Haeseler, and Lars S. Jermiin. Modelfinder: fast model selection for accurate phylogenetic estimates. *Nature Methods*, 14:587–589, 2017.
- [40] Kazutaka Katoh and Daron M. Standley. MAFFT multiple sequence alignment software version 7: improvements in performance and usability. *Molecular Biology and Evolution*, 30(4):772–780, 2013.
- [41] Motoo Kimura. A simple method for estimating evolutionary rates of base substitutions through comparative studies of nucleotide sequences. *Journal of Molecular Evolution*, 16:111–120, 1980.
- [42] Laura Kubatko. An evolving view of species tree inference. *Systematic Biology*, 75(2):195–207, 2026.
- [43] Jack Kyte and Russell F. Doolittle. A simple method for displaying the hydropathic character of a protein. *Journal of Molecular Biology*, 157(1):105–132, 1982. Escala canonica de hidropatia para analisis de perfiles hidrofobicos y dominios transmembrana.
- [44] Eric S. Lander and Michael S. Waterman. Genomic mapping by fingerprinting random clones: A mathematical analysis. *Genomics*, 2(3):231–239, 1988. Modelo matematico fundacional de cobertura de secuenciacion y distribucion de Poisson.
- [45] Ben Langmead and Steven L. Salzberg. Fast gapped-read alignment with bowtie 2. *Nature Methods*, 9(4):357–359, 2012. Algoritmo Bowtie 2 para alineamiento ultrarrapido con gaps sobre FM-index.
- [46] Heng Li and Richard Durbin. Fast and accurate short read alignment with burrows-wheeler transform. *Bioinformatics*, 25(14):1754–1760, 2009. Algoritmo BWA para alineamiento de lecturas cortas mediante la transformada de Burrows-Wheeler.
- [47] Heng Li, Bob Handsaker, Alec Wysoker, Tim Fennell, Jue Ruan, Nils Homer, Gabor Marth, Goncalo Abecasis, and Richard Durbin. The sequence alignment/map format and samtools. *Bioinformatics*, 25(16):2078–2079, 2009. Especificacion canonica de los formatos SAM y BAM y el conjunto de herramientas SAMtools.

- [48] Zeming Lin, Halil Akin, Roshan Rao, Brian Hie, Zhongkai Zhu, Wenting Lu, Nikita Smetanin, Robert Verkuil, Ori Kabeli, Yaniv Shmueli, Allan dos Santos Costa, Maryam Fazel-Zarandi, Tom Sercu, Salvatore Candido, and Alexander Rives. Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science*, 379(6637):1123–1130, 2023. Artículo de ESM-2 y ESMFold para predicción estructural basada en modelos de lenguaje proteico (pLM).
- [49] Christopher A. Lipinski, Franco Lombardo, Beryl W. Dominy, and Paul J. Feeney. Experimental and computational approaches to estimate solubility and permeability in drug discovery and development settings. *Advanced Drug Delivery Reviews*, 23(1-3):3–25, 1997. Artículo clásico de la Regla de 5 de Lipinski para evaluación de drug-likeness oral.
- [50] Wayne P. Maddison. Gene trees in species trees. *Systematic Biology*, 46(3):523–536, 1997.
- [51] MAFFT Project. Mafft version 7: official software documentation. Version 7.526; official project site; released April 2024, 2024. Consulted 2026-08-05.
- [52] Lam-Tung Nguyen, Heiko A. Schmidt, Arndt von Haeseler, and Bui Quang Minh. Iq-tree: a fast and effective stochastic algorithm for estimating maximum-likelihood phylogenies. *Molecular Biology and Evolution*, 32(1):268–274, 2015.
- [53] Marshall W. Nirenberg and J. Heinrich Matthaei. The dependence of cell-free protein synthesis in *e. coli* upon naturally occurring or synthetic polyribonucleotides. *Proceedings of the National Academy of Sciences*, 47(10):1588–1602, 1961. Descubrimiento histórico del código genético y correspondencia codon-aminoácido.
- [54] Laura R. Novick and Kefyn M. Catley. When relationships depicted diagrammatically conflict with prior knowledge: an investigation of students' interpretations of evolutionary trees. *Science Education*, 98(2):269–304, 2014.
- [55] Kevin E. Omland, Lyn G. Cook, and Michael D. Crisp. Tree thinking for all biology: the problem with reading phylogenies as ladders of progress. *BioEssays*, 30(9):854–867, 2008.
- [56] Python Software Foundation. The python tutorial. Python 3.9 official documentation, 2020. Versioned reference for data structures, control flow and exceptions; consulted 2026-08-06.
- [57] G. N. Ramachandran, C. Ramakrishnan, and V. Sasisekharan. Stereochemistry of polypeptide chain configurations. *Journal of Molecular Biology*, 7(1):95–99, 1963. Artículo fundacional sobre los ángulos diedros phi y psi y conformaciones permitidas del esqueleto peptídico.
- [58] David Rogers and Mathew Hahn. Extended-connectivity fingerprints. *Journal of Chemical Information and Modeling*, 50(5):742–754, 2010. Artículo canónico de huellas moleculares extendidas (ECFP) y similitud de Tanimoto.
- [59] Carol A. Rohl, Charlie E. M. Strauss, Kira M. S. Misura, and David Baker. Protein structure prediction using rosetta. *Methods in Enzymology*, 383:66–93, 2004. Metodología de predicción ab initio y ensamblaje de fragmentos en Rosetta.
- [60] Burkhard Rost. Twilight zone of protein sequence alignments. *Protein Engineering*, 12(2):85–94, 1999.
- [61] Naruya Saitou and Masatoshi Nei. The neighbor-joining method: a new method for reconstructing phylogenetic trees. *Molecular Biology and Evolution*, 4(4):406–425, 1987.
- [62] Andrej Sali and Tom L. Blundell. Comparative protein modelling by satisfaction of spatial restraints. *Journal of Molecular Biology*, 234(3):779–815, 1993. Artículo canónico sobre el algoritmo MODELLER y modelado comparativo por restricciones espaciales.
- [63] F. Sanger, G. M. Air, B. G. Barrell, N. L. Brown, A. R. Coulson, J. C. Fiddes, C. A. Hutchison, P. M. Slocumbe, and M. Smith. Nucleotide sequence of bacteriophage ϕ x174 dna. *Nature*, 265:687–695, 1977.
- [64] John SantaLucia. A unified view of polymer, dumbbell, and oligonucleotide DNA nearest-neighbor thermodynamics. *Proceedings of the National Academy of Sciences*, 95(4):1460–1465, 1998.

- [65] John SantaLucia. A unified view of polymer, dumbbell, and oligonucleotide dna nearest-neighbor thermodynamics. *Proceedings of the National Academy of Sciences*, 95(4):1460–1465, 1998. Parametros termodinamicos de vecinos mas proximos y calculo de Tm de oligonucleotidos.
- [66] Claude E. Shannon. A mathematical theory of communication. *The Bell System Technical Journal*, 27:379–423, 1948.
- [67] Brian D. Strahl and C. David Allis. The language of covalent histone modifications. *Nature*, 403(6765):41–45, 2000. Articulo seminal sobre el codigo de histonas y regulacion epigenetica de la cromatina.
- [68] Eric Talevich, Brandon M. Invergo, Peter J. A. Cock, and Brad A. Chapman. Bio.phylo: a unified toolkit for processing, analyzing and visualizing phylogenetic trees in biopython. *BMC Bioinformatics*, 13:209, 2012.
- [69] Ge Tan, Matthieu Muffato, Christian Ledergerber, Javier Herrero, Nick Goldman, Marcel Gil, and Christophe Dessimoz. Current methods for automated filtering of multiple sequence alignments frequently worsen single-gene phylogenetic inference. *Systematic Biology*, 64(5):778–791, 2015.
- [70] Oleg Trott and Arthur J. Olson. Autodock vina: Improving the speed and accuracy of docking with a new scoring function, efficient optimization, and multithreading. *Journal of Computational Chemistry*, 31(2):455–461, 2010. Articulo canonico de AutoDock Vina y funcion de scoring empirica de afinidad libre de Gibbs.
- [71] Alan M. Turing. On computable numbers, with an application to the entscheidungsproblem. *Proceedings of the London Mathematical Society*, 42:230–265, 1936.
- [72] Guido Van Rossum and Fred L. Drake. *Python 3 Reference Manual*. CreateSpace, 2009. Manual oficial del lenguaje Python 3.
- [73] Günter P. Wagner, Koryu Kin, and Vincent J. Lynch. Measurement of mrna abundance using rna-seq data: Rpkm measure is inconsistent among samples. *Theory in Biosciences*, 131(4):281–285, 2012. Articulo canonico sobre las ventajas matematicas de la normalizacion TPM frente a RPKM/FPKM.
- [74] J. D. Watson and F. H. C. Crick. Molecular structure of nucleic acids: A structure for deoxyribose nucleic acid. *Nature*, 171:737–738, 1953.
- [75] Thomas K. F. Wong, Nhan Ly-Trong, Huaiyan Ren, Piyumal Demotte, Hector Baños, et al. Iq-tree 3: phylogenomic inference software using complex evolutionary models. *Molecular Biology and Evolution*, 43(5):msag117, 2026.
- [76] Charles Yanofsky. Attenuation in the control of expression of bacterial operons. *Nature*, 289(5800):751–758, 1981. Mecanismo de atenuacion transcripcional en el operon Trp.
- [77] Yang Zhang and Jeffrey Skolnick. Scoring function for automated assessment of protein structure alignment. *Biophysical Journal*, 87(1):875–881, 2004. Definicion canonica y formulacion analitica de la metrica TM-score.